

A terminal window titled 'bash' with a temperature indicator '℃ 22' in the top right corner. It displays a JSON object representing a closed circuit state. The root object has a 'status' of 'UP' and a 'details' object. The 'details' object contains a 'product' object with 'status' 'UP' and its own 'details' object. This inner 'details' object lists various metrics: 'failureRate' (-1.0%), 'failureRateThreshold' (50.0%), 'slowCallRate' (-1.0%), 'slowCallRateThreshold' (100.0%), 'bufferedCalls' (0), 'slowCalls' (0), 'slowFailedCalls' (0), 'failedCalls' (0), 'notPermittedCalls' (0), and 'state' 'CLOSED'.

```
{
  "status": "UP",
  "details": {
    "product": {
      "status": "UP",
      "details": {
        "failureRate": "-1.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "-1.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 0,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 0,
        "notPermittedCalls": 0,
        "state": "CLOSED"
      }
    }
  }
}
```

Figura 8.5: Respuesta de circuito cerrado

Si algo está mal y el circuito está abierto, responderá con algo como lo siguiente:

A terminal window titled 'bash' with a temperature indicator '℃ 22' in the top right corner. It displays a JSON object representing an open circuit state. The root object has a 'status' of 'UNKNOWN' and a 'details' object. The 'details' object contains a 'product' object with 'status' 'CIRCUIT_OPEN' and its own 'details' object. This inner 'details' object lists various metrics: 'failureRate' (60.0%), 'failureRateThreshold' (50.0%), 'slowCallRate' (0.0%), 'slowCallRateThreshold' (100.0%), 'bufferedCalls' (5), 'slowCalls' (0), 'slowFailedCalls' (0), 'failedCalls' (3), 'notPermittedCalls' (3), and 'state' 'OPEN'.

```
{
  "status": "UNKNOWN",
  "details": {
    "product": {
      "status": "CIRCUIT_OPEN",
      "details": {
        "failureRate": "60.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "0.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 5,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 3,
        "notPermittedCalls": 3,
        "state": "OPEN"
      }
    }
  }
}
```

Figura 8.6: Respuesta de circuito abierto

Con la introducción de Resilience4j, hemos visto un ejemplo de cómo se puede usar el disyuntor para gestionar errores en un cliente REST. Concluamos este capítulo con una introducción sobre cómo se puede usar Spring Cloud para el rastreo distribuido.

Uso de trazado micrométrico y Zipkin para trazado distribuido

Para comprender qué está sucediendo en un sistema distribuido, como un panorama de sistemas de microservicios que cooperan, es fundamental poder rastrear y visualizar cómo fluyen las solicitudes y los mensajes entre los microservicios cuando se procesa una llamada externa al panorama del sistema.



Consulte el Capítulo 1, Introducción a los microservicios, en la sección Seguimiento distribuido , para obtener más información sobre este tema.

A partir de Spring Boot 3, el seguimiento distribuido se gestiona mediante Micrometer Tracing, que reemplaza a Spring Cloud Sleuth. Micrometer Tracing puede marcar solicitudes y mensajes/eventos que forman parte del mismo flujo de procesamiento con un ID de correlación común.

El seguimiento micrométrico también se puede utilizar para decorar registros de registro con identificadores de correlación para facilitar el seguimiento de registros de registro de diferentes microservicios que provienen del mismo flujo de procesamiento.

Zipkin es un sistema de rastreo distribuido (<http://zipkin.io>) A los que Micrometer Tracing puede enviar datos de rastreo para su almacenamiento y visualización. Más adelante, en el Capítulo 19, Registro centralizado con la pila EFK, aprenderemos a encontrar y visualizar registros de un mismo flujo de procesamiento mediante el ID de correlación.

La infraestructura para manejar información de rastreo distribuido en Micrometer Tracing y Zipkin se basa originalmente en Google Dapper (<https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>). En Dapper, la información de seguimiento de un flujo de trabajo completo se denomina árbol de seguimiento, y las subpartes del árbol, como las unidades básicas de trabajo, se denominan tramos. Los tramos, a su vez, pueden constar de subtramos, que forman el árbol de seguimiento. Un ID de correlación se denomina TraceId, y un tramo se identifica por su propio SpanId único , junto con el TraceId del árbol de seguimiento al que pertenece.



Una breve lección de historia sobre la evolución de los estándares (o al menos los esfuerzos comunes para establecer estándares abiertos de facto) para implementar el rastreo distribuido:

Google publicó el artículo sobre Dapper en 2010, después de usarlo internamente desde entonces. 2005.

En 2016, el proyecto OpenTracing se unió a CNCF. OpenTracing está fuertemente influenciado por Dapper y proporciona API independientes del proveedor y bibliotecas específicas del lenguaje para instrumentar el rastreo distribuido.

En 2019, el proyecto OpenTracing se fusionó con el proyecto OpenCensus , creando un nuevo proyecto CNCF: OpenTelemetry. El proyecto OpenCensus ofrece un conjunto de bibliotecas para recopilar métricas y trazas distribuidas.

A continuación se sugieren algunas URL para mayor lectura:

<https://opentracing.io>

<https://opencensus.io>

<https://opentelemetry.io>

En el Capítulo 14, "Comprendiendo el Rastreo Distribuido", veremos cómo usar el Rastreo Micrometer y Zipkin para rastrear el procesamiento que ocurre en nuestro entorno de microservicios. La siguiente es una captura de pantalla de la interfaz de usuario de Zipkin, que visualiza el árbol de rastreo creado como resultado de procesar la creación de un producto agregado:

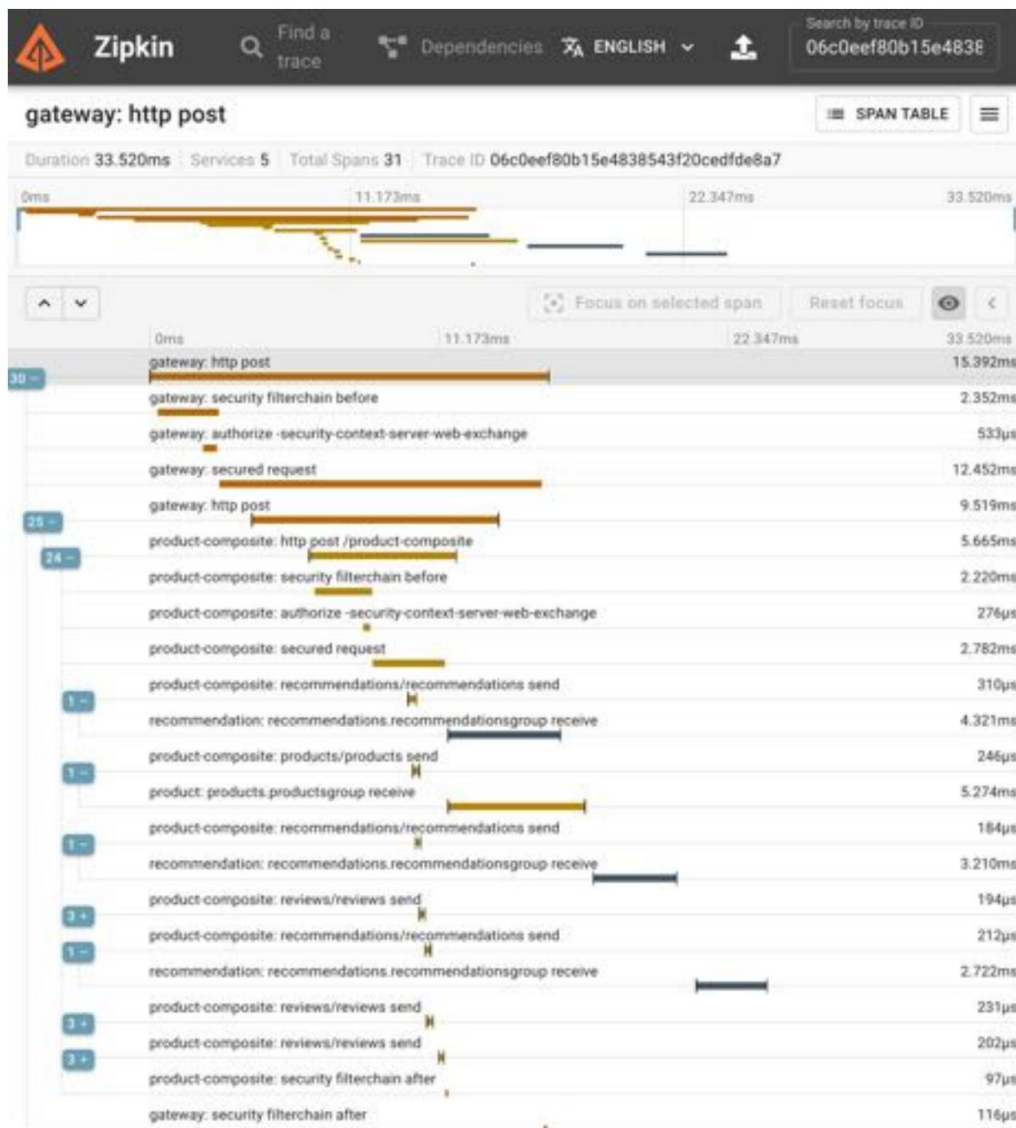


Figura 8.7: Árbol de seguimiento en Zipkin

En la captura de pantalla anterior, podemos ver que se envía una solicitud HTTP POST al servicio compuesto de producto a través de la puerta de enlace (nuestro servidor perimetral) y este responde publicando eventos de creación en los temas de productos, recomendaciones y reseñas. Estos eventos son consumidos por los tres microservicios principales de forma paralela y asíncrona, lo que significa que el servicio compuesto de producto...

El servicio no espera a que los microservicios principales completen su trabajo. Los datos en la creación
Los eventos se almacenan en la base de datos de cada microservicio.

Con la introducción de Micrometer Tracing y Zipkin para el seguimiento distribuido, hemos visto un ejemplo de seguimiento distribuido del procesamiento de una solicitud HTTP sincrónica externa que incluye el paso asincrónico de eventos entre los microservicios involucrados.

Resumen

En este capítulo, hemos visto cómo Spring Cloud ha evolucionado, pasando de estar centrado en OSS de Netflix a tener un alcance mucho mayor, actualmente, y se utiliza junto con herramientas como Resilience4j y Micrometer Tracing. También presentamos cómo los componentes de la última versión de Spring Cloud 2025 pueden usarse para implementar algunos de los patrones de diseño que describimos en el Capítulo 1, Introducción a los Microservicios, en la sección Patrones de diseño para microservicios. Estos patrones de diseño son necesarios para que un entorno de microservicios cooperativos esté listo para producción.

¡Pasa al siguiente capítulo para ver cómo podemos implementar el descubrimiento de servicios usando Netflix Eureka y Spring Cloud LoadBalancer!

Preguntas

1. ¿Cuál es el propósito de Netflix Eureka?
2. ¿Cuáles son las características principales de Spring Cloud Gateway?
3. ¿Qué backends son compatibles con Spring Cloud Config?
4. ¿Cuáles son las capacidades que ofrece Resilience4j?
5. ¿Para qué se utilizan los conceptos de árbol de seguimiento y de intervalo en el seguimiento distribuido y qué es?
¿El documento que los definió originalmente se llamó?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



9

Agregar descubrimiento de servicios Usando Netflix Eureka

En este capítulo, aprenderemos a usar Netflix Eureka como servicio de descubrimiento para microservicios basados en Spring Boot. Para que nuestros microservicios se comuniquen con Netflix Eureka, usaremos el módulo Spring Cloud para clientes de Netflix Eureka. Antes de profundizar en los detalles, explicaremos por qué se necesita un servicio de descubrimiento y por qué un servidor DNS no es suficiente.

En este capítulo se tratarán los siguientes temas:

- Introducción al descubrimiento de servicios
- Configuración de un servidor Netflix Eureka
- Conexión de microservicios a un servidor Eureka de Netflix
- Configuración de la configuración para uso de desarrollo
- Probar Netflix Eureka como servicio de descubrimiento

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter09`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver lo que se necesitó para agregar Netflix Eureka como servicio de descubrimiento al panorama de microservicios, puede compararlo con el código fuente del Capítulo 7, Desarrollo de Microservicios Reactivos. Puede usar su diff favorito. herramienta y compare las dos carpetas, es decir, \$BOOK_HOME/Chapter07 y \$BOOK_HOME/Chapter09.

Presentamos el descubrimiento de servicios

El descubrimiento de servicios es probablemente la función de soporte más importante para que un entorno de microservicios cooperativos esté listo para producción. Netflix Eureka fue el primer servidor de descubrimiento compatible con Spring Cloud.



Nos referimos estrictamente a un servicio de descubrimiento de servicios, pero en lugar de referirnos a él como un servicio de descubrimiento de servicios, simplemente lo llamaremos servicio de descubrimiento. Al referirnos a una implementación real de descubrimiento de servicios, como Netflix Eureka, se utilizará el término "servidor de descubrimiento" .

Veremos lo fácil que es registrar microservicios con Netflix Eureka al usar Spring Cloud.

También aprenderemos cómo un cliente puede usar Spring Cloud LoadBalancer para enviar solicitudes HTTP a una de las instancias registradas en Netflix Eureka. Finalmente, intentaremos escalar los microservicios, tanto verticalmente como horizontalmente, y realizaremos pruebas disruptivas para ver cómo Netflix Eureka puede gestionar diferentes tipos de fallos.

Antes de entrar en los detalles de implementación, veremos los siguientes temas:

- El problema con el descubrimiento de servicios basado en DNS
- Desafíos con el descubrimiento de servicios
- Descubrimiento de servicios con Netflix Eureka en Spring Cloud

El problema con el descubrimiento de servicios basado en DNS

¿Por qué no podemos simplemente iniciar nuevas instancias de un microservicio y confiar en el DNS round-robin?

La idea detrás del DNS round-robin es que cada instancia de un microservicio registre su dirección IP con el mismo nombre en un servidor DNS. Cuando un cliente solicita direcciones IP para el nombre DNS, el servidor DNS devolverá una lista de direcciones IP de las instancias registradas. El cliente puede usar esta lista de direcciones IP para enviar solicitudes a las instancias del microservicio de forma round-robin, utilizando las direcciones IP una tras otra.

¡Hagámoslo y veamos qué pasa! Sigue estos pasos:

1. Suponiendo que ha seguido las instrucciones del Capítulo 7, Desarrollo de microservicios reactivos , inicie el entorno del sistema e inserte algunos datos de prueba con lo siguiente dominio:

```
cd $BOOK_HOME/Capítulo07
./test-em-all.bash inicio
```

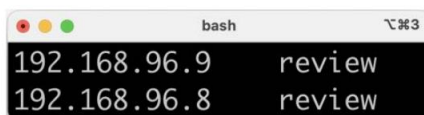
2. Ampliar el microservicio de revisión a dos instancias:

```
docker composer up -d --scale revisión=2
```

3. Solicite al servicio de producto compuesto las direcciones IP que encuentra para el microservicio de revisión :

```
Revisión de hosts de Getent, compuesto de producto de Docker Compose Exec
```

Espere una respuesta como la siguiente:



```
bash ㉿%3
192.168.96.9    review
192.168.96.8    review
```

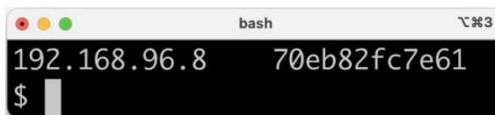
Figura 9.1: Revisar las direcciones IP de los microservicios

Genial, el servicio de producto compuesto ve dos direcciones IP: en mi caso, 192.168.96.9 y 192.168.96.8 – ¡uno para cada instancia del microservicio de revisión !

4. Si lo desea, puede verificar que estas direcciones IP sean correctas con los siguientes comandos. Estos comandos solicitan la dirección IP de cada instancia del microservicio de revisión :

```
docker compose exec --index=1 revisar cat /etc/hosts
docker compose exec --index=2 revisar cat /etc/hosts
```

La última línea de la salida de cada comando debe contener una de las direcciones IP, como se muestra en el código anterior. A continuación, un ejemplo:



```
bash ㉿%3
192.168.96.8    70eb82fc7e61
$
```

Figura 9.2: Salida de la dirección IP

5. Ahora, probemos un par de llamadas al servicio compuesto por productos y veamos si...

utiliza ambas instancias del microservicio de revisión :

```
curl localhost:8080/producto-compuesto/1 -s | jq -r .serviceAddresses.rev
```

Lamentablemente, solo recibiremos respuestas de una de las instancias de microservicio, como en este ejemplo:

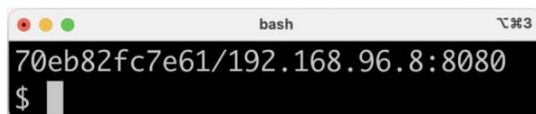


Figura 9.3: Respuesta de una sola instancia de revisión

¡Eso fue decepcionante!

Bien, ¿qué está pasando aquí?

Un cliente DNS solicita a un servidor DNS que resuelva un nombre DNS y recibe una lista de direcciones IP. A continuación, el cliente DNS prueba las direcciones IP recibidas una por una hasta encontrar una que funcione; en la mayoría de los casos, la primera de la lista. Un cliente DNS normalmente conserva una dirección IP que funciona; no aplica un enfoque de rotación por solicitud. Además, ni la implementación típica de un servidor DNS ni el propio protocolo DNS son adecuados para gestionar instancias de microservicios volátiles que aparecen y desaparecen constantemente. Por ello, aunque la rotación por turnos basada en DNS es atractiva en teoría, no resulta muy práctica para el descubrimiento de servicios de instancias de microservicios.

Antes de continuar y aprender cómo gestionar el descubrimiento de servicios de una mejor manera, apaguemos el panorama del sistema:

```
Docker Compose Down
```

Desafíos del descubrimiento de servicios

Por lo tanto, necesitamos algo un poco más potente que un DNS simple para realizar un seguimiento de los micros disponibles.

¡Instancias de servicio!

Debemos tener en cuenta lo siguiente cuando realizamos el seguimiento de muchas partes móviles pequeñas, es decir, instancias de microservicios:

- Se pueden iniciar nuevas instancias en cualquier momento.
- Las instancias existentes pueden dejar de responder y eventualmente bloquearse en cualquier momento.

- Algunas de las instancias fallidas podrían volver a funcionar correctamente después de un tiempo y deberían comenzar a recibir tráfico nuevamente, mientras que otras no lo harán y deberían eliminarse del registro del servicio.
- Algunas instancias de microservicio pueden tardar algún tiempo en iniciarse; es decir, solo porque puedan recibir solicitudes HTTP no significa que se deba enrutar tráfico hacia ellas.
- Particiones de red no deseadas y otros errores relacionados con la red pueden ocurrir en cualquier momento.

Construir un servidor de descubrimiento robusto y resistente no es tarea fácil, como mínimo. ¡Veamos cómo podemos usar Netflix Eureka para afrontar estos desafíos!

Descubrimiento de servicios con Netflix Eureka en Spring Cloud

Netflix Eureka implementa el descubrimiento de servicios del lado del cliente, lo que significa que los clientes ejecutan software que se comunica con el servidor de descubrimiento, Netflix Eureka, para obtener información sobre las instancias de microservicio disponibles. Esto se ilustra en el siguiente diagrama:

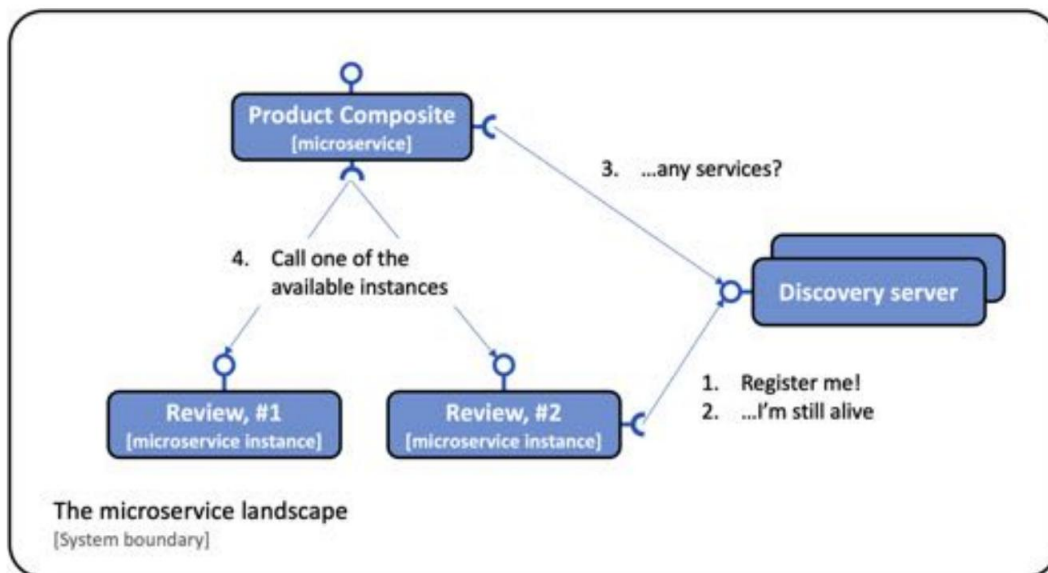


Figura 9.4: Diagrama del servidor de descubrimiento

El proceso es el siguiente:

1. Siempre que se inicia una instancia de microservicio (por ejemplo, el servicio de revisión), se registra a uno de los servidores de Eureka.
2. Periódicamente, cada instancia de microservicio envía un mensaje de latido al servidor Eureka para indicarle que la instancia de microservicio está bien y está lista para recibir solicitudes.

3. Los clientes, por ejemplo, el servicio de compuestos de productos , utilizan una biblioteca de clientes que solicita al servicio Eureka información sobre los servicios disponibles.
4. Cuando el cliente necesita enviar una solicitud a otro microservicio, ya cuenta con una lista de instancias disponibles en su biblioteca cliente y puede seleccionar una sin consultar al servidor de descubrimiento. Normalmente, las instancias disponibles se seleccionan por turnos; es decir, se llaman una tras otra antes de volver a llamar a la primera.



En el Capítulo 17, Implementación de características de Kubernetes para simplificar el panorama del sistema, analizaremos un enfoque alternativo para proporcionar un servicio de descubrimiento utilizando el concepto de servicio del lado del servidor en Kubernetes.

Spring Cloud incluye una abstracción para la comunicación con un servicio de descubrimiento como Netflix Eureka y proporciona una interfaz llamada `DiscoveryClient`. Esta permite interactuar con un servicio de descubrimiento para obtener información sobre los servicios e instancias disponibles. Las implementaciones de la interfaz `DiscoveryClient` también permiten registrar automáticamente una aplicación Spring Boot en el servidor de descubrimiento.

Spring Boot puede encontrar implementaciones de la interfaz `DiscoveryClient` automáticamente durante el inicio, por lo que solo necesitamos crear una dependencia en la implementación correspondiente para conectarnos a un servidor de descubrimiento. En el caso de Netflix Eureka, la dependencia que utilizan nuestros microservicios es `spring-cloud-starter-netflix-eureka-client`.



Spring Cloud también tiene implementaciones de `DiscoveryClient` que admiten el uso de Apache ZooKeeper o HashiCorp Consul como servidor de descubrimiento.

Spring Cloud también viene con una abstracción: la interfaz `LoadBalancerClient` , para los clientes que desean realizar solicitudes a través de un balanceador de carga a instancias registradas en el servicio de descubrimiento. El cliente HTTP reactivo estándar, `WebClient`, se puede configurar para utilizar `LoadBalancerClient` Implementación. Al añadir la anotación `@LoadBalanced` a una declaración `@Bean` que devuelve un objeto `WebClient.Builder` , se inyectará una implementación de `LoadBalancerClient` en la instancia de `Builder` como una función `ExchangeFilterFunction`. Más adelante en este capítulo, en la sección "Conexión de microservicios a un servidor Netflix Eureka" , veremos algunos ejemplos de código fuente que ilustran cómo se puede usar esto.

En resumen, Spring Cloud facilita enormemente el uso de Netflix Eureka como servicio de descubrimiento. Con esta introducción al descubrimiento de servicios y sus desafíos, así como la forma en que Netflix Eureka se puede usar junto con Spring Cloud, estamos listos para aprender a configurar un servidor de Netflix Eureka.

Configuración de un servidor Netflix Eureka

En esta sección, aprenderemos a configurar un servidor Netflix Eureka para el descubrimiento de servicios.

Configurar un servidor Netflix Eureka con Spring Cloud es muy sencillo: solo tienes que seguir estos pasos:

1. Cree un proyecto Spring Boot usando Spring Initializr, como se describe en el Capítulo 3, Creación de un conjunto de microservicios cooperativos, en la sección Uso de Spring Initializr para generar código esqueleto .
2. Agregue una dependencia a `spring-cloud-starter-netflix-eureka-server`.
3. Agregue la anotación `@EnableEurekaServer` a la clase de aplicación.
4. Agregue un Dockerfile, similar a los Dockerfiles que se utilizan para nuestros microservicios, con la excepción de que exportamos el puerto Eureka predeterminado, 8761, en lugar del puerto predeterminado para nuestros microservicios, 8080.
5. Agregue el servidor Eureka a nuestros tres archivos Docker Compose, es decir, `docker-compose.yml`, `docker-compose-partitions.yml` y `docker-compose-kafka.yml`, de la siguiente manera:

```
¡Eureka!:  
compilación: spring-cloud/eureka-server  
límite de memoria: 512 m  
puertos:  
- "8761:8761"
```



Consejo rápido: Mejore su experiencia de codificación con AI Code Explainer y funciones de Copia rápida . Abra este libro en el Lector Packt de última generación. Haga clic en el botón Copiar.

(1) para copiar rápidamente el código en su entorno de codificación, o haga clic en Explicar botón

(2) para que el asistente de IA le explique un bloque de código.



```
function calculate(a, b) {  
  return {sum: a + b};  
};
```

Copy

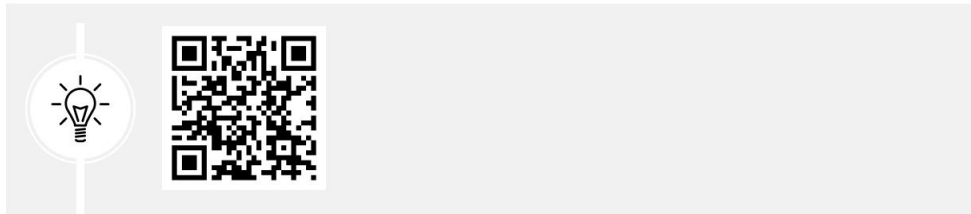
Explain

1

2



El lector Packt de nueva generación se incluye gratis con la compra de este libro. Escanea el código QR o visita packtpub.com/unlock. Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



6. Finalmente, agregue la configuración. Consulte la sección "Configuración para desarrollo " de este capítulo, donde explicaremos la configuración tanto del servidor Eureka como de nuestros microservicios.

¡Eso es todo lo que se necesita!

Puede encontrar el código fuente del servidor Eureka en `$BOOK_HOME/Chapter09/spring-cloud/carpeta eureka-server`.

Ahora que hemos configurado un servidor Netflix Eureka para el descubrimiento de servicios, estamos listos para aprender cómo para conectarle microservicios.

Conexión de microservicios a un servidor Eureka de Netflix

En esta sección, aprenderemos cómo conectar instancias de microservicio a un servidor Netflix Eureka.

Aprenderemos cómo las instancias de microservicio se registran en el servidor Eureka durante su inicio y cómo los clientes pueden usar el servidor Eureka para encontrar las instancias de microservicio que necesitan.
quiero llamar

Para poder registrar una instancia de microservicio en el servidor Eureka, necesitamos hacer lo siguiente:

1. Agregue una dependencia a `spring-cloud-starter-netflix-eureka-client` en el archivo de compilación, `construir.gradle`:

```
Implementación 'org.springframework.cloud:spring-cloud-starter-netflix-eureka-client'
```

2. Al ejecutar pruebas en un solo microservicio, no queremos depender de que el servidor Eureka esté activo. Por lo tanto, deshabilitaremos el uso de Netflix Eureka en todas las pruebas de Spring Boot, es decir, las pruebas JUnit anotadas con `@SpringBootTest`. Esto se puede hacer añadiendo la propiedad `eureka.client.enabled` y estableciéndola en falso en la anotación.

Así:

```
@SpringBootTest(webEnvironment=RANDOM_PORT, propiedades = {"eureka.  
cliente.habilitado=falso"})
```

3. Finalmente, agregue la configuración. Vaya a la sección "Configuración para desarrollo", donde revisaremos la configuración del servidor Eureka.
y nuestros microservicios.

Hay una propiedad en la configuración que es especialmente importante: `spring.application.name`. Se utiliza para asignar a cada microservicio un nombre de host virtual, un nombre que el servicio Eureka utiliza para identificar cada microservicio. Los clientes de Eureka usarán este nombre de host virtual en las URL que se utilizan para realizar llamadas HTTP al microservicio, como veremos más adelante.

Para poder buscar instancias de microservicio disponibles a través del servidor Eureka en el microservicio compuesto por producto, también debemos hacer lo siguiente:

1. Agregue un bean Spring en la clase de aplicación principal, `ProductCompositeServiceApplication`,
on, que crea un generador de `WebClient` que reconoce el balanceador de carga :

```
@Frijol
@LoadBalanced
público WebClient.Builder loadBalancedWebClientBuilder() {
    devolver WebClient.builder();
}
```



Para obtener más información sobre cómo utilizar una instancia de `WebClient` como equilibrador de carga, Cliente de Aker, consulte <https://docs.spring.io/spring-cloud-commons/docs/actual/referencia/html/#webclinet-loadbalancer-client>.

2. El bean `WebClient-builder` puede ser utilizado por la clase de integración,

`ProductCompositeIntegration`, inyectándolo en el constructor:

```
cliente web privado cliente web;

público ProductoCompuestoIntegración(
    WebClient.Builder webClientBuilder,
    ...
){
    este.webClient = webClientBuilder.build();
    ...
}
```

El constructor utiliza el constructor inyectado para crear el `webClient`.

Una vez creado un WebClient , es inmutable. Esto significa que puede ser reutilizado por solicitudes concurrentes sin riesgo de interferir entre sí.

3. Ahora podemos deshacernos de nuestra configuración codificada de microservicios disponibles en application.yml. Se ve así:

```
aplicación:  
  
  producto-servicio:  
    anfitrión: localhost  
    puerto: 7002  
  servicio de recomendación:  
    anfitrión: localhost  
    puerto: 7003  
  servicio de revisión:  
    anfitrión: localhost  
    puerto: 70034
```

4. El código correspondiente en la clase de integración, ProductCompositeIntegration, que gestionaba la configuración predefinida, se simplifica y se reemplaza por una declaración de las URL base a las API de los microservicios principales. Esto se muestra en el siguiente código:

```
cadena final estática privada PRODUCT_SERVICE_URL = "http://producto";  
Cadena final estática privada RECOMENDACIÓN_SERVICIO_URL = "http://  
recomendación";  
cadena final estática privada REVIEW_SERVICE_URL = "http://review";
```

Los nombres de host en las URL anteriores no son nombres DNS reales. Son los nombres de host virtuales que utilizan los microservicios al registrarse en el servidor Eureka; es decir, los valores de la propiedad spring.application.name .

Ahora que hemos visto cómo conectar instancias de microservicio a un servidor Eureka de Netflix, podemos avanzar y aprender cómo configurar el servidor Eureka y las instancias de microservicio que se conectan a él.

Configuración de la configuración para uso de desarrollo

Ahora es el momento de realizar la parte más complicada de configurar Netflix Eureka como servicio de descubrimiento:

configurar una configuración funcional tanto para el servidor Eureka como para sus clientes, nuestras instancias de microservicio.

Netflix Eureka es un servidor de descubrimiento altamente configurable que puede configurarse para diversos casos de uso y ofrece características de ejecución robustas, resilientes y con tolerancia a fallos. Una desventaja de esta flexibilidad y robustez es su abrumadora cantidad de opciones de configuración.

Afortunadamente, Netflix Eureka viene con buenos valores predeterminados para la mayoría de los parámetros configurables, al menos cuando se trata de usarlos en un entorno de producción.

Al usar Netflix Eureka durante el desarrollo, los valores predeterminados provocan tiempos de inicio largos. Por ejemplo, un cliente puede tardar mucho en realizar una primera llamada exitosa a una instancia de microservicio registrada en el servidor Eureka.

Se puede experimentar un tiempo de espera de hasta dos minutos al utilizar los valores de configuración predeterminados. Este tiempo de espera se suma al tiempo que tarda el servicio Eureka y los microservicios en iniciarse.

Este tiempo de espera se debe a que los procesos involucrados necesitan sincronizar la información de registro. Las instancias de microservicio deben registrarse en el servidor Eureka, y el cliente debe recopilar información de este. Esta comunicación se basa principalmente en latidos, que ocurren cada 30 segundos por defecto. También intervienen un par de cachés, lo que ralentiza la propagación de las actualizaciones.

Utilizaremos una configuración que minimice este tiempo de espera, lo cual es útil durante el desarrollo.

En entornos de producción, los valores predeterminados deben usarse como punto de partida.



Solo usaremos una instancia del servidor Netflix Eureka, lo cual es adecuado en un entorno de desarrollo. En un entorno de producción, siempre se recomiendan dos o más instancias para garantizar la alta disponibilidad del servidor Netflix Eureka.

Comencemos a aprender qué tipos de parámetros de configuración necesitamos conocer.

Parámetros de configuración de Eureka

Los parámetros de configuración de Eureka se dividen en tres grupos:

- Parámetros para el servidor Eureka, con el prefijo `eureka.server`.
- Parámetros para clientes de Eureka, con el prefijo `eureka.client`. Esto es para clientes que desean... para comunicarse con el servidor Eureka.
- Parámetros para instancias de Eureka, con el prefijo `eureka.instance`. Esto es para el micros-
Instancias de servicio que desean registrarse en el servidor Eureka.

Algunos de los parámetros disponibles se describen en la documentación de Spring Cloud Netflix: <https://docs.spring.io/spring-cloud-netflix/docs/current/reference/html/>.

Para obtener una lista extensa de los parámetros disponibles, recomiendo leer el código fuente:

- Para conocer los parámetros del servidor Eureka, consulte `org.springframework.cloud.netflix.eureka`.
clase `server.EurekaServerConfigBean` para valores predeterminados y `com.netflix.eureka`.
Interfaz `EurekaServerConfig` para la documentación relevante

- Para conocer los parámetros del cliente de Eureka, consulte [org.springframework.cloud.netflix.eureka](https://docs.spring.io/spring-cloud-netflix/docs/reference/html/#eureka-client).

Clase `EurekaClientConfigBean` para los valores predeterminados y la documentación

- Para los parámetros de instancia de Eureka, consulte [org.springframework.cloud.netflix.eureka](https://docs.spring.io/spring-cloud-netflix/docs/reference/html/#eureka-client).

Clase `eureka.EurekaInstanceConfigBean` para valores predeterminados y documentación

Comencemos a aprender sobre los parámetros de configuración para el servidor Eureka.

Configuración del servidor Eureka

Para configurar el servidor Eureka para su uso en un entorno de desarrollo, se requiere la siguiente configuración se puede utilizar:

```
servidor:

  puerto: 8761

¡Eureka!

  instancia:

    nombre de host: localhost

  cliente:

    registrarseConEureka: falso
    fetchRegistry: falso
    URL del servicio:

      Zona predeterminada: http://${eureka.instance.hostname}:${server.port}/
¡Eureka!

servidor:

  Tiempo de espera en ms cuando la sincronización está vacía: 0

  intervalo de actualización de caché de respuesta: 5000 ms
```

La primera parte de la configuración, para una instancia y un cliente de Eureka, es la configuración estándar para un servidor Eureka independiente. Para más detalles, consulte la documentación de Spring Cloud, a la que hicimos referencia anteriormente. Los dos últimos parámetros utilizados para el servidor Eureka, `waitTimeInMsWhenSyncEmpty` y `response-cache-update-interval-ms`, se utilizan para minimizar el tiempo de inicio.

Con el servidor Eureka configurado, estamos listos para ver cómo se pueden configurar los clientes al servidor Eureka, es decir las instancias del microservicio.

Configuración de clientes en el servidor Eureka

Para poder conectarse al servidor Eureka, los microservicios tienen la siguiente configuración:

```
¡Eureka!:
  cliente:
    URL del servicio:
      Zona predeterminada: http://localhost:8761/eureka/
    initialInstanceInfoReplicationIntervalSeconds: 5
    Intervalo de obtención de registro en segundos: 5
  instancia:
    arrendamientoRenewalIntervalInSeconds: 5
    Duración del vencimiento del arrendamiento en segundos: 5

spring.config.activate.on-profile: docker

eureka.client.serviceUrl.defaultZone: http://eureka:8761/eureka/
```

El parámetro `eureka.client.serviceUrl.defaultZone` se utiliza para encontrar el servidor Eureka. Para ello, se utiliza el nombre de host `localhost` al ejecutarse sin Docker y el nombre de host de Eureka al ejecutarse como contenedores en Docker. Los demás parámetros se utilizan para minimizar el tiempo de inicio y el tiempo que tarda en anular el registro de una instancia de microservicio detenida.

Ahora tenemos todo lo necesario para probar el servicio de descubrimiento utilizando el servidor Eureka de Netflix junto con nuestros microservicios.

Probando el servicio de descubrimiento

Con todos los detalles listos, estamos listos para probar Netflix Eureka:

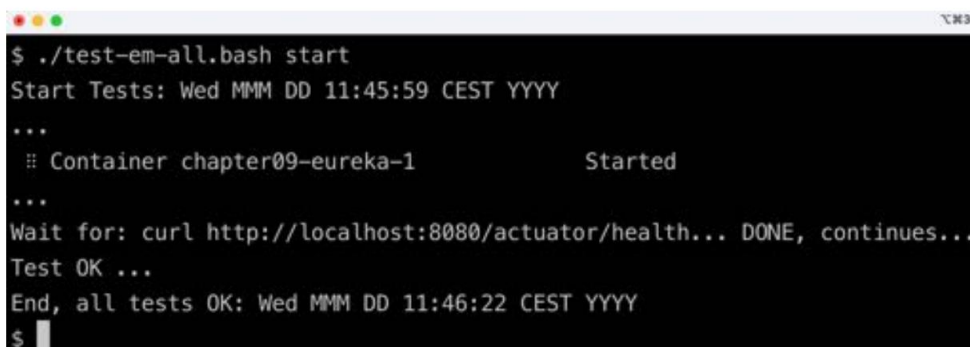
1. Primero, construya las imágenes de Docker con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo09 ./
gradlew compilación y compilación de docker compose
```

2. A continuación, inicie el sistema y ejecute las pruebas habituales con el siguiente comando:

```
./test-em-all.bash inicio
```

Espere un resultado similar al que hemos visto en capítulos anteriores:

A terminal window with a black background and white text. The text shows the execution of a script that starts tests, waits for a health check, and reports success. The output is as follows:

```
$ ./test-em-all.bash start
Start Tests: Wed MMM DD 11:45:59 CEST YYYY
...
:: Container chapter09-eureka-1          Started
...
Wait for: curl http://localhost:8080/actuator/health... DONE, continues...
Test OK ...
End, all tests OK: Wed MMM DD 11:46:22 CEST YYYY
$
```

Figura 9.5: Salida de prueba exitosa

Con el panorama del sistema en funcionamiento, podemos comenzar a probar cómo ampliar el número de instancias para uno de los microservicios.

Ampliación de escala

Ejecute los siguientes comandos para intentar ampliar un servicio:

1. Inicie dos instancias de microservicio de revisión adicionales :

```
docker composer up -d --scale revisión=3
```



Con el comando anterior, solicitamos a Docker Compose que ejecute tres instancias del servicio de revisión . Dado que una instancia ya está en ejecución, se iniciarán dos nuevas.

2. Una vez que las nuevas instancias estén en funcionamiento, navegue a <http://localhost:8761/> y espere algo como lo siguiente:

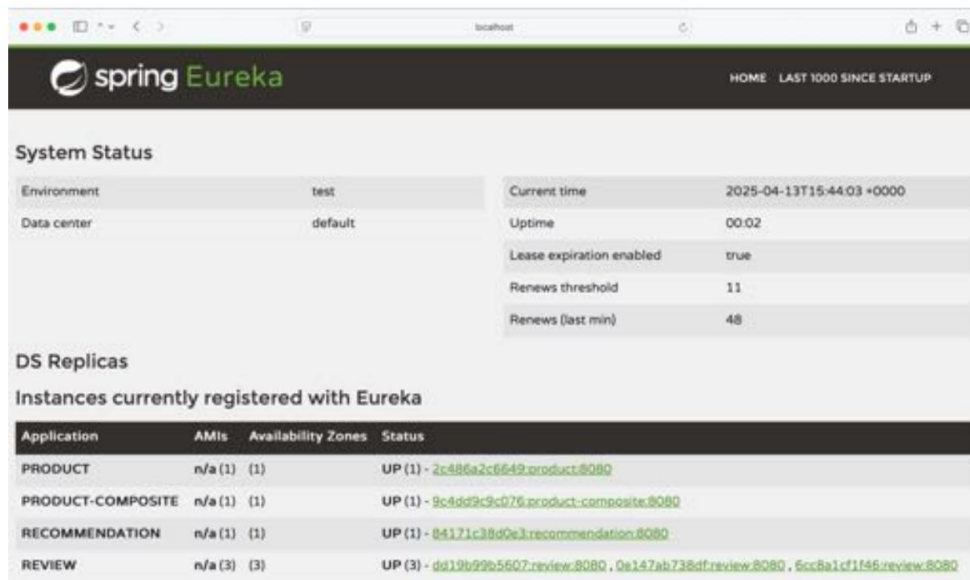


Figura 9.6: Visualización de instancias registradas en Eureka

Verifique que pueda ver tres instancias de revisión en la interfaz web de Netflix Eureka, como se muestra en la captura de pantalla anterior.

3. Una forma de saber cuándo las nuevas instancias están en funcionamiento es ejecutar este comando:

```
Revisión de registros de Docker Compose | grep iniciado
```

Espere un resultado como el siguiente:

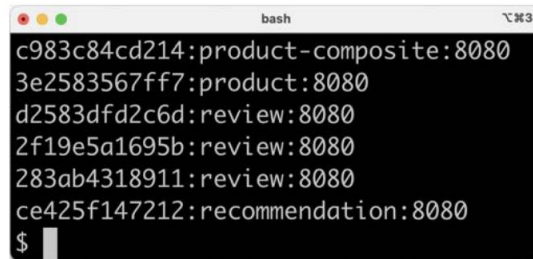


Figura 9.7: Nuevas instancias de revisión

4. También podemos usar una API REST que expone el servicio Eureka. Para obtener una lista de IDs de instancia, Podemos emitir un comando curl , como este:

```
curl -H "accept:aplicación/json" localhost:8761/eureka/apps -s | jq  
-r .aplicaciones.aplicación[].instancia[].idinstancia
```

Espere una respuesta similar a la siguiente:



```
bash  
c983c84cd214:product-composite:8080  
3e2583567ff7:product:8080  
d2583dfd2c6d:review:8080  
2f19e5a1695b:review:8080  
283ab4318911:review:8080  
ce425f147212:recommendation:8080  
$
```

Figura 9.8: Lista de identificadores de instancias de microservicios

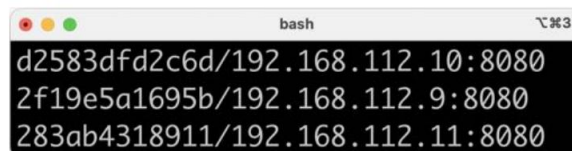
5. Si revisa el script de prueba, test-em-all.bash, encontrará nuevas pruebas que verifican que podemos acceder a la API REST de Eureka y que informa cuatro instancias:

```
# Verificar el acceso a Eureka y que los cuatro microservicios estén # registrados  
en Eureka  
assertCurl 200 "curl -H "accept:aplicación/json" $HOST:8761/eureka/  
aplicaciones -s"  
assertEqual 4 $(echo $RESPONSE | jq ".applications.application | length")
```

6. Ahora que todas las instancias están en funcionamiento, probemos el balanceador de carga del lado del cliente realizando algunas solicitudes y centrándonos en la dirección del servicio de revisión en las respuestas, como se muestra a continuación:

```
curl localhost:8080/producto-compuesto/1 -s | jq -r  
.serviceAddresses.rev
```

Espere respuestas similares a la siguiente:



```
bash  
d2583dfd2c6d/192.168.112.10:8080  
2f19e5a1695b/192.168.112.9:8080  
283ab4318911/192.168.112.11:8080
```

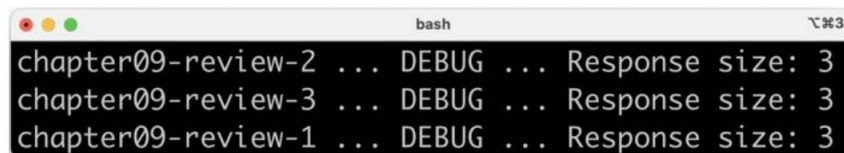
Figura 9.9: Revisar direcciones de servicio

Tenga en cuenta que la dirección del servicio de revisión cambia en cada respuesta; el balanceador de carga utiliza una lógica round-robin para llamar a las instancias de revisión disponibles , ¡una a la vez!

7. También podemos echar un vistazo a los registros de la instancia de revisión con el siguiente comando:

```
Revisión de registros de Docker Compose | grep "Tamaño de respuesta"
```

Verá un resultado similar al siguiente:

A screenshot of a terminal window with a dark background. The window title is 'bash' and it shows three lines of log output. Each line starts with an instance name, followed by '... DEBUG ...', and then 'Response size: 3'. The instance names are 'chapter09-review-2', 'chapter09-review-3', and 'chapter09-review-1' in that order from top to bottom.

```
chapter09-review-2 ... DEBUG ... Response size: 3
chapter09-review-3 ... DEBUG ... Response size: 3
chapter09-review-1 ... DEBUG ... Response size: 3
```

Figura 9.10: Revisar registros de instancias

En la salida anterior, podemos ver cómo las tres instancias de microservicio de revisión, `review_1`, `review_2` y `review_3`, a su vez, han respondido a las solicitudes.

También podemos intentar reducir las instancias, lo que haremos a continuación.

Reducción de escala

Veamos también qué sucede si perdemos una instancia del microservicio de revisión . Ejecute lo siguiente comandos:

1. Podemos simular una instancia que se detiene inesperadamente ejecutando el siguiente comando:

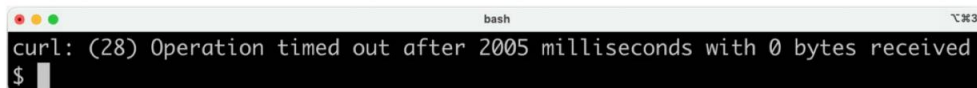
```
docker composer up -d --scale revisión=2
```

2. Tras el cierre de la instancia de revisión , las llamadas a la API pueden fallar durante un breve periodo . Esto se debe al tiempo que tarda la información sobre la instancia perdida en propagarse al cliente (el servicio compuesto por el producto) . Durante este periodo, el balanceador de carga del cliente podría seleccionar la instancia que ya no existe.

Para evitar que esto ocurra, se pueden utilizar mecanismos de resiliencia como tiempos de espera y reintentos. En el Capítulo 13, "Mejora de la resiliencia con Resilience4j", veremos cómo aplicar esto. Por ahora, especifiquemos un tiempo de espera en nuestro comando `curl` , usando la opción `-m 2` para especificar que no esperaremos más de dos segundos por una respuesta:

```
curl localhost:8080/producto-compuesto/1 -m 2
```

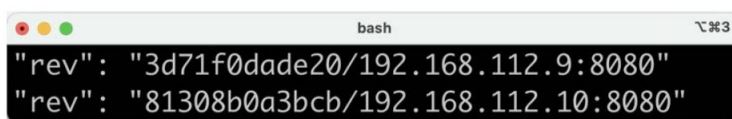
Si se produce un tiempo de espera, es decir, el balanceador de carga del lado del cliente intenta llamar a una instancia que ya no existe, se espera la siguiente respuesta de curl:

A terminal window titled 'bash' with a prompt character '#3'. It displays the output of a curl command: 'curl: (28) Operation timed out after 2005 milliseconds with 0 bytes received'. The prompt '\$ ' is visible on the next line.

```
bash
curl: (28) Operation timed out after 2005 milliseconds with 0 bytes received
$
```

Figura 9.11: Respuesta de curl si se produce un tiempo de espera

Además de eso, debemos esperar respuestas normales de las dos instancias restantes; es decir, el campo `serviceAddresses.rev` debe contener las direcciones de las dos instancias, como en lo siguiente:

A terminal window titled 'bash' with a prompt character '#3'. It displays two lines of JSON output from a curl command. The first line is '"rev": "3d71f0dade20/192.168.112.9:8080"' and the second line is '"rev": "81308b0a3bcb/192.168.112.10:8080"'.

```
bash
"rev": "3d71f0dade20/192.168.112.9:8080"
"rev": "81308b0a3bcb/192.168.112.10:8080"
```

Figura 9.12: Respuestas normales de las instancias restantes

En el ejemplo de salida anterior, se observa que se informan dos nombres de contenedor y direcciones IP diferentes. Esto significa que las solicitudes fueron atendidas por los dos microservicios restantes. instancias.

Después de probar la reducción de las instancias de microservicios, podemos probar algo un poco más disruptivo: detener el servidor Eureka y ver qué sucede cuando el servidor de descubrimiento no está disponible temporalmente.

Pruebas disruptivas con el servidor Eureka

¡Traigamos un poco de desorden a nuestro servidor Eureka y veamos cómo lo gestiona el sistema!

Para empezar, ¿qué pasa si colapsamos el servidor Eureka?

Siempre que los clientes hayan leído la información sobre las instancias de microservicio disponibles desde el servidor Eureka antes de que se detenga, los clientes estarán bien ya que almacenan la información en caché localmente. Sin embargo, las nuevas instancias no estarán disponibles para los clientes y no se les notificará si alguna instancia en ejecución finaliza. Por lo tanto, las llamadas a instancias que ya no se están ejecutando causarán fracasos.

¡Vamos a probarlo!

Deteniendo el servidor Eureka

Para simular un bloqueo del servidor Eureka, siga estos pasos:

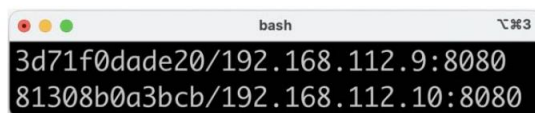
1. Primero, detenga el servidor Eureka y mantenga las dos instancias de revisión en funcionamiento:

```
docker composer arriba -d --scale revisión=2 --scale eureka=0
```

2. Pruebe un par de llamadas a la API y extraiga la dirección del servicio de revisión :

```
curl localhost:8080/producto-compuesto/1 -s | jq -r  
.serviceAddresses.rev
```

La respuesta, al igual que antes de detener el servidor Eureka, contendrá las direcciones de las dos instancias de revisión , de la siguiente manera:



```
bash  
3d71f0dade20/192.168.112.9:8080  
81308b0a3bcb/192.168.112.10:8080
```

Figura 9.13: Respuesta con dos direcciones de instancia de revisión

Esto demuestra que el cliente puede realizar llamadas a instancias existentes, incluso aunque el servidor Eureka ya no esté en ejecución.

Poner en marcha una instancia adicional del servicio del producto

Como prueba final de los efectos de un servidor Eureka colapsado, veamos qué sucede si iniciamos una nueva instancia del microservicio del producto . Realice los siguientes pasos:

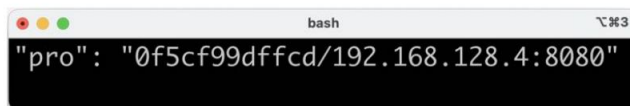
1. Intentemos iniciar una nueva instancia del servicio del producto :

```
docker composer arriba -d --scale revisión=2 --scale eureka=0 --scale producto=2
```

2. Llame a la API un par de veces y extraiga la dirección del servicio del producto con el siguiente comando:

```
curl localhost:8080/producto-compuesto/1 -s | jq  
-r .serviceAddresses.pro
```

Dado que no hay ningún servidor Eureka en ejecución, el cliente no será notificado de la nueva instancia del producto y, por lo tanto, todas las llamadas se dirigirán a la primera instancia, como en el siguiente ejemplo:



```
bash  
"pro": "0f5cf99dffcd/192.168.128.4:8080"
```

Figura 9.14: Dirección de la primera instancia del producto únicamente

Hemos visto algunos de los aspectos más importantes de no tener un servidor Netflix Eureka en funcionamiento. Concluamos la sección sobre pruebas disruptivas reiniciando el servidor Netflix Eureka y observando cómo el sistema gestiona la autorreparación, es decir, la resiliencia.

Iniciando nuevamente el servidor Eureka

En esta sección, finalizaremos las pruebas disruptivas reiniciando el servidor Eureka. También verificaremos que el entorno del sistema se autorrecupere, lo que significa que la nueva instancia del producto...

El microservicio se registra en el servidor Eureka de Netflix y el cliente se actualiza mediante este servidor. Siga estos pasos:

1. Inicie el servidor Eureka con el siguiente comando:

```
docker composer arriba -d --scale revisión=1 --scale eureka=1 --scale producto=2
```

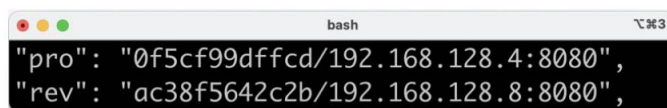
2. Realice la siguiente llamada un par de veces para extraer las direcciones del producto y el Servicio de revisión :

```
curl localhost:8080/producto-compuesto/1 -s | jq -r .serviceAddresses
```

Verifique que ocurra lo siguiente:

- Todas las llamadas van a la instancia de revisión restante , lo que demuestra que el cliente ha detectado que la segunda instancia de revisión ha desaparecido.
- Las llamadas al servicio del producto se equilibran en las dos instancias del producto , lo que demuestra que el cliente ha detectado que hay dos instancias del producto . disponible.

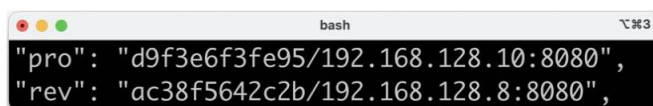
La respuesta debe contener la misma dirección para la instancia de revisión y dos direcciones diferentes para las dos instancias del producto , como se muestra en los siguientes dos ejemplos:



```
bash
{"pro": "0f5cf99dfcd/192.168.128.4:8080",  
"rev": "ac38f5642c2b/192.168.128.8:8080",
```

Figura 9.15: Direcciones de productos y reseñas

Esta es la segunda respuesta:



```
bash
{"pro": "d9f3e6f3fe95/192.168.128.10:8080",  
"rev": "ac38f5642c2b/192.168.128.8:8080",
```

Figura 9.16: Direcciones de productos y reseñas

Las direcciones IP 192.168.128.4 y 192.168.128.10 pertenecen a las dos instancias del producto . 192.168.128.8 es la dirección IP de la única instancia de revisión restante .

En resumen, el servidor Eureka proporciona una implementación robusta y resiliente de un servicio de descubrimiento. Si se desea una mayor disponibilidad, se pueden iniciar y configurar varios servidores Eureka para que se comuniquen entre sí. Puede encontrar información sobre cómo configurar varios servidores Eureka en la documentación de Spring Cloud: <https://docs.spring.io/spring-cloud-netflix/docs/current/reference/html/#spring-cloud->

Conciencia de pares del servidor Eureka.

3. Finalmente, apague el sistema con este comando:

```
Docker Compose Down
```

Con esto finalizan las pruebas del servidor de descubrimiento, Netflix Eureka, donde aprendimos cómo escalar verticalmente y horizontalmente instancias de microservicios y qué sucede si un servidor Netflix Eureka se bloquea y luego vuelve a estar en línea.

Resumen

En este capítulo, aprendimos a usar Netflix Eureka para el descubrimiento de servicios. Primero, analizamos las deficiencias de una solución simple de descubrimiento de servicios basada en DNS y los desafíos que una solución robusta y resiliente debe poder afrontar.

Netflix Eureka es una solución de descubrimiento de servicios muy potente que ofrece características de ejecución robustas, resilientes y con tolerancia a fallos. Sin embargo, su correcta configuración puede ser compleja, especialmente para una experiencia fluida para el desarrollador. Con Spring Cloud, es fácil configurar un servidor Netflix Eureka y adaptar microservicios basados en Spring Boot, tanto para que se registren en Eureka durante el inicio como para que, al actuar como cliente de otros microservicios, realicen un seguimiento de instancias de microservicio disponibles.

Con un servidor de descubrimiento instalado, es hora de ver cómo podemos gestionar el tráfico externo usando Spring Cloud Gateway como servidor perimetral. ¡Ve al siguiente capítulo para descubrir cómo!

Preguntas

1. ¿Qué se necesita para convertir una aplicación Spring Boot creada con Spring Initializr en un servidor Netflix Eureka completo?
2. ¿Qué se necesita para que un microservicio basado en Spring Boot se registre automáticamente?
¿Como servicio de startup con Netflix Eureka?

3. ¿Qué se necesita para que un microservicio basado en Spring Boot pueda llamar a otro microservicio registrado en un servidor Netflix Eureka?
4. Supongamos que tienes un servidor Netflix Eureka en funcionamiento, junto con un...
instancia del microservicio A y dos instancias del microservicio B. Todas las instancias del microservicio
Se registran en el servidor Eureka de Netflix. El microservicio A realiza solicitudes HTTP al microservicio B basándose en la información que obtiene del servidor Eureka. ¿Qué ocurrirá si, a su vez, ocurre lo siguiente?
 - a. El servidor de Netflix Eureka se bloquea.
 - b. Una de las instancias del microservicio B falla.
 - c. Se inicia una nueva instancia del microservicio A
 - d. Se inicia una nueva instancia del microservicio B
 - e. El servidor Netflix Eureka se inicia de nuevo.

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



10

Uso de Spring Cloud Gateway para Ocultar microservicios detrás de un Servidor perimetral

En este capítulo, aprenderemos a usar Spring Cloud Gateway como servidor edge para controlar las API que se exponen desde nuestro entorno de sistemas basado en microservicios. Veremos cómo los microservicios con API públicas son accesibles desde el exterior a través del servidor edge, mientras que los microservicios con API privadas solo son accesibles desde el interior del entorno de microservicios. En nuestro entorno de sistemas, esto significa que el servicio compuesto de producto y el servidor de descubrimiento, Netflix Eureka, se expondrán a través del servidor edge. Los tres servicios principales (producto, recomendación y reseña) permanecerán ocultos.



Spring Cloud Gateway es ideal como servidor edge para desarrollo y pruebas. En un entorno de producción, se suele utilizar un producto de gestión de API completo, como Apigee, Kong Connect, IBM API Connect o MuleSoft Anypoint.

En este capítulo se tratarán los siguientes temas:

- Agregar un servidor de borde a nuestro panorama de sistemas
- Configuración de Spring Cloud Gateway, incluida la configuración de reglas de enrutamiento
- Probar el servidor de borde

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter10`.

Si desea ver los cambios aplicados al código fuente en este capítulo (es decir, ver lo que se necesitó para agregar Spring Cloud Gateway como un servidor perimetral al panorama de microservicios), puede compararlo con el código fuente del Capítulo 9, Agregar descubrimiento de servicios usando Netflix Eureka.

Puede utilizar su herramienta de diferencias favorita y comparar las dos carpetas, `$BOOK_HOME/Chapter09` y `$BOOK_HOME/Chapter10`.

Agregar un servidor perimetral a nuestro panorama de sistemas

En esta sección, veremos cómo se incorpora el servidor perimetral al entorno del sistema y cómo afecta la forma en que los clientes externos acceden a las API públicas que exponen los microservicios. Todas las solicitudes entrantes se enrutarán ahora a través del servidor perimetral, como se ilustra en el siguiente diagrama:

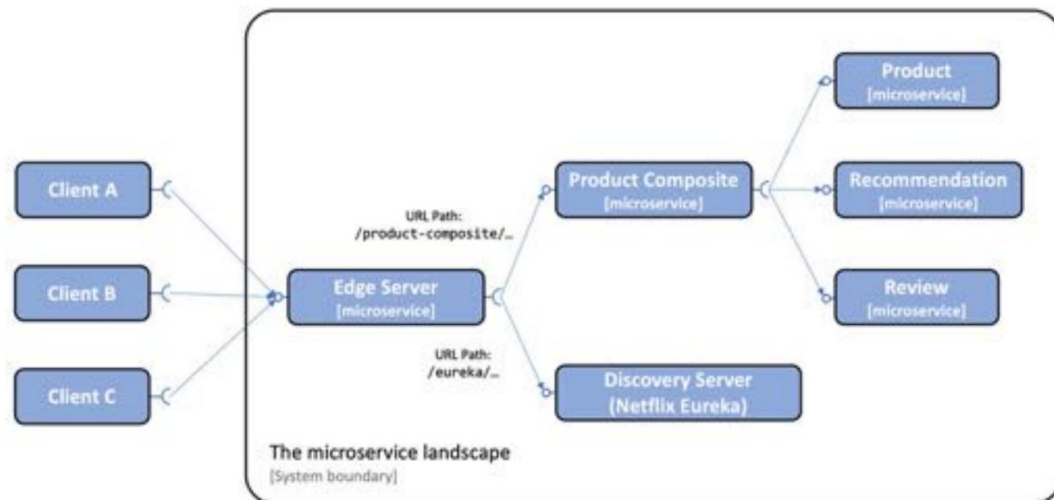


Figura 10.1: Agregar un servidor de borde

Como se puede observar en el diagrama anterior, los clientes externos envían todas sus solicitudes al servidor perimetral. Este puede enrutar las solicitudes entrantes según la ruta URL.

Por ejemplo, las solicitudes con una URL que comienza con `/product-composite/` se enrutan al microservicio compuesto `product`, y una solicitud con una URL que comienza con `/eureka/` se enruta al servidor de descubrimiento basado en Netflix Eureka.



Para que el servicio de descubrimiento funcione con Netflix Eureka, no es necesario exponerlo a través del servidor perimetral. Los servicios internos se comunicarán directamente con Netflix Eureka. El objetivo de exponerlo es que su página web y API sean accesibles para un operador que necesite verificar el estado de Netflix Eureka y ver qué instancias están registradas actualmente en el servicio de descubrimiento.

En el Capítulo 9, Adición de descubrimiento de servicios mediante Netflix Eureka, expusimos tanto el producto compuesto Servicio y el servidor de descubrimiento, Netflix Eureka, al exterior. Cuando presentemos el servidor perimetral en este capítulo, esto ya no será así. Esto se implementa eliminando lo siguiente:

declaraciones de puerto para los dos servicios en los archivos de Docker Compose:

```
producto-compuesto:
  compilación: microservicios/servicio-compuesto-de-producto
  puertos:
    - "8080:8080"

¡Eureka!:
  compilación: spring-cloud/eureka-server
  puertos:
    - "8761:8761"
```

Una vez presentado el servidor de borde, aprenderemos cómo configurar un servidor de borde basado en Spring Cloud Gateway en la siguiente sección.

Configuración de Spring Cloud Gateway

Configurar Spring Cloud Gateway como servidor perimetral es sencillo y se puede realizar con los siguientes pasos:

1. Cree un proyecto Spring Boot usando Spring Initializr, como se describe en el Capítulo 3, Creación de un conjunto de microservicios cooperativos ; consulte Uso de Spring Initializr para generar el esqueleto.
sección de código .
2. Agregue una dependencia en `spring-cloud-starter-gateway-server-webflux`.

3. Para poder ubicar instancias de microservicio a través de Netflix Eureka, agregue también la dependencia `spring-cloud-starter-netflix-eureka-client`.
4. Agregue el proyecto del servidor perimetral al archivo de compilación común, `settings.gradle`:

```
incluir ':spring-cloud:gateway'
```

5. Agregue un `Dockerfile` con el mismo contenido que para los microservicios; consulte el `Dockerfile` contenido en la carpeta `$BOOK_HOME/Chapter10/microservices`.
6. Agregue el servidor perimetral a nuestros tres archivos `Docker Compose`:

```
puerta:
  ambiente:
    - PERFILES_DE_PRIMAVERA_ACTIVOS=docker
  compilación: spring-cloud/gateway
  límite de memoria: 512 m
  puertos:
    - "8080:8080"
```

Del código anterior, podemos ver que el servidor perimetral expone el puerto 8080 al exterior del motor Docker. Para controlar la cantidad de memoria requerida, se establece un límite de memoria de 512 MB se aplica al servidor de borde, de la misma manera que lo hicimos para los otros microservicios.

7. Dado que el servidor perimetral gestionará todo el tráfico entrante, trasladaremos la comprobación de estado compuesta del servicio compuesto del producto al servidor perimetral. Esto se describe en la sección "Añadir una comprobación de estado compuesta".
8. Agregue la configuración de las reglas de enrutamiento y más. Dado que hay mucho que configurar, se trata en una sección aparte: Configuración de Spring Cloud Gateway.

Puede encontrar el código fuente de Spring Cloud Gateway en `$BOOK_HOME/Chapter10/spring-cloud/puerta`.

Agregar una comprobación de salud compuesta

Con un servidor perimetral instalado, las solicitudes de comprobación de estado externas también deben pasar por él. Por lo tanto, la comprobación de estado compuesta, que verifica el estado de todos los microservicios, se ha trasladado del servicio compuesto por producto al servidor perimetral. Consulte el Capítulo 7, Desarrollo de Microservicios Reactivos; consulte la sección "Añadir una API de estado" para obtener detalles de implementación de la comprobación de estado compuesta.

Se ha agregado lo siguiente al servidor perimetral:

1. Se ha agregado la clase `HealthCheckConfiguration`, que declara la salud reactiva contribuyente:

```
@Frijol
ReactiveHealthContributor healthcheckMicroservices() {

    registro final Map<String, ReactiveHealthIndicator> = nuevo
        LinkedHashMap<>();

    registro.put("producto", () ->
        obtenerSalud("http://producto"));
    registro.put("recomendación", () ->
        getHealth("http://recomendación"));
    registry.put("reseña", () -> getHealth("http://
        reseña")); registry.put("compuesto
    del producto", () ->
        getHealth("http://producto-compuesto"));

    devuelve CompositeReactiveHealthContributor.fromMap(registro);
}

private Mono<Health> getHealth(String baseUrl) { String url =
    baseUrl + "/actuator/health"; LOG.debug("Configurando
    una llamada a la API de Salud en la URL: {}",
        direccion URL);
    devolver

    webClient.get().uri(url).retrieve().bodyToMono(String.class).map(s
    -> new
    Health.Builder().up().build()).onErrorResume(ex -> Mono.just(new
    Health.Builder().down(ex).build())) .log(LOG.getName(), FINE);
}
```

Del código anterior, podemos ver que se ha agregado una verificación de estado para el servicio compuesto por producto, en lugar de la verificación de estado utilizada en el Capítulo 7, Desarrollo de microservicios reactivos.

2. La clase de aplicación principal, `GatewayApplication`, declara un bean `WebClient.Builder` para ser utilizados para la implementación del indicador de salud, de la siguiente manera:

```
@Frijol
@LoadBalanced
público WebClient.Builder loadBalancedWebClientBuilder() {
    devolver WebClient.builder();
}
```

Del código fuente anterior, podemos ver que `WebClient.builder` está anotado con `@LoadBalanced`, lo que le permite identificar las instancias de microservicio registradas en el servidor de descubrimiento, Netflix Eureka. Para más detalles, consulte la sección "Descubrimiento de servicios con Netflix Eureka en Spring Cloud" en el capítulo 9, "Añadir descubrimiento de servicios con Netflix Eureka".

Con una verificación de estado compuesta para el servidor perimetral, estamos listos para analizar la configuración que debe establecerse para Spring Cloud Gateway.

Configuración de Spring Cloud Gateway

Al configurar Spring Cloud Gateway, lo más importante es configurar las reglas de enrutamiento. También debemos configurar algunos otros aspectos:

1. Dado que Spring Cloud Gateway usará Netflix Eureka para encontrar los microservicios a los que enrutará el tráfico, debe configurarse como un cliente Eureka de la misma manera que se describe en el Capítulo 9, Agregar descubrimiento de servicios con Netflix Eureka ; consulte Configuración de clientes para el Sección del servidor Eureka .
2. Configure Spring Boot Actuator para su uso en desarrollo, como se describe en el Capítulo 7, Desarrollo de microservicios reactivos ; consulte la sección Agregar una API de estado :

```
management.endpoint.health.show-details: "SIEMPRE"
administración.puntos finales.web.exposición.incluir: ""
```

3. Configure los niveles de registro para que podamos ver los mensajes de registro de partes interesantes del procesamiento interno en Spring Cloud Gateway; por ejemplo, cómo decide a dónde enrutar las solicitudes entrantes:

```
explotación florestal:
nivel:
    raíz: INFO
org.springframework.cloud.gateway.route.
```

Definición de ruta Localizador de ruta: INFORMACIÓN

org.springframework.cloud.gateway: RASTREO

4. Para evitar el reenvío de encabezados sensibles como "X-Forwarded-For" y "Forwarded" desde proxies no confiables, Spring Cloud Gateway requiere que las direcciones IP de los proxies confiables se especifiquen en la propiedad `spring.cloud.gateway.trusted-proxies`. Para fines de desarrollo, se puede configurar para permitir cualquier dirección IP. Tenga en cuenta que esto debe limitarse a las direcciones IP reales que usan los proxies en un entorno de producción. Para el código de ejemplo de este libro, la interfaz de usuario de Swagger y su función "Probar una API" requieren un proxy confiable.

Para obtener más información, consulte <https://spring.io/security/cve-2025-41235>. Para especificar servidores proxy confiables durante el desarrollo, se agrega la siguiente configuración:

```
spring.cloud.gateway.trusted-proxies: ".*"
```

Para obtener el código fuente completo, consulte el archivo de configuración, `src/main/resources/application.yml`.

Reglas de enrutamiento

La configuración de reglas de enrutamiento se puede realizar de dos maneras: programáticamente, mediante un DSL de Java, o mediante configuración. Usar un DSL de Java para configurar reglas de enrutamiento programáticamente puede ser útil cuando las reglas se almacenan en un almacenamiento externo, como una base de datos, o se proporcionan en tiempo de ejecución, por ejemplo, mediante una API RESTful o un mensaje enviado a la puerta de enlace. En casos de uso más estáticos, me resulta más conveniente declarar las rutas en el archivo de configuración, `src/main/resources/application.yml`. Separar las reglas de enrutamiento del código Java permite actualizarlas sin tener que implementar una nueva versión del microservicio.

Una ruta se define de la siguiente manera:

- Predicados, que seleccionan una ruta según la información de la solicitud HTTP entrante
- Filtros, que pueden modificar tanto la solicitud como la respuesta
- Un URI de destino, que describe dónde enviar una solicitud
- Un ID, es decir, el nombre de la ruta

Para obtener una lista completa de predicados y filtros disponibles, consulte la documentación de referencia: <https://cloud.spring.io/spring-cloud-gateway/single/spring-cloud-gateway.html>.

En las siguientes subsecciones, aprenderemos primero a enrutar solicitudes a `product-composite` y al servidor Eureka. Después, veremos cómo usar predicados y filtros, aunque no se usen en este libro.

Enrutamiento de solicitudes a la API compuesta de producto

Si, por ejemplo, queremos enrutar las solicitudes entrantes donde la ruta URL comienza con `/product-composite/` a nuestro servicio `product-composite`, podemos especificar una regla de enrutamiento como esta:

```
spring.cloud.gateway.routes:  
  - id: producto-compuesto  
    uri: lb://producto-compuesto  
  predicados:  
    - Ruta=/product-composite/**
```

A continuación se muestran algunos puntos a tener en cuenta del código anterior:

- `id: product-composite`: El nombre de la ruta es `product-composite`.
- `uri: lb://product-composite`: Si la ruta se selecciona por sus predicados, la solicitud se enrutará al servicio denominado `product-composite` en el servicio de descubrimiento, Netflix Eureka. El protocolo `lb://` se utiliza para indicar a Spring Cloud Gateway que utilice el balanceador de carga del lado del cliente para buscar el destino en el servicio de descubrimiento.
- `predicados: - Path=/product-composite/**` se utiliza para especificar qué solicita esta ruta debe coincidir. `**` coincide con cero o más elementos en la ruta.

Para poder enrutar solicitudes a la interfaz de usuario Swagger configurada en el Capítulo 5, Agregar una descripción de API mediante OpenAPI, se agrega una ruta adicional al servicio compuesto por productos :

```
- id: producto-compuesto-swagger-ui  
  uri: lb://producto-compuesto  
  predicados:  
    - Ruta=/openapi/**
```

Las solicitudes enviadas al servidor perimetral con un URI que comience con `/openapi/` se dirigirán al servicio compuesto del producto .



Cuando la interfaz de usuario Swagger se presenta detrás de un servidor perimetral, debe poder presentar una especificación OpenAPI de la API que contenga la URL del servidor correcta: la URL del servidor perimetral en lugar de la URL del servicio compuesto del producto en sí. Para permitir que el servicio compuesto de producto produzca una URL de servidor correcta en la especificación OpenAPI, se ha agregado la siguiente configuración al servicio compuesto de producto :

estrategia de encabezados de avance del servidor: `marco`



Para obtener más detalles, consulte <https://springdoc.org/index.html#how-can-i-deploy-springdoc-openapi-starter-webmvc-ui-detrás-de-un-proxy-inverso>.

Para verificar que la URL del servidor correcta esté configurada en la especificación OpenAPI, se ha agregado la siguiente prueba al script de prueba, `test-em-all.bash`:

```
assertCurl 200 "curl -s http://$HOST:$PORT/
openapi/v3/api-docs"
assertEqual "http://$HOST:$PORT" "$(echo $RESPONSE
| jq -r .servers[.url])"
```

Enrutamiento de solicitudes a la API y la página web del servidor Eureka

Eureka ofrece una API y una página web a sus clientes. Para separar claramente la API de la página web en Eureka, configuraremos las rutas de la siguiente manera:

- Las solicitudes enviadas al servidor perimetral con la ruta que comienza con `/eureka/api/` deben ser manejado como una llamada a la API de Eureka
- Las solicitudes enviadas al servidor perimetral con la ruta que comienza con `/eureka/web/` deben ser manejado como una llamada a la página web de Eureka

Las solicitudes de API se enrutarán a `http://${app.eureka-server}:8761/eureka`. La regla de enrutamiento para la API de Eureka es la siguiente:

```
- id: eureka-api
  uri: http://${app.eureka-server}:8761
  predicados:
  - Ruta=/eureka/api/{segmento}
  filtros:
  - SetPath=/eureka/{segmento}
```

La parte `{segment}` en el valor Path coincide con cero o más elementos en la ruta y se utilizará para reemplazar la parte `{segment}` en el valor `SetPath`.

Las solicitudes de la página web se enrutarán a `http://${app.eureka-server}:8761`. La página web cargará varios recursos web, como archivos `.js`, `.css` y `.png`. Estas solicitudes se enrutarán a `http://${app.eureka-server}:8761/eureka`. Las reglas de enrutamiento para la página web de Eureka se ven así:
como esto:

```
- id: eureka-web-start
  uri: http://${app.eureka-server}:8761
```

```

predicados:
- Ruta=/eureka/web

filtros:
- EstablecerRuta=/

- id: eureka-web-other
  uri: http://${app.eureka-server}:8761
predicados:
- Ruta=/eureka/**

```

De la configuración anterior, podemos extraer las siguientes notas. El `${app.eureka-server}`

La propiedad se resuelve mediante el mecanismo de propiedad de Spring, dependiendo de qué perfil de Spring esté activo. vated:

1. Al ejecutar los servicios en el mismo host sin usar Docker, por ejemplo, para fines de depuración, la propiedad se traducirá a `localhost` utilizando el perfil predeterminado .
2. Al ejecutar los servicios como contenedores Docker, el servidor Netflix Eureka se ejecutará en un contenedor con el nombre DNS "eureka". Por lo tanto, la propiedad se traducirá a "eureka" mediante el perfil de Docker .

Las partes relevantes del archivo `application.yml` que definen esta traducción se ven así:

```

aplicación.eureka-server: localhost

spring.config.activate.on-profile: docker

aplicación.servidor eureka: eureka

```

Con esto hemos visto cómo enrutar solicitudes tanto al producto compuesto como al servidor Eureka.

Como paso final, veamos cómo se pueden usar los predicados y filtros en Spring Cloud Gateway.

Enrutamiento de solicitudes con predicados y filtros

Para aprender más sobre las capacidades de enrutamiento de Spring Cloud Gateway, probaremos el enrutamiento basado en host, donde Spring Cloud Gateway usa el nombre de host de la solicitud entrante para determinar dónde enrutarla.

Usaremos uno de mis sitios web favoritos para probar el código HTTP: `http://httpstat.us/`.

Una llamada a `http://httpstat.us/${CODE}` simplemente devuelve una respuesta con el código HTTP `${CODE}` y un cuerpo de respuesta, que también contiene el código HTTP y el texto descriptivo correspondiente. Por ejemplo, vea el siguiente comando `curl` :

```
rizo http://httpstat.us/200 -i
```

Esto devolverá el código HTTP 200 y un cuerpo de respuesta con el texto 200 OK.

Supongamos que queremos enrutar llamadas a `http://${hostname}:8080/headerrouting` de la siguiente manera:

- Las llamadas al host `i.feel.lucky` deberían devolver 200 OK
- Las llamadas al host `im.a.teapot` deberían devolver 418 Soy una tetera
- Las llamadas a todos los demás nombres de host deben devolver 501 No implementado

Para implementar estas reglas de enrutamiento en Spring Cloud Gateway, podemos usar el predicado de ruta de host para seleccionar solicitudes con nombres de host específicos y el filtro `SetPath` para establecer el código HTTP deseado en la ruta de la solicitud. Esto se puede hacer de la siguiente manera:

1. Para realizar llamadas a `http://i.feel.lucky:8080/headerrouting` que devuelvan 200 OK, podemos configurar por la siguiente ruta:

```
- id: ruta_del_host_200
  URL: http://httpstat.us
  predicados:
    - Host=tengo.suerte:8080
    - Ruta=/enrutamientodeencabezado/**
  filtros:
    - Establecer ruta=/200
```

2. Para realizar llamadas a `http://im.a.teapot:8080/headerrouting` devolver 418 Soy una tetera,

Podemos configurar la siguiente ruta:

```
- id: ruta_del_host_418
  URL: http://httpstat.us
  predicados:
    - Host=im.a.teatera:8080
    - Ruta=/enrutamientodeencabezado/**
  filtros:
    - Establecer ruta=/418
```

3. Finalmente, para hacer que las llamadas a todos los demás nombres de host devuelvan 501 No implementado, podemos configurar la siguiente ruta:

```
- id: ruta_del_host_501
  URL: http://httpstat.us
  predicados:
    - Ruta=/enrutamientodeencabezado/**
```

```
filtros:  
- Establecer ruta=/501
```

Bueno, eso fue bastante configuración, ¡así que ahora vamos a probarlo!

Probando el servidor de borde

Para probar el servidor edge, realizamos los siguientes pasos:

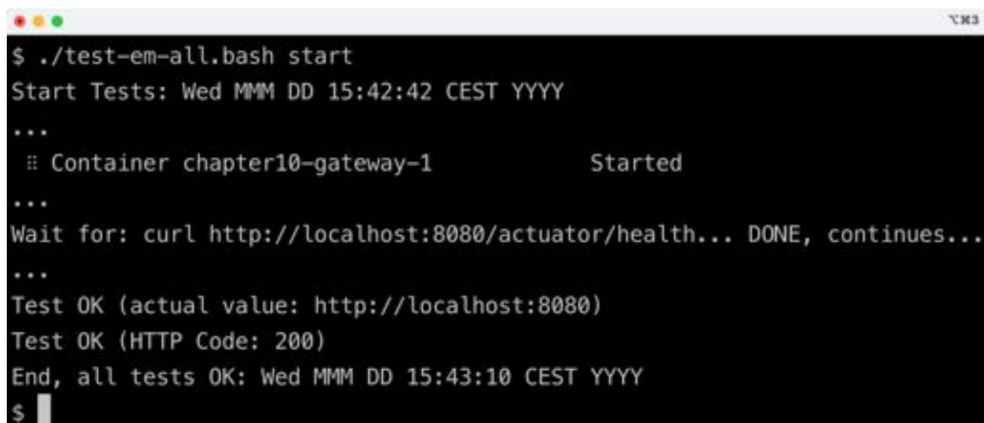
1. Primero, construya las imágenes de Docker con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 10  
./gradlew compilación limpia y compilación de docker compose
```

2. A continuación, inicie el panorama del sistema en Docker y ejecute las pruebas habituales con lo siguiente dominio:

```
./test-em-all.bash inicio
```

3. Espere resultados similares a los que hemos visto en capítulos anteriores:



```
$ ./test-em-all.bash start  
Start Tests: Wed MMM DD 15:42:42 CEST YYYY  
...  
:: Container chapter10-gateway-1          Started  
...  
Wait for: curl http://localhost:8080/actuator/health... DONE, continues...  
...  
Test OK (actual value: http://localhost:8080)  
Test OK (HTTP Code: 200)  
End, all tests OK: Wed MMM DD 15:43:10 CEST YYYY  
$
```

Figura 10.2: Salida de test-em-all.bash

4. En la salida del registro, observe el penúltimo resultado de la prueba: `http://localhost:8080`. Este es el resultado de la prueba que verifica que la URL del servidor en la especificación OpenAPI de Swagger UI se reescribe correctamente como la URL del servidor perimetral.

Con el panorama del sistema, incluido el servidor perimetral, en funcionamiento, exploremos los siguientes temas:

- Examinar lo que expone el servidor perimetral fuera del entorno del sistema en ejecución en el motor Docker

- Probar algunas de las reglas de enrutamiento más utilizadas, como las siguientes:
 - Uso de enrutamiento basado en URL para llamar a nuestras API a través del servidor perimetral
 - Uso de enrutamiento basado en URL para llamar a la interfaz de usuario de Swagger a través del servidor perimetral
 - Utilizando enrutamiento basado en URL para llamar a Netflix Eureka a través del servidor perimetral, ambos usando Su API y su interfaz de usuario basada en web
 - Utilizar el enrutamiento basado en encabezado para ver cómo podemos enrutar solicitudes en función del host nombre en la solicitud

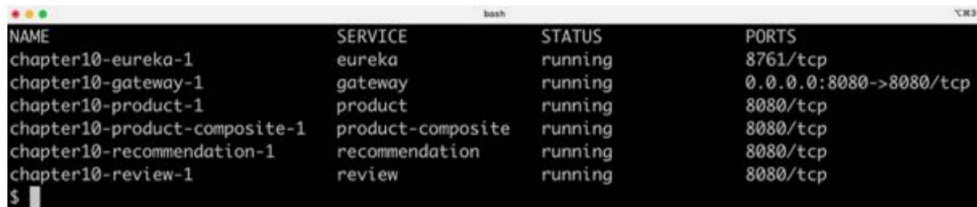
Examinando lo que se expone fuera del motor Docker

Para comprender lo que el servidor perimetral expone al exterior del panorama del sistema, realice los siguientes pasos:

1. Utilice el comando `docker compose ps` para ver qué puertos están expuestos por nuestros servicios:

```
Revisión de recomendaciones de productos de Docker Compose PS Gateway Eureka Product-Composite
```

2. Como podemos ver en la siguiente salida, solo el servidor de borde (llamado puerta de enlace) expone su puerto (8080) fuera del motor Docker:



NAME	SERVICE	STATUS	PORTS
chapter10-eureka-1	eureka	running	8761/tcp
chapter10-gateway-1	gateway	running	0.0.0.0:8080->8080/tcp
chapter10-product-1	product	running	8080/tcp
chapter10-product-composite-1	product-composite	running	8080/tcp
chapter10-recommendation-1	recommendation	running	8080/tcp
chapter10-review-1	review	running	8080/tcp

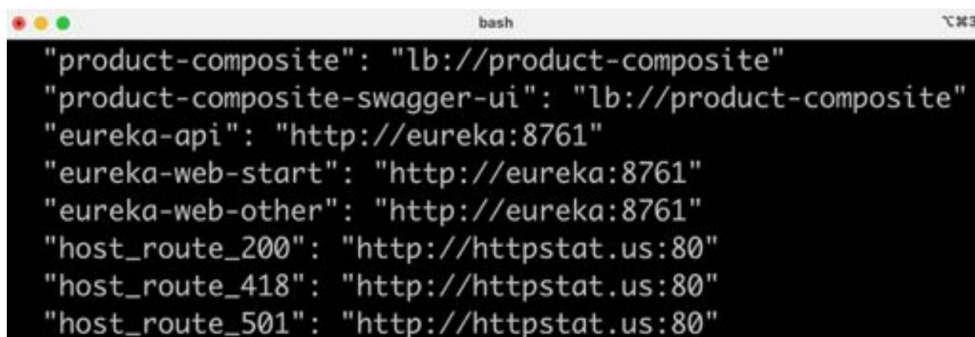
Figura 10.3: Salida de `docker compose ps`

3. Si queremos ver qué rutas ha configurado el servidor de borde, podemos usar `/actuator/`

API de puerta de enlace/rutas . La respuesta de esta API es bastante detallada. Para limitar la respuesta a la información que nos interesa, podemos aplicar un filtro JQ . En el siguiente ejemplo, se seleccionan el ID de la ruta y la URL a la que se enrutará la solicitud:

```
curl localhost:8080/actuator/puerta de enlace/rutas -s | jq '.[] | {".route_id": "\(.uri)"}' | grep -v '\{\'
```

4. Este comando responderá con lo siguiente:



```
bash
"product-composite": "lb://product-composite"
"product-composite-swagger-ui": "lb://product-composite"
"eureka-api": "http://eureka:8761"
"eureka-web-start": "http://eureka:8761"
"eureka-web-other": "http://eureka:8761"
"host_route_200": "http://httpstat.us:80"
"host_route_418": "http://httpstat.us:80"
"host_route_501": "http://httpstat.us:80"
```

Figura 10.4: Reglas de enrutamiento de Spring Cloud Gateway

Esto nos da una buena visión general de las rutas reales configuradas en el servidor perimetral. Ahora, intentemos...
¡Fuera las rutas!

Probando las reglas de enrutamiento

En esta sección, probaremos el servidor perimetral y las rutas que expone al exterior del entorno del sistema. Comenzaremos llamando a la API compuesta del producto y su interfaz de usuario Swagger. Luego, llamaremos a la API de Eureka y visitaremos su página web. Finalmente, concluiremos probando las rutas que...
Se basan en nombres de host.

Llamar a la API compuesta del producto a través del servidor perimetral

Realicemos los siguientes pasos para llamar a la API compuesta del producto a través del servidor perimetral:

1. Para poder ver qué está sucediendo en el servidor perimetral, podemos seguir su salida de registro:

```
registros de docker compose -f --tail=0 puerta de enlace
```

2. Ahora, en una ventana de terminal independiente, realice la llamada a la API compuesta del producto a través de
El servidor de borde:

```
rizo http://localhost:8080/producto-compuesto/1
```

3. Espere el tipo de respuesta normal de la API compuesta del producto :



```
bash
{"productId":1, ... "recommendations":[...],"reviews":[...],"serviceAddresses":{"...}}
$
```

Figura 10.5: Salida de la recuperación del producto compuesto con ID de producto 1

4. Deberíamos poder encontrar la siguiente información en la salida del registro:

```
bash
Pattern "/product-composite/**" matches against value "/product-composite/1"
Route matched: product-composite
...
LoadBalancerClientFilter url chosen: http://2911a48d86ec:8080/product-composite/1
```

Figura 10.6: Salida del registro del servidor perimetral

5. Desde la salida del registro, podemos ver la coincidencia de patrones en función del predicado que especificamos en la configuración, y podemos ver qué instancia de microservicio seleccionó el servidor perimetral de las instancias disponibles en el servidor de descubrimiento; en este caso, reenvía la solicitud a `http://b8013440aea0:8080/product-composite/1`.

Llamar a la interfaz de usuario Swagger a través del servidor perimetral

Para verificar que podemos llegar a la interfaz de usuario Swagger presentada en el Capítulo 5, Agregar una descripción de API usando OpenAPI, a través del servidor perimetral, abra `http://localhost:8080/openapi/swagger-ui.html` URL en un navegador web. La página resultante de la interfaz de usuario de Swagger debería verse así:

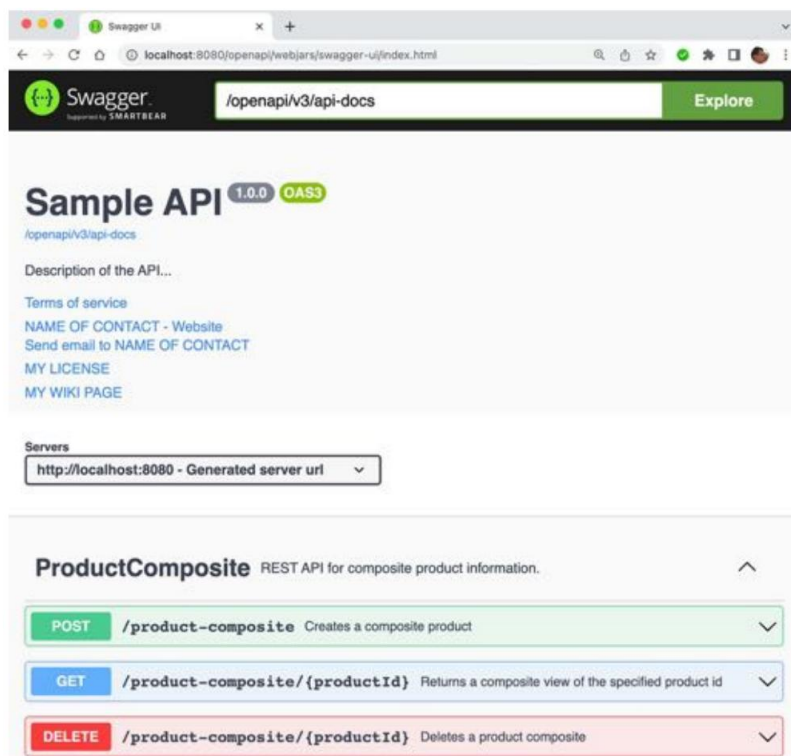


Figura 10.7: La interfaz de usuario de Swagger a través del servidor de borde y la puerta de enlace

Tenga en cuenta la URL del servidor: `http://localhost:8080`; esto significa que la propia URL de la API compuesta del producto, `http://product-service:8080/`, ha sido reemplazada en la especificación OpenAPI devuelta por la interfaz de usuario Swagger.

Si lo desea, puede continuar y probar la API compuesta de producto en la interfaz de usuario Swagger, como hicimos en el Capítulo 5, Cómo agregar una descripción de API usando OpenAPI.

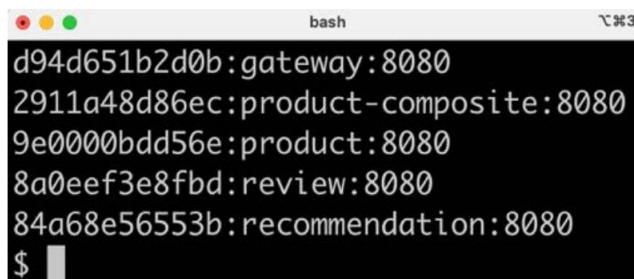
Llamar a Eureka a través del servidor perimetral

Para llamar a Eureka a través de un servidor perimetral, realice los siguientes pasos:

1. Primero, llame a la API de Eureka a través del servidor perimetral para ver qué instancias están actualmente registrado en el servidor de descubrimiento:

```
curl -H "accept:aplicación/json" \  
localhost:8080/eureka/api/apps -s | \  
jq -r .aplicaciones.aplicación[].instancia[].instanceld
```

2. Espere una respuesta similar a la siguiente:



```
bash ㉿#3  
d94d651b2d0b:gateway:8080  
2911a48d86ec:product-composite:8080  
9e0000bdd56e:product:8080  
8a0eef3e8fbd:review:8080  
84a68e56553b:recommendation:8080  
$
```

Figura 10.8: Eureka enumera el servidor de borde y la puerta de enlace en la llamada REST



Tenga en cuenta que el servidor de borde (denominado puerta de enlace) también está presente en la respuesta.

3. A continuación, abra la página web de Eureka en un navegador web utilizando `http://localhost:8080/eureka/web` URL:

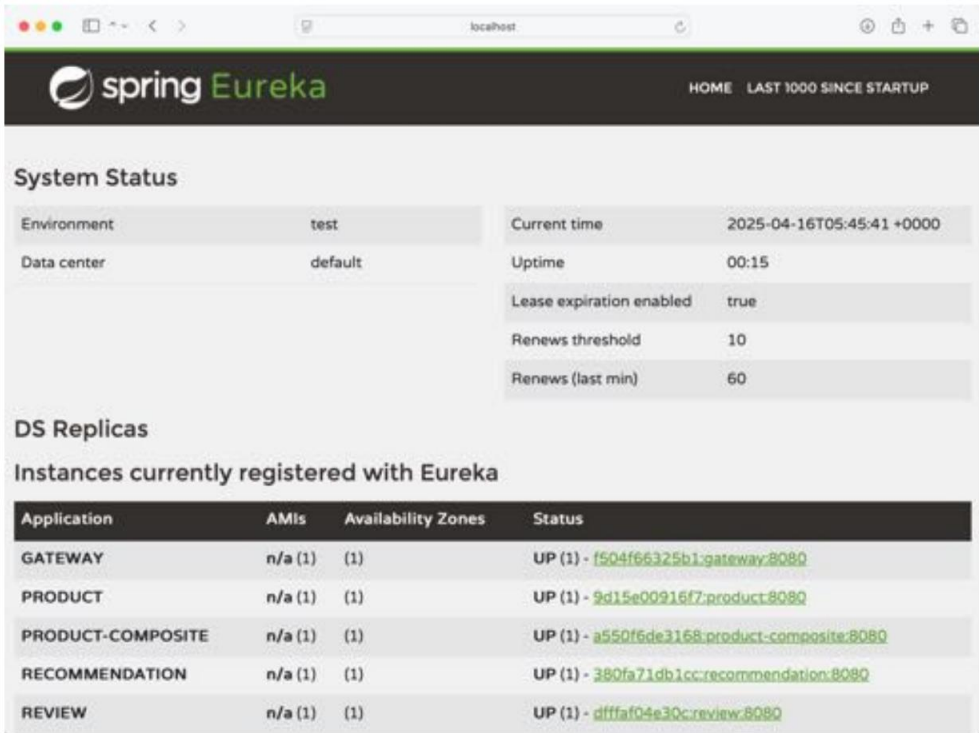


Figura 10.9: Eureka enumera el servidor perimetral y la puerta de enlace en la interfaz web

4. En la captura de pantalla anterior, podemos ver la página web de Eureka informando las mismas instancias disponibles que la respuesta de la API en el paso anterior.

Enrutamiento basado en el encabezado del host

Terminemos probando la configuración de la ruta en función del nombre de host utilizado en las solicitudes para ver los predicados y filtros en uso.

Normalmente, el nombre de host en la solicitud lo establece automáticamente el cliente HTTP en el encabezado del Host . Al probar el servidor perimetral localmente, el nombre de host será localhost ; esto no es tan útil al probar el enrutamiento basado en nombres de host. Sin embargo, podemos hacer trampa especificando otro nombre de host en el encabezado Host al llamar a la API. Veamos cómo se puede hacer:

1. Para solicitar el nombre de host i.feel.lucky , use este código:

```
curl http://localhost:8080/headerrouting -H "Host: i.feel.suerte:8080"
```

2. Espere la respuesta 200 OK.

3. Para el nombre de host im.a.teapot , utilice el siguiente comando:

```
curl http://localhost:8080/headerrouting -H "Host: im.a.teapot:8080"
```

4. Espere la respuesta 418 Soy una tetera.

5. Finalmente, si no se especifica ningún encabezado de Host , utilice localhost como encabezado de Host :

```
rizo http://localhost:8080/headerrouting
```

6. Espere la respuesta 501 No implementado.

También podemos usar i.feel.lucky e im.a.teapot como nombres de host reales en las solicitudes si los añadimos al archivo /etc/hosts , especificando que deben traducirse a la misma dirección IP que localhost, es decir, 127.0.0.1. Para ello, siga estos pasos:

1. Ejecute el siguiente comando para agregar una fila al archivo /etc/hosts con la información requerida. formación:

```
sudo bash -c "echo '127.0.0.1 tengo.suerte soy.una.tetera' >> /etc/anfitriones"
```

2. Ahora podemos realizar el mismo enrutamiento según el nombre de host, pero sin especificar el encabezado del host . Pruébalo ejecutando los siguientes comandos:

```
rizo http://i.feel.lucky:8080/headerrouting  
rizo http://im.a.teapot:8080/headerrouting
```

3. Espere las mismas respuestas que antes, 200 OK y 418 Soy una tetera.
4. Finalice las pruebas apagando el sistema con el siguiente comando:

```
Docker Compose Down
```

5. Limpia el archivo `/etc/hosts` de la traducción de nombres DNS que añadimos para los nombres de host `i.feel.lucky` e `im.a.teapot`. Edita el archivo `/etc/hosts` y elimina la línea que añadimos:

```
127.0.0.1 me.siento.afortunada soy.una.tetera
```

Estas pruebas de las capacidades de enrutamiento en el servidor de borde finalizan el capítulo.

Resumen

En este capítulo, hemos visto cómo Spring Cloud Gateway puede usarse como servidor perimetral para controlar qué servicios pueden llamarse desde fuera del entorno del sistema. Basándonos en predicados, filtros y URI de destino, podemos definir reglas de enrutamiento de forma muy flexible. Si lo deseamos, podemos configurar Spring Cloud Gateway para que use un servicio de descubrimiento como Netflix Eureka para buscar las instancias de microservicio de destino.

Un tema importante que aún debemos abordar es cómo evitar el acceso no autorizado a las API expuestas por el servidor perimetral y cómo podemos evitar que terceros intercepten el tráfico.

En el próximo capítulo, veremos cómo podemos proteger el acceso al servidor perimetral utilizando mecanismos de seguridad estándar, como HTTPS, OAuth y OpenID Connect.

Preguntas

1. ¿Cómo se llaman los elementos que se utilizan para crear una regla de enrutamiento en Spring Cloud Gateway?
2. ¿Para qué se utilizan los elementos mencionados anteriormente?
3. ¿Cómo podemos indicarle a Spring Cloud Gateway que localice instancias de microservicio a través de un servicio de descubrimiento como Netflix Eureka?
4. En un entorno Docker, ¿cómo podemos garantizar que las solicitudes HTTP externas al motor Docker solo puedan llegar al servidor perimetral?
5. ¿Cómo cambiamos las reglas de enrutamiento para que el servidor perimetral acepte llamadas al servicio compuesto de producto en la URL `http://$HOST:$PORT/api/product` en lugar de la URL `http://$HOST:$PORT/product-composite` utilizada actualmente?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



11

Asegurar el acceso a las API

En este capítulo, veremos cómo proteger el acceso a las API y páginas web expuestas por el servidor perimetral presentado en el capítulo anterior. Aprenderemos a usar HTTPS para proteger contra el espionaje en el acceso externo a nuestras API, y a usar OAuth 2.0 y OpenID Connect para autenticar y autorizar el acceso de usuarios y aplicaciones cliente a nuestras API. Finalmente, usaremos la autenticación HTTP básica para proteger el acceso al servidor de descubrimiento, Netflix Eureka.

En este capítulo se tratarán los siguientes temas:

- Una introducción a los estándares OAuth 2.0 y OpenID Connect
- Una discusión general sobre cómo proteger el panorama del sistema
- Protección de la comunicación externa con HTTPS
- Asegurar el acceso al servidor de descubrimiento, Netflix Eureka
- Agregar un servidor de autorización local a nuestro panorama de sistemas
- Autenticación y autorización del acceso a la API mediante OAuth 2.0 y OpenID Connect
- Pruebas con el servidor de autorización local
- Pruebas con un proveedor externo de OpenID Connect, Auth0

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter11`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver lo que se necesitó para asegurar el acceso a las API en el panorama de microservicios, puede compararlo con el código fuente del Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral.

Puede utilizar su herramienta de diferencias favorita y comparar las dos carpetas \$BOOK_HOME/Chapter10 y \$BOOK_HOME/Chapter11.

Introducción a OAuth 2.0 y OpenID Connect

Antes de presentar OAuth 2.0 y OpenID Connect, aclaremos qué entendemos por autenticación y autorización.

La autenticación consiste en identificar a un usuario mediante la validación de las credenciales proporcionadas por este, como su nombre de usuario y contraseña. La autorización consiste en dar acceso a diversas partes de, en nuestro caso, una API a un usuario autenticado.

OAuth 2.0 es un estándar abierto para la delegación de autorizaciones, y OpenID Connect es un complemento de OAuth 2.0 que permite a las aplicaciones cliente verificar la identidad de los usuarios basándose en la autenticación realizada por el servidor de autorización. Analicemos brevemente OAuth 2.0 y OpenID Connect por separado para comprender sus funciones.

Presentamos OAuth 2.0

OAuth 2.0 es un estándar abierto ampliamente aceptado para la autorización que permite a un usuario dar su consentimiento para que una aplicación cliente de terceros acceda a recursos protegidos en su nombre. Otorgar a una aplicación cliente de terceros el derecho a actuar en nombre de un usuario, por ejemplo, llamando a una API, se conoce como delegación de autorización.

Entonces ¿qué significa esto?

Comencemos ordenando los conceptos utilizados:

- Propietario del recurso: el usuario final.
- Cliente: la aplicación cliente de terceros, por ejemplo, una aplicación web o una aplicación móvil nativa que desea llamar a algunas API protegidas en nombre del usuario final.
- Servidor de recursos: El servidor que expone las API que queremos proteger.
- Servidor de autorización: El servidor de autorización emite tokens al cliente después de la re-
El propietario de la fuente, es decir, el usuario final, ha sido autenticado. La gestión de la información del usuario y su autenticación suelen delegarse, en segundo plano, a un proveedor de identidad (IdP).

Un cliente se registra en el servidor de autorización y se le asigna un ID de cliente y un secreto de cliente.

El secreto del cliente debe estar protegido por el cliente, como una contraseña. Un cliente también se registra con un conjunto de URL de redirección permitidas que el servidor de autorización utilizará después de que un usuario se haya autenticado.

Diseñado para enviar códigos de autorización y tokens que se han emitido a la aplicación cliente.

El siguiente es un ejemplo ilustrativo. Supongamos que un usuario accede a una aplicación cliente de terceros y esta desea llamar a una API protegida para atender al usuario. Para poder acceder a estas API, la aplicación cliente necesita una forma de indicarles que está actuando.

En nombre del usuario. Para evitar soluciones donde el usuario deba compartir sus credenciales con el

Aplicación cliente para autenticación, un token de acceso es emitido por un servidor de autorización que le otorga a la aplicación cliente acceso limitado a un conjunto seleccionado de API en nombre del usuario.

Esto significa que el usuario nunca tiene que revelar sus credenciales a la aplicación cliente. El usuario también puede dar su consentimiento para que la aplicación cliente acceda a API específicas en su nombre. Un token de acceso representa un conjunto de derechos de acceso con límite temporal, expresado como alcances en términos de OAuth 2.0 . El servidor de autorización también puede emitir un token de actualización a una aplicación cliente. La aplicación cliente puede utilizar un token de actualización para obtener nuevos tokens de acceso sin tener que...
para involucrar al usuario.

La especificación OAuth 2.0 define cuatro flujos de concesión de autorización para emitir tokens de acceso, que se explican a continuación:

- Flujo de concesión de código de autorización: Este es el flujo de concesión más seguro, pero también el más complejo.

Este flujo de concesión requiere que el usuario interactúe con el servidor de autorización mediante un navegador web para autenticarse y dar consentimiento a la aplicación cliente, como lo ilustra el siguiente diagrama:

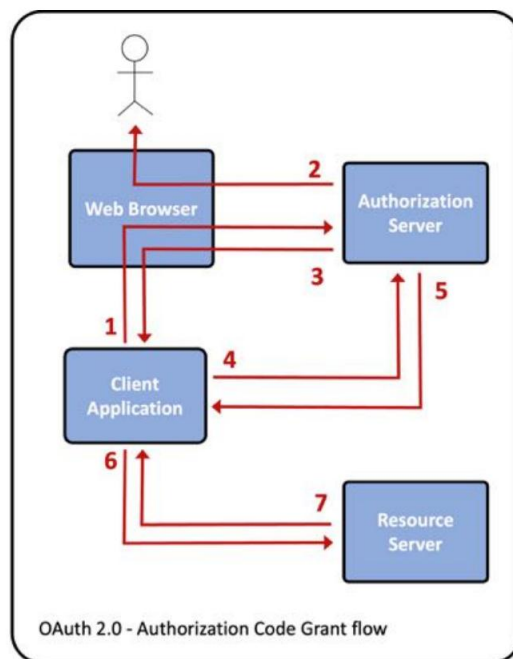


Figura 11.1: OAuth 2.0: flujo de concesión de código de autorización

Esto es lo que sucede en este diagrama:

1. La aplicación cliente inicia el flujo de concesión enviando al usuario a la autoridad. Servidor de ubicación en el navegador web.
2. El servidor de autorización autentica al usuario y solicitará su consentimiento.
3. El servidor de autorización redirigirá al usuario a la aplicación cliente con un código de autorización. El servidor de autorización utilizará una URI de redirección especificada por el cliente en el paso 1 para saber dónde enviar el código de autorización. Dado que el código de autorización se envía a la aplicación cliente mediante el navegador web, es decir, a un entorno inseguro donde código JavaScript malicioso podría acceder a él, solo se permite su uso una vez y durante un breve periodo.

4. Para intercambiar el código de autorización por un token de acceso, se espera que la aplicación cliente vuelva a llamar al servidor de autorización. La aplicación cliente debe presentar su ID de cliente y su secreto de cliente junto con el código de autorización para el servidor de autorización. Dado que el secreto de cliente es confidencial y debe protegerse, esto...
La llamada debe ejecutarse desde el código del lado del servidor.
5. El servidor de autorización emite un token de acceso y lo envía de vuelta al cliente.
aplicación. El servidor de autorización también puede, opcionalmente, emitir y devolver un token de actualización.
6. Utilizando el token de acceso, el cliente puede enviar una solicitud a la API protegida expuesta por el servidor de recursos.
7. El servidor de recursos valida el token de acceso y atiende la solicitud si la validación es correcta. Los pasos 6 y 7 pueden repetirse mientras el token de acceso sea válido. Una vez vencido el periodo de validez del token de acceso, el cliente puede usar su token de actualización para obtener uno nuevo.

Flujo de concesión implícita : Este flujo también se basa en el navegador web, pero está diseñado para aplicaciones cliente que no pueden proteger un secreto de cliente, por ejemplo, una aplicación web de una sola página. El navegador web recibe un token de acceso del servidor de autorización en lugar de un código de autorización. Dado que el flujo de concesión implícita es menos seguro que el flujo de concesión de código de autorización, el cliente no puede solicitar un token de actualización.



El flujo de concesión implícito ha quedado obsoleto y se ha eliminado del borrador de especificación de OAuth 2.1 (ver a continuación) y, por lo tanto, ya no debe utilizarse.

Flujo de concesión de credenciales de contraseña del propietario del recurso: Si una aplicación cliente no puede interactuar con un navegador web, puede recurrir a este flujo de concesión. En este flujo, el usuario debe compartir sus credenciales con la aplicación cliente, y esta las usará para obtener un token de acceso.



El flujo de concesión de credenciales de contraseña del propietario del recurso ha quedado obsoleto y se ha eliminado del borrador de especificación de OAuth 2.1 (ver a continuación) y, por lo tanto, ya no debe utilizarse.

- Flujo de concesión de credenciales de cliente: en el caso en que una aplicación cliente necesite llamar a una API no relacionada con un usuario específico, puede usar este flujo de concesión para adquirir un token de acceso usando su ID de cliente propio y secreto de cliente.

La especificación completa se puede encontrar aquí: <https://tools.ietf.org/html/rfc6749>. También hay una serie de especificaciones adicionales que detallan varios aspectos de OAuth 2.0; para obtener una descripción general, consulte <https://www.oauth.com/oauth2-servers/map-oauth-2-0-specs/>. Una especificación adicional que merece atención especial es RFC 7636: Clave de prueba para intercambio de código por parte de clientes públicos OAuth (o PKCE para abreviar), <https://tools.ietf.org/html/rfc7636>. Esta especificación describe cómo un cliente público no seguro, como una aplicación nativa móvil o una aplicación de escritorio, puede utilizar el flujo de concesión del código de autorización de forma segura agregando una capa adicional de seguridad.

La especificación OAuth 2.0 se publicó en 2012 y, a lo largo de los años, se han aprendido muchas lecciones sobre qué funciona y qué no. En 2019, se comenzó a trabajar en la creación de OAuth 2.1, consolidando las mejores prácticas y experiencias derivadas del uso de OAuth 2.0. Puede encontrar una versión preliminar aquí: <https://tools.ietf.org/html/draft-ietf-oauth-v2-1-13>.

En mi opinión, las mejoras más importantes en OAuth 2.1 son las siguientes:

PKCE está integrado en el flujo de concesión del código de autorización. Los clientes públicos requerirán el uso de PKCE para mejorar su seguridad, como se describió anteriormente. Para clientes confidenciales, donde el servidor de autorización puede verificar sus credenciales, el uso de PKCE no es obligatorio, solo se recomienda.

- El flujo de concesión implícito está obsoleto y se omite de la especificación debido a su menor Naturaleza segura.
- El flujo de concesión de credenciales de contraseña del propietario del recurso también está obsoleto y se omite. la especificación, por las mismas razones.

Dada la dirección de la próxima especificación OAuth 2.1, en este libro solo utilizaremos el flujo de concesión de código de autorización y el flujo de concesión de credenciales de cliente.



Para automatizar pruebas con API protegidas por OAuth 2.0, el flujo de concesión de credenciales de cliente resulta muy práctico, ya que no requiere interacción manual mediante un navegador web. Utilizaremos este flujo de concesión más adelante en este capítulo con nuestro script de prueba; consulte la sección "Cambios en el script de prueba".

Presentamos OpenID Connect

OpenID Connect (abreviado como OIDC) es, como ya se mencionó, un complemento de OAuth 2.0 que permite a las aplicaciones cliente verificar la identidad de los usuarios. OIDC añade un token adicional, un token de identificación, que la aplicación cliente recibe del servidor de autorización tras completar un flujo de concesión.

El token de identificación se codifica como un token web JSON (JWT) y contiene varias declaraciones, como el ID y la dirección de correo electrónico del usuario. El token de identificación se firma digitalmente mediante firmas web JSON.

Esto permite que una aplicación cliente confíe en la información del token de identificación validando su firma digital utilizando claves públicas del servidor de autorización.

Opcionalmente, los tokens de acceso también pueden codificarse y firmarse de la misma manera que los tokens de identificación, aunque esto no es obligatorio según la especificación. Además, OIDC define un punto final de descubrimiento, que es una forma estandarizada de establecer URL a puntos finales importantes, como solicitar códigos de autorización y tokens, o obtener las claves públicas para verificar un JWT firmado digitalmente. Finalmente, también define un punto final de información de usuario, que puede utilizarse para obtener información adicional sobre un usuario autenticado al que se le asigna un token de acceso.

Para obtener una descripción general de las especificaciones disponibles, consulte <https://openid.net/developers/specs/>.



En este libro, solo utilizaremos servidores de autorización que cumplan con la especificación OpenID Connect. Esto simplificará la configuración de los servidores de recursos mediante sus endpoints de descubrimiento. También utilizaremos la compatibilidad opcional con tokens de acceso JWT firmados digitalmente para simplificar la verificación de la autenticidad de los tokens de acceso por parte de los servidores de recursos. Consulte la sección "Cambios en el servidor perimetral y en el servicio compuesto por producto" a continuación.

Con esto concluye nuestra introducción a los estándares OAuth 2.0 y OpenID Connect. Más adelante en este capítulo, aprenderemos más sobre cómo usar estos estándares. En la siguiente sección, presentaremos una visión general de cómo se protegerá el entorno del sistema.

Asegurar el panorama del sistema

Para proteger el panorama del sistema como se describe en la introducción de este capítulo, realizaremos los siguientes pasos:

1. Cifre las solicitudes y respuestas externas hacia y desde nuestra API externa mediante HTTPS para protegerse contra escuchas clandestinas.
2. Autenticar y autorizar a los usuarios y aplicaciones cliente que acceden a nuestras API mediante OAuth 2.0 y OpenID Connect.
3. Acceso seguro al servidor de descubrimiento, Netflix Eureka, mediante autenticación básica HTTP.

Solo aplicaremos HTTPS para la comunicación externa a nuestro servidor perimetral, utilizando HTTP simple para la comunicación dentro de nuestro sistema.



En el capítulo sobre mallas de servicios (Capítulo 18, Uso de una malla de servicios para mejorar la observabilidad y la administración) que aparecerá más adelante en este libro, veremos cómo podemos obtener ayuda de un producto de malla de servicios para aprovisionar automáticamente HTTPS para proteger la comunicación dentro de un entorno de sistema.

Para fines de prueba, agregaremos un servidor de autorización OAuth 2.0 local a nuestro sistema.

Toda comunicación externa con el servidor de autorización se enrutará a través del servidor de borde.

El servidor perimetral y el servicio compuesto del producto actuarán como servidores de recursos OAuth 2.0; es decir, requerirán un token de acceso OAuth 2.0 válido para permitir el acceso.

Para minimizar la sobrecarga de validar los tokens de acceso, asumiremos que están codificados como JWT firmados y que el servidor de autorización expone un punto final que los servidores de recursos pueden usar para acceder a las claves públicas, también conocidas como un conjunto de claves web JSON, o jwk-set para abreviar, necesarias para validar la firma.

El panorama del sistema se verá así:

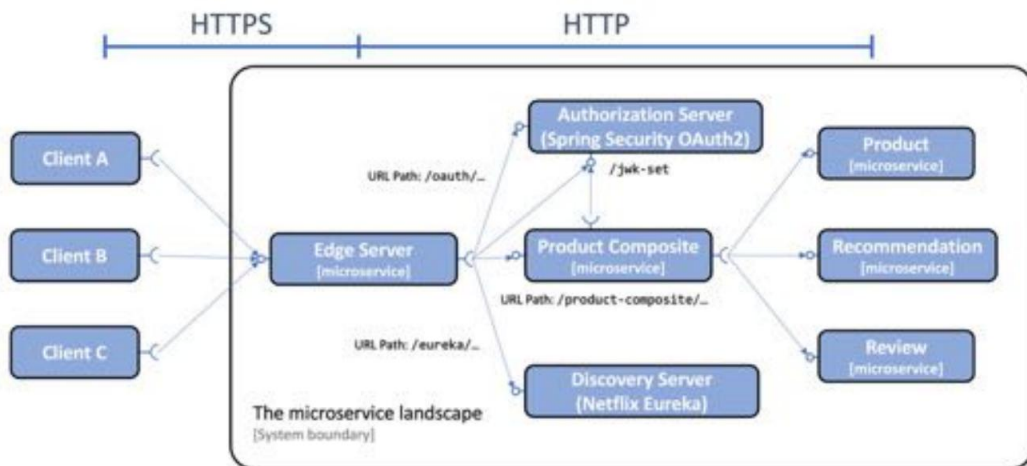


Figura 11.2: Agregar un servidor de autorización al panorama del sistema

Del diagrama anterior podemos observar lo siguiente:

- HTTPS se utiliza para la comunicación externa, mientras que HTTP de texto simple se utiliza dentro del sistema.
paisaje del tem
- Se accederá al servidor de autorización local OAuth 2.0 externamente a través del borde
servidor

- Tanto el servidor de borde como el microservicio compuesto del producto validarán los tokens de acceso como JWT firmados
- El servidor perimetral y el microservicio compuesto del producto obtendrán las claves públicas del servidor de autorización desde su punto final jwk-set y las usarán para validar la firma de los tokens de acceso basados en JWT.



Tenga en cuenta que nos centraremos en proteger el acceso a las API mediante HTTP, no en cubrir las mejores prácticas generales para proteger las aplicaciones web; por ejemplo, la gestión de los riesgos de seguridad de las aplicaciones web señalados por el Proyecto Top Ten de OWASP. Consulte <https://owasp.org/www-proyecto-top-ten/> Para más información.

Con esta descripción general de cómo se protegerá el panorama del sistema, comencemos a ver cómo podemos proteger la comunicación externa contra escuchas clandestinas mediante HTTPS.

Protegiendo la comunicación externa con HTTPS

En esta sección, aprenderemos cómo evitar el espionaje de las comunicaciones externas, por ejemplo, desde internet, a través de las API públicas expuestas por el servidor perimetral. Utilizaremos HTTPS para cifrar la comunicación. En esencia, HTTPS utiliza el protocolo Secure Sockets Layer (SSL) para cifrar la comunicación. SSL es un protocolo que proporciona privacidad, autenticación e integridad de datos en las comunicaciones. Con el tiempo, SSL fue reemplazado por su equivalente más seguro y moderno, Transport Layer Security (TLS).

Para utilizar HTTPS, necesitamos hacer lo siguiente:

- **Crear un certificado:** Crearemos nuestro propio certificado autofirmado, suficiente para el desarrollo.
fines de desarrollo
- **Configurar el servidor de borde:** debe configurarse para aceptar solo servidores externos basados en HTTPS.
tráfico que utiliza el certificado

El certificado autofirmado se crea con el siguiente comando:

```
herramienta de claves -genkeypair -alias localhost -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore edge.p12 -validity 3650
```



El código fuente viene con un archivo de certificado de muestra, por lo que no es necesario ejecutar este comando para ejecutar los siguientes ejemplos.

El comando solicitará varios parámetros. Cuando se me pidió una contraseña, la introduje .

Para el resto de los parámetros, simplemente ingresé un valor vacío para aceptar el valor predeterminado. El archivo de certificado creado, `edge.p12`, se guarda en la carpeta de proyectos de la puerta de enlace, `src/main/resources/` almacén de claves. Esto significa que el archivo del certificado se guardará en el archivo `.jar` durante la compilación y estará disponible en la ruta de clases en tiempo de ejecución en `keystore/edge.p12`.



Proporcionar certificados mediante la ruta de clases es suficiente durante el desarrollo, pero no es aplicable a otros entornos, como por ejemplo, un entorno de producción. Consulte la sección "Reemplazo de un certificado autofirmado en tiempo de ejecución" para saber cómo reemplazar este certificado con un certificado externo en tiempo de ejecución.

Configurar la comunicación segura con SSL y TLS es inherentemente complejo. Esta complejidad se agrava porque las diferentes bibliotecas, e incluso las versiones principales de la misma, suelen requerir enfoques de configuración únicos. Los paquetes SSL se introdujeron en Spring Boot 3.1 para simplificar este proceso, un concepto diseñado para optimizar y centralizar la configuración de SSL y TLS. Los paquetes SSL admiten archivos Java KeyStore en formato JKS o PKCS12 y certificados codificados en PEM. Una vez configurado un paquete SSL, puede ser reutilizado por múltiples tipos de conexiones, como un servidor web, una base de datos, un gestor de colas o sus clientes. En este caso, el servidor perimetral utilizará un paquete SSL para proteger la comunicación con su servidor web integrado. Para más información.

Para saber cómo se pueden usar los paquetes SSL en Spring Boot, consulte <https://docs.spring.io/spring-boot/referencia/caracteristicas/ssl.html>.

Para configurar el servidor perimetral para usar el certificado y HTTPS, se agrega lo siguiente a la aplicación.

yaml en el proyecto de puerta de enlace :

```
puerto del servidor: 8443

spring.ssl.bundle.jks.gateway:
  llave:
    alias: localhost
  almacén de claves:
    tipo: PKCS12
    ubicación: classpath:keystore/edge.p12
    contraseña: contraseña

server.ssl.bundle: puerta de enlace
```

Algunas notas del código fuente anterior:

- En primer lugar, se declara un paquete SSL para un almacén de claves Java, llamado puerta de enlace .

El paquete SSL indica la ruta al certificado en el parámetro `keystore.location` , establecido en `classpath:keystore/edge.p12`. Esto significa que el certificado se obtendrá en la ruta de clase de la ubicación `keystore/edge.p12`.

- La contraseña del certificado se especifica en el parámetro `keystore.password` .
- Para indicar que el servidor de borde habla HTTPS y no HTTP, también cambiamos el puerto de 8080 a 8443 en el parámetro `server.port` .
- En segundo lugar, el parámetro `server.ssl.bundle` se utiliza para especificar que el paquete SSL de la puerta de enlace será utilizado por el servidor web incorporado.

Además de estos cambios en el servidor perimetral, también se requieren cambios en los siguientes archivos para reflejar los cambios en el puerto y el protocolo HTTP, reemplazando HTTP por HTTPS y 8080 por 8443:

- Los tres archivos Docker Compose, `docker-compose*.yaml`
- El script de prueba, `test-em-all.bash`

Como se mencionó anteriormente, proporcionar certificados mediante la ruta de clases solo es suficiente durante el desarrollo. Veamos cómo podemos reemplazar este certificado con un certificado externo en tiempo de ejecución.

Reemplazo de un certificado autofirmado en tiempo de ejecución

Colocar un certificado autofirmado en el archivo `.jar` solo es útil para desarrollo. Para que una solución funcione en entornos de ejecución, por ejemplo, para pruebas o producción, debe ser posible usar certificados firmados por CA autorizadas (abreviatura de Autoridades de Certificación).

También debe ser posible especificar los certificados que se utilizarán durante el tiempo de ejecución sin la necesidad de reconstruir los archivos `.jar` y, al usar Docker, la imagen de Docker que contiene el archivo `.jar` . Al usar Docker Compose para administrar el contenedor Docker, podemos asignar un volumen del contenedor a un certificado que reside en el host Docker. También podemos configurar variables de entorno para el contenedor Docker que apunten al certificado externo en el volumen Docker.



En el Capítulo 15, Introducción a Kubernetes, aprenderemos sobre Kubernetes, donde veremos soluciones más potentes sobre cómo manejar secretos, como certificados, que son adecuados para ejecutar contenedores Docker en un clúster; es decir, donde los contenedores se programan en un grupo de hosts Docker y no en un solo host Docker.

Los cambios descritos en este tema no se han aplicado al código fuente en el repositorio de GitHub del libro; ¡debe realizarlos usted mismo para verlos en acción!

Para reemplazar el certificado empaquetado en el archivo .jar , realice los siguientes pasos:

1. Cree un segundo certificado y configure la contraseña como testtest cuando se le solicite:

```
cd $BOOK_HOME/Capítulo 11  
almacén de claves mkdir  
herramienta de claves -genkeypair -alias localhost -keyalg RSA -tamaño de clave 2048 -tipo de almacén  
PKCS12 -almacén de claves keystore/edge-test.p12 -validez 3650
```

2. Actualice el archivo de Docker Compose, docker-compose.yml, con las variables de entorno para la ubicación, la contraseña del nuevo certificado y un volumen que asigne a la carpeta donde se encuentra el nuevo certificado. La configuración del servidor perimetral se verá así después del cambio:

```
puerta:  
  ambiente:  
    - PERFILES_DE_PRIMAVERA_ACTIVOS=docker  
    - UBICACIÓN DEL ALMACÉN DE CLAVES DE PUERTA DE ENLACE JKS DEL PAQUETE SSL DEL SERVIDOR=  
      archivo:/almacén de claves/edge-test.p12  
    - CONTRASEÑA DEL ALMACÉN DE LLAVES DE LA PUERTA DE ENLACE JKS DEL PAQUETE SSL DEL SERVIDOR=  
      pruebaprueba  
  volúmenes:  
    - $PWD/almacén de claves:/almacén de claves  
  compilación: spring-cloud/gateway  
  límite de memoria: 512 m  
  puertos:  
    - "8443:8443"
```

3. Si el servidor perimetral está en funcionamiento, es necesario reiniciarlo con los siguientes comandos:

```
docker composer up -d --scale gateway=0  
docker composer up -d --scale gateway=1
```



El comando docker compose restart gateway podría parecer una buena opción para reiniciar el servicio de puerta de enlace, pero en realidad no tiene en cuenta los cambios en docker-compose.yml . Por lo tanto, no es útil. comando en este caso.

4. ¡El nuevo certificado ya está en uso!

Con esto concluye la sección sobre cómo proteger la comunicación externa con HTTPS. En la siguiente sección, aprenderemos a proteger el acceso al servidor de descubrimiento, Netflix Eureka, mediante HTTP. Autenticación básica.

Asegurar el acceso al servidor de descubrimiento

Anteriormente, aprendimos a proteger la comunicación externa con HTTPS. Ahora usaremos la autenticación HTTP básica para restringir el acceso a las API y páginas web en el servidor de descubrimiento, Netflix Eureka. Esto significa que necesitaremos un nombre de usuario y una contraseña para acceder. Se requieren cambios tanto en el servidor Eureka como en los clientes Eureka, que se describen a continuación.

Cambios en el servidor Eureka

Para proteger el servidor Eureka, se han aplicado los siguientes cambios al código fuente:

1. En build.gradle, se ha agregado una dependencia para Spring Security:

```
implementación 'org.springframework.boot:spring-boot-starter-security'
```

2. Se ha agregado la configuración de seguridad a la clase SecurityConfig :

- a. El usuario se define de la siguiente manera:

```
@Frijol
público InMemoryUserDetailsManager userDetailsService() {
    Detalles del usuario usuario = Usuario.withDefaultPasswordEncoder()
        .username(nombre de usuario)
        .password(contraseña)
        .roles("USUARIO")
        .construir();
    devuelve nuevo InMemoryUserDetailsManager(usuario);
}
```

- b. El nombre de usuario y la contraseña se inyectan en el constructor desde la configuración.

archivo de uración:

```
Configuración de seguridad pública (
    @Value("${app.eureka-username}") Cadena nombre de usuario,
    @Value("${app.eureka-password}") Contraseña de cadena
){
    este.nombreusuario = nombredeusuario;
    this.password = contraseña;
}
```

- c. Todas las API y páginas web están protegidas mediante autenticación básica HTTP mediante de la siguiente definición:

```
@Bean
public SecurityFilterChain configure(HttpSecurity http) lanza una excepción
{

    // Deshabilitar CSRF para permitir que los servicios se registren a sí mismos
    con Eureka

    http .csrf(csrf ->
        csrf.disable()) .authorizeHttpRequests(solicitudes ->
            solicitudes.cualquierSolicitud().autenticada())
        .httpBasic(Customizer.withDefaults()); devolver
        http.build();
}
```

3. Las credenciales del usuario se configuran en el archivo de configuración, application.yml:

```
aplicación:
  eureka-nombre de usuario: u
  contraseña de eureka: p
```

4. Finalmente, la clase de prueba, EurekaServerApplicationTests, utiliza las credenciales de la Archivo de configuración al probar las API del servidor Eureka:

```
@Value("${app.eureka-username}") cadena
privada nombre de usuario;

@Value("${app.eureka-password}")
contraseña de cadena privada ;

@Autowired
void setTestRestTemplate(TestRestTemplate testRestTemplate) { this.testRestTemplate
    = testRestTemplate.withBasicAuth(
        nombre de usuario, contraseña);
}
```

Los pasos anteriores son los necesarios para restringir el acceso a las API y páginas web del servidor de descubrimiento, Netflix Eureka. Ahora utilizará autenticación básica HTTP y requerirá que el usuario proporcione un nombre de usuario y una contraseña para acceder. El último paso es configurar los clientes de Netflix Eureka para que proporcionen credenciales al acceder al servidor.

Cambios en los clientes de Eureka

Para los clientes de Eureka, las credenciales se pueden especificar en la URL de conexión al servidor de Eureka.

Esto se especifica en el archivo de configuración de cada cliente, `application.yml`, de la siguiente manera:

```
aplicación:
  eureka-nombre de usuario: u
  contraseña de eureka: p

¡Eureka!:
  cliente:
    URL del servicio:
      Zona predeterminada: "http://${app.eureka-username}:${app.eureka-
        contraseña}@${app.eureka-server}:8761/eureka/"
```

Con esto concluye la sección sobre cómo restringir el acceso al servidor Netflix Eureka. En la sección "Probar el servidor de descubrimiento protegido", realizaremos pruebas para verificar que el acceso esté protegido. En la siguiente sección, aprenderemos a agregar un servidor de autorización local al sistema.

Agregar un servidor de autorización local

Para poder ejecutar pruebas localmente y de forma totalmente automatizada con APIs protegidas mediante OAuth 2.0 y OpenID Connect, añadiremos a nuestro sistema un servidor de autorización que cumpla con estas especificaciones.

Históricamente, Spring Security no ha proporcionado un servidor de autorización listo para usar. Sin embargo, en abril de 2020, un proyecto comunitario, Spring Authorization Server, liderado por

El equipo de Spring Security anunció el lanzamiento de un servidor de autorización. En agosto de 2021, el proyecto Spring Authorization Server dejó de ser experimental y se incorporó a la cartera de proyectos de Spring. Para más información, consulte <https://spring.io/blog/2020/04/15/anuncio-del-servidor-de-autorización-spring> y <https://spring.io/blog/2021/08/17/el-servidor-de-autorización-de-spring-se-mueve-oficialmente-a-spring-projects>.

El Servidor de Autorización Spring admite tanto el uso del punto de descubrimiento de OpenID Connect como la firma digital de tokens de acceso. También proporciona un punto de descubrimiento al que se puede acceder mediante la información de descubrimiento para obtener claves que verifiquen la firma digital de un token. Gracias a estas funciones, puede utilizarse como servidor de autorización en pruebas locales y automatizadas que verifican el correcto funcionamiento del sistema.

El servidor de autorización de este libro se basa en el servidor de autorización de muestra proporcionado por el proyecto Spring Authorization Server; consulte <https://github.com/spring-projects/spring-servidor-de-autorización/árbol/principal/muestras/servidor-de-autorización-predeterminado>.

Se han aplicado los siguientes cambios al proyecto de muestra:

- El archivo de compilación se ha actualizado para seguir la estructura de los archivos de compilación de otros proyectos. en este libro.
- El puerto está configurado en 9999.
- Se ha agregado un Dockerfile con la misma estructura que para los otros proyectos de este libro.
- El servidor de autorización se ha integrado con Eureka para el descubrimiento de servicios en el De la misma manera que los otros proyectos de este libro.
- Se ha agregado acceso público a los puntos finales del actuador.



ADVERTENCIA: Como ya se advirtió en el Capítulo 7, Desarrollo de Microservicios Reactivos , permitir el acceso público a los puntos finales del actuador es muy útil durante el desarrollo, pero revelar demasiada información en los puntos finales del actuador en sistemas de producción puede representar un problema de seguridad. Por lo tanto, planifique minimizar la información expuesta por los puntos finales del actuador en producción.

- Se han agregado pruebas unitarias que verifican el acceso a los puntos finales más críticos de acuerdo con la especificación OpenID Connect.
 - El nombre de usuario y la contraseña para el usuario registrado individual se establecen en u y p, respectivamente.
- Se registran dos clientes OAuth: lector y escritor. Al cliente lector se le otorga un ámbito de lectura de producto , y al cliente escritor se le otorgan ambos ámbitos . Los clientes están configurados para tener su secreto de cliente establecido en secreto -lector y secreto -escritor, respectivamente.
- Las URI de redireccionamiento permitidas para los clientes se establecen en <https://my.redirect.uri> y <https://localhost:8443/openapi/swagger-ui/oauth2-redirect.html>. La primera URL se usará en las pruebas que se describen a continuación, y la segunda URL la usará la interfaz de usuario de Swagger componente.

Por seguridad, el servidor de autorización no permite redireccionar URLs que comiencen con `https://localhost`. El servidor de autorización se ha personalizado para aceptar `https://localhost` con fines de desarrollo y pruebas. La personalización aplicada se describe aquí: [https://docs.spring.io/spring-authorization-server/docs/1.0.0/reference/html/protocol-endpoints.html#oauth2-authorization-punto final-personalización-autorización-solicitud-validación](https://docs.spring.io/spring-authorization-server/docs/1.0.0/reference/html/protocol-endpoints.html#oauth2-authorization-punto-final-personalización-autorización-solicitud-validación).

El código fuente del servidor de autorización está disponible en `$BOOK_HOME/Chapter11/spring-cloud/authorization-server`.

Para incorporar el servidor de autorización en el panorama del sistema, se han aplicado cambios en los siguientes archivos:

- El servidor se ha agregado al archivo de compilación común, `settings.gradle`.
- Se ha agregado el servidor a los tres archivos Docker Compose, `docker-compose*.yaml`.
- El servidor de borde, `spring-cloud/gateway`.
- Se ha agregado una comprobación de estado para el servidor de autorización en `HealthCheckConfiguration`.

Se han añadido rutas al servidor de autorización para las URI que empiezan por `/oauth`, `/login` y `/error` en el archivo de configuración `application.yaml`. Estas URI se utilizan para emitir tokens a los clientes, autenticar usuarios y mostrar mensajes de error.

- Dado que el servidor perimetral no debe proteger estos tres URI, se configuran en la nueva clase `SecurityConfig` para permitir todas las solicitudes.

Con una comprensión de cómo se agrega un servidor de autorización local al panorama del sistema, avancemos y veamos cómo usar OAuth 2.0 y OpenID Connect para autenticar y autorizar. acceso a las API.

Protección de API mediante OAuth 2.0 y OpenID Connect

Con el servidor de autorización en su lugar, podemos mejorar el servidor de borde y el producto compuesto. El servicio se convertirá en servidores de recursos OAuth 2.0, por lo que requerirá un token de acceso válido para permitir el acceso. El servidor perimetral se configurará para aceptar cualquier token de acceso que pueda validar mediante la firma digital proporcionada por el servidor de autorización. El servicio compuesto por producto también requerirá que el token de acceso contenga ámbitos OAuth 2.0 válidos:

- El alcance del producto: lectura será necesario para acceder a las API de solo lectura
- El alcance del producto: escritura será necesario para acceder a las API de creación y eliminación.

El servicio compuesto por productos también se mejorará con una configuración que permite que su componente Swagger UI interactúe con el servidor de autorización para emitir un token de acceso. Esto permitirá a los usuarios de la página web de Swagger UI probar la API protegida.

También necesitamos mejorar el script de prueba, `test-em-all.bash`, para que adquiera tokens de acceso y los utilice cuando realiza las pruebas.

Cambios tanto en el servidor de borde como en el servicio compuesto por productos

Se han aplicado los siguientes cambios en el código fuente tanto para el servidor perimetral como para el servicio compuesto del producto :

- Se han agregado dependencias de Spring Security a `build.gradle` para admitir OAuth 2.0

servidores de recursos:

```
implementación 'org.springframework.boot:spring-boot-starter-security'

implementación 'org.springframework.security:spring-security-oauth2-
servidor de recursos

implementación 'org.springframework.security:spring-security-oauth2-
José'
```

- Se han agregado configuraciones de seguridad a las nuevas clases `SecurityConfig` en ambos proyectos:

```
@Configuración
@EnableWebFluxSecurity
clase pública SecurityConfig {

    @Frijol
    Cadena de filtros web de seguridad springSecurityFilterChain(
        SeguridadHttp del servidor http) {
        http
        .csrf(csrf -> csrf.disable())
        .authorizeExchange(cambio -> cambio
            .pathMatchers("/actuador/**").permitAll()
            .anyExchange().autenticado()))
        .oauth2ResourceServer(servidor -> servidor.jwt(
            Personalizador.withDefaults()));

        devolver http.build();
    }
}
```

- Las explicaciones del código fuente anterior son las siguientes:
 - La anotación `@EnableWebFluxSecurity` habilita la compatibilidad de Spring Security con API basadas en Spring WebFlux.
 - `.pathMatchers("/actuator/**").permitAll()` se utiliza para permitir el acceso sin restricciones a URL que deberían estar desprotegidas; por ejemplo, los puntos finales del actuador, en este caso. Consulte el código fuente para ver las URL que se consideran desprotegidas. Tenga cuidado con las URL que se exponen sin protección. Por ejemplo, el actuador. Los puntos finales deben protegerse antes de pasar a producción.
 - `.anyExchange().authenticated()` garantiza que el usuario esté autenticado antes permitiéndose el acceso a todas las demás URL.
 - `.oauth2ResourceServer(...)` especifica que la autorización se basará en OAuth 2.0 tokens de acceso codificados como JWT.
 - El punto final de descubrimiento OIDC del servidor de autorización se ha registrado en el archivo de configuración, `application.yml`:

```
aplicación.auth-server: localhost

spring.security.oauth2.resourceserver.jwt.url del emisor:
http://${app.auth-server}:9999

spring.config.activate.on-profile: docker

app.auth-server: servidor de autenticación
```



Más adelante en este capítulo, al iniciar el entorno del sistema, puede probar el punto final de descubrimiento. Por ejemplo, puede encontrar el punto final que devuelve las claves necesarias para verificar la firma digital de un token mediante este comando:

```
docker compose exec auth-server curl localhost:9999/
configuración conocida/openid -s | jq -r .jwks_uri
```

También necesitamos realizar algunos cambios que sólo se apliquen al servicio compuesto por producto .

Cambios únicamente en el servicio compuesto por producto

Además de los cambios comunes aplicados en la sección anterior, también se han aplicado los siguientes cambios al servicio compuesto por producto :

- La configuración de seguridad en la clase `SecurityConfig` se ha refinado al requerir

OAuth 2.0 establece el alcance del token de acceso para permitir el acceso:

```
.pathMatchers(POST, "/producto-compuesto/**")
    .hasAuthority("SCOPE_product:write")
.pathMatchers(BORRAR, "/producto-compuesto/**")
    .hasAuthority("SCOPE_product:write")
.pathMatchers(GET, "/producto-compuesto/**")
    .hasAuthority("SCOPE_product:read")
```



Por convención, los ámbitos de OAuth 2.0 deben tener el prefijo `SCOPE_` cuando se verifica la autoridad mediante Spring Security.

Se ha añadido un método, `logAuthorizationInfo()`, para registrar las partes relevantes del token de acceso codificado con JWT en cada llamada a la API. El token de acceso se puede obtener mediante el `SecurityContext` estándar de Spring Security, que , en un entorno reactivo, se puede obtener mediante el método auxiliar estático `ReactiveSecurityContextHolder`.

`getContext()`. Consulte la clase `ProductCompositeServiceImpl` para obtener más detalles.

Se ha deshabilitado el uso de OAuth al ejecutar pruebas de integración basadas en Spring. Para evitar que OAuth se active al ejecutar pruebas de integración, lo deshabilitamos de la siguiente manera:

- Se agrega una configuración de seguridad, `TestSecurityConfig`, para utilizarla durante las pruebas.

Permite el acceso a todos los recursos:

```
http.csrf(csrf -> csrf.disable().authorizeExchange(
    intercambio -> intercambio.anyExchange().permitAll()));
```

- En cada clase de prueba de integración de Spring, configuramos `TestSecurityConfig` para anular

La configuración de seguridad existente con lo siguiente:

```
@SpringBootTest(
    clases = {TestSecurityConfig.class},
    propiedades = {"spring.main.allow-bean-definition-
        anulación=verdadero"})
```

Cambios para permitir que Swagger UI adquiera tokens de acceso

Para permitir el acceso a las API protegidas desde el componente Swagger UI, se han aplicado los siguientes cambios en el servicio compuesto del producto :

Las páginas web expuestas por el componente Swagger UI se han configurado para que sean públicas . Se ha añadido la siguiente línea a la clase SecurityConfig :

```
.pathMatchers("/openapi/**").permitAll()
```

- Se ha mejorado la especificación OpenAPI de la API para exigir que security_auth

Se aplica el esquema de seguridad.

Se ha agregado la siguiente línea a la definición de la interfaz ProductCompositeService en el proyecto API:

```
@SecurityRequirement(nombre = "security_auth")
```

Para definir la semántica del esquema de seguridad security_auth , se ha añadido la clase OpenApiConfig al proyecto compuesto por producto . Su aspecto es el siguiente:

```
@SecurityScheme( nombre = "security_auth", tipo = SecuritySchemeType.OAUTH2,
    flujos = @OAuthFlows( código
        de autorización = @OAuthFlow( URL de
            autorización = "${springdoc.oAuthFlow.authorizationUrl}",
            URL de token = "${
                {springdoc.oAuthFlow.tokenUrl}", ámbitos = {

                @OAuthScope(nombre = "producto:lectura", descripción = "alcance
                    de lectura"),
                @OAuthScope(nombre = "producto:escritura", descripción = "ámbito
                    de escritura")
            }
        )
    )
))
clase pública OpenApiConfig {}
```

- De la definición de clase anterior, podemos ver lo siguiente:

- El esquema de seguridad se basará en OAuth 2.0.
- Se utilizará el flujo de concesión del código de autorización.

- Las URL necesarias para adquirir un código de autorización y tokens de acceso serán proporcionadas por la configuración utilizando los parámetros `springdoc.oauthFlow.authorizationUrl` y `springdoc.oauthFlow.tokenUrl`.
- Una lista de alcances (producto: lectura y producto: escritura) que requerirá Swagger UI para poder llamar a las API.
- Por último, se agregan algunas configuraciones a `application.yml`:

interfaz de usuario swagger:

URL de redirección de oauth2: `/swagger-ui/oauth2-redirect.html`

oauth:

ID de cliente: `escritor`

`clientSecret`: `escritor secreto`

`usarAutenticaciónBásicaConCódigoDeAccesoConcesión`: `verdadero`

Flujo de OAuth:

URL de autorización: `https://localhost:8443/oauth2/authorize`

URL del token: `https://localhost:8443/oauth2/token`

- De la configuración anterior podemos observar lo siguiente:
 - La URL de redireccionamiento que Swagger UI utilizará para adquirir el código de autorización.
 - Su ID de cliente y secreto de cliente.
 - Utilizará la autenticación básica HTTP al identificarse para la autorización.
- servidor.
- Los valores de los parámetros `"authorizationUrl"` y `"tokenUrl"` utilizados por la clase `OpenApiConfig` descrita anteriormente. Tenga en cuenta que estas URL las utiliza el navegador web, no el servicio compuesto por el producto, por lo que deben poder resolverse desde el navegador web.

Para permitir el acceso sin protección a las páginas web de Swagger UI, el servidor perimetral también se ha configurado para permitir el acceso sin restricciones a las URL dirigidas al componente Swagger UI. Se ha añadido lo siguiente a la clase `SecurityConfig` del servidor perimetral :

```
.pathMatchers("/openapi/**").permitAll()
```

Con estos cambios implementados, tanto el servidor perimetral como el servicio compuesto por producto pueden actuar como servidores de recursos OAuth 2.0, y el componente Swagger UI puede actuar como cliente OAuth. El último paso para implementar el uso de OAuth 2.0 y OpenID Connect es actualizar el script de prueba para que adquiera tokens de acceso y los utilice al ejecutar las pruebas.

Cambios en el script de prueba

Para empezar, necesitamos obtener un token de acceso antes de poder llamar a cualquiera de las API, excepto a la API de salud. Esto se hace, como ya se mencionó, mediante el flujo de credenciales de cliente de OAuth 2.0. Para poder llamar a las API de creación y eliminación, adquirimos un token de acceso como cliente de escritura, de la siguiente manera:

```
TOKEN DE ACCESO=$(curl -k https://escritor:escritor-secreto@$HOST:$PORT/
oauth2/token -d tipo_concesión=credenciales_del_cliente -d ámbito="producto:lectura producto:escritura" -s |
jq .access_token -r)
```

Del comando anterior, podemos ver que utiliza autenticación básica HTTP, pasando su ID de cliente y secreto de cliente como `writer:secret-writer@` antes del nombre de host.

Para verificar que la autorización basada en alcance funciona, se han agregado dos pruebas al script de prueba:

```
# Verificar que una solicitud sin token de acceso falla en 401, No autorizado
assertCurl 401 "curl -k https://$HOST:$PORT/compuesto-de-producto/$PROD_ID_
REVS_RECS -s"

# Verifique que el cliente lector con solo alcance de lectura pueda llamar a la API de lectura
pero no eliminar API

TOKEN DE ACCESO DEL LECTOR=$(curl -k https://lector:lector-secreto@$HOST:$PORT/
oauth2/token -d tipo_concesión=credenciales_del_cliente -d ámbito="producto:lectura" -s | jq .token_de_acceso -r)

READER_AUTH="-H \"Autorización: Portador $READER_ACCESS_TOKEN\""

assertCurl 200 "curl -k https://$HOST:$PORT/compuesto-del-producto/$PROD_ID_
REVS_RECS $READER_AUTH -s"
assertCurl 403 "curl -k https://$HOST:$PORT/compuesto-del-producto/$PROD_ID_
REVS_RECS $READER_AUTH -X ELIMINAR -s"
```

El script de prueba utiliza las credenciales del cliente lector para adquirir un token de acceso:

- La primera prueba llama a una API sin proporcionar un token de acceso. Se espera que la API devuelva el estado HTTP 401 no autorizado.
- La segunda prueba verifica que el cliente lector pueda llamar a una API de solo lectura.

La última prueba llama a una API de actualización mediante el cliente de lectura, al que solo se le otorga un alcance de lectura. Se espera que una solicitud enviada a la API de eliminación devuelva el estado HTTP 403 "Prohibido".

Para ver el código fuente completo, consulte `test-em-all.bash`.

Con el script de prueba actualizado para adquirir y usar tokens de acceso OAuth 2.0, estamos listos para probarlo.

¡En la siguiente sección!

Pruebas con el servidor de autorización local

En esta sección, probaremos el entorno del sistema seguro; es decir, probaremos todos los componentes de seguridad en conjunto. Utilizaremos el servidor de autorización local para emitir tokens de acceso. Se realizarán las siguientes pruebas:

1. Primero, construimos desde la fuente y ejecutamos el script de prueba para asegurarnos de que todo encaja.
2. A continuación, probaremos la API y la página web del servidor de descubrimiento protegido.
3. Después de eso, aprenderemos cómo adquirir tokens de acceso utilizando credenciales de cliente OAuth 2.0 y flujos de concesión de códigos de autorización.
4. Con los tokens de acceso emitidos, probaremos las API protegidas. También verificaremos que un token de acceso emitido para un cliente lector no pueda usarse para llamar a una API de actualización.
5. Finalmente, también verificaremos que Swagger UI pueda emitir tokens de acceso y llamar a las API.

Construir y ejecutar las pruebas automatizadas

Para construir y ejecutar pruebas automatizadas, realizamos los siguientes pasos:

1. Primero, construya las imágenes de Docker desde la fuente con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 11
./gradlew compilación y compilación de docker compose
```

2. A continuación, inicie el panorama del sistema en Docker y ejecute las pruebas habituales con lo siguiente dominio:

```
./test-em-all.bash inicio
```



Tenga en cuenta las nuevas pruebas negativas al final que verifican que obtenemos un 401 Código no autorizado devuelto cuando no está autenticado y 403 Prohibido cuando no esté autorizado.

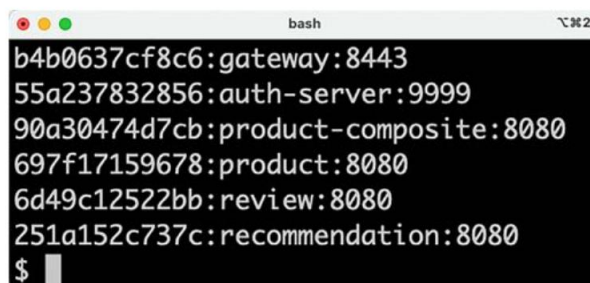
Prueba del servidor de descubrimiento protegido

Con el servidor de descubrimiento protegido, Eureka, en funcionamiento, tenemos que proporcionar credenciales válidas para poder acceder a sus API y páginas web.

Por ejemplo, solicitar instancias registradas al servidor Eureka se puede hacer mediante el siguiente comando curl, donde suministramos el nombre de usuario y la contraseña directamente en la URL:

```
curl -H "accept:application/json" https://u:p@localhost:8443/eureka/api/
aplicaciones -ks | jq -r '.aplicaciones.aplicación[].instancia[].idinstancia'
```

Un ejemplo de respuesta es el siguiente:

A terminal window titled 'bash' with a dark background. It displays the output of a curl command, listing several Eureka service instances with their IDs and instance IDs. The output is as follows:

```
b4b0637cf8c6:gateway:8443
55a237832856:auth-server:9999
90a30474d7cb:product-composite:8080
697f17159678:product:8080
6d49c12522bb:review:8080
251a152c737c:recommendation:8080
$
```

Figura 11.3: Servicios registrados en Eureka mediante una llamada API

Al acceder a la página web <https://localhost:8443/eureka/web>, primero tenemos que aceptar una conexión no segura, ya que nuestro certificado es autofirmado, y a continuación, tenemos que proporcionar credenciales válidas, tal y como se especifica en el archivo de configuración (u como nombre de usuario y p como contraseña):

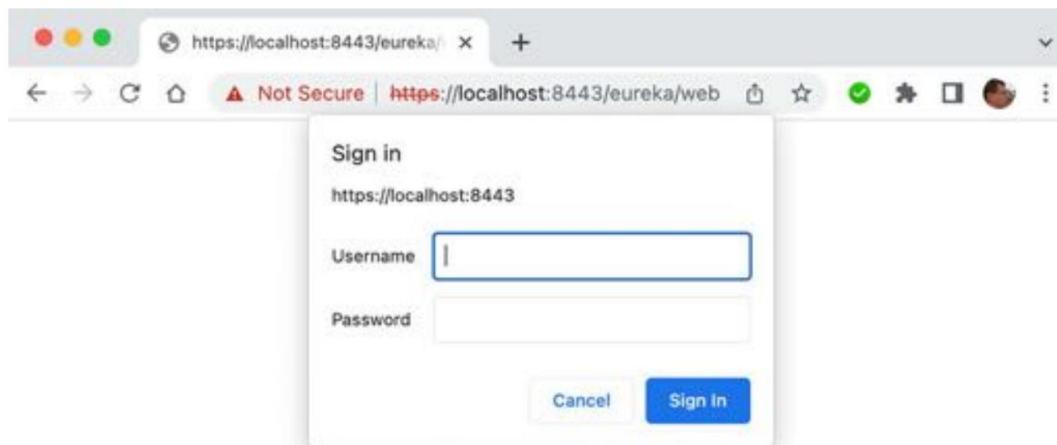
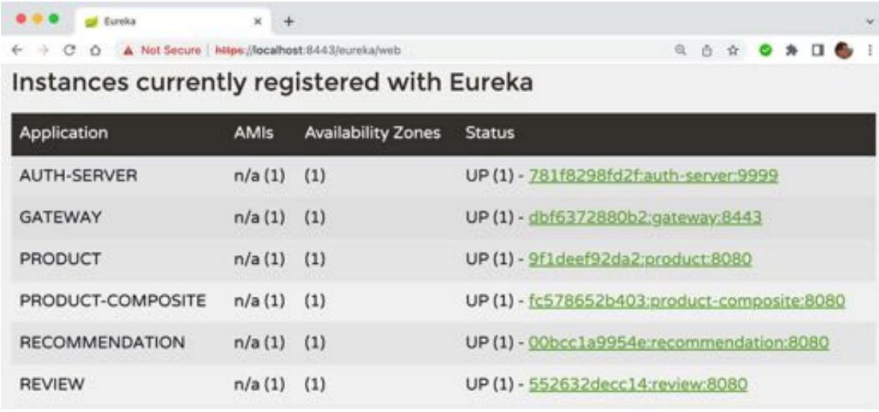


Figura 11.4: Eureka requiere autenticación

Después de iniciar sesión correctamente, veremos la página web familiar del servidor Eureka:



The screenshot shows a web browser window with the title 'Eureka'. The address bar shows 'https://localhost:8443/eureka/web'. The page content is titled 'Instances currently registered with Eureka'. It contains a table with the following data:

Application	AMIs	Availability Zones	Status
AUTH-SERVER	n/a (1)	(1)	UP (1) - 781f8298fd2f:auth-server:9999
GATEWAY	n/a (1)	(1)	UP (1) - dbf6372880b2:gateway:8443
PRODUCT	n/a (1)	(1)	UP (1) - 9f1deef92da2:product:8080
PRODUCT-COMPOSITE	n/a (1)	(1)	UP (1) - fc578652b403:product-composite:8080
RECOMMENDATION	n/a (1)	(1)	UP (1) - 00bcc1a9954e:recommendation:8080
REVIEW	n/a (1)	(1)	UP (1) - 552632decc14:review:8080

Figura 11.5: Servicios registrados en Eureka utilizando la página web

Después de asegurarnos de que el acceso al servidor Eureka esté protegido, aprenderemos cómo emitir OAuth tokens de acceso.

Adquisición de tokens de acceso

Ahora estamos listos para adquirir tokens de acceso mediante los flujos de concesión definidos por OAuth 2.0. Primero probaremos el flujo de concesión de credenciales de cliente y, a continuación, el flujo de concesión de código de autorización.

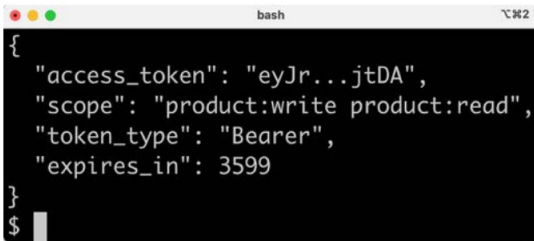
Adquisición de tokens de acceso mediante el flujo de concesión de credenciales del cliente

Para obtener un token de acceso para el cliente escritor, es decir, tanto con el producto:lectura como con el producto:escritura ámbitos, emita el siguiente comando:

```
curl -k https://escritor:escritor-secreto@localhost:8443/oauth2/token -d concesión_
tipo=credenciales_de_cliente -d alcance="producto:leer producto:escribir" -s | jq .
```

El cliente se identifica mediante autenticación básica HTTP, indicando su ID de cliente (escritor) y su secreto de cliente (secret).

A continuación, se muestra un ejemplo de respuesta:



```
bash
{
  "access_token": "eyJr...jtDA",
  "scope": "product:write product:read",
  "token_type": "Bearer",
  "expires_in": 3599
}
```

Figura 11.6: Respuesta del token de muestra

En la captura de pantalla, podemos ver que obtuvimos la siguiente información en la respuesta:

- El token de acceso en sí.
- Los alcances otorgados al token. El cliente escritor tiene los alcances `product:write` y `product:read`. También tiene el alcance `openid`, que permite acceder a información sobre el ID del usuario, como su dirección de correo electrónico.
- El tipo de token que obtuvimos; Portador significa que se le debe dar acceso al portador de este token. según los alcances otorgados al token.
- La cantidad de segundos que el token de acceso es válido: 3599 segundos en este caso.

Para obtener un token de acceso para el cliente lector, es decir, solo con el alcance `product:read`, simplemente reemplace `writer` con `reader` en el comando anterior, lo que dará como resultado lo siguiente:

```
curl -k https://lector:lector-secreto@localhost:8443/oauth2/token -d grant_
tipo=credenciales_de_cliente -d alcance="producto:leer" -s | jq .
```

Adquisición de tokens de acceso mediante el flujo de concesión de código de autorización

Para obtener un token de acceso mediante el flujo de concesión del código de autorización, necesitamos un navegador web. Este flujo de concesión es un poco más complejo para garantizar su seguridad en un entorno parcialmente inseguro (el navegador web).

En el primer paso no seguro, utilizaremos el navegador web para obtener un código de autorización de un solo uso, que se canjeará por un token de acceso. El código de autorización se transferirá del navegador web a una capa segura (por ejemplo, el código del servidor), que puede realizar una nueva solicitud al servidor de autorización para intercambiar el código de autorización por un token de acceso. En este intercambio seguro, el servidor debe proporcionar un secreto de cliente para verificar su identidad.

Realice los siguientes pasos para ejecutar el flujo de concesión del código de autorización:

1. Para obtener un código de autorización para el cliente lector, utilice la siguiente URL en un navegador web que acepte el uso de certificados autofirmados, por ejemplo, Chrome: `https://localhost:8443/oauth2/autorizar?tipo_de_respuesta=código&id_de_cliente=lector&uri_de_redirección=https://mi.redirect.uri&scope=producto:lectura&estado=35725`.

2. Cuando el navegador web le solicite iniciar sesión, utilice las credenciales especificadas en la configuración del servidor de autorización, u y p:

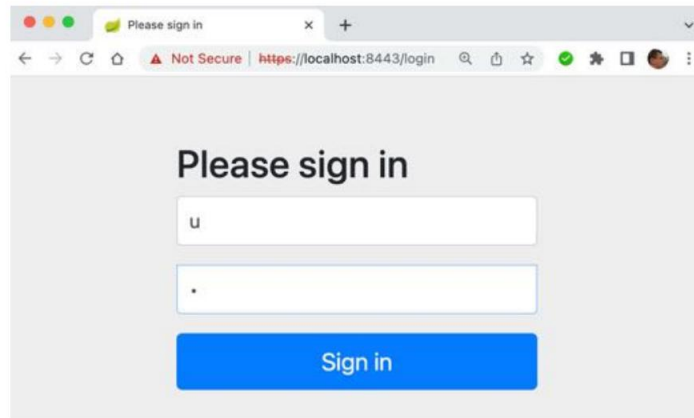


Figura 11.7: Prueba del flujo de concesión del código de autorización

3. A continuación, se nos solicitará que demos el consentimiento del cliente lector para llamar a las API en nuestro nombre:

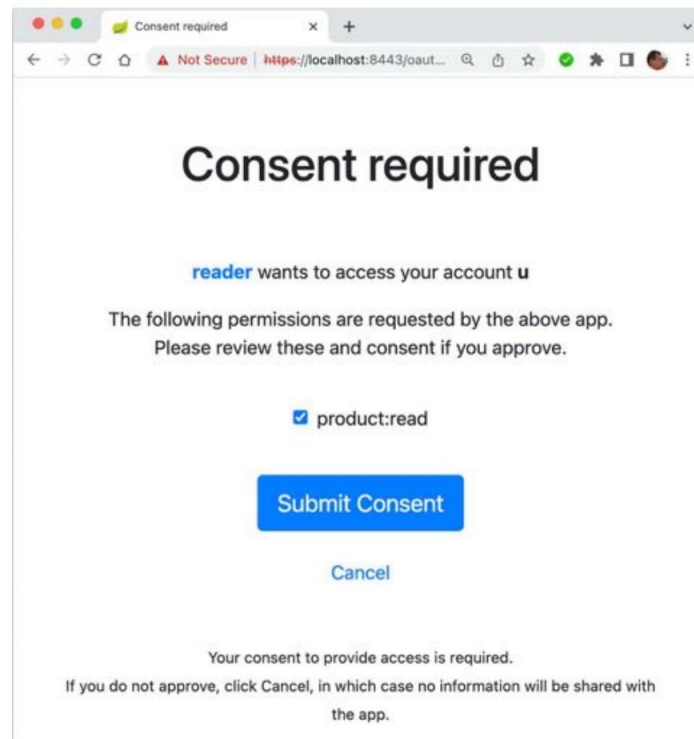


Figura 11.8: Página de consentimiento del flujo de concesión de código de autorización

4. Después de hacer clic en el botón Enviar consentimiento , recibiremos la siguiente respuesta:

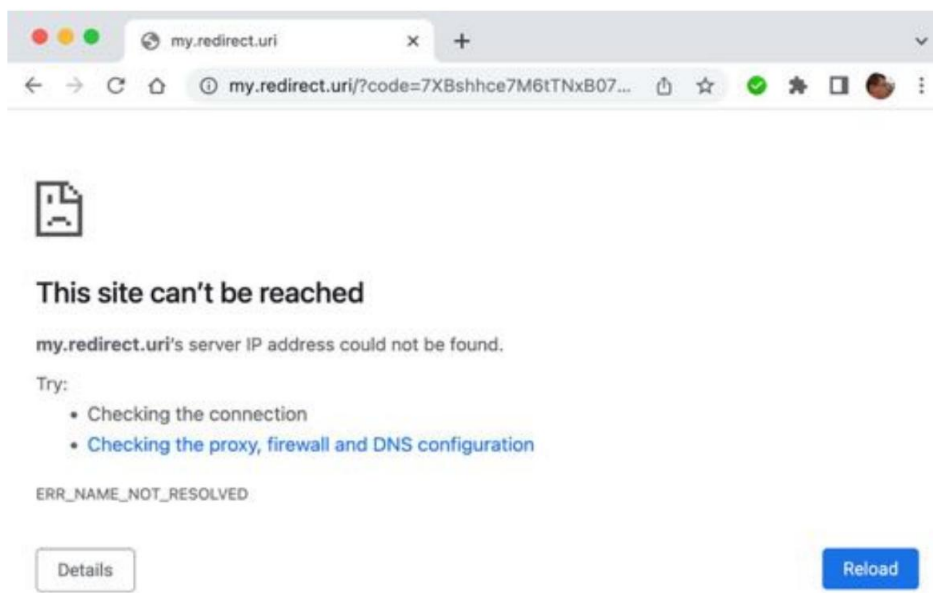


Figura 11.9: Página de redirección del flujo de concesión del código de autorización

5. A primera vista, esto podría parecer un poco decepcionante. La URL que el servidor de autorización envía al navegador web se basa en la URI de redirección especificada por el cliente en la solicitud inicial. Copie la URL en un editor de texto y encontrará algo similar a lo siguiente:

```
https://my.redirect.uri/?code=7XBs...0mmyk&state=35725
```

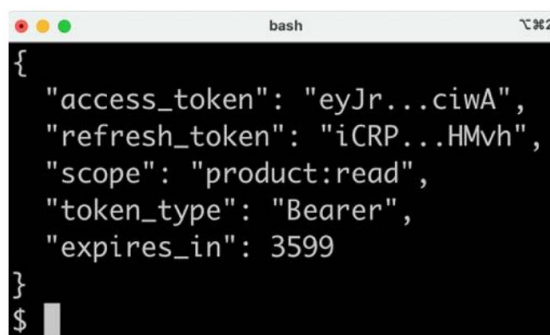
¡Genial! Podemos encontrar el código de autorización en la URL de redirección, en el parámetro de solicitud de código . Extraiga el código de autorización del parámetro de código y defina una variable de entorno, CODE, con su valor:

```
CÓDIGO=7XBs...0mmyk
```

6. A continuación, imagine que es el servidor backend que intercambia el código de autorización con un token de acceso utilizando el siguiente comando curl :

```
curl -k https://lector:lector-secreto@localhost:8443/oauth2/token \
-d tipo_de_concesión=código_de_autorización \
-d id_cliente=lector \
-d redirect_uri=https://mi.redirect.uri \
-d código=$CODE -s | jq .
```

Un ejemplo de respuesta es el siguiente:

A screenshot of a terminal window with a dark background and light text. The window title is "bash". It displays a JSON object representing an OAuth2 token response. The JSON has five key-value pairs: "access_token" with a long alphanumeric string, "refresh_token" with another long alphanumeric string, "scope" with the value "product:read", "token_type" with the value "Bearer", and "expires_in" with the value 3599. The JSON is enclosed in curly braces, and a dollar sign prompt is visible at the bottom left.

```
{
  "access_token": "eyJr...ciwA",
  "refresh_token": "iCRP...HMvh",
  "scope": "product:read",
  "token_type": "Bearer",
  "expires_in": 3599
}
```

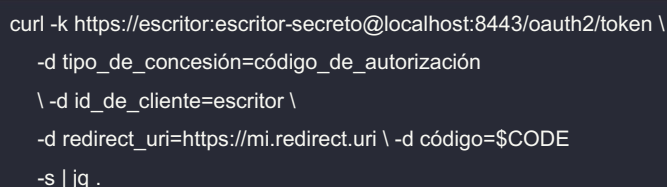
Figura 11.10: Token de acceso al flujo de concesión de código de autorización

En la captura de pantalla, podemos ver que obtuvimos información similar en la respuesta a la que obtuvimos del flujo de credenciales del cliente, con las siguientes excepciones:

- Dado que usamos un flujo de concesión más seguro, también obtuvimos un token de actualización emitido.
- Dado que solicitamos un token de acceso para el cliente lector, solo obtuvimos un producto: lectura. alcance, sin producto:escribir alcance

7. Para obtener un código de autorización para el cliente escritor, utilice la siguiente URL: https://localhost:8443/oauth2/authorize?response_type=code&client_id=writer&redirect_uri=https://my.redirect.uri&scope=product:read+product:write&state=72489.

8. Para intercambiar el código por un token de acceso para el cliente escritor, ejecute el siguiente comando:

A screenshot of a terminal window with a dark background and light text. It shows a single line of a curl command used to exchange an authorization code for an access token. The command includes several options: -k for insecure SSL, -d for data, and -s for silent mode. The data includes the client ID, redirect URI, and the authorization code stored in a variable named CODE.

```
curl -k https://escritor:escritor-secreto@localhost:8443/oauth2/token \
  -d tipo_de_concesión=código_de_autorización \
  -d id_de_cliente=escritor \
  -d redirect_uri=https://mi.redirect.uri \ -d código=$CODE
-s | jq .
```

9. Verifique que la respuesta ahora también contenga el alcance product:write .

Llamar a API protegidas mediante tokens de acceso

Ahora, usemos los tokens de acceso que hemos adquirido para llamar a las API protegidas.

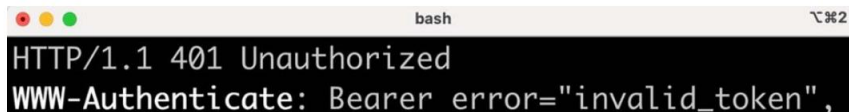
Se espera que un token de acceso OAuth 2.0 se envíe como un encabezado de autorización HTTP estándar, donde el token de acceso tiene el prefijo Bearer.

Ejecute los siguientes comandos para llamar a las API protegidas:

1. Primero, llame a una API para recuperar un producto compuesto sin un token de acceso válido:

```
ACCESS_TOKEN=un token no válido  
rizo https://localhost:8443/producto-compuesto/1 -k -H  
"Autorización: Portador $ACCESS_TOKEN" -i
```

Debería devolver la siguiente respuesta:



```
bash  
HTTP/1.1 401 Unauthorized  
WWW-Authenticate: Bearer error="invalid_token",
```

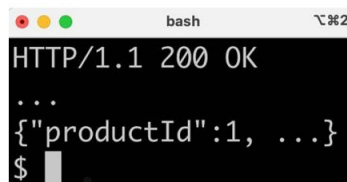
Figura 11.11: Un token no válido da como resultado una respuesta 401 No autorizado

¡El mensaje de error indica claramente que el token de acceso no es válido!

2. A continuación, intente usar la API para recuperar un producto compuesto utilizando uno de los tokens de acceso adquiridos para el cliente lector de la sección anterior:

```
ACCESS_TOKEN={token de acceso de un lector}  
rizo https://localhost:8443/producto-compuesto/1 -k -H  
"Autorización: Portador $ACCESS_TOKEN" -i
```

Ahora obtendremos el código de estado 200 OK y se devolverá el cuerpo de respuesta esperado:



```
bash  
HTTP/1.1 200 OK  
...  
{"productId":1, ...}  
$
```

Figura 11.12: Un token de acceso válido da como resultado una respuesta 200 OK

3. Si intentamos acceder a una API de actualización, por ejemplo, la API de eliminación, con un token de acceso adquirido para el cliente lector, la llamada fallará:

```
ACCESS_TOKEN={token de acceso de un lector}  
rizo https://localhost:8443/producto-compuesto/999 -k -H  
"Autorización: Portador $ACCESS_TOKEN" -X DELETE -i
```

Fallará con una respuesta similar a la siguiente:



```
bash
HTTP/1.1 403 Forbidden
WWW-Authenticate: Bearer error="insufficient_scope", error_description="The request requires higher privileges than provided by the access token.", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"
$
```

Figura 11.13: Un alcance insuficiente da como resultado un resultado 403 Prohibido

De la respuesta de error se desprende claramente que tenemos prohibido llamar a la API ya que la solicitud requiere privilegios superiores a los que se le otorgan a nuestro token de acceso.

4. Si repetimos la llamada a la API de eliminación, pero con un token de acceso adquirido para el escritor Cliente, la llamada tendrá éxito con 202 Aceptado en la respuesta.



La operación de eliminación debe devolver 202 incluso si el producto con el ID de producto especificado no existe en la base de datos subyacente, ya que la eliminación

La operación es idempotente, como se describe en el Capítulo 6, "Añadir persistencia".

Consulte la sección "Añadir nuevas API" .

Si observa la salida del registro utilizando `docker compose logs -f product-composite` comando, debería poder encontrar información de autorización como la siguiente:



```
bash
chapter11-product-composite-1 | ... Authorization info: Subject: u,
scopes: [product:write, product:read], expires 2023-04-07T05:27:38Z:
issuer: http://auth-server:9999, audience: [writer]
$
```

Figura 11.14: Información de autorización en la salida del registro

Esta información se extrajo en el servicio compuesto del producto a partir de la codificación JWT. token de acceso ed; el servicio compuesto por producto no necesitaba comunicarse con el ¡Servidor de autorización para obtener esta información!

Con estas pruebas, hemos visto cómo adquirir un token de acceso con las credenciales del cliente y los flujos de concesión del código de autorización. También hemos visto cómo se pueden usar los ámbitos para limitar lo que un cliente puede hacer con un token de acceso específico; por ejemplo, usarlo solo para operaciones de lectura.

Prueba de la interfaz de usuario de Swagger con OAuth 2.0

En esta sección, aprenderemos cómo utilizar el componente Swagger UI para acceder a la API protegida.

La configuración descrita en la sección anterior Cambios en el servicio compuesto de producto únicamente nos permite emitir un token de acceso para Swagger UI y usarlo al llamar a las API desde Swagger UI.

Para probarlo, realice los siguientes pasos:

1. Abra la página de inicio de Swagger UI yendo a la siguiente URL en un navegador web: `https://localhost:8443/openapi/swagger-ui.html`.
2. En la página de inicio, ahora podemos ver un nuevo botón, junto a la lista desplegable Servidores , con el texto Autorizar.
3. Haga clic en el botón Autorizar para iniciar un flujo de concesión de código de autorización.
4. La interfaz de usuario de Swagger mostrará una lista de ámbitos a los que solicitará acceso al servidor de autorización. Seleccione todos los ámbitos haciendo clic en el enlace con el texto " Seleccionar todo" y luego haciendo clic en el botón Autorizar :

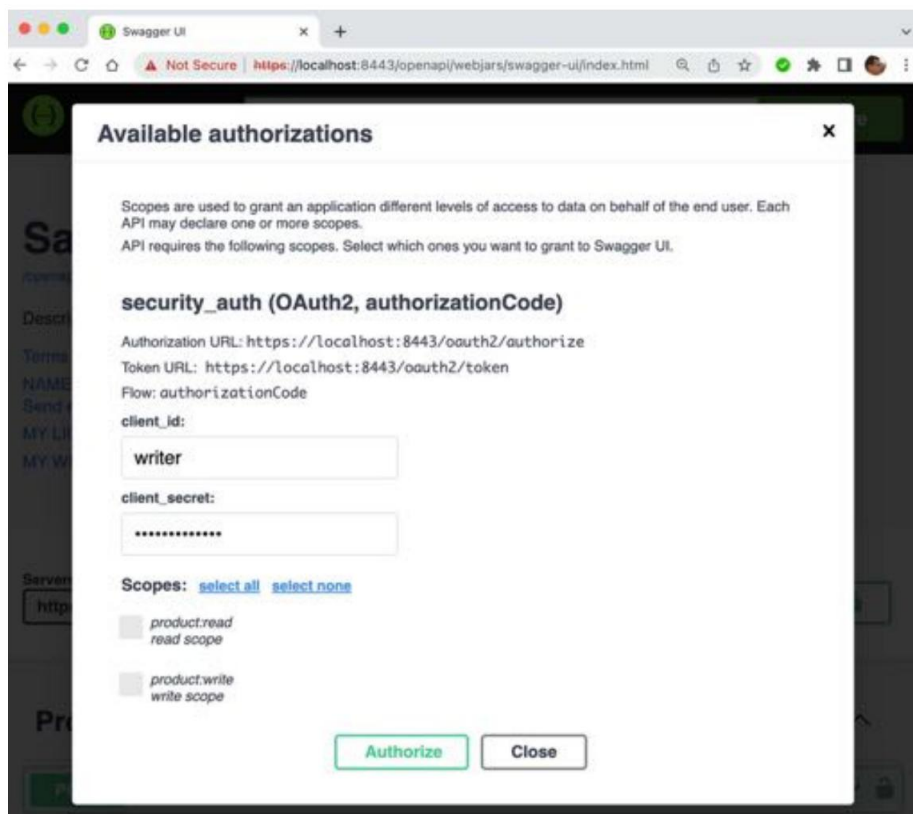


Figura 11.15: Interfaz de usuario de Swagger solicitando ámbitos OAuth

Se le redirigirá al servidor de autorización. Si aún no ha iniciado sesión desde el navegador web que utiliza, el servidor le solicitará sus credenciales, como se indica en la sección "Adquisición de tokens de acceso mediante el flujo de concesión del código de autorización".

5. Inicie sesión con el nombre de usuario *u* y la contraseña *p*.
6. A continuación, el servidor de autorización le solicitará su consentimiento. Seleccione ambos ámbitos y haga clic en el botón Enviar consentimiento.
7. La interfaz de usuario de Swagger completará el proceso de autorización mostrando información sobre el flujo de concesión completado. Haga clic en el botón Cerrar para volver a la página de inicio.

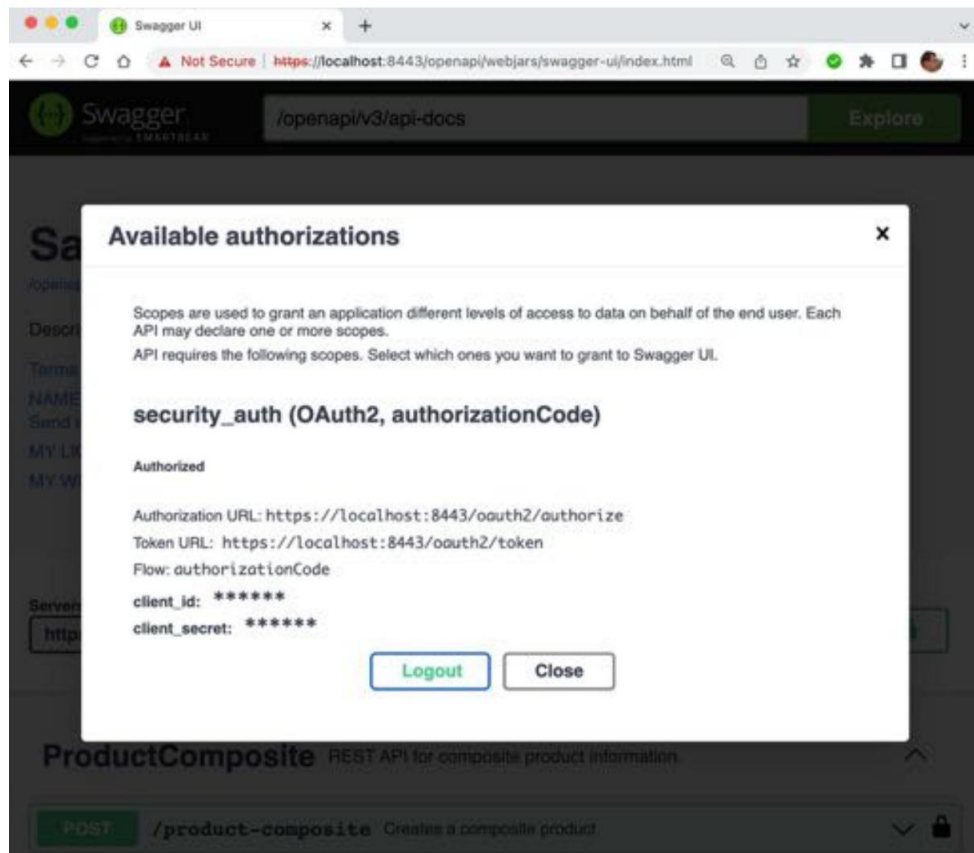


Figura 11.16: Interfaz de usuario de Swagger que resume el flujo de concesión de OAuth

8. Ahora puede probar las API de la misma manera que se describe en el Capítulo 5, "Añadir una descripción de API con OpenAPI". Swagger UI añadirá el token de acceso a las solicitudes. Si observa detenidamente el comando curl que se muestra debajo del encabezado "Respuestas", encontrará... el token de acceso.

Con esto finalizamos las pruebas que realizaremos con el servidor de autorización local. En la siguiente sección, lo reemplazaremos con un proveedor externo compatible con OpenID Connect.

Pruebas con un proveedor externo de OpenID Connect

Entonces, el proceso OAuth funciona correctamente con un servidor de autorización que controlamos nosotros mismos. Pero ¿qué pasa si lo reemplazamos con un proveedor certificado de OpenID Connect? En teoría, debería funcionar de inmediato.

Averigüémoslo, ¿de acuerdo?



Para obtener una lista de implementaciones certificadas de OpenID Connect, consulte <https://openid.net/desarrolladores/certificados/>.

Utilizaremos Auth0, <https://auth0.com/>, para nuestras pruebas con un proveedor externo de OpenID. Para poder usar Auth0 en lugar de nuestro propio servidor de autorización, revisaremos los siguientes temas:

- Configurar una cuenta con un cliente lector y escritor y un usuario en Auth0
- Aplicar los cambios necesarios para utilizar Auth0 como proveedor de OpenID
- Ejecutar el script de prueba para verificar que funciona
- Adquirir tokens de acceso utilizando los siguientes flujos de concesión:
 - Flujo de concesión de credenciales del cliente
 - Flujo de concesión de códigos de autorización
- Llamar a API protegidas utilizando los tokens de acceso adquiridos de los flujos de concesión
- Usar el punto final de información del usuario para obtener más información sobre un usuario

Repasemos cada uno de ellos en las siguientes secciones.

Configuración de una cuenta en Auth0

La mayor parte de la configuración requerida en Auth0 se gestionará mediante un script que utiliza la API de administración de Auth0. Sin embargo, debemos realizar algunos pasos manuales hasta que Auth0 haya creado un ID y un secreto de cliente que podamos usar para acceder a la API de administración. El servicio de Auth0 es multiusuario, lo que nos permite crear nuestro propio dominio de objetos OAuth, incluyendo clientes, propietarios de recursos y servidores de recursos.

Realice los siguientes pasos manuales para registrarse en una cuenta gratuita en Auth0 y crear un cliente que podamos usar para acceder a la API de administración:

1. Abra <https://auth0.com> URL en su navegador.

2. Haga clic en el botón Registrarse :

- a. Regístrese con un correo electrónico de su elección.
- b. Seleccione el rol de codificación seleccionando la casilla Sí, codificación.
- c. Seleccione la casilla de verificación Necesito configuración avanzada para seleccionar la región y el nombre de su inquilino.
- d. Haga clic en el botón Siguiente .
- e. Seleccione la Región e ingrese el nombre del nuevo Dominio de Inquilino ; en mi caso, dev-ml-1.eu.auth0.com. Haga clic en el botón "Crear cuenta" .
- f. Busque en su buzón un correo electrónico con el asunto Verificar su correo electrónico y utilice el Instrucciones en el correo electrónico para verificar su cuenta.

3. Después de registrarse, se le dirigirá a una página de incorporación .

4. En el menú de la izquierda, haga clic en Aplicaciones para expandirlo y, a continuación, en API para encontrar la API de administración, Auth0 Management API. Esta API se creó automáticamente al crear su inquilino. La usaremos para crear las definiciones necesarias en el inquilino.

5. Haga clic en API de administración de Auth0 y seleccione la pestaña Prueba .

6. Aparecerá un botón grande con el texto "Crear y autorizar aplicación de prueba" . Haz clic en él. para crear un cliente que pueda usarse para acceder a la API de administración.

7. Una vez creada, se muestra una página con el encabezado " Solicitando tokens a Auth0 de mi aplicación". Como paso final, debemos otorgarle al cliente creado permiso para usar el administrador. API de gestión.

8. Haga clic en la pestaña Aplicaciones de máquina a máquina , junto a la pestaña Prueba .

9. Aquí, encontraremos el cliente de prueba, Auth0 Management API (Aplicación de Prueba), y veremos que está autorizado para usar la API de administración. Si hacemos clic en la flecha hacia abajo junto al botón "Autorizado " , se mostrarán los privilegios disponibles.

10. Haga clic en la opción "Todos" y luego en el botón "Actualizar" . La pantalla debería ser similar a la siguiente captura de pantalla:

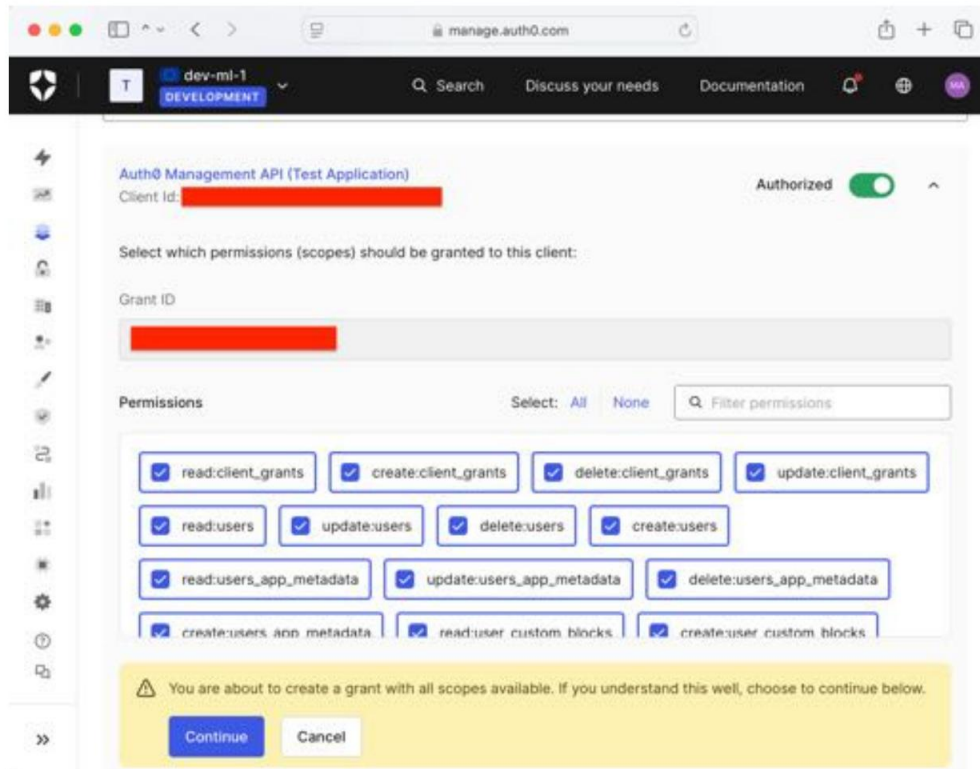


Figura 11.17: Permisos del cliente de la API de administración de Auth0

11. Haga clic en el botón Continuar después de comprender que ahora tiene una herramienta muy poderosa. Cliente con acceso a todas las API de gestión dentro de su inquilino.

12. Ahora solo necesitamos obtener el ID y el secreto del cliente creado. La forma más sencilla de hacerlo es seleccionar Aplicaciones en el menú de la izquierda (en la opción Aplicaciones del menú principal) y luego seleccionar la aplicación Auth0 Management API (Aplicación de prueba). Debería aparecer una pantalla similar a la siguiente:

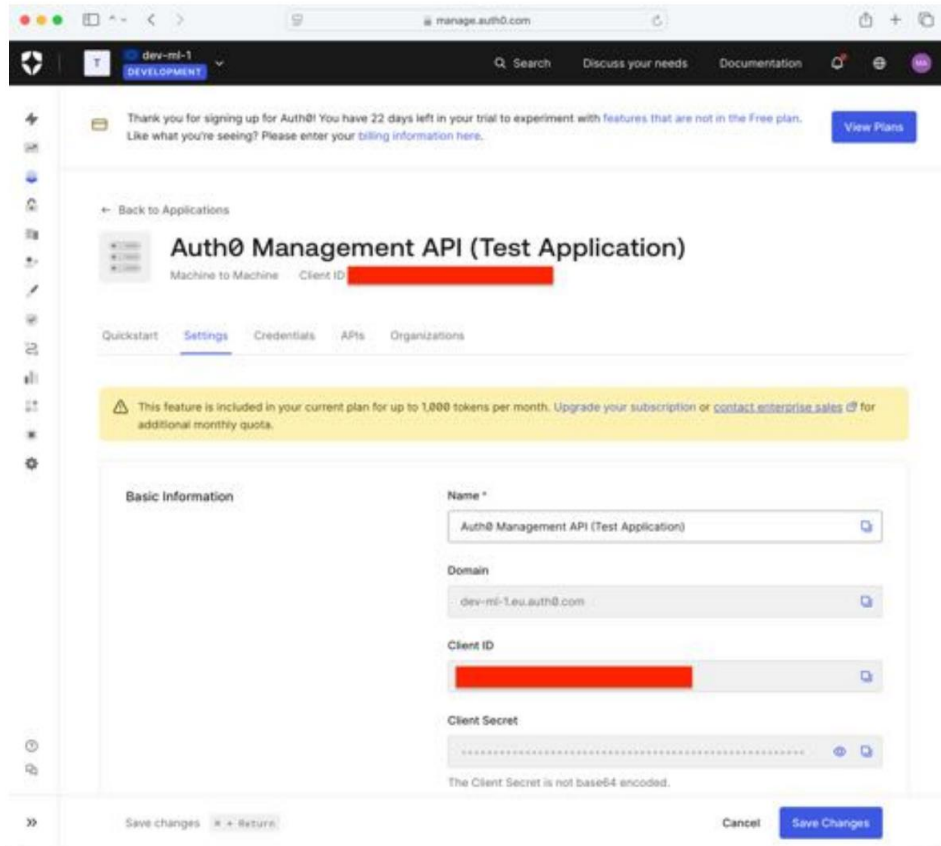


Figura 11.18: Información de la aplicación cliente de la API de administración de Auth0

13. Abra el archivo `$BOOK_HOME/Chapter11/auth0/env.bash` y copie los siguientes valores de la pantalla anterior:
- Dominio en el valor de la variable `TENANT`
 - ID de cliente en el valor de la variable `MGM_CLIENT_ID`
 - Secreto del cliente en el valor de la variable `MGM_CLIENT_SECRET`
14. Complete los valores requeridos en el archivo `env.bash` especificando una dirección de correo electrónico y una contraseña, en las variables `USER_EMAIL` y `USER_PASSWORD`, de un usuario de prueba que el script creará para nosotros.



Especificar una contraseña para un usuario de esta manera no se considera una buena práctica desde el punto de vista de la seguridad. Auth0 permite registrar usuarios que podrán configurar la contraseña ellos mismos, pero su configuración es más compleja. Para más información, consulte <https://auth0.com/docs/connections/database/> cambio de contraseña. Dado que esto solo se utiliza para fines de prueba, está bien especificar una contraseña como ésta.

15. Ahora podemos ejecutar el script que creará las siguientes definiciones para nosotros:

- Dos aplicaciones, lector y escritor, o clientes en la terminología OAuth
- La API compuesta de producto , un servidor de recursos en la terminología OAuth, con el Ámbitos de OAuth producto:lectura y producto:escritura
- Un usuario, un propietario de recursos en la terminología OAuth, que usaremos para probar la autorización.
flujo de concesión del código de autorización
- Finalmente, otorgaremos a la aplicación de lectura el alcance producto:lectura , y a la aplicación de escritura los alcances producto:lectura y producto:escritura.

16. Ejecute los siguientes comandos:

```
cd $BOOK_HOME/Capítulo11/auth0
./setup-tenant.bash
```

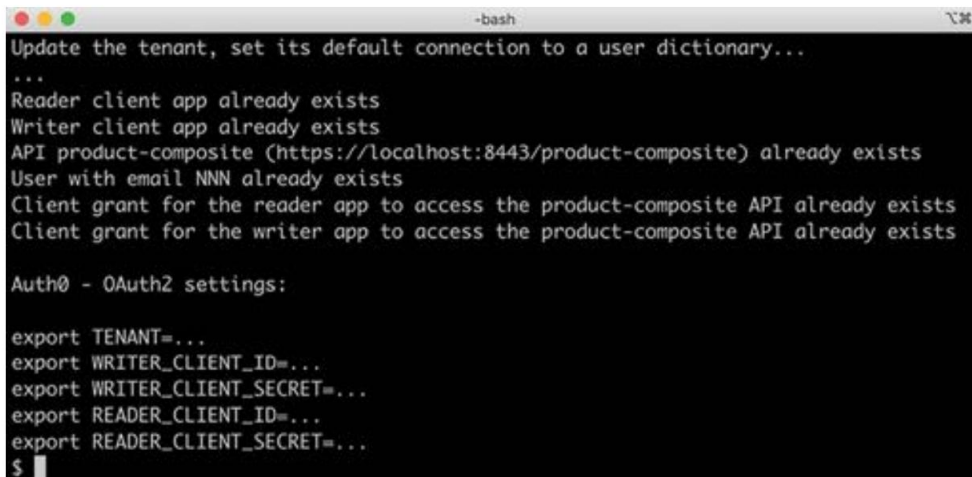
17. Espere el siguiente resultado (detalles eliminados del siguiente resultado):

```
bash
Update the tenant, set its default connection to a user dictionary...
...
Creates reader client app...
...
Creates writer client app...
...
Creates API product-composite (https://localhost:8443/product-composite)...
...
Creates user with email NNN...
...
Create client grant for the reader app to access the product-composite API...
...
Create client grant for the writer app to access the product-composite API...
...
Auth0 - OAuth2 settings:

export TENANT=...
export WRITER_CLIENT_ID=...
export WRITER_CLIENT_SECRET=...
export READER_CLIENT_ID=...
export READER_CLIENT_SECRET=...
$
```

Figura 11.19: Salida de setup-tenant.bash la primera vez que se ejecuta

18. Guarde una copia de los comandos de exportación impresos al final de la salida; los usaremos varias veces más adelante en este capítulo.
19. Busque en su buzón el correo electrónico especificado para el usuario de prueba. Recibirá un correo con el asunto " Verifique su correo electrónico". Siga las instrucciones del correo electrónico para verificar el correo electrónico del usuario de prueba.
dirección de correo electrónico.
20. Tenga en cuenta que el script es idempotente, lo que significa que puede ejecutarse varias veces sin dañar la configuración.
Si se ejecuta de nuevo, debería responder con lo siguiente:



```
-bash
Update the tenant, set its default connection to a user dictionary...
...
Reader client app already exists
Writer client app already exists
API product-composite (https://localhost:8443/product-composite) already exists
User with email NNN already exists
Client grant for the reader app to access the product-composite API already exists
Client grant for the writer app to access the product-composite API already exists

Auth0 - OAuth2 settings:

export TENANT=...
export WRITER_CLIENT_ID=...
export WRITER_CLIENT_SECRET=...
export READER_CLIENT_ID=...
export READER_CLIENT_SECRET=...
$
```

Figura 11.20: Salida de setup-tenant.bash la próxima vez que se ejecuta

21. Puede ser muy útil poder ejecutar el script nuevamente, por ejemplo, para obtener acceso al ID de cliente y secreto de cliente del lector y escritor.



Si necesita eliminar los objetos creados por setup-tenant.bash, puede ejecutar el script reset-tenant.bash .

Con una cuenta Auth0 creada y configurada, podemos avanzar y aplicar los cambios de configuración necesarios en el panorama del sistema.

Aplicar los cambios necesarios para utilizar Auth0 como proveedor de OpenID

En esta sección, aprenderemos qué cambios de configuración son necesarios para reemplazar el servidor de autorización local con Auth0. Solo necesitamos cambiar la configuración de los dos servicios que actúan como servidores de recursos OAuth: el servicio compuesto por producto y el servicio de puerta de enlace .

También necesitamos modificar ligeramente nuestro script de prueba para que adquiera los tokens de acceso de Auth0 en lugar de obtenerlos de nuestro servidor de autorización local. Empecemos con los servidores de recursos OAuth, los servicios de producto compuesto y de puerta de enlace .



Los cambios descritos en este tema no se han aplicado al código fuente en el repositorio Git del libro; ¡debe realizarlos usted mismo para verlos en acción!

Cambiar la configuración en los servidores de recursos OAuth

Como ya se describió, al utilizar un proveedor de OpenID Connect, solo tenemos que configurar la URI base en el punto final de descubrimiento estandarizado en los servidores de recursos OAuth.

En los proyectos de producto compuesto y puerta de enlace , actualice el punto de conexión de detección de OIDC para que apunte a Auth0 en lugar de a nuestro servidor de autorización local. Realice el siguiente cambio en la aplicación. archivo yml en ambos proyectos:

1. Localice la propiedad `spring.security.oauth2.resourceserver.jwt.issuer-uri`.
2. Reemplace su valor con `https://${TENANT}/`, donde `${TENANT}` debe reemplazarse con el nombre de dominio de su inquilino; en mi caso, es `dev-ml-1.eu.auth0.com`. No olvide el símbolo / final.

En mi caso, la configuración del punto final de descubrimiento de OIDC se verá así:

```
spring.security.oauth2.resourceserver.jwt.issuer-uri: https://dev-ml-1.eu.auth0.com/
```



Si tienes curiosidad, puedes ver qué hay en el documento de descubrimiento ejecutando el siguiente comando:

```
curl https://${TENANT}/.well-known/openid-configuration -s | jq
```

Reconstruya los servicios de producto compuesto y de puerta de enlace de la siguiente manera:

```
cd $BOOK_HOME/Capítulo 11
./gradlew build && docker compose up -d --build pasarela compuesta del producto
```

Con los servicios de producto compuesto y de puerta de enlace actualizados, podemos avanzar y también actualizar el script de prueba.

Cambiar el script de prueba para que adquiera tokens de acceso de Auth0

También necesitamos actualizar el script de prueba para que adquiera tokens de acceso del proveedor OIDC Auth0. Esto se hace realizando los siguientes cambios en test-em-all.bash:

1. Busque el siguiente comando:

```
TOKEN DE ACCESO=$(curl -k https://escritor:escritor-secreto@$HOST:$PORT/  
oauth2/token -d tipo_concesión=credenciales_del_cliente -d ámbito="producto:lectura producto:escritura"  
-s | jq .access_token -r)
```

2. Reemplázelo con estos comandos:

```
exportar INQUILINO=...  
exportar WRITER_CLIENT_ID=...  
exportar WRITER_CLIENT_SECRET=...  
  
ACCESO_TOKEN=$(curl -X POST https://$INQUILINO/oauth/token \  
-d tipo_de_concesión=credenciales_del_cliente \  
-d audiencia=https://localhost:8443/producto-compuesto \  
-d alcance=producto:lectura+producto:escritura \  
-d id_de_cliente=$ID_DE_CLIENTE_DE_ESCRITOR \  
-d secreto_del_cliente=$SECRETO_DEL_CLIENTE_DEL_ESCRITOR -s | jq -r .token_de_acceso)
```



Tenga en cuenta que, según el comando anterior, Auth0 requiere que especifiquemos la audiencia a la que se dirige el token de acceso solicitado, como una capa adicional de seguridad. La audiencia es la API a la que planeamos llamar usando el token de acceso. Dado que una implementación de API verifica el campo de audiencia, esto evitaría... la situación en la que alguien intenta utilizar un token de acceso emitido para otro propósito de obtener acceso a una API.

3. Establezca los valores para el entorno TENANT, WRITER_CLIENT_ID y WRITER_CLIENT_SECRET.

Las variables de ment en los comandos anteriores con los valores devueltos por el setup-tenant. script bash .



Como se mencionó anteriormente, ¡puedes ejecutar el script nuevamente para adquirir estos valores sin correr el riesgo de sufrir efectos secundarios negativos!

4. A continuación, busque el siguiente comando:

```
TOKEN DE ACCESO DEL LECTOR=$(curl -k https://lector:lector-secreto@$HOST:
$PORT/oauth2/token -d tipo_concesión=credenciales_del_cliente -d alcance="producto:lectura" -s |
jq .token_de_acceso -r)
```

5. Reemplácelo con este comando:

```
exportar READER_CLIENT_ID=...
exportar READER_CLIENT_SECRET=...
TOKEN DE ACCESO AL LECTOR=$(curl -X POST https://$INQUILINO/oauth/token \
  -d tipo_de_concesión=credenciales_del_cliente \
  -d audiencia=https://localhost:8443/producto-compuesto \
  -d alcance=producto:lectura \
  -d id_de_cliente=$ID_DE_CLIENTE_DE_LECTOR \
  -d secreto_del_cliente=$SECRETO_DEL_CLIENTE_DEL_LECTOR -s | jq -r .token_de_acceso)
```



Tenga en cuenta que aquí solo solicitamos el alcance del producto: lectura y no el alcance del producto: escritura .

6. Establezca los valores para las variables de entorno READER_CLIENT_ID y READER_CLIENT_SECRET en los comandos anteriores con los valores devueltos por el script setup-tenant.bash .

Ahora, los tokens de acceso son emitidos por Auth0 en lugar de nuestro servidor de autorización local, y nuestras implementaciones de API pueden verificarlos utilizando la información del servicio de descubrimiento de Auth0 configurado en los archivos application.yml . Como antes, las implementaciones de API pueden usar los ámbitos de los tokens de acceso para autorizar al cliente a realizar la llamada a la API, o no.

Con esto, ya tenemos todos los cambios necesarios. Ejecutemos algunas pruebas para verificar que podemos obtener tokens de acceso de Auth0.

Ejecución del script de prueba con Auth0 como proveedor de OpenID Connect

¡Ahora estamos listos para probar Auth0!

Ejecute las pruebas habituales, pero esta vez, utilizando Auth0 como proveedor de OpenID Connect, con el siguiente comando:

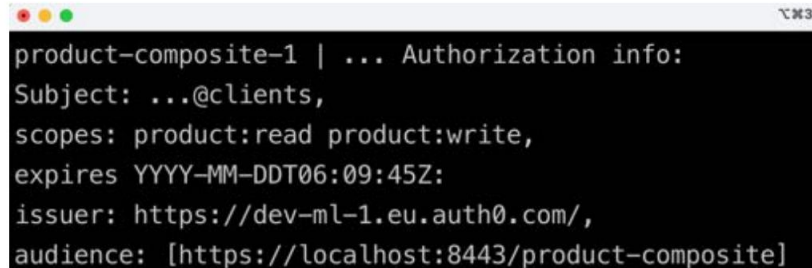
```
./pruébalos-a-todos.bash
```

En los registros, podrá encontrar información de autorización de los tokens de acceso emitidos por Auth0. Ejecute este comando:

```
Registros de Docker Compose, producto-compuesto | grep "Información de autorización"
```

Espere los siguientes resultados del comando:

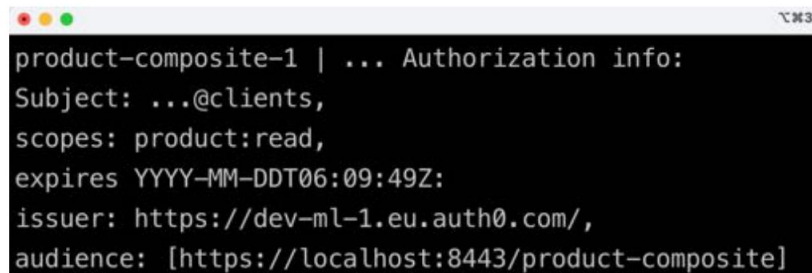
1. A partir de las llamadas que utilizan un token de acceso con los ámbitos `product:read` y `product:write`, veremos ambos ámbitos enumerados de la siguiente manera:



```
product-composite-1 | ... Authorization info:
Subject: ...@clients,
scopes: product:read product:write,
expires YYYY-MM-DDT06:09:45Z:
issuer: https://dev-ml-1.eu.auth0.com/,
audience: [https://localhost:8443/product-composite]
```

Figura 11.21: Información de autorización para el cliente escritor de Auth0 en la salida del registro

2. A partir de las llamadas que utilizan un token de acceso con solo el alcance `product:read`, veremos que solo Ese alcance se enumera a continuación:



```
product-composite-1 | ... Authorization info:
Subject: ...@clients,
scopes: product:read,
expires YYYY-MM-DDT06:09:49Z:
issuer: https://dev-ml-1.eu.auth0.com/,
audience: [https://localhost:8443/product-composite]
```

Figura 11.22: Información de autorización para el cliente lector de Auth0 en la salida del registro



Como podemos ver en la salida del registro, ahora también obtenemos información sobre el público objetivo de este token de acceso. Para reforzar la seguridad, podríamos añadir una prueba a nuestro servicio que verifique su URL, `https://localhost:8443/product-composite`. En este caso, forma parte de la lista de audiencia. Esto, como se mencionó anteriormente, evitaría que alguien intentase usar un token de acceso emitido para un fin distinto al de acceder a nuestra API.

Con las pruebas automatizadas trabajando en conjunto con Auth0, podemos avanzar y aprender a adquirir tokens de acceso mediante los diferentes tipos de flujo de concesión. Comencemos con el flujo de concesión de credenciales de cliente.

Adquisición de tokens de acceso mediante el flujo de concesión de credenciales del cliente

Si desea adquirir un token de acceso de Auth0 usted mismo, puede hacerlo ejecutando el siguiente comando, utilizando el flujo de concesión de credenciales del cliente:

```
exportar INQUILINO=...
exportar WRITER_CLIENT_ID=...
exportar WRITER_CLIENT_SECRET=...
curl -X POST https://$TENANT/oauth/token \
  -d tipo_de_concesión=credenciales_del_cliente \
  -d audiencia=https://localhost:8443/producto-compuesto \
  -d alcance=producto:lectura+producto:escritura \
  -d id_de_cliente=$ID_DE_CLIENTE_DE_ESCRITOR \
  -d secreto_de_cliente=$SECRETO_DE_CLIENTE_DE_ESCRITOR
```

Establezca los valores para las variables de entorno `TENANT`, `WRITER_CLIENT_ID` y `WRITER_CLIENT_SECRET` en los comandos anteriores con los valores devueltos por el script `setup-tenant.bash`.

Siguiendo las instrucciones de la sección `Llamar a API protegidas mediante tokens de acceso`, debería poder llamar a las API utilizando el token de acceso adquirido.

Adquisición de tokens de acceso mediante el flujo de concesión de código de autorización

En esta sección, aprenderemos a obtener un token de acceso de Auth0 mediante el flujo de concesión de código de autorización.

Como ya se describió, primero necesitamos obtener un código de autorización mediante un navegador web. A continuación, podemos usar código del lado del servidor para intercambiar el código de autorización por un token de acceso.

Realice los siguientes pasos para ejecutar el flujo de concesión de código de autorización con Auth0:

1. Para obtener un código de autorización para el cliente de la aplicación predeterminada, utilice la siguiente URL en la web navegador:

```
https://${TENANT}/authorize?audience=https://localhost:8443/product-composite&scope=openid
correo electrónico producto:leer producto:escribir&respuesta_
tipo=código&id_de_cliente=${ID_DE_CLIENTE_DE_ESCRITOR}&uri_de_redirección=https://mi.redirección.
uri&estado=845361.
```

2. Reemplace `${TENANT}` y `${WRITER_CLIENT_ID}` en la URL anterior con el nombre de dominio del inquilino y el ID del cliente del escritor devueltos por el script `setup-tenant.bash`.
3. Auth0 debería presentar la siguiente pantalla de inicio de sesión:

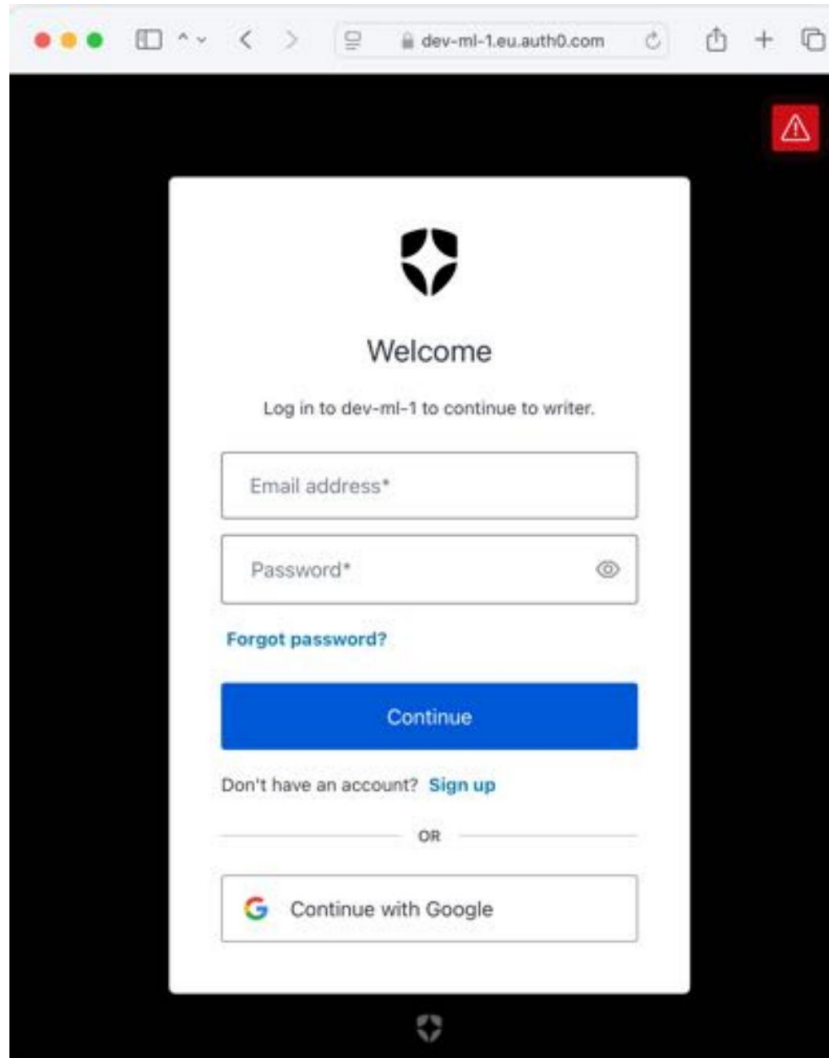


Figura 11.23: Flujo de concesión de código de autorización con Auth0, la pantalla de inicio de sesión

4. Después de iniciar sesión correctamente, Auth0 le solicitará que dé su consentimiento a la aplicación cliente:

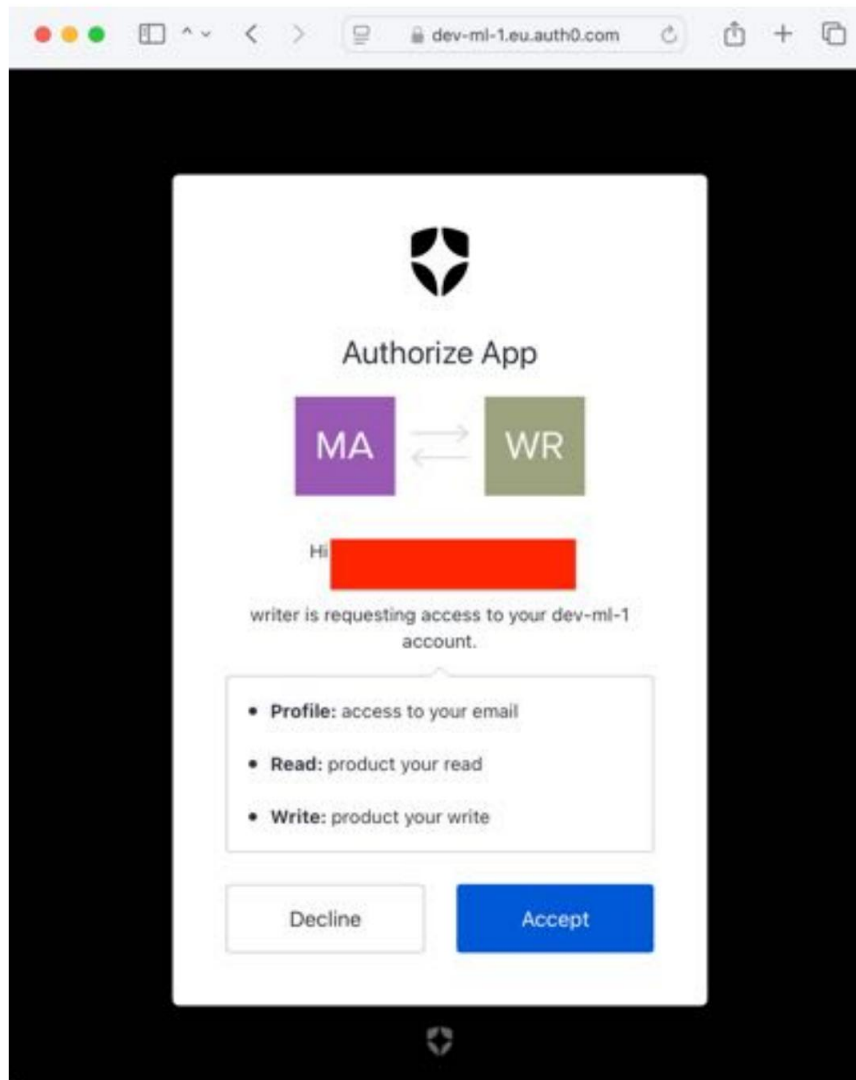


Figura 11.24: Flujo de concesión de código de autorización con Auth0, la pantalla de consentimiento

5. El código de autorización ahora está en la URL del navegador, tal como cuando probamos el

Flujo de concesión de código de autorización con nuestro servidor de autorización local:

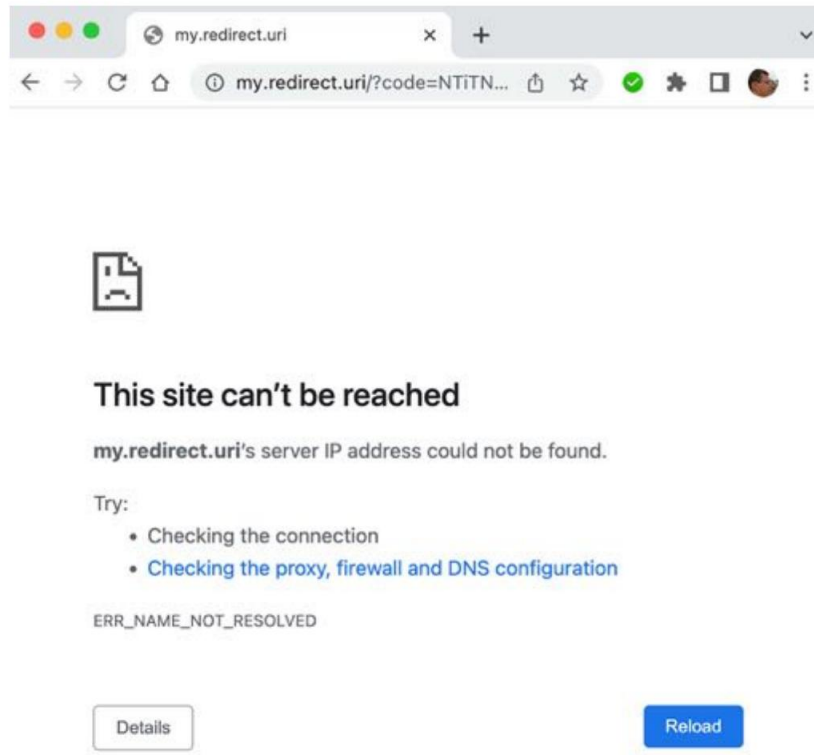


Figura 11.25: Flujo de concesión de código de autorización con Auth0, token de acceso

6. Extraiga el código y ejecute el siguiente comando para obtener el token de acceso:

```
CÓDIGO=...

exportar INQUILINO=...

exportar ID_CLIENTE_ESCRITOR=...

exportar WRITER_CLIENT_SECRET=... curl -X

POST https://$TENANT/oauth/token \

  -d tipo_de_concesión=código_de_autorización \ -d

  id_de_cliente=$ID_DE_CLIENTE_DE_ESCRITOR \ -d

  secreto_de_cliente=$SECRETO_DE_CLIENTE_DE_ESCRITOR \

  -d código=$CÓDIGO \

  -d redirect_uri=https://mi.redirect.uri -s | jq .
```

7. Establezca los valores de las variables de entorno `TENANT`, `WRITER_CLIENT_ID` y `WRITER_CLIENT_SECRET` en los comandos anteriores en los valores devueltos por `setup-tenant`.
`script bash`.

Ahora que hemos aprendido cómo adquirir tokens de acceso utilizando ambos flujos de concesión, estamos listos para intentar llamar a la API externa utilizando un token de acceso adquirido de Auth0 en la siguiente sección.

Llamar a API protegidas mediante los tokens de acceso Auth0

Podemos usar tokens de acceso emitidos por Auth0 para llamar a nuestras API, tal como cuando usamos tokens de acceso emitidos por nuestro servidor de autorización local.

Para una API de solo lectura, ejecute el siguiente comando:

```
TOKEN DE ACCESO=...  
curl https://localhost:8443/product-composite/1 -k -H "Autorización:  
Portador $ACCESS_TOKEN" -i
```

Para una API de actualización, ejecute el siguiente comando:

```
TOKEN DE ACCESO=...  
curl https://localhost:8443/product-composite/999 -k -H "Autorización:  
Portador $ACCESS_TOKEN" -X DELETE -i
```

Dado que hemos solicitado ambos ámbitos, `producto:lectura` y `producto:escritura`, se espera que ambas llamadas API anteriores devuelvan 200 OK.

Obtener información adicional sobre el usuario

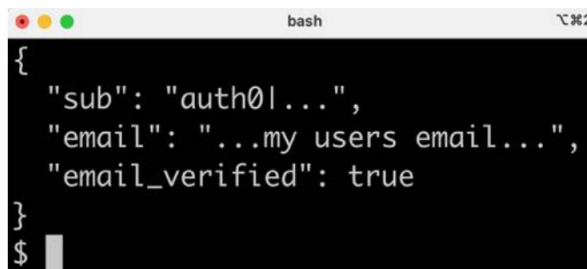
En la salida del registro de las Figuras 11.21 y 11.22 de la sección "Ejecución del script de prueba con Auth0 como proveedor de OpenID Connect", no se pudo obtener información sobre el usuario que inició la solicitud a la API. Si desea que la implementación de su API obtenga más información sobre el usuario, puede llamar al punto final `userinfo` de Auth0. La URL del punto final `userinfo` se encuentra en la respuesta a una solicitud al punto final de descubrimiento de OIDC, como se describe en la sección "Cambiar la configuración en los servidores de recursos de OAuth". Para obtener información de usuario relacionada con un token de acceso, realice la siguiente solicitud:

```
Exportar INQUILINO=...  
curl -H "Autorización: Portador $ACCESS_TOKEN" https://$TENANT/userinfo -s | jq
```

Establezca los valores de la variable de entorno `TENANT` en los comandos anteriores en los valores devueltos por el script `setup-tenant.bash`.

Tenga en cuenta que este comando solo se aplica a los tokens de acceso emitidos mediante el flujo de concesión de código de autorización. Los tokens de acceso emitidos mediante el flujo de concesión de credenciales de cliente no contienen información del usuario y generarán una respuesta de error si se intentan.

Un ejemplo de respuesta es el siguiente:

A terminal window titled 'bash' with a temperature icon and '2' in the top right corner. The terminal displays a JSON object: { "sub": "auth0|...", "email": "...my users email...", "email_verified": true }. The cursor is at the end of the object, and a dollar sign (\$) is visible on the next line.

```
{
  "sub": "auth0|...",
  "email": "...my users email...",
  "email_verified": true
}
$
```

Figura 11.26: Solicitud de información de usuario adicional de Auth0

Este punto final también se puede utilizar para verificar que el usuario no haya revocado el token de acceso en Auth0.

Finalice las pruebas apagando el sistema con el siguiente comando:

```
Docker Compose Down
```

Con esto concluye la sección, donde aprendimos a reemplazar el servidor de autorización OAuth 2.0 local por una alternativa externa. También vimos cómo reconfigurar el entorno de microservicios para validar tokens de acceso mediante un proveedor OIDC externo.

Resumen

En este capítulo, aprendimos cómo usar Spring Security para proteger nuestras API.

Vimos lo fácil que es habilitar HTTPS para evitar el espionaje de terceros con Spring Security. Con Spring Security, también aprendimos que es sencillo restringir el acceso al servidor de descubrimiento, Netflix Eureka, mediante la autenticación HTTP básica. Finalmente, vimos cómo podemos usar Spring Security para simplificar el uso de OAuth 2.0 y OpenID Connect y permitir que aplicaciones cliente de terceros accedan a nuestras API en nombre de un usuario, sin que este tenga que compartir credenciales con las aplicaciones cliente. Aprendimos a configurar un servidor de autorización OAuth 2.0 local basado en Spring Security y a modificar la configuración para poder usar un proveedor externo de OpenID Connect, Auth0.

Sin embargo, una preocupación es cómo gestionar la configuración requerida. Cada instancia de microservicio debe contar con su propia configuración, lo que dificulta obtener una visión general de la configuración actual. Actualizar la configuración que afecta a varios microservicios también será un desafío. A la configuración dispersa se suma el hecho de que algunas de las configuraciones que hemos visto hasta ahora contienen información confidencial, como credenciales o certificados. Parece que necesitamos una mejor manera de gestionar la configuración de varios microservicios que cooperan, así como una solución para gestionar las partes confidenciales de la configuración.

En el próximo capítulo, exploraremos Spring Cloud Config Server y veremos cómo se puede utilizar para manejar este tipo de problemas.

Preguntas

1. ¿Cuáles son los beneficios y desventajas de utilizar certificados autofirmados?
2. ¿Cuál es el propósito de los códigos de autorización OAuth 2.0?
3. ¿Cuál es el propósito de los ámbitos de OAuth 2.0?
4. ¿Qué significa cuando un token es un JWT?
5. ¿Cómo podemos confiar en la información almacenada en un JWT?
6. ¿Es adecuado utilizar el flujo de concesión de código de autorización OAuth 2.0 con una aplicación móvil nativa?
7. ¿Qué añade OpenID Connect a OAuth 2.0?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



12

Configuración centralizada

En este capítulo, aprenderemos a usar Spring Cloud Config Server para centralizar la gestión de la configuración de nuestros microservicios. Como ya se describió en el Capítulo 1, Introducción a los Microservicios, un mayor número de microservicios suele conllevar un mayor número de archivos de configuración que deben administrarse y actualizarse.

Con Spring Cloud Config Server, podemos almacenar los archivos de configuración de todos nuestros microservicios en un repositorio de configuración central, lo que simplifica enormemente su gestión. Nuestros microservicios se actualizarán para recuperar su configuración del servidor de configuración al inicio.

En este capítulo se tratarán los siguientes temas:

- Introducción a Spring Cloud Config Server
- Configuración de un servidor de configuración
- Configuración de clientes de un servidor de configuración
- Estructuración del repositorio de configuración
- Probando Spring Cloud Config Server

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter12`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver lo que se necesitó para agregar un servidor de configuración al entorno de microservicios, puede compararlo con el código fuente del Capítulo 11, "Securing Access to APIs". Puede usar su herramienta de comparación preferida y comparar las carpetas \$BOOK_HOME/Chapter11 y \$BOOK_HOME/Chapter12.

Introducción a Spring Cloud Config Server

Spring Cloud Config Server (abreviado como servidor de configuración) se agregará al panorama de microservicios existente detrás del servidor de borde, de la misma manera que para los otros microservicios:

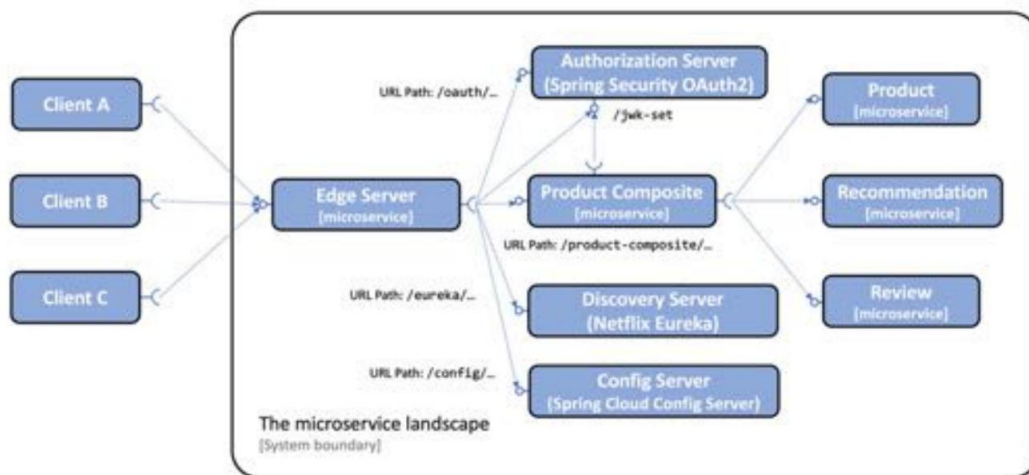


Figura 12.1: Agregar un servidor de configuración al panorama del sistema

 Consejo rápido: ¿Necesitas ver una versión en alta resolución de esta imagen? Abre este libro. en el Packt Reader de próxima generación o vértelo en copia PDF/ePub.

 El lector Packt de última generación y una copia gratuita en PDF/ePub de este libro se incluyen con la compra. Escanea el código QR o visita packtpub.com/unlock. Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



A la hora de configurar un servidor de configuración, hay varias opciones a considerar:

- Seleccionar un tipo de almacenamiento para el repositorio de configuración
- Decidir sobre la conexión inicial del cliente, ya sea al servidor de configuración o al de descubrimiento.
servidor
- Proteger la configuración, tanto contra el acceso no autorizado a la API como evitando almacenar información confidencial en texto sin formato en el repositorio de configuración.

Repasemos cada opción una por una y también presentemos la API expuesta por el servidor de configuración.

Seleccionar el tipo de almacenamiento del repositorio de configuración

Como ya se describió en el Capítulo 8, Introducción a Spring Cloud, el servidor de configuración permite almacenar archivos de configuración en diferentes backends. A continuación, se muestran algunos ejemplos:

- Repositorio Git
- Sistema de archivos local
- Bóveda de HashiCorp
- Base de datos JDBC

En este capítulo, usaremos un sistema de archivos local. Para usarlo, el servidor de configuración debe iniciarse con el perfil nativo de Spring habilitado. La ubicación de los repositorios de configuración se especifica mediante la propiedad `spring.cloud.config.server.native.searchLocations`.



Usar el sistema de archivos local es fácil de configurar y, por lo tanto, preferible en un entorno de desarrollo. En un entorno de producto, se recomienda usar una alternativa más segura, por ejemplo, HashiCorp Vault. Para obtener una introducción sobre cómo empezar a usar HashiCorp Vault, consulte <https://docs.spring.io/spring-cloud-config/referencia/servidor/repositorio-de-entorno/vault-backend.html>.

Decidir sobre la conexión inicial del cliente

De forma predeterminada, un cliente se conecta primero al servidor de configuración para obtener su configuración. Según la configuración, se conecta al servidor de descubrimiento (Netflix Eureka en nuestro caso) para registrarse. También es posible hacerlo a la inversa; es decir, el cliente primero se conecta al servidor de descubrimiento para encontrar una instancia del servidor de configuración y luego se conecta al servidor de configuración para obtener su configuración. Ambos enfoques tienen ventajas y desventajas.

En este capítulo, los clientes se conectarán primero al servidor de configuración. Con este enfoque, será posible almacenar la configuración del servidor de descubrimiento en dicho servidor.

Para obtener más información sobre la otra alternativa, consulte <https://docs.spring.io/spring-cloud-config/docs/actual/referencia/html/#discovery-first-bootstrap>.



Una preocupación al conectarse primero al servidor de configuración es que este puede convertirse en un punto único de fallo. Si los clientes se conectan primero a un servidor de descubrimiento, como Netflix Eureka, se pueden registrar varias instancias del servidor de configuración, lo que evita un punto único de fallo. Más adelante, en este libro, a partir del capítulo 15, « Introducción a Kubernetes», veremos cómo evitar un punto único de fallo ejecutando varios contenedores (por ejemplo, servidores de configuración) detrás de cada servicio de Kubernetes.

Asegurar la configuración

En general, la información de configuración se tratará como información confidencial. Esto significa que debemos protegerla tanto en tránsito como en reposo. En tiempo de ejecución, no es necesario exponer el servidor de configuración al exterior a través del servidor de borde. Sin embargo, durante el desarrollo, resulta útil poder acceder a la API del servidor de configuración para comprobar la configuración. En entornos de producción, se recomienda bloquear el acceso externo al servidor de configuración.

Asegurar la configuración en tránsito

Cuando un microservicio o cualquier persona que use la API del servidor de configuración solicita la información de configuración, estará protegida contra escuchas por parte del servidor perimetral, ya que ya está utiliza HTTPS.

Para garantizar que el usuario de la API sea un cliente conocido, utilizaremos la autenticación HTTP básica. Podemos configurar la autenticación HTTP básica usando Spring Security en el servidor de configuración y especificando las variables de entorno `SPRING_SECURITY_USER_NAME` y `SPRING_SECURITY_USER_PASSWORD` con las credenciales permitidas.

Asegurar la configuración en reposo

Para evitar que alguien con acceso al repositorio de configuración robe información confidencial, como contraseñas, el servidor de configuración admite el cifrado de la información de configuración almacenada en disco. El servidor de configuración admite el uso de claves simétricas y asimétricas. Las claves asimétricas son más seguras, pero más difíciles de administrar.

En este capítulo, usaremos una clave simétrica. Esta clave se proporciona al servidor de configuración al iniciarse mediante la variable de entorno `ENCRYPT_KEY`. La clave cifrada es simplemente una cadena de texto sin formato que debe protegerse igual que cualquier información confidencial.

Para obtener más información sobre el uso de claves asimétricas, consulte https://docs.spring.io/spring-cloud-config/docs/current/reference/html/#_key_management.

Presentación de la API del servidor de configuración

El servidor de configuración expone una API REST que sus clientes pueden utilizar para recuperar su configuración.

En este capítulo, utilizaremos los siguientes puntos finales en la API:

- `/actuator`: El punto final del actuator estándar expuesto por todos los microservicios. Como siempre, deben usarse con precaución. Son muy útiles durante el desarrollo, pero deben bloquearse antes de usarse en producción.
- `/encrypt` y `/decrypt`: puntos finales para cifrar y descifrar información confidencial. Estos también deben bloquearse antes de usarse en producción.
- `/[microservice]/[profile]`: Devuelve la configuración para el microservicio especificado y el perfil de resorte especificado.

Veremos algunos ejemplos de uso de la API cuando probemos el servidor de configuración.

Configuración de un servidor de configuración

Configurar un servidor de configuración basándose en las decisiones analizadas es sencillo:

1. Cree un proyecto Spring Boot con Spring Initializr, como se describe en el Capítulo 3, "Creación de un conjunto de microservicios cooperativos". Consulte "Uso de Spring Initializr para generar código esqueleto" . sección.
2. Agregue las dependencias, `spring-cloud-config-server` y `spring-boot-starter-security`, al archivo de compilación de Gradle, `build.gradle`.
3. Agregue la anotación `@EnableConfigServer` a la clase de aplicación,

Aplicación del servidor de configuración:

```
@EnableConfigServer
@SpringBootApplication
class pública ConfigServerApplication {
```

4. Agregue la configuración del servidor de configuración al archivo de propiedades predeterminado, application.yml:

```
puerto del servidor: 8888

spring.cloud.config.server.native.searchLocations: archivo:${PWD}/
repositorio de configuración

management.endpoint.health.show-details: "SIEMPRE"
administración.puntos finales.web.exposición.incluir: ""

explotación florestal:
  nivel:
    raíz: información

---

spring.config.activate.on-profile: docker
spring.cloud.config.server.native.searchLocations: archivo:/config-repo
```

La configuración más importante es especificar dónde encontrar el repositorio de configuración, indicado mediante la propiedad `spring.cloud.config.server.native.searchLocations`.

5. Agregue una regla de enrutamiento al servidor perimetral para que la API del servidor de configuración sea accesible desde fuera del panorama de microservicios.
6. Agregue un Dockerfile y una definición del servidor de configuración a los tres archivos Docker Compose.
7. Externalice los parámetros de configuración sensibles al archivo de entorno estándar de Docker Compose, `.env`. Los parámetros se describen en la sección Configuración del servidor de configuración para Úselo con la sección Docker.
8. Agregue el servidor de configuración al archivo de compilación común, `settings.gradle`:

```
incluir 'spring-cloud:config-server'
```

El código fuente de Spring Cloud Config Server se puede encontrar en `$BOOK_HOME/Chapter12/spring-nube/config-servidor`.

Ahora, veamos cómo configurar la regla de enrutamiento mencionada en el paso 5 y cómo configurar el servidor de configuración agregado en Docker Compose, como se describe en los pasos 6 y 7.

Configuración de una regla de enrutamiento en el servidor perimetral

Para acceder a la API del servidor de configuración desde fuera del entorno de microservicios, añadimos una regla de enrutamiento al servidor perimetral. Todas las solicitudes al servidor perimetral que comiencen con `/config` se enrutarán al servidor de configuración con la siguiente regla de enrutamiento:

```
- id: servidor de configuración
  uri: ${spring.cloud.config.uri}
  predicados:
    - Ruta=/config/**
  filtros:
    - RewritePath=/config/(?<segmento>.*), /${segmento}
```

El filtro `RewritePath` en la regla de enrutamiento eliminará la parte principal, `/config`, de la URL entrante antes de enviarla al servidor de configuración.

El servidor perimetral también está configurado para permitir todas las solicitudes al servidor de configuración, delegando las comprobaciones de seguridad en este. Se añade la siguiente línea a la clase `SecurityConfig` del servidor perimetral:

```
.pathMatchers("/config/**").permitAll()
```

Con esta regla de enrutamiento en su lugar, podemos usar la API del servidor de configuración; por ejemplo, ejecutar el siguiente comando para solicitar la configuración del servicio del producto cuando usa el contenedor

Perfil de resorte:

```
curl https://dev-usr:dev-pwd@localhost:8443/config/product/docker -ks | jq
```

Ejecutaremos este comando cuando probemos el servidor de configuración más adelante.

Configuración del servidor de configuración para su uso con Docker

El `Dockerfile` del servidor de configuración se ve igual que el de los otros microservicios, excepto por el hecho de que expone el puerto 8888 en lugar del puerto 8080.

Cuando se trata de agregar el servidor de configuración a los archivos de Docker Compose, se ve un poco diferente De lo que hemos visto para los otros microservicios:

```
servidor de configuración:
  compilación: spring-cloud/config-server
  límite de memoria: 512 m
  ambiente:
    - SPRING_PROFILES_ACTIVE=docker,nativo
```

```
- CLAVE DE CIFRADO=${CLAVE DE CIFRADO DEL SERVIDOR DE CONFIGURACIÓN}
- NOMBRE DE USUARIO DE SPRING_SECURITY=${CONFIG_SERVER_USR}
- CONTRASEÑA DE USUARIO DE SPRING_SECURITY=${CONFIG_SERVER_PWD}
volúmenes:
- $PWD/config-repo:/config-repo
```

Aquí están las explicaciones del código fuente anterior:

1. El perfil Spring, nativo, se agrega para indicar al servidor de configuración que el repositorio de configuración se basa en archivos locales.
2. La variable de entorno ENCRYPT_KEY se utiliza para especificar la clave de cifrado simétrico que utilizará el servidor de configuración para cifrar y descifrar información de configuración confidencial.
3. Las variables de entorno SPRING_SECURITY_USER_NAME y SPRING_SECURITY_USER_PASSWORD se utilizan para especificar las credenciales que se utilizarán para proteger las API mediante autenticación básica. Autenticación HTTP.
4. La declaración de volúmenes hará que la carpeta config-repo sea accesible en el contenedor Docker.

Docker Compose obtiene los valores de las tres variables de entorno anteriores, marcadas en el archivo Docker Compose con \${...}, del archivo .env :

```
CONFIG_SERVER_ENCRYPT_KEY=mi-clave-de-cifrado-muy-segura
CONFIG_SERVER_USR=dev-usr
CONFIG_SERVER_PWD=contraseña de desarrollo
```



La información almacenada en el archivo .env , es decir, el nombre de usuario, la contraseña y la clave de cifrado, es confidencial y debe protegerse si se utiliza para fines distintos al desarrollo y las pruebas. Además, tenga en cuenta que perder la clave de cifrado provocará que la información cifrada en el repositorio de configuración no se pueda descifrar.

Configuración de clientes de un servidor de configuración

Para obtener sus configuraciones del servidor de configuración, nuestros microservicios deben actualizarse . Los microservicios usarán el mecanismo de reintento de Spring para reconectarse a un servidor de configuración. Esto es especialmente útil cuando todo el conjunto de microservicios y su servidor de configuración se inician simultáneamente, por ejemplo, al usar el comando docker compose up .

En este escenario, muchos de los clientes intentarán conectarse al servidor de configuración antes de que esté listo, y la lógica de reintento hará que los clientes se conecten al servidor de configuración correctamente una vez que esté en funcionamiento.



Inicialmente, Spring Cloud Config ofrecía un mecanismo de arranque para que los clientes se conectaran al servidor. A partir de Spring Boot 2.4, los datos de configuración se pueden importar mediante la propiedad `spring.config.import`. Esta se ha convertido en la forma predeterminada de conectarse al servidor de Spring Cloud Config.

Sin embargo, en escenarios donde necesitamos definir múltiples URL de servidor de configuración y habilitar reintentos para cada uno, el uso de la propiedad `spring.config.import` no funciona. En su lugar, debemos usar el mecanismo de arranque anterior. Esto se habilita añadiendo la dependencia `spring-cloud-starter-bootstrap`.

El método de arranque para conectarse a Spring Cloud Config Server sigue siendo totalmente compatible, sin planes de desuso. Para más información, consulte <https://github.com/spring-cloud/spring-cloud-config/issues/2244#issuecomment-1497695548>.

Esto se puede hacer con los siguientes pasos:

1. Agregue `spring-cloud-starter-config`, `spring-cloud-starter-bootstrap` y dependencias de `spring-retry` al archivo de compilación de Gradle, `build.gradle`.
2. Mueva el archivo de configuración, `application.yml`, al repositorio de configuración y cámbiele el nombre con el nombre del cliente según lo especificado por la propiedad `spring.application.name`.
3. Agregue un nuevo archivo `bootstrap.yml` a la carpeta `src/main/resources`. Este archivo se usará para almacenar la configuración necesaria para conectarse al servidor de configuración. Consulte la sección "Configuración de la información de conexión" para obtener una explicación de su contenido.
4. Agregue credenciales para acceder al servidor de configuración a los archivos de Docker Compose, por ejemplo, El servicio del producto :

```
producto:
  ambiente:
    - CONFIG_SERVIDOR_USR=${CONFIG_SERVIDOR_USR}
    - CONTRASEÑA_DEL_SERVIDOR_DE_CONFIGURACIÓN=${CONTRASEÑA_DEL_SERVIDOR_DE_CONFIGURACIÓN}
```

5. Deshabilite el uso del servidor de configuración al ejecutar pruebas automatizadas basadas en Spring Boot.

Esto se logra añadiendo `spring.cloud.config.enabled=false` a las anotaciones `@DataMongoTest`, `@DataJpaTest` y `@SpringBootTest`. Su aspecto es el siguiente:

```
@DataMongoTest(propiedades = {"spring.cloud.config.enabled=false"})

@DataJpaTest(propiedades = {"spring.cloud.config.enabled=false"})

@SpringBootTest(webEnvironment=PUERTO_ALEATORIO,
    propiedades = {"eureka.client.enabled=false",
        "spring.cloud.config.enabled=false"})
```

Configuración de la información de conexión. Como se mencionó

anteriormente, el archivo `src/main/resources/bootstrap.yml` ahora contiene la configuración del cliente necesaria para conectarse al servidor de configuración. Este archivo tiene el mismo contenido para todos los clientes del servidor de configuración, excepto el nombre de la aplicación, especificado por la propiedad `spring.application.name` (en el siguiente ejemplo, establecida en `product`).

```
spring.application.name: producto

spring.cloud.config:
  failFast: verdadero
  rever:
    Intervalo inicial: 3000
    multiplicador: 1.3
    intervalo máximo: 10000
    Intentos máximos: 20
  uri: http://localhost:8888 nombre de
  usuario: ${CONFIG_SERVER_USR}
  contraseña: ${CONFIG_SERVER_PWD}

---

spring.config.activate.on-profile: docker spring.cloud.config.uri:
http://config-server:8888
```

Esta configuración hará que el cliente haga lo siguiente:

1. Conéctese al servidor de configuración usando la URL `http://localhost:8888` cuando se ejecuta fuera de Docker, y usando la URL `http://config-server:8888` cuando se ejecuta en un Docker recipiente.

2. Utilice la autenticación básica HTTP, basada en el valor de CONFIG_SERVER_USR y CONFIG_

Propiedades de SERVER_PWD , como nombre de usuario y contraseña del cliente.

3. Intente volver a conectarse al servidor de configuración durante el inicio hasta 20 veces, si es necesario.

4. Si el intento de conexión falla, el cliente esperará inicialmente 3 segundos antes de intentarlo.

Para reconectarse.

5. El tiempo de espera para reintentos posteriores aumentará en un factor de 1,3.

6. El tiempo máximo de espera entre intentos de conexión será de 10 segundos.

7. Si el cliente no puede conectarse al servidor de configuración después de 20 intentos, su inicio fallará.

Esta configuración generalmente es buena para la resiliencia ante problemas de conectividad temporales con el servidor de configuración; por ejemplo, cuando todo el panorama de microservicios y su servidor de configuración se inician a la vez, como se describe.

Estructuración del repositorio de configuración

Tras transferir los archivos de configuración del código fuente de cada cliente al repositorio de configuración, tendremos una configuración común en muchos de ellos; por ejemplo, para la configuración de los endpoints de los actuadores y la conexión a Eureka, RabbitMQ y Kafka. Las partes comunes se han incluido en un archivo de configuración común llamado `application.yml`. Este archivo es compartido por todos los clientes. El repositorio de configuración contiene los siguientes archivos:

repositorio-de-configuración/

├─ aplicación.yml

├─ auth-server.yml

├─ eureka-server.yml

├─ puerta de enlace.yml

├─ producto-compuesto.yml

├─ producto.yml

├─ recomendación.yml

└─ revisión.yml

El repositorio de configuración se puede encontrar en `$BOOK_HOME/Chapter12/config-repo`.

Probando Spring Cloud Config Server

Ahora es el momento de probar el servidor de configuración:

1. Primero, compilaremos desde la fuente y ejecutaremos el script de prueba para asegurarnos de que todo encaja.

juntos.

2. A continuación, probaremos la API del servidor de configuración para recuperar la configuración de nuestros microservicios.
3. Finalmente, veremos cómo podemos cifrar y descifrar información sensible, por ejemplo, contraseñas.

Creación y ejecución de pruebas automatizadas

Ahora crearemos y ejecutaremos pruebas de verificación del panorama del sistema, de la siguiente manera:

1. Primero, construya las imágenes de Docker con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 12
./gradlew compilación y compilación de docker compose
```

2. A continuación, inicie el panorama del sistema en Docker y ejecute las pruebas habituales con lo siguiente dominio:

```
./test-em-all.bash inicio
```

Obtener la configuración mediante la API del servidor de configuración

Como se describió anteriormente, podemos acceder a la API del servidor de configuración a través del servidor perimetral usando el prefijo URL /config. También debemos proporcionar las credenciales especificadas en el archivo .env para la autenticación HTTP básica. Por ejemplo, para recuperar la configuración utilizada para el producto. servicio cuando se ejecuta como un contenedor Docker, es decir, habiendo activado el perfil Spring de Docker , ejecute el siguiente comando:

```
curl https://dev-usr:dev-pwd@localhost:8443/config/product/docker -ks | jq
```

Espera una respuesta con la siguiente estructura (muchas de las propiedades en la respuesta se reemplazan por ... para aumentar la legibilidad):

```
{
  "nombre": "producto",
  "perfiles": [
    "estibador"
  ],
  ...
  "PropertySources": [
    {
      "nombre": "...archivo [/config-repo/product.yml]...",
      "fuente": {
        "spring.config.activate.on-profile": "docker",
```

```

        "servidor.puerto": 8080,
        ...
    },
    {
        "nombre": "...archivo [/config-repo/product.yml]...", "fuente": {

            "servidor.puerto": 7002,
            ...
        }
    },
    {
        "nombre": "...archivo [/config-repo/application.yml]...", "fuente": {

            "spring.config.activate.on-profile": "docker",
            ...
        }
    },
    {
        "nombre": "...archivo [/config-repo/application.yml]...", "fuente": {

            ...
            "app.eureka-password": "p",
            "spring.rabbitmq.password": "invitado"
        }
    }
]
}

```

Las explicaciones de esta respuesta son las siguientes:

La respuesta contiene propiedades de varias fuentes de propiedades, una por archivo de propiedades y perfil de Spring que coincidió con la solicitud de API. Las fuentes de propiedades se devuelven en orden de prioridad; si una propiedad se especifica en varias fuentes, la primera propiedad en la respuesta tiene prioridad. La respuesta de ejemplo anterior contiene las siguientes fuentes de propiedades, en el siguiente orden de prioridad:

- /config-repo/product.yml, para el perfil de Spring de Docker
- /config-repo/product.yml, para el perfil de Spring predeterminado

- /config-repo/application.yml, para el perfil de Spring de Docker
- /config-repo/application.yml, para el perfil Spring predeterminado

Por ejemplo, el puerto utilizado será 8080 y no 7002, ya que "server.port": 8080 se especifica antes de "server.port": 7002 en la respuesta anterior.

La información confidencial, como las contraseñas de Eureka y RabbitMQ, se devuelve en texto plano (por ejemplo, "p" y "guest"), pero se cifra en el disco. En el archivo de configuración, application.yml, se especifican de la siguiente manera:

```
aplicación:
  contraseña de eureka:
    '{cifrar}'
    bf298f6d5f878b342f9e44bec08cb9ac00b4ce57e98316f030194a225fac89fb'
  primavera.rabbitmq:
    contraseña: '{cipher}17fcf0ae5b8c5cf87de6875b699be4a1746dd49
    3a99d926c7a26a68c422117ef'
```

Cifrado y descifrado de información confidencial

La información se puede cifrar y descifrar mediante los puntos de conexión /encrypt y /decrypt expuestos por el servidor de configuración. El punto de conexión /encrypt permite crear valores cifrados que se colocarán en el archivo de propiedades del repositorio de configuración. Consulte el ejemplo de la sección anterior, donde las contraseñas de Eureka y RabbitMQ se almacenan cifradas en el disco. El punto de conexión /decrypt permite verificar la información cifrada almacenada en el disco del repositorio de configuración.

1. Para cifrar la cadena "Hola mundo", ejecute el siguiente comando:

```
curl -k https://dev-usr:dev-pwd@localhost:8443/config/encrypt
--data-urlencode "hola mundo"
```



Es importante utilizar el indicador --data-urlencode al usar curl para llamar al punto final /encrypt, para garantizar el manejo correcto de caracteres especiales.

como por ejemplo +.

2. Espere una respuesta similar a la siguiente:

```
bash
d91001603dcdcf3eb1392ccbd40ff201cdcf7b9af2fcaab3da39e37919033b206$
$
```

Figura 12.2: Un valor cifrado de un parámetro de configuración

3. Para descifrar el valor cifrado, ejecute el siguiente comando:

```
curl -k https://dev-usr:dev-pwd@localhost:8443/config/decrypt -d  
d91001603dcd3eb1392ccbd40ff201cdcf7b9af2fcaab3da39e37919033b206
```

4. Espere la cadena "hola mundo" como respuesta:



Figura 12.3: Un valor descifrado de un parámetro de configuración

5. Si desea usar un valor cifrado en un archivo de configuración, debe anteponerlo con {cipher} y encerrarlo entre comillas. Por ejemplo, para almacenar la versión cifrada de "Hola mundo", agregue la siguiente línea en un archivo de configuración basado en YAML:

```
mi-secreto: '{cipher}  
d91001603dcd3eb1392ccbd40ff201cdcf7b9af2fcaab3da39e37919033b206'
```

6. Cuando el servidor de configuración detecta valores en el formato '{cipher}...', intenta descifrarlos. ellos utilizando su clave de cifrado antes de enviarlos a un cliente.
7. Estas pruebas concluyen el capítulo sobre configuración centralizada. Termine cerrando
- Abajo en el panorama del sistema:

```
Docker Compose Down
```

Resumen

En este capítulo, vimos cómo usar Spring Cloud Config Server para centralizar la gestión de la configuración de nuestros microservicios. Podemos colocar los archivos de configuración en un repositorio común y compartir las configuraciones comunes en un único archivo, manteniendo la configuración específica de cada microservicio en archivos específicos. Los microservicios se actualizaron para recuperar su configuración del servidor de configuración al inicio y se configuraron para gestionar interrupciones temporales mientras recuperan su configuración del servidor de configuración.

El servidor de configuración puede proteger la información de configuración al requerir el uso autenticado de su API con autenticación HTTP básica y puede evitar la interceptación de datos al exponer su API externamente a través del servidor perimetral que usa HTTPS. Para evitar que intrusos que accedieron a los archivos de configuración en disco accedan a información confidencial, como contraseñas, podemos usar el punto final /encrypt del servidor de configuración para cifrar la información y almacenarla cifrada en el disco.

Si bien exponer las API del servidor de configuración externamente es útil durante el desarrollo, se deben bloquear antes de usarlas en producción.

En el próximo capítulo, aprenderemos cómo podemos usar Resilience4j para mitigar los posibles inconvenientes del uso excesivo de la comunicación sincrónica entre microservicios.

Preguntas

1. ¿Qué llamada API podemos esperar de un servicio de revisión al servidor de configuración durante el inicio para recuperar su configuración?

2. El servicio de revisión se inició utilizando el siguiente comando: `docker compose up -d`.

¿Qué información de configuración debemos esperar de una llamada API al servidor de configuración mediante el siguiente comando?

```
curl https://dev-usr:dev-pwd@localhost:8443/config/application/  
predeterminado -ks | jq
```

3. ¿Qué tipos de backend de repositorio admite Spring Cloud Config?

4. ¿Cómo podemos cifrar información confidencial en el disco usando Spring Cloud Config Server?

5. ¿Cómo podemos proteger la API del servidor de configuración contra el uso indebido?

6. Mencione algunas ventajas y desventajas para los clientes que se conectan primero al servidor de configuración en comparación con... a aquellos que primero se conectan al servidor de descubrimiento.

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



13

Mejorar la resiliencia mediante Resilience4j

En este capítulo, aprenderemos a usar Resilience4j para aumentar la resiliencia de nuestros microservicios; es decir, cómo mitigar errores y recuperarse de ellos. Como ya explicamos en el Capítulo 1, Introducción a los Microservicios, en la sección "Interruptor de circuito" , y en el Capítulo 8, Introducción a Spring Cloud, en la sección "Uso de Resilience4j para mejorar la resiliencia", un interruptor de circuito puede utilizarse para minimizar el daño que un microservicio descendente lento o que no responde puede causar en un entorno de sistemas a gran escala de microservicios que se comunican sincrónicamente. Veremos cómo el interruptor de circuito en Resilience4j puede utilizarse junto con un limitador de tiempo y un mecanismo de reintento para evitar dos de Las situaciones de error más comunes:

- Microservicios que comienzan a responder lentamente o no responden en absoluto
- Solicitudes que fallan aleatoriamente de vez en cuando, por ejemplo, debido a problemas de red temporales. problemas

En este capítulo se tratarán los siguientes temas:

- Presentación de los tres mecanismos de Resilience4j: disyuntor, limitador de tiempo y reintento
- Agregar los mecanismos al código fuente
- Probar los mecanismos cuando se implementan en el panorama del sistema

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter13`.

Si desea ver los cambios aplicados al código fuente en este capítulo (es decir, ver lo que se necesitó para agregar resiliencia con Resilience4j), puede compararlo con el código fuente del Capítulo 12, Configuración centralizada. Puede usar su herramienta de comparación preferida y comparar las dos carpetas. `$BOOK_HOME/Capítulo 12` y `$BOOK_HOME/Capítulo 13`.

Presentando los mecanismos de resiliencia de Resilience4j

El disyuntor, el limitador de tiempo y los mecanismos de reintento son potencialmente útiles en cualquier sistema sincrónico.

Comunicación entre dos componentes de software, por ejemplo, microservicios. En este capítulo, aplicaremos estos mecanismos en un solo lugar, en llamadas del servicio compuesto por producto al servicio del producto.

Esto se ilustra en la siguiente figura:

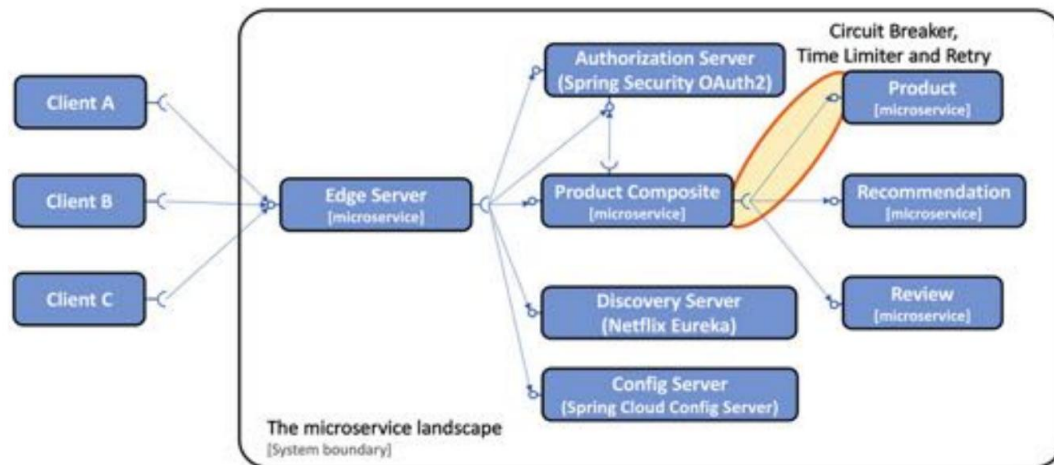


Figura 13.1: Adición de capacidades de resiliencia al panorama del sistema

Tenga en cuenta que las llamadas sincrónicas a los servidores de descubrimiento y configuración desde los otros microservicios no se muestran en el diagrama anterior (para facilitar la lectura).



Con el lanzamiento de Spring Cloud Hoxton en noviembre de 2019, se añadió el proyecto Spring Cloud Circuit Breaker . Este proporciona una capa de abstracción para interruptores de circuito. Resilience4j puede configurarse para su uso interno. Este proyecto no proporciona otros mecanismos de resiliencia, como reintentos, limitadores de tiempo, mamparos o limitadores de velocidad, de forma integrada, como sí lo hace el proyecto Resilience4j. Para más información sobre el proyecto, consulte <https://spring.io/projects/spring-cloud-circuitbreaker>.

También existen otras alternativas. Por ejemplo, el proyecto Reactor incluye compatibilidad integrada con reintentos y tiempos de espera; véase `Mono.retryWhen()` y `Mono.timeout()`. Spring también tiene un mecanismo de reintento (ver <https://github.com/spring-projects/spring-retry>) y estilo), pero no admite una programación reactiva modelo.

Sin embargo, ninguna de las alternativas ofrece un enfoque tan cohesivo e integrado para proporcionar un conjunto de mecanismos de resiliencia como Resilience4j, específicamente en un entorno Spring Boot, donde las dependencias, las anotaciones y la configuración se utilizan de forma elegante y consistente. Finalmente, cabe destacar que las anotaciones de Resilience4j funcionan independientemente del estilo de programación utilizado, ya sea reactivo o imperativo.

Presentación del disyuntor

Repasemos rápidamente el diagrama de estados de un disyuntor del Capítulo 8, Introducción a Spring. Nube, en la sección Uso de Resilience4j para mejorar la resiliencia :

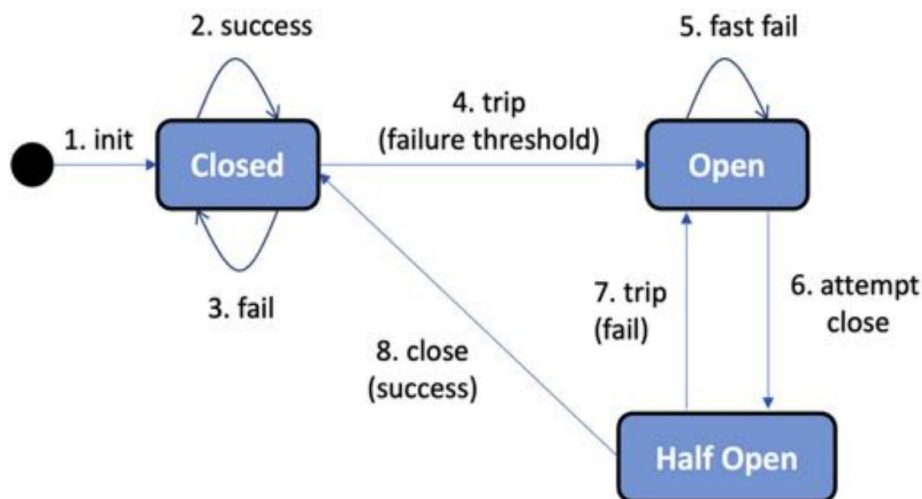


Figura 13.2: Diagrama de estados del disyuntor

Las características principales de un disyuntor son las siguientes:

- Si un disyuntor detecta demasiadas fallas, abrirá su circuito, es decir, no permitirá nuevas fallas.
llamadas.

Cuando el circuito está abierto, un interruptor automático ejecuta una lógica de fallo rápido. Esto significa que no espera a que se produzca un nuevo fallo, por ejemplo, un tiempo de espera, en las llamadas subsiguientes. En su lugar, redirige directamente la llamada a un método de respaldo. Este método puede aplicar diversas lógicas de negocio para generar una respuesta óptima. Por ejemplo, un método de respaldo puede devolver datos de una caché local o simplemente devolver un mensaje de error inmediato. Esto evitará que un microservicio deje de responder si los servicios de los que depende dejan de responder con normalidad. Esto es especialmente útil bajo alta carga.

Tras un tiempo, el disyuntor estará semiabierto, lo que permitirá nuevas llamadas para comprobar si el problema que causó los fallos ha desaparecido. Si el disyuntor detecta nuevos fallos, abrirá el circuito de nuevo y volverá a la lógica de fallo rápido. De lo contrario, cerrará el circuito y volverá a la operación normal. Esto hace que un microservicio sea resiliente a fallos o autoreparación, una capacidad indispensable en un entorno de microservicios que se comunican sincrónicamente entre sí.

Resilience4j expone información sobre los disyuntores en tiempo de ejecución de varias maneras:

- El estado actual de un disyuntor se puede monitorear utilizando el punto final de salud del actuador del microservicio , /
actuador/health
- El disyuntor también publica eventos en un punto final del actuador , por ejemplo, el estado
transiciones y eventos /actuador/interruptor
- Finalmente, los disyuntores se integran con el sistema de métricas de Spring Boot, que se puede utilizar para
publicar métricas en herramientas de monitoreo como Prometheus.

En este capítulo , probaremos los endpoints de estado y eventos . En el Capítulo 20, Monitoreo de Microservicios, veremos Prometheus en acción y cómo puede recopilar métricas expuestas por Spring Boot, por ejemplo, las métricas de nuestro disyuntor.

Para controlar la lógica de un interruptor, se puede configurar Resilience4j mediante archivos de configuración estándar de Spring Boot. Utilizaremos los siguientes parámetros de configuración:

- Tipo de ventana deslizante: Para determinar si es necesario abrir un interruptor, Resilience4j utiliza una ventana deslizante que cuenta los eventos más recientes para tomar la decisión. Las ventanas deslizantes pueden basarse en un número fijo de llamadas o en un tiempo transcurrido fijo.
Este parámetro se utiliza para configurar el tipo de ventana deslizante. Utilizaremos una ventana deslizante basada en conteos, estableciendo este parámetro en COUNT_BASED.

Tamaño de la ventana deslizante: Este es el número de llamadas en estado cerrado, que se utilizan para determinar si el circuito debe abrirse. Estableceremos este parámetro en 5.

- `FailureRateThreshold`: Este es el umbral, en porcentaje, de llamadas fallidas que provocarán la apertura del circuito. Fijaremos este parámetro al 50 %. Este ajuste, junto con `slideDownSize` a 5, significa que si tres o más de las últimas cinco llamadas son fallidas, el circuito se abrirá.

- `automaticTransitionFromOpenToHalfOpenEnabled`: Esto determina si el circuito El interruptor pasará automáticamente al estado semiabierto una vez finalizado el período de espera. De lo contrario, esperará la primera llamada una vez finalizado el período de espera hasta que pase al estado semiabierto. Estableceremos este parámetro como verdadero.

- `waitDurationInOpenState`: Especifica cuánto tiempo permanece el circuito abierto, es decir, antes de pasar al estado semiabierto. Fijaremos este parámetro en 10000. ms. Esta configuración, junto con la habilitación de la transición automática al estado semiabierto, establecida por el parámetro anterior, significa que el disyuntor mantendrá el circuito abierto durante 10 segundos y luego pasará al estado semiabierto.

- `NúmeroPermitidoDeLlamadasEnEstadoMedioAbierto`: Este es el número de llamadas en estado semiabierto, que se utilizan para determinar si el circuito se abrirá de nuevo o volverá al estado normal, cerrado. Fijaremos este parámetro en 3, lo que significa que el interruptor automático decidirá si el circuito se abrirá o cerrará en función de las tres primeras llamadas después de que el circuito haya pasado al estado semiabierto. Dado que el `UmbralDeTasaDeFallo...`
 Si los parámetros se configuran al 50%, el circuito se abrirá nuevamente si fallan 2 o las 3 llamadas. De lo contrario, El circuito se cerrará.

- `ignoreExceptions`: Permite especificar excepciones que no se contabilizan como fallos. Las excepciones comerciales esperadas, como "no encontrado" o "entrada no válida", son excepciones típicas que el disyuntor debe ignorar; los usuarios que buscan datos inexistentes o introducen datos no válidos no deben provocar la apertura del circuito. Configuraremos este parámetro como una lista que contenga las excepciones `"NotFoundException"` e `"InvalidInputException"`.

Finalmente, para configurar `Resilience4j` para informar el estado del disyuntor en la salud del actuador punto final de la forma correcta, se establecen los siguientes parámetros:

- `registerHealthIndicator = true` permite que `Resilience4j` complete el punto final de salud con Información sobre el estado de sus disyuntores.

- `allowHealthIndicatorToFail = false` indica a Resilience4j que no afecte el estado del endpoint de salud . Esto significa que el endpoint de salud seguirá reportando "ACTIVO" incluso si uno de los interruptores del componente está abierto o semiabierto. Es fundamental que el estado del componente no se reporte como "INACTIVO" solo porque uno de sus interruptores no esté cerrado. Esto significa que el componente se considera correcto, aunque uno de los componentes del que depende no lo esté.



En realidad, este es el valor principal de un disyuntor, por lo que establecer este valor en verdadero Esto prácticamente arruinaría el valor de activar un interruptor automático. En versiones anteriores de Resilience4j, este comportamiento era el correcto. En versiones más recientes, esto se ha corregido y el valor predeterminado para este parámetro es "falso" . Sin embargo, dado que considero muy importante comprender la relación entre el estado del componente y el estado de sus interruptores automáticos, lo he añadido a la configuración.

- Por último, también debemos configurar Spring Boot Actuator para agregar la información del estado del disyuntor que Resilience4j produce en respuesta a una solicitud a su punto final de estado :

`gestión.salud.interruptores.habilitados: verdadero`

Para obtener una lista completa de los parámetros de configuración disponibles, consulte <https://resilience4j.readme.io/docs/disyuntor#crear-y-configurar-un-disyuntor>.

Presentamos el limitador de tiempo

Para ayudar a un disyuntor a gestionar servicios lentos o que no responden, un mecanismo de tiempo de espera puede ser útil . El mecanismo de tiempo de espera de Resilience4j, llamado `TimeLimiter`, se puede configurar mediante los archivos de configuración estándar de Spring Boot. Usaremos el siguiente parámetro de configuración:

- `timeoutDuration`: Esto especifica cuánto tiempo espera una instancia de `TimeLimiter` una llamada a

Se completará antes de que se genere una excepción de tiempo de espera. Lo estableceremos en 2 s.

Presentamos el mecanismo de reintento

El mecanismo de reintento es muy útil para fallos aleatorios e infrecuentes, como fallos temporales de red. Permite simplemente reintentar una solicitud fallida varias veces con un retardo configurable entre intentos. Una restricción muy importante en el uso del mecanismo de reintento es que los servicios que reintenta deben ser idempotentes ; es decir, invocar el servicio una o varias veces con los mismos parámetros de solicitud produce el mismo resultado.

Por ejemplo, leer información es idempotente, pero crearla no suele serlo. No conviene que un mecanismo de reintento cree dos órdenes accidentalmente solo porque la respuesta de la primera se perdió en la red.

Resilience4j expone la información de reintento de la misma manera que lo hace para los interruptores automáticos en cuanto a eventos y métricas, pero no proporciona información de estado. Se puede acceder a los eventos de reintento en el punto final del actuador `/actuador/retryevents`. Para controlar la lógica de reintento, Resilience4j se puede configurar mediante los archivos de configuración estándar de Spring Boot. Utilizaremos los siguientes parámetros de configuración:

- **Intentos Máximos:** El número de intentos antes de rendirse, incluyendo la primera llamada. Fijaremos este parámetro en 3, lo que permitirá un máximo de dos reintentos tras una llamada inicial fallida.
- **waitDuration:** El tiempo de espera antes del siguiente reintento. Fijaremos este valor en 1000 ms, lo que significa que esperaremos 1 segundo entre reintentos.
- **RetryExceptions:** Lista de excepciones que activarán un reintento. Solo activaremos reintentos en excepciones de tipo `InternalServerError`, es decir, cuando las solicitudes HTTP respondan con un error 500. código de estado.



Tenga cuidado al configurar los ajustes de reintento y del disyuntor para que, por ejemplo, el disyuntor no abra el circuito antes de que se complete el número de reintentos previsto.

Para obtener una lista completa de los parámetros de configuración disponibles, consulte <https://resilience4j.readme.io/docs/reintentar#crear-y-configurar-reintentar>.

Con esta introducción, estamos listos para ver cómo agregar estos mecanismos de resiliencia al código fuente en el servicio compuesto por producto .

Añadiendo los mecanismos de resiliencia al código fuente

Antes de añadir los mecanismos de resiliencia al código fuente, añadiremos código que permita forzar la ocurrencia de un error, ya sea como un retraso o como un fallo aleatorio. A continuación, añadiremos un disyuntor junto con un limitador de tiempo para gestionar APIs lentas o que no responden, así como un mecanismo de reintento que pueda gestionar fallos aleatorios. Añadir estas funciones de Resilience4j sigue el mismo método que Spring Boot, que hemos utilizado en los capítulos anteriores:

- Agregue una dependencia de inicio de Resilience4j en el archivo de compilación
- Agregar anotaciones en el código fuente donde se aplicarán los mecanismos de resiliencia
- Agregar una configuración que controle el comportamiento de los mecanismos de resiliencia

Gestionar los desafíos de resiliencia es responsabilidad de la capa de integración; por lo tanto, los mecanismos de resiliencia se ubicarán en la clase `ProductCompositeIntegration`. El código fuente de la lógica de negocio, implementado en la clase `ProductCompositeServiceImpl`, no detectará la presencia de los mecanismos de resiliencia.

Una vez que tengamos los mecanismos en su lugar, finalmente ampliaremos nuestro script de prueba, `test-em-all.bash`, con pruebas que verifican automáticamente que el disyuntor funciona como se espera cuando se implementa en el panorama del sistema.

Añadiendo retrasos programables y errores aleatorios

Para poder probar nuestros mecanismos de resiliencia, necesitamos una forma de controlar cuándo se producen errores. Una forma sencilla de lograrlo es añadir parámetros de consulta opcionales a la API utilizada para recuperar un producto y un producto compuesto.



El código y los parámetros de API añadidos en esta sección para forzar la aparición de retrasos y errores solo deben utilizarse durante el desarrollo y las pruebas, no en producción. Al aprender sobre el concepto de malla de servicios en el Capítulo 18, "Uso de una malla de servicios para mejorar la observabilidad y la gestión", descubriremos mejores métodos que pueden utilizarse en producción para introducir retrasos y errores de forma controlada. Al usar una malla de servicios, podemos introducir retrasos y errores, normalmente utilizados para verificar la resiliencia, sin afectar el código fuente de los microservicios.

La API de producto compuesta simplemente transferirá los parámetros a la API de producto. Se han añadido los siguientes parámetros de consulta a ambas API:

- **delay:** Hace que la API `getProduct` del microservicio del producto retrase su respuesta. El parámetro se especifica en segundos. Por ejemplo, si el parámetro se establece en 3, se producirá un retraso de 3 segundos antes de que se devuelva la respuesta.
- **faultPercentage:** Hace que la API `getProduct` del microservicio de producto genere una excepción aleatoriamente con la probabilidad especificada en el parámetro de consulta, del 0 al 100 %. Por ejemplo, si el parámetro se establece en 25, provocará que una de cada cuatro llamadas a la API, en promedio, falle con una excepción. En estos casos, se devolverá el error HTTP 500 (Error interno del servidor).

Cambios en las definiciones de API

Los dos parámetros de consulta que introdujimos anteriormente, `delay` y `faultPercentage`, se han definido en el proyecto de API en las siguientes dos interfaces Java:

- Servicio de producto compuesto:

```
Mono<ProductAggregate> obtenerProducto(
    @PathVariable int ID del producto,
    @RequestParam(valor = "retraso", requerido = falso, valor
        predeterminado = "0") int retraso,
    @RequestParam(valor = "porcentajeDeFallo", requerido = falso,
        valorPredeterminado = "0") int porcentajeDeFallo
);
```

- ProductoServicio:

```
Mono<Producto> obtenerProducto(
    @PathVariable int ID del producto,
    @RequestParam(valor = "retraso", requerido = falso, valor
        predeterminado = "0") int retraso,
    @RequestParam(valor = "porcentajeDeFallo", requerido = falso,
        valorPredeterminado = "0") int porcentajeDeFallo
);
```

Los parámetros de consulta se declaran opcionales con valores predeterminados que deshabilitan el uso de mecanismos de error. Esto significa que si no se utiliza ninguno de los parámetros de consulta en una solicitud, no se aplicará un retraso ni se generará un error.

Cambios en el microservicio compuesto por producto. El microservicio compuesto por producto simplemente pasa los parámetros a la API del producto. La implementación del servicio recibe la solicitud de la API y pasa los parámetros al componente de integración que realiza la llamada a la API del producto.

- La llamada de la clase `ProductCompositeServiceImpl` al componente de integración se ve así:

```
público Mono<ProductAggregate> obtenerProducto(int productId,
    int retraso, int porcentajeDeFallo) {
    devolver Mono.zip(
        ...
        integración.getProduct(productId, delay, faultPercent),
        ....
    );
}
```

- La llamada de la clase `ProductCompositeIntegration` a la API del producto se ve así:

```
p blico Mono<Producto> obtenerProducto(int idProducto, int retraso,
    int porcentajeDeFalla) {

    URI url =
        UriComponentsBuilder.fromUriString( PRODUCT_SERVICE_URL + "/"
            product/{productId}?delay={delay}" +
            "&faultPercent={faultPercent}") .build(productId,
            delay, faultPercent); return webClient.get().uri(url).retrieve()...
```

Cambios en el microservicio del producto

El microservicio de producto implementa el generador de retrasos y errores aleatorios en la clase `ProductServiceImpl`, extendiendo el flujo existente que se utiliza para leer la informaci n del producto desde la base de datos MongoDB. Su aspecto es el siguiente:

```
p blico Mono<Producto> obtenerProducto(int idProducto, int retraso,
    int porcentajeDeFalla) {

    ...

    devolver repositorio.findByProductId(productId)
        .map(e -> throwErrorIfBadLuck(e,
            faultPercent)) .delayElement(Duraci n.deSegundos(retraso))
    ...

}
```

 Consejo r pido: Mejora tu experiencia de programaci n con las funciones AI Code Explainer y Quick Copy . Abre este libro en el Packt Reader de  ltima generaci n. Haz clic en el bot n Copiar.

(1) para copiar r pidamente el c digo en su entorno de codificaci n, o haga clic en el bot n Explicar

(2) para que el asistente de IA le explique un bloque de c digo.



```
function calculate(a, b) {
    return {sum: a + b};
};
```

Copy

Explain

1

2



 El lector Packt de próxima generación se incluye de forma gratuita con la compra de este libro.

Escanee el código QR O vaya a packtpub.com/unlock, Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



Cuando el flujo devuelve una respuesta del repositorio de Spring Data, primero aplica el método `throwErrorIfBadLuck` para determinar si se debe lanzar una excepción. A continuación, aplica un retraso mediante la función `delayElement` de la clase `Mono`.

El generador de errores aleatorios, `throwErrorIfBadLuck()`, crea un número aleatorio entre 1 y 100 y genera una excepción si es mayor o igual al porcentaje de fallo especificado. Si no se genera ninguna excepción, la entidad del producto se pasa en el flujo. El código fuente se ve así:

```
EntidadProductoprivada throwErrorIfBadLuck(
    Entidad ProductEntity, int faultPercent) {

    si (porcentajeDeFallo == 0) {
        entidad de retorno ;
    }

    int umbralAleatorio = obtenerNumeroAleatorio(1, 100);

    si (porcentajeDeFallo < UmbralAleatorio) {
        LOG.debug("Tuvimos suerte, no ocurrió ningún error, {} < {}",
            porcentaje de falla, umbral aleatorio);

    } demás {
        LOG.info("Mala suerte, ocurrió un error, {} >= {}",
            porcentaje de falla, umbral aleatorio);

        lanzar nueva RuntimeException("Algo salió mal...");
    }

    entidad de retorno ;
}
```

```

    }

    privado final Aleatorio randomNumberGenerator = nuevo Aleatorio();

    privado int obtenerNumeroAleatorio(int min, int max) {

        si (máx < min) {
            lanzar nueva IllegalArgumentException("Max debe ser mayor que min");
        }

        devuelve randomNumberGenerator.nextInt((max - min) + 1) + min;
    }

```

Con los retrasos programables y las funciones de error aleatorio implementadas, estamos listos para comenzar a agregar Mecanismos de resiliencia del código. Comenzaremos con el disyuntor y el limitador de tiempo.

Adición de un disyuntor y un limitador de tiempo

Como mencionamos anteriormente, necesitamos agregar dependencias, anotaciones y configuración. También necesitamos agregar código para implementar la lógica de respaldo para escenarios de falla rápida. Veremos cómo hacerlo en las siguientes secciones.

Agregar dependencias al archivo de compilación

Para agregar un disyuntor y un limitador de tiempo, debemos agregar dependencias a las bibliotecas de Resilience4j correspondientes en el archivo de compilación, build.gradle. De la documentación del producto (<https://resilience4j.readme.io/docs/getting-started-3#setup>), Podemos aprender que es necesario agregar las siguientes tres dependencias. Usaremos la última versión disponible (v2.3.0, al momento de escribir este capítulo):

```

extensión {
    resilience4jVersión = "2.3.0"
}

dependencias {
    Implementación "io.github.resilience4j:resilience4j-spring-boot2:
        ${resilience4jVersion}"
    Implementación "io.github.resilience4j:resilience4j-reactor:
        ${resilience4jVersion}"
    implementación 'org.springframework.boot:spring-boot-starter-aop'
    ...

```

Para evitar que Spring Cloud anule la versión utilizada con la versión anterior de Resilience4j que incluye, también importamos un archivo resilience4j-bom (lista de materiales), como se describe en el proyecto de demostración de Spring Boot 3 en <https://github.com/resilience4j/resilience4j-spring-boot3-demo>.

Agregamos este archivo bom al archivo bom existente para Spring Cloud en la sección dependencyManagement :

```
gestión de dependencias {  
    importaciones {  
        mavenBom "org.springframework.cloud:dependencias-de-la-nube-de-spring:  
            ${springCloudVersion}"  
        mavenBom "io.github.resilience4j:resilience4j-bom:  
            ${resilience4jVersion}"  
    }  
}
```

Agregar anotaciones en el código fuente

El disyuntor se puede aplicar anotando el método que se espera que proteja con `@CircuitBreaker(...)`, que, en este caso, es el método `getProduct()` en la Clase `ProductCompositeIntegration`. El disyuntor se activa por una excepción, no por un tiempo de espera. Para activar el disyuntor tras un tiempo de espera, añadiremos un limitador de tiempo que se puede aplicar con la anotación `@TimeLimiter(...)`. El código fuente es el siguiente:

```
@TimeLimiter(nombre = "producto")  
@CircuitBreaker( nombre = "producto",  
                fallbackMethod = "getProductFallbackValue")  
  
público Mono<Producto> obtenerProducto(  
    int IDProducto, int Retraso, int PorcentajeDeFalla) {  
    ...  
}
```

El nombre del disyuntor y la anotación del limitador de tiempo, "producto", se utilizan para identificar la configuración que se aplicará. El parámetro `fallbackMethod` de la anotación del disyuntor se utiliza para especificar el método de reserva que se debe llamar (`getProductFallbackValue`, en este caso) cuando el disyuntor esté abierto; en breve proporcionaremos información sobre su uso.

Para activar el disyuntor, el método anotado debe invocarse como un bean Spring. En nuestro caso, es la clase de integración que Spring inyecta en la clase de implementación del servicio, `ProductCompositeServiceImpl`, y, por lo tanto, se utiliza como un bean Spring:

```
integración final privada ProductCompositeIntegration ;

público ProductoCompuestoServicioImpl(
    Integración de ProductCompositeIntegration ) {
    esto.integracion = integración;
}

público Mono<ProductAggregate> obtenerProducto(int IdProducto, int retraso,
                                              int porcentajeDeFalla) {

    devolver Mono.zip(
        ...,
        integración.getProduct(productId, delay, faultPercent),
        ...
    );
}
```

Añadiendo lógica de respaldo rápido ante fallos

Para poder aplicar la lógica de respaldo cuando el disyuntor está abierto (es decir, cuando una solicitud falla rápidamente), podemos especificar un método de respaldo en la anotación `CircuitBreaker`, como se muestra en el código fuente anterior. El método debe seguir la firma del método al que se aplica el disyuntor y también tener un último argumento adicional para pasar la excepción que lo activó. En nuestro caso, la firma del método de respaldo se ve así:

```
privado Mono<Producto> getProductFallbackValue(int productId,
                                              int retraso, int porcentajeDeFallo, CallNotPermittedException ej) {
```

El último parámetro especifica que queremos poder gestionar excepciones del tipo `CallNotPermittedException`. Solo nos interesan las excepciones que se lanzan cuando el interruptor está abierto para poder aplicar la lógica de fallo rápido. Cuando el interruptor está abierto, no permite llamadas al método subyacente; en su lugar, lanza inmediatamente una excepción `CallNotPermittedException`. Por lo tanto, solo nos interesa capturar las excepciones `CallNotPermittedException`.

La lógica de respaldo puede buscar información basada en el ID del producto en fuentes alternativas, por ejemplo, una caché interna. En nuestro caso, devolveremos valores codificados basados en el ID del producto para simular un acierto en la caché. Para simular un error en la caché, lanzaremos una excepción de no encontrado si el ID del producto es 13. La implementación del método de respaldo se ve así:

```

privado Mono<Producto> getProductFallbackValue(int productId,
    int retraso, int porcentajeDeFallo, CallNotPermittedException ej) {

    si (productId == 13) {
        Cadena errMsg = "Id. del producto: " + ID del producto
            + " ¡No se encontró en el caché de reserva!";
        lanzar nueva NotFoundException(errMsg);
    }

    devolver Mono.just(new Product(productId, "Producto de respaldo"
        + productId, productId, serviceUtil.getServiceAddress()));
}

```

Agregar configuración Finalmente,

la configuración del disyuntor y del limitador de tiempo se agrega al archivo product-composi- ite.yml en el repositorio de configuración , de la siguiente manera:

```

resiliencia4j.limitador de tiempo:
  instancias:
    producto:
      Tiempo de espera: 2 s

gestión.salud.interruptores.habilitados: verdadero

resiliencia4j.disyuntor:
  instancias:
    producto:
      permitir que el indicador de salud falle: falso
      registerHealthIndicator: verdadero
      tipoDeVentanaDeslizante: BASADO EN
      CONTADORES tamañoDeVentanaDeslizante: 5
      Umbral de tasa de fallos: 50
      Duración de espera en estado abierto: 10000
      NúmeroDeLlamadasPermitidoEnEstadoMedioAbierta: 3
      TransiciónAutomáticaDeAbiertaAMedioAbiertaHabilitada: verdadero
  ignorarExcepciones:
    - se.magnus.api.exceptions.InvalidInputException
    - se.magnus.api.exceptions.NotFoundException

```

Los valores en la configuración ya han sido descritos en los apartados anteriores, Introducción al disyuntor y Introducción al limitador de tiempo.

Agregar un mecanismo de reintento

Al igual que con el disyuntor, se configura un mecanismo de reintento añadiendo dependencias , anotaciones y configuración. Las dependencias se añadieron previamente en la sección "Añadir dependencias al archivo de compilación" , por lo que solo necesitamos añadir la anotación y configurar la configuración.

Añadiendo la anotación de reintento

El mecanismo de reintento se puede aplicar a un método anotándolo con `@Retry(name="nnn")`, Donde nnn es el nombre de la entrada de configuración que se usará para este método. Consulte la sección " Añadir configuración" para obtener más detalles. En nuestro caso, el método es el mismo que para el disyuntor y el limitador de tiempo: `getProduct()` en `ProductCompositeIntegration`. clase:

```
@Retry(nombre = "producto")
@TimeLimiter(nombre = "producto")
@CircuitBreaker(nombre = "producto", método de respaldo =
    "obtenerValorDeReemplazoDeProducto")
público Mono<Producto> obtenerProducto(int idProducto, int retraso,
    int porcentajeDeFalla) {
```

Añadiendo configuración

La configuración para el mecanismo de reintento se agrega de la misma manera que para el disyuntor y el limitador de tiempo en el archivo `product-composite.yml` en el repositorio de configuración, de la siguiente manera:

```
resiliencia4j.retry:
  instancias:
    producto:
      Intentos máximos: 3
      Duración de espera: 1000
      Excepciones de reintento:
        -org.springframework.web.reactive.function.client .
        Excepción de respuesta del cliente web$Error interno del servidor
```

Los valores reales se analizaron en la sección anterior Introducción al mecanismo de reintento .



Al utilizar varios mecanismos de Resilience4j (en nuestro caso, el disyuntor, el limitador de tiempo y el mecanismo de reintento), es importante comprender el orden en que se aplican estos aspectos. Para más información, consulte <https://resilience4j.readme.io/docs/getting-started-3#aspect-order>.

Estas son todas las dependencias, anotaciones, código fuente y configuración necesarias. Para finalizar, ampliaremos el script de prueba con pruebas que verifiquen que el disyuntor funciona correctamente en un entorno de sistema implementado.

Agregar pruebas automatizadas

Se han agregado pruebas automatizadas para el disyuntor al script de prueba `test-em-all.bash` en una función separada, `testCircuitBreaker()`:

```
función testCircuitBreaker() {
    echo "¡Iniciar pruebas del disyuntor!"
    ...
}
...
prueba de disyuntor
...
echo "Fin, todas las pruebas están bien:" `date`
```

Para poder realizar algunas de las verificaciones requeridas, necesitamos tener acceso al actuador. Puntos finales del microservicio compuesto por producto , que no se exponen a través del servidor perimetral. Por lo tanto, accederemos a los puntos finales del actuador ejecutando un comando en el microservicio compuesto por producto mediante el comando Docker Compose `exec` . La imagen base utilizada por los microservicios, Eclipse Temurin, incluye `wget`, por lo que podemos simplemente ejecutar un comando `wget` en el contenedor compuesto por producto para obtener la información necesaria. El comando se ve así:

```
docker compose exec -T producto-compuesto wget -qO - http://producto-compuesto:8080/actuador/salud
```



El argumento `-T` se utiliza para deshabilitar el uso de una terminal para el comando `exec`. Esto es importante para posibilitar la ejecución del script de prueba `test-em-all.bash` en un entorno donde no existen terminales, por ejemplo, en una canalización de compilación automatizada utilizada para CI/CD.

Para poder extraer la información que necesitamos para nuestras pruebas, podemos enviar la salida a la herramienta jq .

Por ejemplo, para extraer el estado actual del disyuntor, podemos ejecutar el siguiente comando:

```
docker compose exec -T producto-compuesto wget -qO - http://producto-compuesto:8080/
actuador/salud | jq -r .componentes.circuitBreakers.
detalles.producto.detalles.estado
```

Devolverá CERRADO, ABIERTO o MEDIO ABIERTO, según el estado actual.

La prueba comienza haciendo exactamente esto, es decir, verificando que el disyuntor esté cerrado antes de ejecutar las pruebas:

```
assertEqual "CERRADO" "$(docker compose exec -T product-composite wget -qO - http://
product-composite:8080/actuador/health | jq -r .components.
Disyuntores.detalles.producto.detalles.estado)"
```

A continuación, la prueba forzará la apertura del disyuntor ejecutando tres comandos seguidos, todos los cuales fallarán debido a un tiempo de espera causado por una respuesta lenta del servicio del producto (el retraso el parámetro se establece en 3 segundos):

```
para ((n=0; n<3; n++))
hacer
    assertCurl 500 "curl -k https://$HOST:$PORT/compuesto-de-producto/$PROD_
ID_REVS_RECS?retardo=3 $AUTH -s"
    mensaje=$(echo $RESPUESTA | jq -r .mensaje)
    assertEquals "No se observó ningún elemento ni señal terminal dentro
2000ms" "${mensaje:0:57}"
hecho
```



Un breve recordatorio de la configuración: El tiempo de espera del servicio del producto está establecido en 2 segundos, por lo que un retraso de 3 segundos provocará un tiempo de espera. El interruptor está configurado para evaluar las últimas cinco llamadas al cerrarse. Las pruebas del script que preceden a las pruebas específicas del interruptor ya han realizado un par de llamadas exitosas. El umbral de fallo está establecido en el 50 %. Tres llamadas con un retraso de tres segundos son suficientes para abrir el circuito.

Con el circuito abierto, esperamos un comportamiento rápido, es decir, no tendremos que esperar a que se agote el tiempo de espera para obtener una respuesta. También esperamos que se invoque el método de reserva para obtener una respuesta óptima. Esto también debería aplicarse a una llamada normal, es decir, sin solicitar un retraso. Esto se verifica con el siguiente código:

```
assertEqual "ABIERTO" "$(docker compose exec -T producto-compuesto wget -qO - http://producto-compuesto:8080/actuador/salud | jq -r .componentes.Disyuntores.detalles.producto.detalles.estado)"

assertCurl 200 "curl -k https://$HOST:$PORT/compuesto-del-producto/$PROD_ID_REVS_RECS?retardo=3 $AUTH -s"
assertEqual "Producto de respaldo$PROD_ID_REVS_RECS" "$(echo "$RESPONSE" | jq -r .name)"

assertCurl 200 "curl -k https://$HOST:$PORT/compuesto-del-producto/$PROD_ID_REVS_RECS $AUTH -s"
assertEqual "Producto de respaldo$PROD_ID_REVS_RECS" "$(echo "$RESPONSE" | jq -r .name)"
```



El ID del producto 1 se almacena en una variable, \$PROD_ID_REVS_RECS, para facilitar la modificación del script si es necesario.

También podemos verificar que la lógica de error simulado no encontrado funciona como se espera en el método de respaldo; es decir, el método de respaldo devuelve 404, NOT_FOUND para el ID de producto 13:

```
assertCurl 404 "curl -k https://$HOST:$PORT/compuesto-de-producto/$PROD_ID_NO_ENCONTRADO $AUTH -s"
assertEqual "¡No se encontró el ID del producto: $PROD_ID_NOT_FOUND en la caché de respaldo!" "$(echo $RESPONSE | jq -r .message)"
```



El ID del producto 13 se almacena en una variable, \$PROD_ID_NOT_FOUND.

Según la configuración, el disyuntor cambiará a semiabierto después de 10 segundos. Para verificarlo, la prueba espera 10 segundos:

```
echo "Dormiré durante 10 segundos esperando que el CB se abra a medias..."
dormir 10
```

Luego de verificar el estado esperado (semiabierto), la prueba ejecuta tres solicitudes normales para que el disyuntor vuelva a su estado normal, el cual también se verifica:

```
assertEqual "MEDIO_ABIERTO" "$(docker compose exec -T producto-compuesto wget -qO - http://
producto-compuesto:8080/actuador/salud | jq -r .componentes.
Disyuntores.detalles.producto.detalles.estado)"

para ((n=0; n<3; n++))
hacer
    assertCurl 200 "curl -k https://$HOST:$PORT/producto-compuesto/$PROD_
ID_REVS_RECS $AUTH -s"
    assertEquals "nombre del producto C" "$(echo "$RESPONSE" | jq -r .name)"
hecho

assertEqual "CERRADO" "$(docker compose exec -T product-composite wget -qO - http://
product-composite:8080/actuador/health | jq -r .components.
Disyuntores.detalles.producto.detalles.estado)"
```

El código de prueba también verifica que se haya obtenido una respuesta con datos de la base de datos subyacente. Para ello, compara el nombre del producto devuelto con el valor almacenado en la base de datos. Para el producto con ID 1, el nombre es "nombre de producto C".



Un breve recordatorio de la configuración: El disyuntor está configurado para evaluar las tres primeras llamadas cuando está semiabierto. Por lo tanto, necesitamos ejecutar tres solicitudes con más del 50 % de éxito antes de que se cierre el circuito.

La prueba finaliza utilizando la API del actuador /actuador/circuitbreakerevents , expuesta por el interruptor para revelar eventos internos. Esta API permite determinar las transiciones de estado que ha realizado el interruptor. Esperamos que las tres últimas transiciones de estado sean las siguientes:

- Primera transición de estado: cerrado a abierto
- Próxima transición de estado: Abierto a semiabierto
- Última transición de estado: Semiabierto a cerrado

Esto se verifica mediante el siguiente código:

```
assertEqual "CERRADO A ABRIR" "$(docker compose exec -T producto-
compuesto wget -qO - http://producto-compuesto:8080/actuador/
eventos_de_interruptor/producto/TRANSICIÓN_DE_ESTADO | jq -r
.circuitBreakerEvents[-3].stateTransition)"
```

```
assertEqual "ABIERTO_A_MITAD_ABIERTO" "$(docker compose exec -T producto-  
compuesto wget -qO - http://producto-compuesto:8080/  
actuador/circuitbreakerevents/producto/STATE_TRANSITION | jq  
-r .circuitBreakerEvents[-2].stateTransition)"  
  
assertEqual "MEDIO_ABIERTO_A_CERRADO" "$(docker compose exec -T producto-  
compuesto wget -qO - http://producto-compuesto:8080/  
actuador/eventosinterruptor/producto/TRANSICIÓN_DE_ESTADO | jq  
-r .circuitBreakerEvents[-1].stateTransition)"
```



La expresión `jq`, `circuitBreakerEvents[-1]`, representa la última entrada en la matriz de eventos del disyuntor. `[-2]` es el penúltimo evento y `[-3]` el anteúltimo. Juntos, son los tres eventos más recientes, los que nos interesan.

Hemos añadido bastantes pasos al script de prueba, pero con esto podemos verificar automáticamente que el comportamiento básico esperado de nuestro interruptor automático se cumple. En la siguiente sección, lo probaremos. Realizaremos pruebas tanto automáticas ejecutando el script de prueba como manualmente ejecutando los comandos del script.

Probando el disyuntor y el mecanismo de reintento

Ahora es el momento de probar el disyuntor y el mecanismo de reintento. Comenzaremos, como siempre, compilando las imágenes de Docker y ejecutando el script de prueba, `test-em-all.bash`. Después, ejecutaremos manualmente las pruebas descritas anteriormente para asegurarnos de comprender qué sucede. Realizaremos las siguientes pruebas manuales:

- Pruebas de días felices del disyuntor para verificar que el circuito esté cerrado en condiciones normales operaciones
- Pruebas negativas del disyuntor para verificar que el circuito se abre cuando comienzan las cosas salir mal
- Volver al funcionamiento normal para verificar que el circuito vuelva a su estado cerrado una vez resueltos los problemas.
- Probar el mecanismo de reintento con errores aleatorios

Construir y ejecutar las pruebas automatizadas

Para construir y ejecutar las pruebas automatizadas, necesitamos hacer lo siguiente:

1. Primero, construya las imágenes de Docker con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 13
./gradlew compilación y compilación de docker compose
```

2. A continuación, inicie el panorama del sistema en Docker y ejecute las pruebas habituales con lo siguiente dominio:

```
./test-em-all.bash inicio
```



Cuando el script de prueba imprime ¡ Iniciar pruebas de disyuntor!, ¡ las pruebas que describimos anteriormente se habrán ejecutado!

Verificar que el circuito esté cerrado en condiciones normales de funcionamiento.

Antes de poder llamar a la API, necesitamos un token de acceso. Ejecute los siguientes comandos para obtenerlo.
token de acceso:

```
desconfigurar ACCESS_TOKEN
TOKEN DE ACCESO=$(curl -k https://escritor:escritor-secreto@localhost:8443/
oauth2/token -d tipo_concesión=credenciales_del_cliente -d ámbito="producto:lectura
producto:escritura" -s | jq -r .token_de_acceso)
eco $ACCESS_TOKEN
```



Un token de acceso emitido por el servidor de autorización tiene una validez de una hora. Por lo tanto, si después de un tiempo empieza a recibir errores 401 (no autorizado) , probablemente sea el momento de adquirirlo. un nuevo token de acceso.

Pruebe una solicitud normal y verifique que devuelva el código de respuesta HTTP 200:

```
curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/
producto-compuesto/1 -w "%{http_code}\n" -o /dev/null -s
```



La opción -w "%{http_code}\n" se usa para imprimir el estado de retorno HTTP. Mientras el comando devuelva 200, no nos interesa el cuerpo de la respuesta, por lo que lo suprimimos con la opción -o /dev/null .

Verifique que el disyuntor esté cerrado mediante la API de estado :

```
docker compose exec producto-compuesto wget -qO - http://producto-compuesto:8080/
actuator/health | jq -r .components.circuitBreakers.
detalles.producto.detalles.estado
```

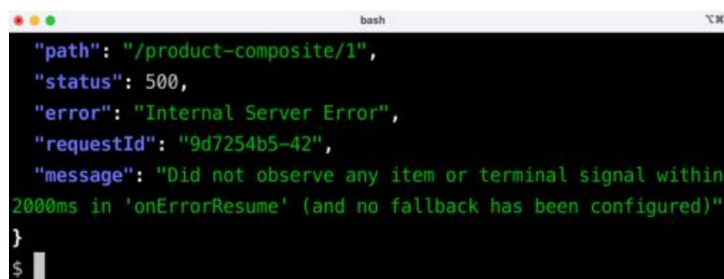
Esperamos que responda con CERRADO.

Forzar la apertura del disyuntor cuando las cosas salen mal

¡Ahora es el momento de que las cosas salgan mal! Con esto quiero decir que es hora de probar algunas pruebas negativas para verificar que el circuito se abre cuando las cosas empiezan a salir mal. Llama a la API tres veces e indica al servicio del producto que provoque un tiempo de espera en cada llamada; es decir, que retrase la respuesta tres segundos. Esto debería ser suficiente para activar el disyuntor.

```
curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/
producto-compuesto/1?retraso=3 -s | jq .
```

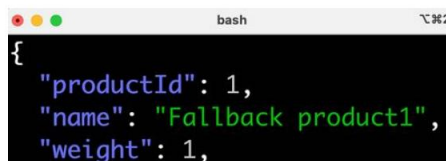
Esperamos una respuesta como la siguiente cada vez:



```
bash
{"path": "/product-composite/1",
 "status": 500,
 "error": "Internal Server Error",
 "requestId": "9d7254b5-42",
 "message": "Did not observe any item or terminal signal within
2000ms in 'onErrorResume' (and no fallback has been configured)"
}
```

Figura 13.3: Respuesta después de un tiempo de espera

El disyuntor ya está abierto, así que si realiza un cuarto intento (dentro del intervalo de espera , es decir, 10 segundos), verá el comportamiento de fallo rápido y el método de respaldo en acción. Recibirá una respuesta inmediata en lugar de un mensaje de error una vez que el limitador de tiempo se active después de 2 segundos.



```
bash
{
  "productId": 1,
  "name": "Fallback product1",
  "weight": 1,
```

Figura 13.4: Respuesta cuando el disyuntor está abierto

La respuesta provendrá del método de respaldo. Esto se puede reconocer observando el valor del campo de nombre, Fallback product1.



Los métodos de fallo rápido y de respaldo son capacidades clave de un interruptor automático. Una configuración con un tiempo de espera de tan solo 10 segundos en estado abierto requiere bastante rapidez para poder ver la lógica de fallo rápido y los métodos de respaldo en acción. Una vez en estado semiabierto, siempre puede enviar tres nuevas solicitudes que provoquen un tiempo de espera, forzando al interruptor automático a volver al estado abierto, y luego intentar rápidamente la cuarta solicitud. Entonces, debería obtener una respuesta de fallo rápido del método de respaldo. También puede aumentar el tiempo de espera a uno o dos minutos, pero puede ser bastante tedioso esperar tanto tiempo antes de que el circuito cambie al estado semiabierto.

Espera 10 segundos para que el disyuntor pase a estado semiabierto y luego ejecute el siguiente comando para verificar que el circuito ahora esté en estado semiabierto:

```
docker compose exec producto-compuesto wget -qO - http://producto-compuesto:8080/actuador/health | jq -r .components.circuitBreakers.
detalles.producto.detalles.estado
```

Se espera que responda con HALF_OPEN.

Cerrando nuevamente el disyuntor

Una vez que el disyuntor está en un estado semiabierto, espera tres llamadas para ver si debe abrir el circuito nuevamente o volver a la normalidad cerrándolo.

Envíemos tres solicitudes normales para cerrar el disyuntor:

```
curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/
producto-compuesto/1 -w "%{http_code}\n" -o /dev/null -s
```

Todos deberían responder con 200. Verifique que el circuito esté cerrado nuevamente usando la API de salud :

```
docker compose exec producto-compuesto wget -qO - http://producto-compuesto:8080/actuador/health | jq -r .components.circuitBreakers.
detalles.producto.detalles.estado
```

Esperamos que responda con CERRADO.

Concluya enumerando las últimas tres transiciones de estado utilizando el siguiente comando:

```
docker compose exec producto-compuesto wget -qO - http://producto-compuesto:8080/actuador/circuitbreakerevents/producto/STATE_TRANSITION | jq -r
'.circuitBreakerEvents[-3].stateTransition, .circuitBreakerEvents[-2].stateTransition, .circuitBreakerEvents[-1].stateTransition'
```

Espere que responda con lo siguiente:



Figura 13.5: Cambios de estado del disyuntor

Esta respuesta nos indica que hemos llevado a nuestro disyuntor a través de una vuelta completa de su diagrama de estados:

- De cerrado a abierto cuando los errores de tiempo de espera comienzan a impedir que las solicitudes tengan éxito
- De abierto a medio abierto para ver si el error ha desaparecido
- De medio abierto a cerrado cuando el error desaparece, es decir, cuando volvemos a la normalidad.

operación

Con esto, hemos terminado de probar el disyuntor; sigamos adelante y veamos el mecanismo de reintento en acción.

Probar reintentos causados por errores aleatorios

Simulemos que hay un problema aleatorio (con suerte, temporal) con nuestro producto o servicio.

La comunicación con él.

Podemos hacerlo usando el parámetro `faultPercent`. Si lo establecemos en 25, esperamos que falle una de cada cuatro solicitudes, en promedio. Esperamos que el mecanismo de reintento se active para ayudarnos reintentando automáticamente las solicitudes fallidas. Una forma de comprobar si el mecanismo de reintento se ha activado es medir el tiempo de respuesta del comando `curl`.

Una respuesta normal debería tardar unos 100 ms. Dado que hemos configurado el mecanismo de reintento para que espere 1 segundo (consulte el parámetro `waitDuration` en la sección sobre la configuración del mecanismo de reintento), esperamos que el tiempo de respuesta aumente 1 segundo por cada reintento. Para forzar la aparición de un error aleatorio, ejecute el siguiente comando un par de veces:

```
tiempo curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://  
localhost:8443/producto-compuesto/1?porcentajeDeFallo=25 -w "%{http_code}\n" -o /dev/null -s
```

El comando debería responder con 200, lo que indica que la solicitud se realizó correctamente. Un tiempo de respuesta con el prefijo "real" (por ejemplo, "real 0m0.078s") significa que el tiempo de respuesta fue de 0.078 s o 78 ms. Una respuesta normal, es decir, sin reintentos, debería indicar un tiempo de respuesta de aproximadamente 100 ms, como se muestra a continuación:



```
bash  ~%2
200
real    0m0.066s
```

Figura 13.6: Tiempo transcurrido para una solicitud sin reintento

Una respuesta después de un reintento debería tardar un poco más de 1 segundo y verse como sigue:



```
bash  ~%2
200
real    0m1.072s
```

Figura 13.7: Tiempo transcurrido para una solicitud con un reintento



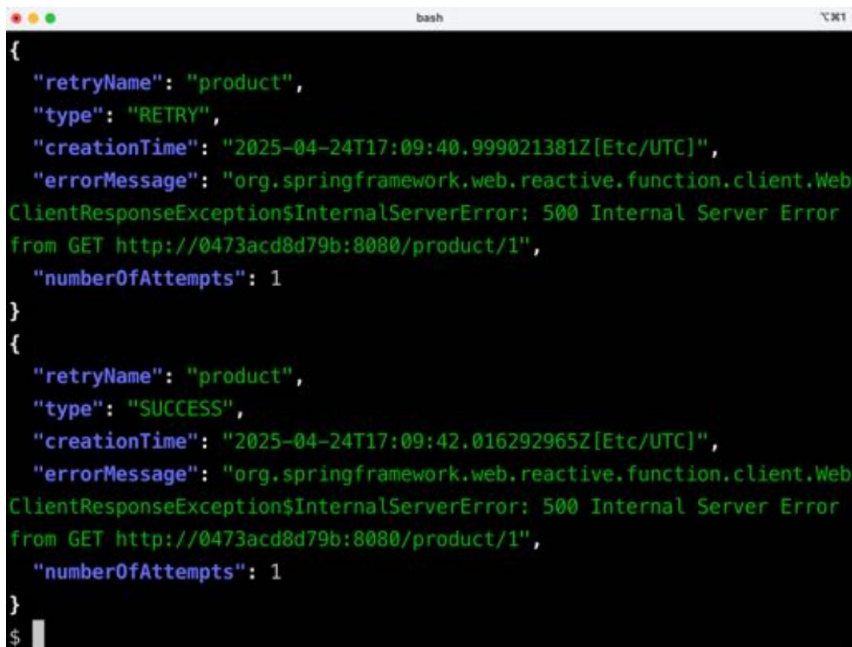
¡El código de estado HTTP 200 indica que la solicitud fue exitosa, aunque fue necesario un reintento antes de tener éxito!

Después de notar un tiempo de respuesta de 1 segundo, que indica que la solicitud requirió 1 reintento para tener éxito, ejecute el siguiente comando para ver los últimos 2 eventos de reintento:

```
docker compose exec producto-compuesto wget -qO - http://producto-compuesto:8080/actuator/retryevents |
jq '.retryEvents[-2], .retryEvents[-1]
```

Deberías poder ver la solicitud fallida y el siguiente intento exitoso. El tiempo de creación

Se espera que las marcas de tiempo difieran en un segundo. Espere una respuesta como la siguiente:



```
{
  "retryName": "product",
  "type": "RETRY",
  "creationTime": "2025-04-24T17:09:40.999021381Z[Etc/UTC]",
  "errorMessage": "org.springframework.web.reactive.function.client.WebClientResponseExceptions$InternalServerError: 500 Internal Server Error from GET http://0473acd8d79b:8080/product/1",
  "numberOfAttempts": 1
}
{
  "retryName": "product",
  "type": "SUCCESS",
  "creationTime": "2025-04-24T17:09:42.016292965Z[Etc/UTC]",
  "errorMessage": "org.springframework.web.reactive.function.client.WebClientResponseExceptions$InternalServerError: 500 Internal Server Error from GET http://0473acd8d79b:8080/product/1",
  "numberOfAttempts": 1
}
$
```

Figura 13.8: Eventos de reintento capturados después de una solicitud con un reintento



Si realmente no tiene suerte, obtendrá 2 fallas seguidas y luego obtendrá un tiempo de respuesta de 2 segundos en lugar de 1. Si repite el comando anterior, podrá ver que el campo `numberOfAttempts` se cuenta para cada intento de reintento, que está configurado en 1 en este caso: `"numberOfAttempts": 1`. Si las llamadas continúan fallando, el disyuntor se activará y abrirá su circuito; es decir, las llamadas posteriores aplicarán la lógica de falla rápida y se aplicará el método de respaldo.

Con esto concluye el capítulo. Siéntete libre de experimentar con los parámetros en la configuración para comprender mejor los mecanismos de resiliencia.

No olvides apagar el sistema paisajístico:

```
Docker Compose Down
```

Resumen

En este capítulo, vimos Resilience4j y su disyuntor, limitador de tiempo y mecanismo de reintento en acción.

Un microservicio con dependencias síncronas con otros servicios puede dejar de responder o incluso bloquearse si estos servicios dejan de responder como se espera, especialmente bajo una carga alta. Este tipo de escenarios de error se pueden evitar mediante un disyuntor, que aplica lógica de fallo rápido e invoca métodos de respaldo cuando está abierto. Un disyuntor también puede hacer que un microservicio sea resiliente al permitir solicitudes cuando está medio abierto para verificar si el servicio que falla está operando normalmente de nuevo y cerrar el circuito en caso afirmativo. Para ayudar a un disyuntor a gestionar servicios que no responden, un limitador de tiempo se puede utilizar para maximizar el tiempo que espera un disyuntor antes de activarse.

Un mecanismo de reintento permite reintentar solicitudes que fallan aleatoriamente, por ejemplo, debido a problemas temporales de red. Es fundamental aplicar reintentos solo a servicios idempotentes, es decir, servicios que pueden procesar la misma solicitud dos o más veces.

Los interruptores automáticos y los mecanismos de reintento se implementan siguiendo las convenciones de Spring Boot: declarando dependencias y añadiendo anotaciones y configuración. Resilience4j expone información sobre sus interruptores automáticos y mecanismos de reintento en tiempo de ejecución, mediante endpoints de actuadores. Para los interruptores automáticos, se dispone de información sobre el estado, los eventos y las métricas. Para los reintentos, se dispone de información sobre los eventos y las métricas.

Hemos visto el uso de puntos finales tanto para la salud como para los eventos en este capítulo, pero tendremos que esperar hasta el Capítulo 20, Monitoreo de microservicios, antes de usar cualquiera de las métricas.

En el próximo capítulo, cubriremos la última parte del uso de Spring Cloud, donde aprenderemos cómo rastrear cadenas de llamadas a través de un conjunto de microservicios cooperativos utilizando Spring Cloud Sleuth y Zipkin.

¡Dirígete al Capítulo 14, Comprensión del seguimiento distribuido, para comenzar!

Preguntas

1. ¿Cuáles son los estados de un disyuntor y cómo se utilizan?
2. ¿Cómo podemos manejar errores de tiempo de espera en el disyuntor?
3. ¿Cómo podemos aplicar la lógica de respaldo cuando un disyuntor falla rápidamente?
4. ¿Cómo pueden interferir entre sí un mecanismo de reintento y un disyuntor?
5. Proporcione un ejemplo de un servicio al que no pueda aplicar un mecanismo de reintento.

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



14

Entendiendo la distribución Rastreo

En este capítulo, aprenderemos a usar el rastreo distribuido para comprender mejor cómo cooperan nuestros microservicios; por ejemplo, al atender una solicitud enviada a la API externa. Poder usar el rastreo distribuido es esencial para gestionar un entorno de sistemas de microservicios que cooperan. Como ya se describió en el Capítulo 8, Introducción a Spring Cloud, se usará Micrometer Tracing para recopilar información de rastreo, y Zipkin para almacenarla y visualizarla .

En este capítulo aprenderemos sobre los siguientes temas:

- Introducción al rastreo distribuido con Micrometer Tracing y Zipkin.
- Cómo agregar seguimiento distribuido al código fuente.
- Cómo agregar información a los rastros mediante programación.

Cómo realizar seguimiento distribuido, visualizando tanto las solicitudes de API exitosas como las fallidas .

Veremos cómo se puede visualizar el procesamiento síncrono y asíncrono.

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter14`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver el proceso para añadir el seguimiento distribuido con Micrometer Tracing y Zipkin, puede compararlo con el código fuente del Capítulo 13, "Mejora de la resiliencia con Resilience4j". Puede usar su herramienta de comparación preferida y comparar las carpetas \$BOOK_HOME/Chapter13 y \$BOOK_HOME/Chapter14.

Introducción al rastreo distribuido con Micrometer Rastreo y Zipkin

Para recapitular el Capítulo 8, Introducción a Spring Cloud, en referencia a la sección " Uso del Rastreo Micrométrico y Zipkin para el rastreo distribuido ", la información de rastreo de un flujo de trabajo completo se denomina rastreo o árbol de rastreo, y las subpartes del árbol (por ejemplo, las unidades básicas de trabajo) se denominan tramos. Los tramos pueden constar de subtramos que forman el árbol de rastreo. Se pueden añadir metadatos a un rastreo y a sus tramos como pares clave-valor denominados etiquetas. La interfaz de usuario de Zipkin permite visualizar un árbol de rastreo y sus tramos de la siguiente manera:



Figura 14.1: Ejemplo de una traza con sus tramos

El seguimiento de Micrometer se utiliza para recopilar información de seguimiento, propagar contextos de seguimiento (por ejemplo, identificadores de seguimiento y de intervalo) en llamadas a otros microservicios y exportar la información de seguimiento a herramientas de análisis de seguimiento como Zipkin. El manejo de la información de seguimiento se realiza de forma interna mediante un rastreador. Micrometer admite la configuración automática de rastreadores basada en OpenTelemetry (<https://opentelemetry.io/>) o OpenZipkin Brave (<https://github.com/openzipkin/brave>). La exportación de la información de seguimiento a una herramienta de análisis de seguimiento se realiza mediante un reportero. Antes de la existencia del estándar OpenTelemetry, el formato Zipkin para enviar información de seguimiento a las herramientas de análisis de seguimiento era muy común. Hoy en día, cada vez más herramientas de análisis de seguimiento admiten el formato OpenTelemetry. Con el tiempo, es probable que el estándar OpenTelemetry se convierta en el formato más utilizado.

De forma predeterminada, los encabezados de seguimiento se propagan entre microservicios mediante encabezados de contexto de seguimiento W3C (<https://www.w3.org/TR/trace-context/>) —y lo más importante, el encabezado `traceparent`—, pero se puede configurar para usar los encabezados B3 de OpenZipkin. En este capítulo, usaremos los encabezados de contexto de seguimiento del W3C. En el capítulo 18, usaremos los encabezados B3.

Un ejemplo de encabezado `traceparent` del contexto de seguimiento del W3C se ve así:

```
traceparent: "00-2425f26083814f66c985c717a761e810-fbec8704028cfb20-01"
```

El valor del encabezado `traceparent` contiene cuatro partes, separadas por un "-":

- "00" indica la versión utilizada. Siempre será "00" con la especificación actual.
- "124...810" es el ID de seguimiento.
- "fbe...b20" es el identificador del intervalo.
- "01", la última parte, contiene varios indicadores. El único indicador compatible con la especificación actual es "muestreado", con el valor "01". Esto significa que quien llama está registrando los datos de seguimiento de esta solicitud. Configuraremos nuestros microservicios para que registren los datos de seguimiento de todas las solicitudes, por lo que este indicador siempre tendrá el valor "01".

El uso de los encabezados Brave B3 de OpenZipkin se verá así:

```
X-B3-TraceId: "64436ea679e8eb6e6fa028bb3459e703"
```

```
X-B3-SpanId: "120678270898ddd5"
```

```
X-B3-ParentSpanId: "3c431d3d01987c22"
```

```
X-B3-Muestreo: "1"
```

Los nombres de los encabezados se explican por sí solos, y podemos ver que los encabezados no solo se proporcionan para el ID de seguimiento y de intervalo, sino también para el ID del intervalo principal.

Spring Boot crea automáticamente seguimientos y tramos para el tráfico entrante, tanto para las solicitudes HTTP entrantes como para los mensajes recibidos por Spring Cloud Stream. Si una solicitud entrante contiene un ID de seguimiento, este se utilizará al crear tramos; de lo contrario, se creará uno nuevo. Los ID de seguimiento y tramo se propagan automáticamente al tráfico saliente, ya sea como solicitudes HTTP o mediante el envío de mensajes mediante Spring Cloud Stream.

Si es necesario, se puede añadir información de seguimiento adicional mediante programación, ya sea añadiendo intervalos personalizados o etiquetas personalizadas a todos los intervalos creados por un microservicio. Esto se realiza mediante Micrometer Observability (<https://micrometer.io/docs/observation>). y su API de observación.

La versión inicial de Micrometer Tracing, junto con Spring Boot 3, presentó algunas limitaciones para la compatibilidad con el rastreo distribuido en clientes reactivos. Esto afecta a los microservicios utilizados en este libro que utilizan Project Reactor. En la sección "Añadir soluciones alternativas para la falta de compatibilidad con clientes reactivos", aprenderemos a mitigar estas deficiencias.

Zipkin incluye soporte nativo para almacenar información de seguimiento, ya sea en memoria o en una base de datos como Apache Cassandra, Elasticsearch o MySQL. Además, dispone de varias extensiones. Para más información, consulte https://zipkin.io/pages/extensions_choices.html. En este capítulo, almacenaremos la información de seguimiento en la memoria.

Con la introducción de Micrometer Tracing y Zipkin, veamos qué cambios se requieren en el código fuente para habilitar el rastreo distribuido.

Añadiendo seguimiento distribuido al código fuente

En esta sección, aprenderemos a actualizar el código fuente para habilitar el rastreo distribuido. Esto se puede hacer siguiendo estos pasos:

1. Agregue dependencias a los archivos de compilación para incorporar Micrometer Tracing con una implementación de trazador y un reportero.
2. Agregue un servidor Zipkin a los archivos de Docker Compose.
3. Configure los microservicios para enviar información de seguimiento a Zipkin.
4. Agregar soluciones alternativas a la falta de soporte de clientes reactivos.
5. Agregue código para crear intervalos personalizados y etiquetas personalizadas en intervalos existentes.

Repasaremos cada uno de los pasos por turno.

Agregar dependencias a los archivos de compilación

Para poder utilizar Micrometer Tracing y la capacidad de exportar información de seguimiento a Zipkin, necesitamos agregar una dependencia al trazador y al reportero seleccionados a los archivos de compilación del proyecto Gradle, `build.gradle`.

Esto se logra agregando las siguientes dos líneas:

```
Implementación 'io.micrometer:micrometer-tracing-bridge-otel'  
implementación 'io.opentelemetry:opentelemetry-exporter-zipkin'
```

Para el servicio de revisión, también se ha añadido una dependencia que permite el seguimiento de la información relativa a las operaciones de la base de datos SQL. Su aspecto es el siguiente:

```
implementación 'net.ttddyy.observation.datasource-micrometer-spring-boot:1.0.0'
```

Esta biblioteca puede crear intervalos para las operaciones SQL ejecutadas por el servicio de revisión. Estos intervalos contendrán información sobre la consulta SQL ejecutada y su tiempo de ejecución.

Agregar configuración para Micrometer Tracing y Zipkin

La configuración para usar Micrometer Tracing y Zipkin se ha añadido al archivo de configuración común, config-repo/application.yml.

En el perfil predeterminado, se especifica que la información de seguimiento se enviará a Zipkin mediante la siguiente URL:

```
gestión.zipkin.tracing.punto final: http://zipkin:9411/api/v2/spans
```

De forma predeterminada, Micrometer Tracing solo envía el 10 % de los rastros a Zipkin. Para garantizar que todos los rastros se envíen a Zipkin, se añade la siguiente propiedad al perfil predeterminado:

```
gestión.rastreo.muestreo.probabilidad: 1.0
```

También queremos que los identificadores de seguimiento y de intervalo se escriban en los registros; esto nos permitirá correlacionar la salida de registros de los microservicios cooperativos que, por ejemplo, cumplen una solicitud enviada a la API externa.



Investigaremos cómo utilizar esto en el Capítulo 19, Registro centralizado con la pila EFK.

Los identificadores de seguimiento y de intervalo se escriben en los registros de forma predeterminada cuando se utiliza el seguimiento micrométrico. Para simplificar el análisis de la salida del registro en el Capítulo 19, Registro centralizado con la pila EFK, se ha personalizado el formato del registro con la siguiente configuración:

```
registro.patrn.correlación: "[${spring.application.name:},  
%X{traceId:-},%X{spanId:-}] "  
logging.include-nombre-de-aplicación: falso
```

Para obtener más información sobre cómo personalizar el formato de registro para la información de seguimiento, consulte <https://docs.spring.io/spring-boot/reference/actuator/tracing.html#actuator.micrometer-tracing.logging>.



Al usar Spring Cloud Configuration Server, la propiedad `logging.include-application-name` debe ubicarse en el archivo `bootstrap.yml`, dentro de la carpeta `src/main/resources` de cada microservicio. Para mantener unidas las dos nuevas propiedades de registro, también se ha incluido allí la propiedad `logging.pattern.correlation`. En el Capítulo 17, donde Spring Cloud Configuration Server se reemplaza por las características correspondientes de Kubernetes, estas propiedades se moverán al archivo de propiedades común `config-repo/application.yml`.

Con el formato de registro anterior, la salida del registro se verá así:

```
2025-04-22T14:02:07.417Z INFO ... [producto-compuesto,01234,56789] ...
```

Donde `product-composite` es el nombre del microservicio, `01234` es el ID de seguimiento y `56789` es el ID del intervalo.

Para reducir la salida del registro, también cambiamos el nivel de registro de `DEBUG` a `INFO` para el archivo de configuración de cada microservicio en el repositorio de configuración. Esto facilita la verificación de que los ID de seguimiento y de intervalo se agreguen correctamente. Este cambio se aplica con la siguiente línea:

```
se.magnus: INFORMACIÓN
```

Para el microservicio compuesto de producto, el nivel de registro de la clase `HttpWebHandlerAdapter` se ha cambiado de `TRACE` a `INFO` por el mismo motivo:

```
org.springframework.web.server.adapter.HttpWebHandlerAdapter: INFORMACIÓN
```

Agregar Zipkin a los archivos de Docker Compose

Para ejecutar el servidor Zipkin como contenedor Docker, usaremos una imagen de Docker, `openzipkin/zipkin`, publicada por el proyecto Zipkin. Consulte <https://hub.docker.com/r/openzipkin/zipkin> Para más detalles, consulte la definición del servidor Zipkin:

```
cremallera:
  imagen: openzipkin/zipkin:3.5.0
  reiniciar: siempre
  límite de memoria: 1024m
  ambiente:
    - TIPO_DE_ALMACENAMIENTO=mem
  puertos:
    - 9411:9411
```

Expliquemos el código fuente anterior:

- Se especifica que la versión de la imagen de Docker, openzipkin/zipkin, es la versión 3.5.0.
- La variable de entorno `STORAGE_TYPE=mem` se utiliza para especificar que Zipkin mantendrá todos rastrear información en la memoria.

El límite de memoria de Zipkin se ha incrementado a 1024 MB, en comparación con los 512 MB de los demás contenedores. Esto se debe a que, dado que Zipkin está configurado para guardar toda la información de seguimiento en memoria, con el tiempo consumirá más memoria que los demás contenedores.

En caso de que Zipkin se quede sin memoria y se detenga, hemos aplicado una política de reinicio que solicita al motor Docker que siempre reinicie el contenedor. Esto se aplicará tanto si el contenedor falla como si se reinicia el motor Docker.

- Zipkin expone el puerto HTTP 9411 para que los navegadores web accedan a su interfaz de usuario web.



Si bien está bien almacenar la información de seguimiento en Zipkin en la memoria para actividades de desarrollo y prueba, Zipkin debe configurarse para almacenar la información de seguimiento en una base de datos en un entorno de producción.

Agregar soluciones alternativas a la falta de soporte de clientes reactivos

Como se mencionó anteriormente, las versiones actuales de Spring Boot, Project Reactor y Micrometer Tracing aún no funcionan a la perfección. Por lo tanto, se han aplicado un par de soluciones alternativas al código fuente de los clientes reactivos: los cuatro microservicios y la puerta de enlace. Los problemas se relacionan principalmente con la complejidad de propagar contextos de seguimiento (por ejemplo, los identificadores de seguimiento y de intervalo) entre los diferentes subprocesos involucrados en el procesamiento asíncrono reactivo, especialmente si partes del procesamiento implican procesamiento síncrono imperativo.



Si todo el procesamiento de una solicitud se realiza con una implementación sincrónica, utilizando un mismo hilo para todo el procesamiento, la propagación del contexto de seguimiento no es un problema. La variable `ThreadLocal` se puede usar para almacenar el contexto de seguimiento. El contexto de seguimiento puede... se recuperará de la variable `ThreadLocal` en cualquier lugar de la implementación ya que todo el código se ejecuta en un mismo hilo.

Un ejemplo de esta combinación de procesamiento reactivo e imperativo es la implementación del microservicio de revisión, donde las llamadas a la base de datos SQL subyacente se realizan de forma sincrónica.



Si le interesa comprender los desafíos con más detalle, consulte la serie de blogs de tres partes «Uniendo los caminos reactivos e imperativos», publicada por el equipo de Spring. Puede encontrarla aquí: [https://spring.io/blog/2023/03/30/Propagación de contexto con el proyecto Reactor 3: puente unificado entre reactivo](https://spring.io/blog/2023/03/30/Propagación-de-contexto-con-el-proyecto-Reactor-3:puente-unificado-entre-reactivo).

Cabe destacar que el predecesor de Micrometer Tracing, Spring Cloud Sleuth, ofrece mejor compatibilidad con clientes reactivos que las versiones actuales de Spring Boot, Project Reactor y Micrometer Tracing. Esperamos que las próximas versiones solucionen pronto estas deficiencias.



Si tiene una base de código que ya utiliza Spring Cloud Sleuth, puede encontrar pautas de migración aquí: [https://github.com/micrometer-metrics/tracing/wiki/Guía de migración de Spring Cloud Sleuth 3.1](https://github.com/micrometer-metrics/tracing/wiki/Guía-de-migración-de-Spring-Cloud-Sleuth-3.1).

Para abordar muchos de los desafíos de la propagación de contexto, podemos activar la propagación automática de contexto llamando al método `Hooks.enableAutomaticContextPropagation()` en el método `main()` de un cliente reactivo .

Para más detalles, consulte la serie de blogs "Unificación de puentes entre reactivo e imperativo", mencionada anteriormente. Para el servicio compuesto por producto, el proceso es el siguiente:

```
público estático void main(String[] args) {
    Ganchos.enableAutomaticContextPropagation();
    SpringApplication.run(ProductCompositeServiceApplication.class, argumentos);
}
```

Sin embargo, para el servicio compuesto por producto , persiste un problema. Para garantizar que una instancia de WebClient esté correctamente instrumentada para la observación (por ejemplo, para poder propagar los identificadores de seguimiento y de intervalo actuales como encabezados en una solicitud saliente), se espera que la instancia de WebClient.Builder se inyecte mediante cableado automático. Desafortunadamente, al usar Eureka para el descubrimiento de servicios , se recomienda crear la instancia de WebClient.Builder como un bean anotado con `@LoadBalanced` , como se indica a continuación:

```
@Frijol
@LoadBalanced
público WebClient.Builder loadBalancedWebClientBuilder() {
    devolver WebClient.builder();
}
```

Existe un conflicto al crear una instancia de WebClient al usar Eureka y Micrometer Tracing. Para resolverlo, se puede reemplazar el bean `@LoadBalanced` por una función de filtro de intercambio compatible con balanceadores de carga:

`ReactorLoadBalancerExchangeFilterFunction`.

Se puede configurar una función de filtro de intercambio en una instancia de `WebClient.Builder` con cableado automático de la siguiente manera:

```
@Autowired
Función privada ReactorLoadBalancerExchangeFilterFunction lbFunction;

@Frijol
público WebClient webClient(WebClient.Builder constructor) {
    devuelve constructor.filter(lbFunction).build();
}
```

Esto significa que la clase de aplicación, `ProductCompositeServiceApplication`, registra un bean `WebClient` en lugar de un bean `WebClient.Builder`. Esto afecta a la clase `ProductCompositeIntegration`. Ahora necesita conectar automáticamente un bean `WebClient` en lugar de un bean `WebClient.Builder`.

Para obtener acceso a la función `ReactorLoadBalancerExchangeFilterFunction`, se ha agregado una dependencia de `org.springframework.cloud:spring-cloud-starter-loadbalancer` al archivo de compilación, `build.gradle`.

Agregar intervalos personalizados y etiquetas personalizadas a intervalos existentes

Además de utilizar la compatibilidad integrada para crear trazas y intervalos durante el procesamiento continuo, también podemos usar la API de Observación de Micrometer `Observability` para, por ejemplo, añadir nuestros propios intervalos o añadir etiquetas personalizadas a intervalos existentes creados por un microservicio. La API de Observación se implementa tras una interfaz Java del mismo nombre.

Veamos primero cómo agregar un intervalo personalizado y, después, cómo podemos agregar etiquetas personalizadas a todos los intervalos creados por un microservicio.

Para que cualquier observación se realice mediante la API de Observación, los objetos de observación deben registrarse en un bean `ObservationRegistry`. Un bean `ObservationRegistry` puede inyectarse mediante autoconexión, como en el siguiente ejemplo:

```
@Componente
clase pública MiComponente {

    registro privado final de observación ;

    público MyComponent( Registro de Observación registro) {
        este.registro = registro;
    }
}
```

Agregar un lapso personalizado

Para agregar un intervalo personalizado, la interfaz `Observation` proporciona un método estático, `createNotStarted()`, que permite crear un intervalo. Para ejecutarlo, se puede usar el método `observe()` para proporcionar el código que debe cubrir. Si el código no devuelve ningún valor, se puede especificar como una función ejecutable ; de lo contrario, se debe especificar como una función proveedora .

Un ejemplo de la creación de un lapso personalizado para la ejecución de una función de Proveedor minimalista.

La acción se ve así:

```
int y = Observation.createNotStarted("mi observación", registro)
    .observar() -> {
        int x = 1;
        LOG.info(" Devolverá {}", x);
        devuelve 1;
    };
LOG.info("Obtuve {}", y);
```

El parámetro de registro es un bean de tipo `ObservationRegistry` y se conecta automáticamente como se describe más temprano.

La ejecución de este código dará como resultado un registro como este:

```
AAAA-MM-DDT14:02:07.417Z INFO 1 ---           paralelo-6] [producto-
[ compuesto,9761b2b2b2da59c5096e78252c48ab3d,d8bcbd9cde9fe2d7]
smmcpsProductCompositeServiceImpl : Devolverá 1
AAAA-MM-DDT14:02:07.417Z INFO 1 ---           paralelo-6] [producto-
[compuesto,9761b2b2b2da59c5096e78252c48ab3d,4c8ea2820fb74ec9]
smmcpsProductCompositeServiceImpl : Obtuvo 1
```

Desde la salida del registro, podemos ver que ambas declaraciones de registro hacen referencia al mismo ID de seguimiento, 9761b2b2b2da59c5096e78252c48ab3d, pero especifica diferentes ID de intervalo, siendo d8bcbd9cde9fe2d7 el ID de intervalo del intervalo personalizado.

Si deseamos agregar metadatos sobre el lapso, podemos especificarlo añadiendo un nombre contextual y un conjunto de etiquetas como pares clave-valor. El nombre contextual será el nombre del lapso y puede usarse para identificarlo al visualizar un árbol de seguimiento en Zipkin. Si los posibles valores de la información están limitados a un número finito de alternativas, la etiqueta debe especificarse mediante el método `lowCardinalityKeyValue()` . Para valores ilimitados, debe usarse el método `highCardinalityKeyValue()` . Al llamar a estos métodos, se agregarán etiquetas al lapso actual, mientras que los valores de baja cardinalidad también se etiquetarán con las métricas creadas por la observación.



Investigaremos cómo utilizar métricas en el Capítulo 20, Monitoreo de microservicios.

Un ejemplo de cómo especificar información como etiquetas se ve así:

```
int y = Observation.createNotStarted("mi observación", registro)
    .contextualName(" información del producto")
    .lowCardinalityKeyValue("url-de-plantilla",
                           "/producto-compuesto/{ID-producto}")
    .highCardinalityKeyValue("url-actual",
                             "/producto-compuesto/12345678")
    .observar() -> {
        int x = 1;
        LOG.info(" Devolverá {}", x);
        devuelve x;
    };
LOG.info("Obtuve {}", y);
```

Del ejemplo anterior podemos ver lo siguiente:

- El nombre contextual se establece como "información del producto".
- Una clave, "template-url", con pocos valores posibles, se especifica con el método `lowCardinalityKeyValue()`.
En nuestro caso, solo tiene tres valores posibles para los métodos `create`, `get` y `delete`.
- Se ha especificado una clave, "actual-url", con una cantidad ilimitada de valores según el `productId` especificado, con el método `highCardinalityKeyValue()`.

Apliquemos esto al servicio compuesto de producto para crear un intervalo personalizado que proporcione `productId` como etiqueta. Dado que el número de ID de producto es ilimitado, lo especificaremos mediante el método `highCardinalityKeyValue()`. Crearemos un intervalo que especifique el ID de producto actual para cada uno de los tres métodos de la API para crear, recuperar y eliminar productos compuestos. La creación de un intervalo personalizado que contenga una etiqueta de alta cardinalidad se gestiona mediante una clase de utilidad, `ObservationUtil`. Esta clase se encuentra en `se.magnus.microservices.composite`.

Paquete `product.services.tracing`. El método de utilidad `observe()` de esta clase tiene el siguiente aspecto:

```
público <T> T observar(String nombreObservación, String nombreContextual,
                      Cadena highCardinalityKey, Cadena highCardinalityValue,
```

```

Proveedor<T> proveedor) {
    devolver Observation.createNotStarted(observacionName, registro)
        .nombreContextual(nombreContextual)
        .highCardinalityKeyValue(clavedecardinalidadalta, valordecardinalidadalta)
        .observe(proveedor);
}

```

El método observe() encapsula la llamada al método Observation.createNotStarted(). El uso de este método se explica en el ejemplo anterior, por lo que no necesita más explicación.

Este método de utilidad se utiliza en la clase ProductCompositeServiceImpl mediante un método auxiliar, observaciónWithProductInfo(), que aplica valores comunes a la Clase ProductCompositeServiceImpl :

```

privado <T> T observaciónConInfoProducto(int InformaciónProducto, Proveedor<T>
proveedor) {
    devolver observaciónUtil.observe(
        "observación compuesta",
        "información del producto",
        "ID del producto",
        Cadena.valueOf(productInfo),
        proveedor);
}

```

Finalmente, el método auxiliar es utilizado por los tres métodos de la API: createProduct(), getProduct() y deleteProduct(). El intervalo personalizado se crea envolviendo el código existente en cada método. El código existente se ha trasladado a los métodos "internos" correspondientes para simplificar la estructura de la solución. Estos métodos se invocan mediante el método "observaciónWithProductInfo()" . Para getProduct(), la implementación ahora se ve así:

```

público Mono<ProductAggregate> obtenerProducto(int productId, ...) {
    devolver observaciónConProductInfo(productId,
        () -> getProductInternal(productId, ...));
}

privado Mono<ProductAggregate> getProductInternal(int productId, ...) {

    devolver observaciónConProductInfo(productId, () -> {

```

```
LOG.info(" Obtendrá información compuesta del producto para product.id={}",
ID del producto);
devolver Mono.zip(
    valores -> createProductAggregate(...
    integración.getProduct(productId, ...),
    integración.getRecommendations(productId).collectList(),
    integración.getReviews(productId).collectList())
    ...);
}
```

Si lo comparamos con la implementación correspondiente en el Capítulo 13, podemos ver que los cambios necesarios para crear un intervalo personalizado se limitan a agregar un nuevo método "interno" y llamarlo desde el método `observerWithProductInfo()` :

```
público Mono<ProductAggregate> obtenerProducto(int productId, ...) {

LOG.info(" Obtendrá información compuesta del producto para product.id={}",
ID del producto);
devolver Mono.zip(
    valores -> createProductAggregate(...
    integración.getProduct(productId, ...),
    integración.getRecommendations(productId).collectList(),
    integración.getReviews(productId).collectList())
    ...);
}
```

Por lo tanto, añadir un intervalo personalizado al código existente se puede realizar con un cambio mínimo, siempre que se gestione la configuración de un método de utilidad adecuado. Cuando probemos el rastreo distribuido más adelante en este capítulo, veremos este intervalo personalizado en acción. Con un intervalo personalizado, veamos cómo podemos añadir etiquetas personalizadas a cualquier intervalo creado en un microservicio.

Agregar etiquetas personalizadas a intervalos existentes

Si queremos añadir información personalizada a todos los intervalos creados por un microservicio, podemos usar un `ObservationFilter`. Debe registrarse en el bean `ObservationRegistry` mediante un bean `ObservationRegistryCustomizer` .

Apliquemos un filtro que registre la versión actual del microservicio compuesto por producto como etiqueta en cada intervalo que cree. Necesitamos hacer lo siguiente:

- Actualice el archivo de compilación para que Gradle cree información de compilación, incluida la especificación de la versión actual. ificado por la propiedad de versión en el archivo build.gradle
- Cree un filtro que agregue la versión actual como una etiqueta de baja cardinalidad a todos los intervalos
- Cree un bean de configuración de registro que registre el filtro

Para que Gradle cree información de compilación, se agrega lo siguiente al archivo de compilación, build.gradle:

```
springBoot
    { información de construcción()
}
```

Esta adición generará el archivo build/resources/main/META-INF/build-info.properties al ejecutar el comando de compilación ./gradlew . Este archivo especificará la versión actual de la siguiente manera:

```
build.version=1.0.0-SNAPSHOT
```

El archivo de información de compilación se incluirá en el archivo JAR del microservicio, y se podrá acceder a su información mediante un bean BuildProperties .

El filtro se ve así:

```
clase pública BuildInfoObservationFilter implementa ObservationFilter {

    propiedades de construcción finales privadas propiedades de construcción;

    público BuildInfoObservationFilter(BuildProperties buildProperties) {
        esto.buildProperties = buildProperties;
    }

    @Override
    público Observation.Context mapa(final Observation.Context contexto) {
        KeyValuebuildVersion = KeyValue.of("build.version",
                                           buildProperties.getVersion());
        devolver contexto.addLowCardinalityKeyValue(buildVersion);
    }
}
```

Del código fuente anterior, podemos ver lo siguiente:

- Se inyecta un bean `BuildProperties` en el constructor del filtro
- El método `map()` del filtro recupera la versión del microservicio usando `BuildProperties`

bean y lo establece como una etiqueta de baja cardinalidad en el contexto de observación proporcionado

El bean de configuración del registro se ve así:

```
@Configuration(proxyBeanMethods = falso)
la clase pública ObservationRegistryConfig implementa
ObservationRegistryCustomizer<ObservationRegistry> {

    propiedades de construcción finales privadas propiedades de construcción;

    público ObservationRegistryConfig(BuildProperties buildProperties) {
        esto.buildProperties = buildProperties;
    }

    @Anular
    public void customize(final ObservationRegistry registro) {
        registro.observationConfig().observationFilter(nuevo
        BuildInfoObservationFilter(propiedades de construcción));
    }
}
```

Del código fuente anterior, podemos aprender lo siguiente:

- También se inyecta un bean `BuildProperties` en el constructor de la clase de configuración.
- En el método `customize()`, se crea y registra el filtro. El bean `BuildProperties` también se inyecta aquí.

El filtro y el bean de configuración de registro se pueden encontrar en `se.magnus.microservices`.

Paquete `composite.product.services.tracing`. Cuando probemos el rastreo distribuido más adelante en este capítulo, veremos este filtro de observación en acción.

Para conocer más formas de gestionar intervalos personalizados (por ejemplo, configurar predicados para determinar cuándo aplicar un filtro de observación o usar anotaciones para describir una observación), consulte <https://micrometer.io/docs/observación>.

Eso es lo que se necesita para agregar seguimiento distribuido usando Micrometer Tracing y Zipkin, ¡así que probémoslo en la siguiente sección!

Probando el rastreo distribuido

Con los cambios necesarios en el código fuente implementados, podemos probar el rastreo distribuido. Para ello, seguiremos los siguientes pasos:

1. Construir, iniciar y verificar el panorama del sistema.
 2. Envíe una solicitud de API exitosa y vea qué información de seguimiento podemos encontrar en Zipkin relacionada con esta solicitud de API.
 3. Envíe una solicitud de API fallida y vea qué información de error podemos encontrar.
 4. Envíe una solicitud de API exitosa que active el procesamiento asíncrono y vea cómo se rastrea.
- La información está representada.

Discutiremos estos pasos en detalle en las próximas secciones.

Puesta en marcha del panorama del sistema

Comencemos con el entorno del sistema. Construya las imágenes de Docker con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 14
./gradlew compilación y compilación de docker compose
```

Inicie el paisaje del sistema en Docker y ejecute las pruebas habituales con el siguiente comando:

```
./test-em-all.bash inicio
```

Antes de poder llamar a la API, necesitamos un token de acceso. Ejecute los siguientes comandos para obtenerlo:

```
desconfigurar ACCESS_TOKEN
TOKEN DE ACCESO=$(curl -k https://escritor:escritor-secreto@localhost:8443/
oauth2/token -d tipo_concesión=credenciales_del_cliente -d ámbito="producto:lectura
producto:escritura" -s | jq -r .token_de_acceso)
eco $ACCESS_TOKEN
```



Como se mencionó en capítulos anteriores, un token de acceso emitido por el servidor de autorización tiene una validez de una hora. Por lo tanto, si después de un tiempo empieza a recibir errores 401 No autorizado, probablemente sea momento de adquirir un nuevo token de acceso.

Envío de una solicitud de API exitosa

Ahora estamos listos para enviar una solicitud normal a la API. Ejecute el siguiente comando:

```
curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/  
producto-compuesto/1 -w "%{http_code}\n" -o /dev/null -s
```

Espere que el comando devuelva el código de estado HTTP en caso de éxito, 200.

Ahora podemos iniciar la interfaz de usuario de Zipkin para ver qué información de seguimiento se ha enviado a Zipkin:

1. Abra la siguiente URL en su navegador web: <http://localhost:9411/zipkin/>.
2. Para encontrar la información de seguimiento de nuestra solicitud, podemos buscar los seguimientos que hayan pasado por el servicio de puerta de enlace . Siga estos pasos:
 - a. Haga clic en el signo más grande (un signo + blanco sobre un fondo rojo) y seleccione serviceName y luego puerta de enlace.
 - b. Haga clic en el botón EJECUTAR CONSULTA .
 - c. Haga clic en el encabezado Hora de inicio para ver los resultados ordenados del más reciente al más reciente (debería verse una flecha hacia abajo a la izquierda del encabezado Hora de inicio).

La respuesta al encontrar rastros debería verse como la siguiente captura de pantalla:

The screenshot shows the Zipkin web interface. At the top, there's a navigation bar with the Zipkin logo, a search bar, and links for 'Find a trace', 'Dependencies', and 'ENGLISH'. Below the navigation bar, there's a search input field containing 'serviceName gateway' and a red '+' button. To the right of the search field is a 'RUN QUERY' button. Below the search field, there's a section for '10 Results' with buttons for 'EXPAND ALL' and 'COLLAPSE ALL'. The results are displayed in a table with columns: 'Root', 'Start Time', 'Spans', and 'Duration'. The 'Start Time' column is sorted by 'Start Time' (indicated by a red box around the 'Start Time' header and a red box around the 'Start Time' column header). The table shows three results for the 'gateway' service, each with a 'SHOW' button.

Root	Start Time	Spans	Duration
gateway: http get	a minute ago (04/23 11:13:30.849)	21	26.751ms
gateway: http post	a minute ago (04/23 11:13:20.613)	12	22.687ms
gateway: http get	2 minutes ago (04/23 11:12:17.377)	20	37.461ms

Figura 14.2: Búsqueda de rastros distribuidos usando Zipkin

3. La información de seguimiento de nuestra solicitud API anterior es la primera en la lista. Haga clic en su botón MOSTRAR para ver detalles relacionados con el seguimiento:

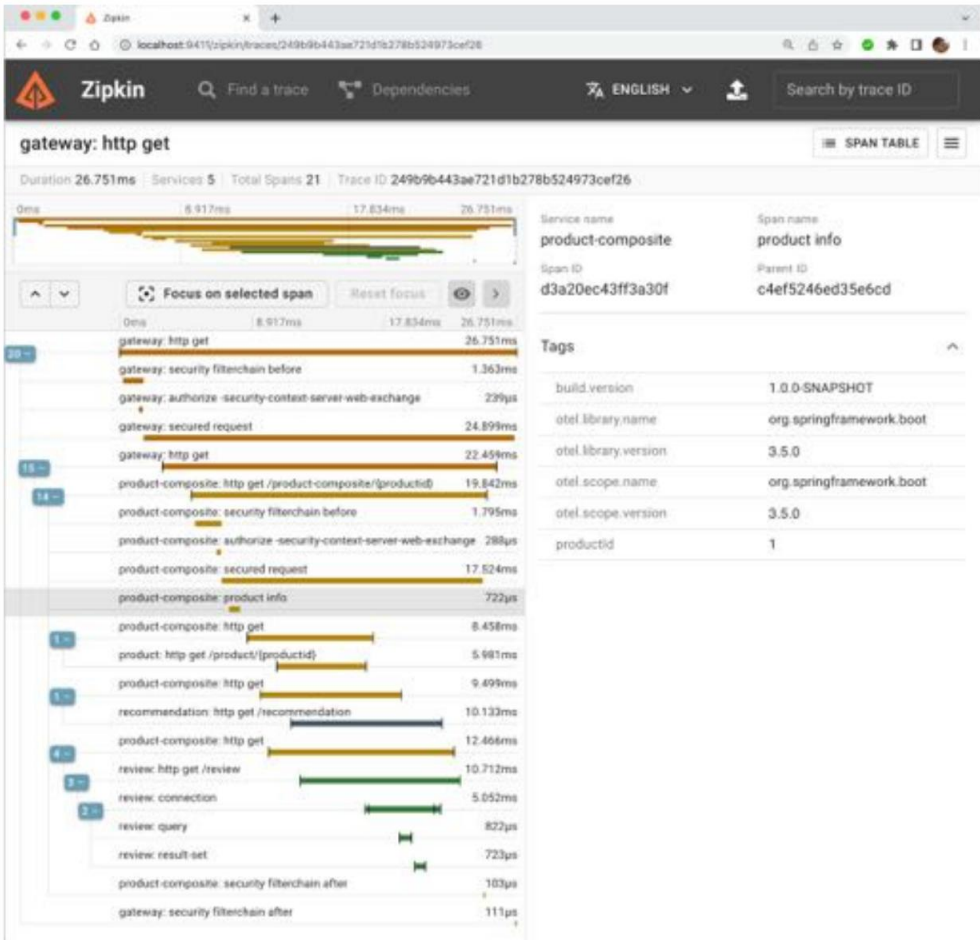


Figura 14.3: Muestra de traza distribuida visualizada en Zipkin

En la vista de información detallada, podemos observar lo siguiente:

- a. La solicitud fue recibida por el servicio de puerta de enlace .
- b. El servicio de puerta de enlace delegó el procesamiento de la solicitud al producto-servicio compuesto .
- c. El servicio compuesto por producto , a su vez, envió tres solicitudes paralelas a los servicios principales: producto, recomendación y reseña. Consulte los intervalos denominados "product-com-posite: http get".

- d. Una vez que el servicio compuesto de producto recibió la respuesta de los tres servicios principales, creó una respuesta compuesta y la envió de vuelta al llamador a través del servicio de puerta de enlace.
 - e. El intervalo personalizado creado en la sección anterior se llama product-composite: product info. Haga clic en él para ver sus etiquetas. En la vista de detalles a la derecha, podemos ver tanto la etiqueta creada por el intervalo personalizado, productId = 1, como la etiqueta creada por el filtro de observación build.version = 1.0.0-SNAPSHOT.
 - f. Para verificar que la etiqueta creada por el filtro de observación funciona como se espera, haga clic en otros intervalos creados por el servicio compuesto por producto y verifique que la compilación. La versión está presente.
4. Seleccione el intervalo denominado revisión: consulta para ver un intervalo informado por el microservicio de revisión capa de base de datos:

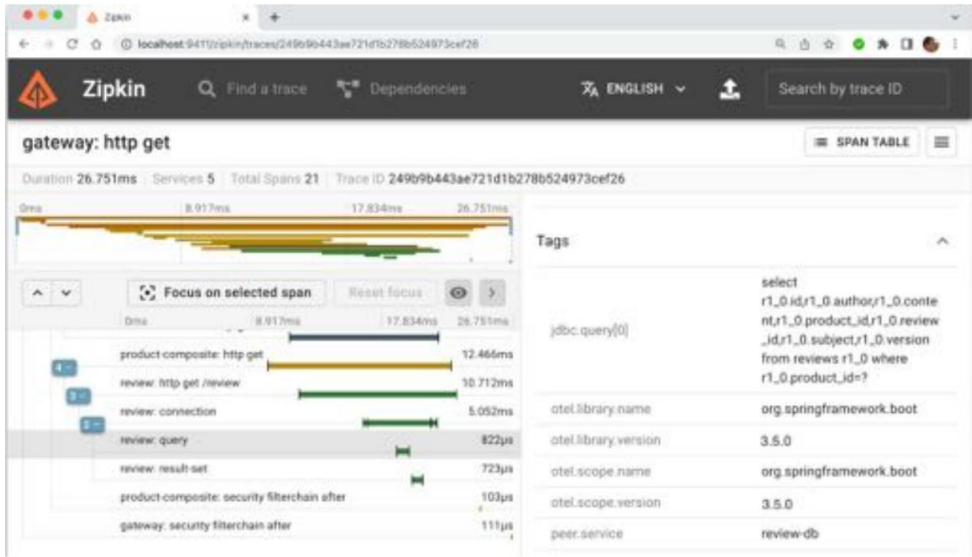


Figura 14.4: Un lapso que describe la ejecución de una consulta SQL

5. En la lista de etiquetas del intervalo, podemos ver la consulta SQL que ha realizado. También podemos Observa su tiempo de ejecución de 0,8 ms. ¡Información muy valiosa!

Para comprender mejor cómo se propagan los ID de seguimiento y de intervalo entre microservicios, podemos cambiar la configuración de registro del servicio compuesto por producto para que los encabezados HTTP de las solicitudes salientes se escriban en su registro. Esto se puede lograr siguiendo estos pasos:

1. Agregue las siguientes dos líneas al archivo de configuración `config-repo/product-composite.yml`

archivo:

```
spring.codec.log-request-details: verdadero
logging.level.org.springframework.web.reactive.function.client:
Funciones de intercambio: TRACE
```

2. Las dos líneas ya existen en el archivo de configuración, pero están comentadas. Van precedidas del siguiente comentario:

```
# Para ver los encabezados de seguimiento, descomente las dos líneas siguientes y reinicie
el servicio compuesto del producto
```

3. Busque el comentario anterior en el archivo de configuración y descomente las dos líneas siguientes: Bájalo.

4. Después de eso, reinicie el servicio compuesto por producto :

```
docker compose reiniciar producto-compuesto
```

5. Muestra la salida del registro del servicio compuesto por producto :

```
registros de docker compose -f --tail 0 producto-compuesto
```

6. Vuelva a ejecutar la solicitud curl anterior y verá la salida del registro que contiene el encabezado HTTP `traceparent` mencionado anteriormente; por ejemplo, una solicitud enviada a Servicio de recomendación :

```
producto-compuesto-1 | AAAA-MM-DDT09:24:50.849Z TRAZA 1 --- [ paralelo-2]
[producto-compuesto,e1420dcc38901378e888b8ce7022510e,0686
7b65cf84b552] oswrfclient.Funciones de intercambio: [14606b71]
HTTP GET http://d40874197b77:8080/recomendación?productId=1,
encabezados=[traceparent:"00-
e1420dcc38901378e888b8ce7022510e-06867b65cf84b552-01"]
```

7. En la salida del registro de muestra, podemos ver el valor del encabezado HTTP `traceparent` , donde el ID de seguimiento se establece en `e1420dcc38901378e888b8ce7022510e` y el ID de intervalo se establece en `06867b65cf84b552`.
8. Si no desea conservar el registro del encabezado HTTP `traceparent` , comente las dos líneas en `config-repo/product-composite.yml` y reinicie el servicio `product-composite` .

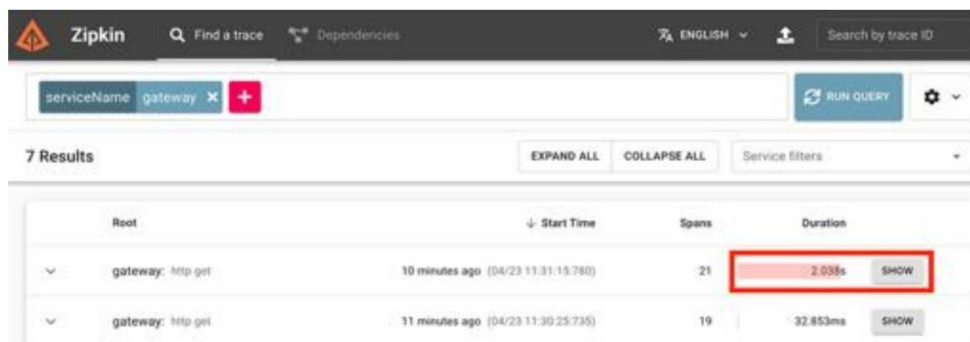
Envío de una solicitud de API fallida

Veamos cómo se ve la información de seguimiento si realizamos una solicitud de API fallida; por ejemplo, al buscar un producto que provoca un tiempo de espera:

1. Envíe una solicitud de API para el ID de producto 1, aplique un retraso de tres segundos, que se activará el limitador de tiempo y verificar que devuelva el código de estado HTTP 500:

```
curl -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/producto-compuesto/1?delay=3 -w "%{http_code}\n" -o /dev/null -s
```

2. En la interfaz de Zipkin, regrese a la página de búsqueda (use el botón `Atrás` del navegador web) y haga clic de nuevo en el botón `EJECUTAR CONSULTA`. Para ver los resultados ordenados del más reciente al más reciente, haga clic en el encabezado `Hora de inicio`. Obtendrá un resultado similar a la siguiente captura de pantalla:



7 Results

Root	Start Time	Spans	Duration
gateway: http get	10 minutes ago (04/23 11:31:15.780)	21	2.038s
gateway: http get	11 minutes ago (04/23 11:30:25.735)	19	32.853ms

Figura 14.5: Encontrar una solicitud fallida usando Zipkin

3. Debería ver la solicitud fallida en la parte superior de la lista de resultados. Tenga en cuenta que su barra de duración es... Rojo, indica que se ha producido un error. Haga clic en el botón MOSTRAR para ver los detalles.

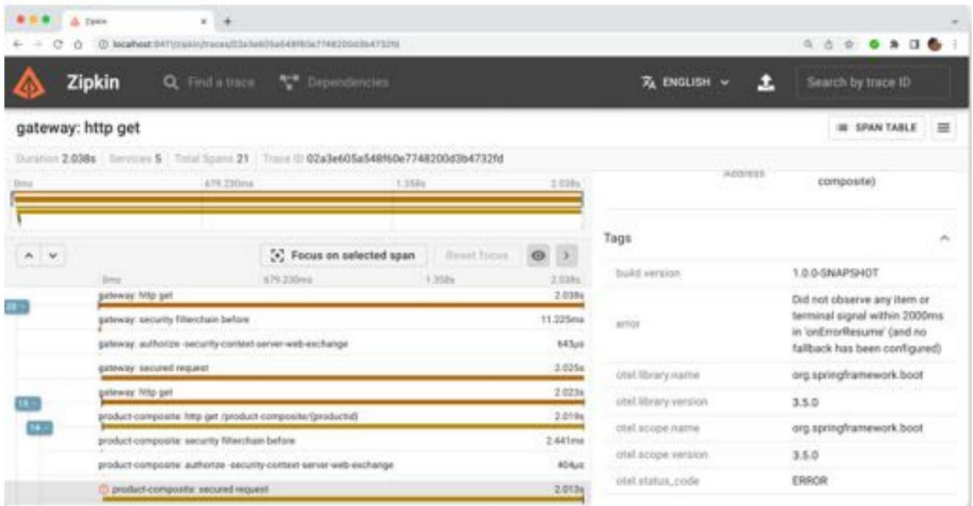


Figura 14.6: Visualización de un seguimiento de una solicitud fallida mediante Zipkin

- Aquí podemos ver un lapso, producto-compuesto: solicitud segura, con un símbolo de error delante (un círculo rojo con un signo de exclamación).
4. Haga clic en el intervalo para ver sus etiquetas. Encontrará una etiqueta llamada "error" que indica claramente que el error se debe a un tiempo de espera agotado después de dos segundos.

Envío de una solicitud de API que activa el procesamiento asíncrono

El tercer tipo de envío de solicitudes que resulta interesante ver representado en la interfaz de usuario de Zipkin es una solicitud cuyo procesamiento parcial se realiza de forma asíncrona. Probemos con una solicitud de eliminación, donde el proceso de eliminación en los servicios principales se realiza de forma asíncrona. El servicio compuesto por producto envía un evento de eliminación a cada uno de los tres servicios principales a través del intermediario de mensajes, y cada servicio principal lo recibe y lo procesa de forma asíncrona. Gracias al seguimiento micrométrico, la información de seguimiento se añade a los eventos enviados al intermediario de mensajes, lo que genera una visión coherente del procesamiento total de la solicitud de eliminación.

Realice los siguientes pasos:

1. Ejecute el siguiente comando para eliminar el producto con un ID de producto de 12345 y verifique que devuelva el código de estado HTTP para la solicitud aceptada, 202:

```
curl -X DELETE -H "Autorización: Portador $ACCESS_TOKEN" -k https://localhost:8443/producto-compuesto/12345 -w "%{http_code}\n" -o /dev/nulo -s
```



Recuerde que la operación de eliminación es idempotente, es decir, ¡tendrá éxito incluso si el producto no existe!

2. En la interfaz de Zipkin, regrese a la página de búsqueda (use el botón Atrás del navegador web) y haga clic de nuevo en el botón EJECUTAR CONSULTA. Para ver los resultados ordenados del más reciente al más reciente, haga clic en el encabezado Hora de inicio. Obtendrá un resultado como el de la siguiente captura de pantalla:

Zipkin interface showing search results for 'gateway' service. The table displays 10 results, with the first two visible. The first trace is 'gateway: http delete' with a duration of 39.273ms and a 'SHOW' button highlighted in red.

Root	Start Time	Spans	Duration
gateway: http delete	a minute ago (04/23 11:42:54.382)	21	39.273ms
gateway: http get	12 minutes ago (04/23 11:32:25.960)	21	2.027s

Figura 14.7: Encontrar una solicitud de eliminación usando Zipkin

3. Debería ver la solicitud de eliminación en la parte superior de la lista de resultados. Tenga en cuenta que el nombre del servicio raíz, gateway, tiene como sufijo el método HTTP utilizado, delete. Haga clic en el botón MOSTRAR. Para ver detalles:

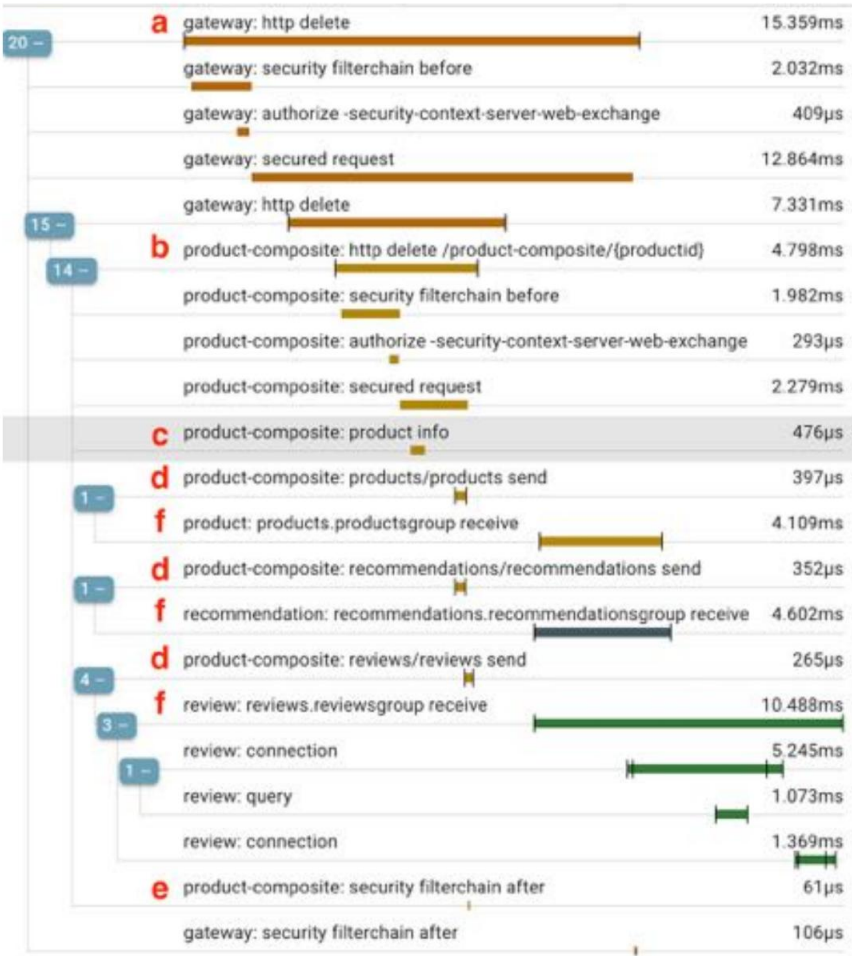


Figura 14.8: Visualización de una solicitud de eliminación mediante Zipkin

4. Aquí podemos ver la información de seguimiento para procesar la solicitud de eliminación:
- La solicitud fue recibida por el servicio de puerta de enlace .
 - El servicio de puerta de enlace delegó el procesamiento de la solicitud al producto-servicio compuesto .

- El servicio compuesto de producto ha creado un intervalo personalizado denominado product-composite: product info, como se esperaba.

El servicio compuesto por producto , a su vez, publicó tres eventos en el agente de mensajes (RabbitMQ, en este caso). Consulte los intervalos con nombres con el sufijo "send".

El servicio compuesto por producto ya está listo y devuelve un código de estado HTTP de éxito , 200, a través del servicio de puerta de enlace al emisor. Tenga en cuenta que esto se realiza antes de que los servicios principales completen todo el procesamiento.

Los servicios principales (producto, recomendación y reseña) reciben los eventos de eliminación y comienzan a procesarlos de forma asíncrona, es decir, de forma independiente. Consulte los intervalos cuyos nombres terminan con el sufijo "receive" .

5. Para confirmar la participación del agente de mensajes, haga clic en el primer segmento del producto:

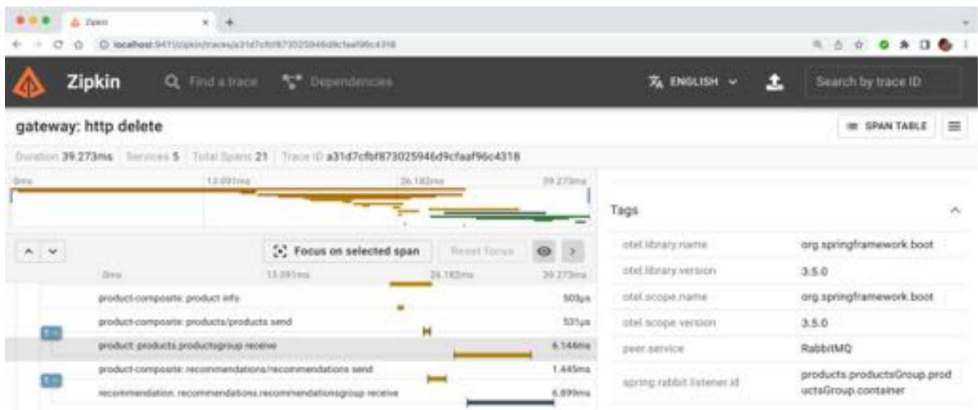


Figura 14.9: Visualización de información sobre el procesamiento asíncrono de un evento mediante Zipkin

6. El intervalo seleccionado contiene una etiqueta llamada peer.service que revela que se utiliza RabbitMQ, y la etiqueta spring.rabbit.listener.id señala que los mensajes son recibidos por el grupo de consumidores productsGroup.
7. Finalmente, para ver el encabezado traceparent en un mensaje enviado a través de RabbitMQ, podemos inspeccionar el mensaje almacenado en la cola de auditoría de los productos. Abra <http://localhost:15672/#/colas/%2F/productos.auditGroup> en su navegador web.

8. Haga clic en el botón Obtener mensaje(s) para ver el mensaje más antiguo en la cola.
La página web debería verse así:

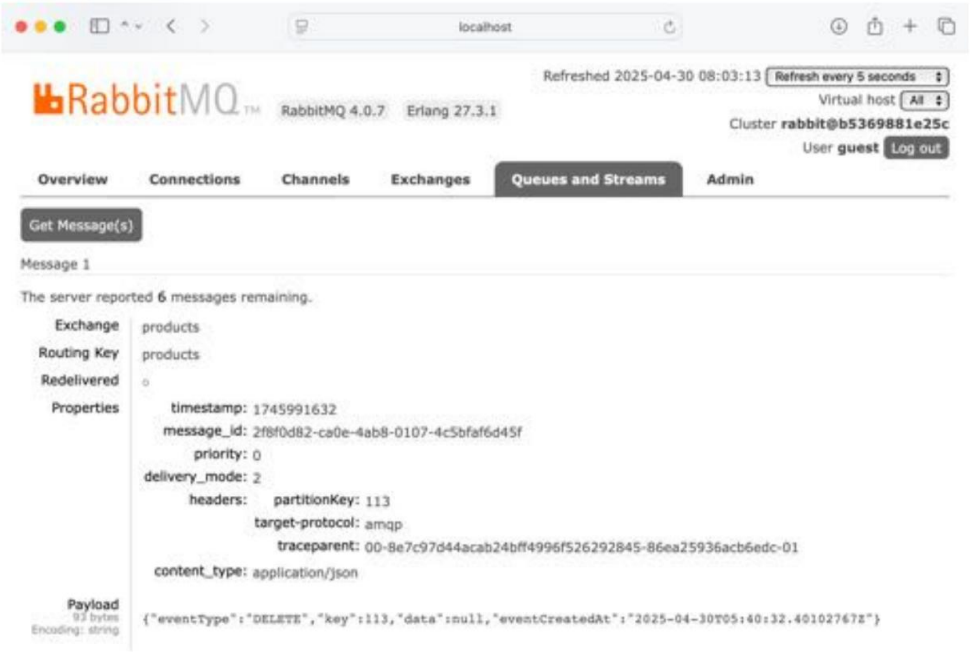


Figura 14.10: Un mensaje con un encabezado traceparent en RabbitMQ

9. En la sección Propiedades , encontrará el encabezado traceparent de este mensaje. En este caso, el ID de seguimiento es 99a9f2501e4d454643184c6b1cb0a232 y el ID de intervalo es 7458430fe56d6df1.

¡Con esto finalizan las pruebas de seguimiento distribuido para este capítulo!



¡La interfaz de usuario de Zipkin contiene mucha más funcionalidad para encontrar rastros de interés!

Para familiarizarse con la interfaz de Zipkin, pruebe la función de consulta haciendo clic en el signo más y seleccionando tagQuery. Por ejemplo, para encontrar solicitudes que fallaron con un error 403 - Prohibido , establezca su valor en tagQuery=http.status_code=403, buscando así los rastros que fallaron con un error 403. También puede intentar establecer límites para el rango de retrosección (hora de inicio y fin) y el número máximo de resultados haciendo clic en el icono de engranaje a la derecha del botón EJECUTAR CONSULTA .

Finalice las pruebas desactivando el panorama del sistema. Ejecute el siguiente comando:

```
Docker Compose Down
```

Resumen

En este capítulo, aprendimos a usar el rastreo distribuido para comprender cómo interactúan nuestros microservicios. Aprendimos a usar el rastreo micrométrico para recopilar información de rastreo y a usar Zipkin. para almacenar y visualizar la información de seguimiento.

Vimos cómo la adición de Micrometer Tracing a los microservicios se ve afectada al añadir un par de dependencias a los archivos de compilación y configurar algunos parámetros. Los microservicios reactivos no son totalmente compatibles actualmente, pero aprendimos a solucionar los problemas más importantes.

Si es necesario, podemos usar la API de Observación para crear intervalos personalizados o agregar etiquetas a todos los intervalos creados por un microservicio. También vimos cómo la interfaz de usuario de Zipkin facilita la identificación de qué parte de un flujo de trabajo complejo causó un tiempo de respuesta inesperadamente largo o un error. Tanto los flujos de trabajo síncronos como los asíncronos se pueden visualizar con la interfaz de usuario de Zipkin.

En el siguiente capítulo, aprenderemos sobre los orquestadores de contenedores, específicamente Kubernetes. Aprenderemos a usar Kubernetes para implementar y administrar microservicios, a la vez que mejoramos características importantes del entorno de ejecución, como la escalabilidad, la alta disponibilidad y la resiliencia.

Preguntas

1. ¿Cuál es el propósito de la configuración de gestión, seguimiento, muestreo y probabilidad?
¿parámetro?
2. ¿Cómo puedes identificar la solicitud de mayor duración después de ejecutar `test-em-all.bash`?
¿guión de prueba?
3. ¿Cómo podemos encontrar solicitudes que han sido interrumpidas por un tiempo de espera introducido en el Capítulo 1?
13. ¿Cómo mejorar la resiliencia utilizando Resilience4j?
4. ¿Cómo se ve el seguimiento de una solicitud de API cuando se introduce el disyuntor en
El capítulo 13, Mejorando la resiliencia usando Resilience4j, ¿está abierto?
5. ¿Cómo podemos localizar las API que fallaron porque el llamador no estaba autorizado para realizar la operación?
¿pedido?
6. ¿Cómo podemos agregar información de seguimiento mediante programación?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



Parte 3

Desarrollo de peso ligero Uso de microservicios Kubernetes

Esta parte te ayudará a comprender la importancia de Kubernetes como plataforma de ejecución para cargas de trabajo en contenedores. Aprenderás a configurar Kubernetes en un entorno de desarrollo local e implementar microservicios en Kubernetes. Finalmente, aprenderás a usar algunas de las características más importantes de Kubernetes en lugar de las correspondientes de Spring Cloud para proporcionar un entorno de microservicios más ligero (es decir, más fácil de mantener y administrar).

Esta parte del libro incluye los siguientes capítulos:

- Capítulo 15, Introducción a Kubernetes
- Capítulo 16, Implementación de nuestros microservicios en Kubernetes
- Capítulo 17, Implementación de funciones de Kubernetes para simplificar el panorama del sistema
- Capítulo 18, Uso de una malla de servicios para mejorar la observabilidad y la gestión
- Capítulo 19, Registro centralizado con la pila EFK
- Capítulo 20, Monitoreo de microservicios

15

Introducción a Kubernetes

En este capítulo, comenzaremos a aprender sobre Kubernetes, el orquestador de contenedores más popular y utilizado al momento de escribir este libro. Dado que los temas sobre los orquestadores de contenedores en general y sobre Kubernetes en sí son demasiado extensos para abarcarlos en un solo capítulo, me centraré en presentar las áreas que he considerado más importantes en mi uso de Kubernetes durante los últimos años.

En este capítulo se tratarán los siguientes temas:

- Introducción a los conceptos de Kubernetes
- Introducción a los objetos de la API de Kubernetes
- Presentación de los componentes de tiempo de ejecución de Kubernetes
- Creación de un clúster de Kubernetes local
- Probar una implementación de muestra y acostumbrarse a la herramienta CLI de Kubernetes `kubectl`
- Administrar un clúster local de Kubernetes

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Los ejemplos de código de este capítulo provienen del código fuente de `$BOOK_HOME/Capítulo 15`. El código fuente del ejemplo de implementación en Kubernetes que se realizará en este capítulo se encuentra en la carpeta `$BOOK_HOME/Capítulo 15/kubernetes/first-attempts`.

Introducción a los conceptos de Kubernetes

A alto nivel, como orquestador de contenedores, Kubernetes crea un clúster de servidores (físicos o virtuales) que ejecutan contenedores como un gran servidor lógico que ejecuta contenedores. Como operador, declaramos un estado deseado al clúster de Kubernetes mediante la creación de objetos mediante la API de Kubernetes. **Kubernetes compara continuamente el estado deseado con el actual. Si detecta diferencias, Toma medidas para garantizar que el estado actual sea el mismo que el estado deseado.**

Uno de los propósitos principales de un clúster de Kubernetes es implementar y ejecutar contenedores, pero también soportar actualizaciones continuas sin tiempo de inactividad mediante técnicas como las implementaciones azul/verde y canarias . Kubernetes puede programar contenedores, es decir, pods que contienen uno o más contenedores co-ubicados, en los nodos disponibles en el clúster. Para poder monitorear el estado de los contenedores en ejecución, Kubernetes supone que los contenedores implementan una sonda de actividad. Si una sonda de actividad informa un contenedor en mal estado, Kubernetes reiniciará el contenedor. Los contenedores se pueden escalar en el clúster de forma manual o automática mediante un escalador automático horizontal. Para optimizar el uso de los recursos de hardware disponibles en un clúster (por ejemplo, memoria y CPU), los contenedores se pueden configurar con cuotas que especifican la cantidad de recursos que necesita un contenedor. Por otro lado, los límites con respecto a cuánto se le permite consumir a un contenedor se pueden especificar en el pod o para un grupo de pods en el nivel de espacio de nombres . Los espacios de nombres se presentarán a medida que avancemos en este capítulo. Esto es de especial importancia si varios equipos comparten un clúster de Kubernetes común.

Otro objetivo principal de Kubernetes es proporcionar detección de servicios de los pods en ejecución y sus contenedores. Los objetos de servicio de Kubernetes se pueden definir para la detección de servicios y también equilibrarán la carga de las solicitudes entrantes en los pods disponibles. Los objetos de servicio se pueden exponer al exterior de un clúster de Kubernetes. Sin embargo, como veremos, un objeto de entrada es, en muchos casos, más adecuado para gestionar el tráfico entrante externo a un grupo de servicios. Para ayudar a Kubernetes a determinar si un contenedor está listo para aceptar solicitudes entrantes, este puede implementar una sonda de preparación.

Internamente, un clúster de Kubernetes proporciona una gran red IP plana donde cada pod tiene su propia dirección IP y puede comunicarse con todos los demás pods, independientemente del nodo en el que se ejecuten. Para admitir múltiples proveedores de red, Kubernetes permite el uso de complementos de red que cumplen con la especificación de la Interfaz de Red de Contenedores (CNI) (<https://github.com/containernetworking/>) (cni). Los pods no están aislados por defecto; aceptan todas las solicitudes entrantes. Los complementos CNI compatibles con el uso de definiciones de políticas de red pueden usarse para restringir el acceso a los pods; por ejemplo, solo permiten el tráfico de pods en el mismo espacio de nombres.

Para permitir que varios equipos trabajen en el mismo clúster de Kubernetes de forma segura, se puede aplicar el control de acceso basado en roles (RBAC) (<https://kubernetes.io/docs/reference/access-authn-authz/rbac/>). Por ejemplo, se puede autorizar a los administradores a acceder a recursos a nivel de clúster, mientras que el acceso de los miembros del equipo se puede restringir a recursos creados en un espacio de nombres propiedad de los equipos.

En total, estos conceptos proporcionan una plataforma para ejecutar contenedores que es escalable, segura, altamente disponible y resistente.

Analicemos un poco más los objetos API que están disponibles en Kubernetes y, luego, los componentes de tiempo de ejecución que conforman un clúster de Kubernetes.

Presentación de los objetos de la API de Kubernetes

Kubernetes define una API que se utiliza para gestionar diferentes tipos de objetos o recursos, como también se les conoce. Algunos de los tipos más utilizados, según se mencionan en la API, son los siguientes:

- **Nodo:** Un nodo representa un servidor, virtual o físico, en el clúster.
- **Pod:** Un Pod representa el componente implementable más pequeño posible en Kubernetes y consta de uno o más contenedores ubicados conjuntamente. Los contenedores comparten la misma dirección IP y rango de puertos. Esto significa que los contenedores en la misma instancia de Pod pueden comunicarse entre sí a través del host local, pero deben ser conscientes de posibles colisiones de puertos. Normalmente, un Pod consta de un contenedor, pero existen casos prácticos para ampliar la funcionalidad del contenedor principal ejecutando el segundo contenedor en un Pod. En el Capítulo 18, "Uso de una Malla de Servicios para Mejorar la Observabilidad y la Gestión", se utilizará un segundo contenedor en los Pods, ejecutando un sidecar que integra el contenedor principal en la malla de servicios.

Implementación : Una implementación se utiliza para implementar y actualizar pods. Los objetos de implementación transfieren la responsabilidad de crear y supervisar los pods a un ReplicaSet. Al crear una implementación por primera vez, el trabajo del objeto de implementación se limita a crear el objeto ReplicaSet. Al realizar una actualización continua de una implementación, la función del objeto de implementación es más compleja.

- **ReplicaSet:** un ReplicaSet se utiliza para garantizar que una cantidad específica de pods se esté ejecutando al mismo tiempo. En todo momento. Si se elimina un Pod, el ReplicaSet lo reemplazará con uno nuevo.

Servicio : Un servicio es un punto final de red estable que permite conectarse a uno o varios pods. A cada servicio se le asigna una dirección IP y un nombre DNS en la red interna del clúster de Kubernetes. La dirección IP del servicio se mantendrá igual durante su vida útil. Las solicitudes enviadas a un servicio se reenviarán a uno de los pods disponibles mediante un balanceo de carga por turnos. De forma predeterminada, un servicio solo se expone dentro del clúster mediante una dirección IP del clúster. También es posible exponer un servicio fuera del clúster, ya sea en un puerto dedicado en cada nodo del clúster o, mejor aún, a través de un balanceador de carga externo compatible con Kubernetes, es decir, que pueda proporcionar automáticamente una dirección IP pública o un nombre DNS para el servicio. Los proveedores de nube que ofrecen Kubernetes como servicio suelen ser compatibles con este tipo de balanceador de carga.

- **Ingress:** Ingress puede gestionar el acceso externo a los servicios en un clúster de Kubernetes, generalmente mediante HTTP o HTTPS. Por ejemplo, puede enrutar el tráfico a los servicios subyacentes según rutas URL o encabezados HTTP, como el nombre de host. En lugar de exponer varios servicios externamente, ya sea mediante puertos de nodo o balanceadores de carga, suele ser más práctico configurar un Ingress delante de los servicios. Para gestionar la comunicación definida por los objetos de Ingress, es necesario que se ejecute un controlador de Ingress en el clúster. Veremos un ejemplo de un controlador de Ingress más adelante.
- **Espacio de nombres:** Un espacio de nombres se utiliza para agrupar y, en ciertos niveles, aislar recursos en un clúster de Kubernetes. Los nombres de los recursos deben ser únicos en sus espacios de nombres, pero no entre ellos.
- **ConfigMap:** Un ConfigMap se utiliza para almacenar la configuración utilizada por los contenedores. Los ConfigMaps pueden mapearse en un contenedor en ejecución como variables de entorno o archivos.
- **Secreto:** los secretos se utilizan para almacenar datos confidenciales utilizados por los contenedores, como las credenciales. Los secretos se pueden poner a disposición de los contenedores de la misma forma que los ConfigMaps. Cualquier persona con acceso de lectura completo al servidor API puede acceder a los valores de los secretos creados, por lo que no son tan seguros como su nombre podría indicar. Los valores de los secretos se almacenan codificados en Base64, por lo que no están cifrados.



Existen algunas extensiones del recurso Secret estándar que ofrecen alternativas más seguras. Por ejemplo, HashiCorp ofrece una alternativa basada en HashiCorp Vault. Consulte <https://developer.hashicorp.com/vault/tutoriales/introducción-a-kubernetes/operador-de-secretos-de-vault>.

- **DaemonSet:** Esto garantiza que un pod se ejecute en cada nodo del conjunto de nodos del clúster. En el capítulo 19, Registro centralizado con la pila EFK, veremos un ejemplo de un recopilador de registros, Fluentd, que se ejecuta en cada nodo de trabajo como un DaemonSet.

Para obtener una lista completa de los objetos de recursos que cubre la API de Kubernetes en v1.32, consulte <https://kubernetes.io/docs/referencia/generated/kubernetes-api/v1.32/>.

El siguiente diagrama resume los recursos de Kubernetes que participan en el manejo de solicitudes entrantes:

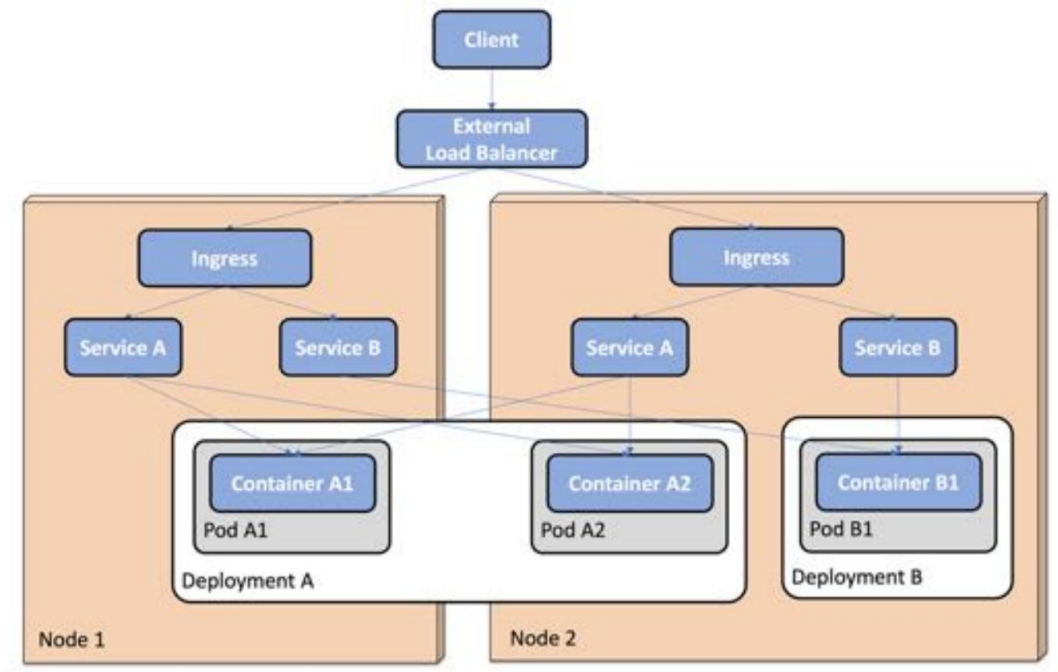


Figura 15.1: Descripción general de los recursos de Kubernetes

En el diagrama anterior podemos ver lo siguiente:

- Se han implementado dos implementaciones, Implementación A e Implementación B, en un clúster con dos nodos, Nodo 1 y Nodo 2.
- La implementación A contiene dos pods, Pod A1 y Pod A2
- La implementación B contiene un Pod, Pod B1
- El Pod A1 está programado para el Nodo 1
- Los Pod A2 y Pod B1 están programados para el Nodo 2
- Cada implementación tiene un servicio correspondiente implementado, Servicio A y Servicio B, y Están disponibles en todos los nodos
- Se define un Ingress para enrutar las solicitudes entrantes a los dos Servicios
- Un cliente normalmente envía solicitudes al clúster a través de un balanceador de carga externo

Estos objetos no son, en sí mismos, componentes en ejecución; son definiciones de diferentes tipos de estados deseados. Para reflejar el estado deseado en el estado actual del clúster, Kubernetes incluye una arquitectura compuesta por varios componentes de tiempo de ejecución, como se describe en [Siguiente sección](#).

Presentación de los componentes de tiempo de ejecución de Kubernetes

Un clúster de Kubernetes contiene dos tipos de nodos: nodos maestros y nodos de trabajo. Los nodos maestros gestionan el clúster, mientras que el propósito principal de los nodos de trabajo es ejecutar la carga de trabajo real; por ejemplo, los contenedores que implementamos en el clúster. Kubernetes se compone de varios componentes de tiempo de ejecución. Los componentes más importantes son los siguientes:

- Hay componentes que se ejecutan en nodos maestros y que constituyen el plano de control:
 - Servidor API, el punto de entrada al plano de control. Esto expone una API RESTful, que, por ejemplo, la herramienta CLI de Kubernetes conocida como `kubectl` utiliza.
 - etcd, un almacén de claves/valores altamente disponible y distribuido, utilizado como base de datos para Todos los datos del clúster.

Un administrador de controladores, que contiene varios controladores que evalúan continuamente el estado deseado frente al estado actual de los objetos definidos en la base de datos etcd. Siempre que el estado deseado o actual cambia, un controlador responsable de ese tipo de estado toma medidas para cambiar el estado actual al deseado. Por ejemplo, un controlador de replicación responsable de la gestión de pods reaccionará si se añade un nuevo pod a través del servidor API o se elimina un pod en ejecución y garantizará el inicio de nuevos pods. Otro ejemplo de controlador es el controlador de nodos. Es responsable de actuar si un nodo deja de estar disponible, garantizando que los pods que se ejecutan en un nodo con fallos se reprogramen en otros nodos.

el clúster.

Un programador , responsable de asignar los pods recién creados a un nodo con capacidad disponible, por ejemplo, en términos de memoria y CPU. Las reglas de afinidad permiten controlar la asignación de los pods a los nodos. Por ejemplo, los pods que realizan muchas operaciones de E/S de disco pueden asignarse a un grupo de nodos de trabajo con discos SSD rápidos. Se pueden definir reglas de antiafinidad para separar los pods, por ejemplo, para evitar programar pods de la misma implementación al mismo nodo de trabajo.

- También hay componentes que se ejecutan en todos los nodos y que constituyen el plano de datos:
 - kubelet, un agente de nodo que se ejecuta como un proceso directamente en el sistema operativo de los nodos y no como un contenedor. Un kubelet garantiza que los pods programados en su nodo tengan sus contenedores en funcionamiento y en buen estado. Actúa como un conducto entre el servidor API y el entorno de ejecución del contenedor en su nodo.
 - kube-proxy, un proxy de red que habilita el concepto de Servicio en Kubernetes y es capaz de reenviar solicitudes a los Pods apropiados, generalmente de manera rotatoria si hay más de un Pod disponible para el Servicio específico. kube-proxy se implementa como un DaemonSet.
- Tiempo de ejecución del contenedor, que es el software que ejecuta los contenedores en un nodo. Históricamente, Kubernetes usaba Docker Engine, pero hoy en día, cualquier implementación de Se puede utilizar la interfaz de tiempo de ejecución de contenedores (CRI) de Kubernetes , como cri-o (<https://cri-o.io>) y containerd (<https://containerd.io/>). La compatibilidad con Docker Engine se eliminó en Kubernetes v1.24.



containerd es, en realidad, el motor de contenedores de Docker. Se separó de Docker en 2017 y hoy es un proyecto CNCF graduado.

- DNS de Kubernetes, que es un servidor DNS que se utiliza en la red interna del clúster. Trabajo. A los servicios y pods se les asigna un nombre DNS, y los pods se configuran para usar este servidor DNS para resolver los nombres DNS internos. El servidor DNS se implementa como un objeto de implementación y un objeto de servicio.

El siguiente diagrama resume los componentes de tiempo de ejecución de Kubernetes descritos anteriormente:

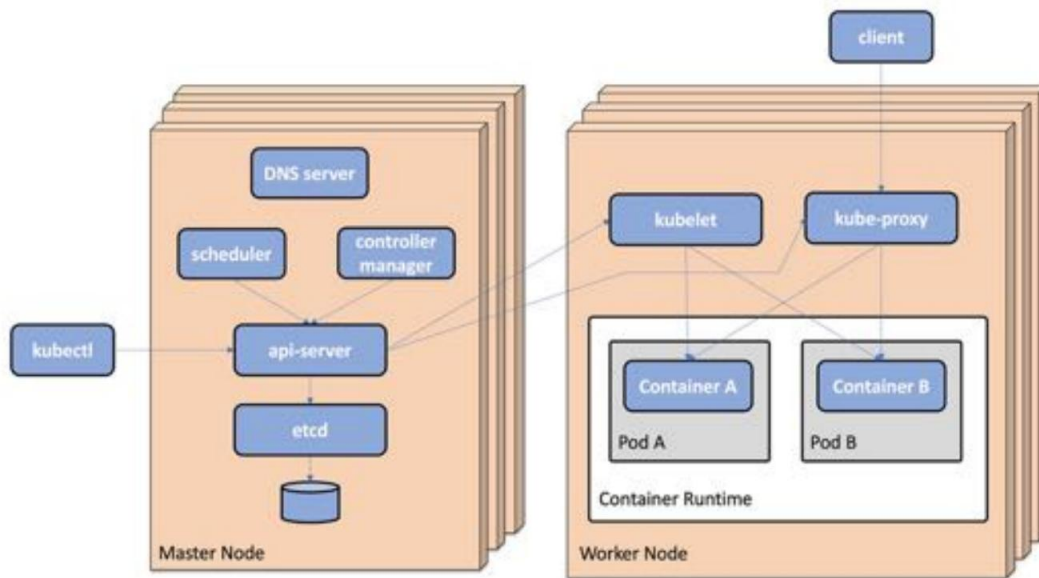


Figura 15.2: Descripción general de los componentes de tiempo de ejecución de Kubernetes

Basándonos en el diagrama, podemos imaginar la siguiente secuencia de acontecimientos:

1. Un operador usa `kubectl` para enviar a Kubernetes un nuevo estado deseado, que contiene manifiestos que declaran un nuevo objeto de Implementación, Servicio e Ingress. El Ingress define una ruta al objeto de Servicio, y este se define para seleccionar los pods configurados por el objeto de Implementación.
2. `kubectl` habla con el servidor API y almacena el nuevo estado deseado como objetos en `etcd` base de datos.
3. Varios controladores reaccionarán a la creación de los nuevos objetos y tomarán las siguientes medidas:

comportamiento:

- Para el objeto de implementación:
 - Los nuevos objetos `ReplicaSet` y `Pod` se registrarán en el servidor API.
 - El programador verá los nuevos `Pods` y los programará en los nodos de trabajo apropiados.

En cada nodo de trabajo, el agente de `kubelet` lanzará los contenedores según lo descrito por los pods. `Kubelet` utilizará el entorno de ejecución del contenedor en el nodo de trabajo para administrarlos.

- Para el objeto Servicio:
 - Se registrará un nombre DNS en el servidor DNS interno para el objeto de Servicio y los kube-proxies podrán enrutar solicitudes que utilicen el Nombre DNS a uno de los Pods disponibles.



Tenga en cuenta que se puede acceder a los pods desde cualquier nodo del clúster, por lo que no es necesario que kube-proxy se ejecute en el mismo nodo que el pod para poder reenviarle solicitudes.

- Para el objeto Ingress:
 - Un controlador de Ingress configurará rutas según el objeto de Ingress y estará listo para aceptar solicitudes desde fuera del clúster de Kubernetes. Las solicitudes externas que coincidan con las rutas definidas por el objeto Ingress serán reenviadas por el controlador de Ingress al objeto Servicio. El proxy de kube las reenviará a un pod, como se describió anteriormente.

Ahora que entendemos los componentes de tiempo de ejecución de Kubernetes y qué admiten y en qué se ejecutan, pasemos a la creación de un clúster de Kubernetes con minikube.

Creación de un clúster de Kubernetes con minikube

¡Ya estamos listos para crear un clúster de Kubernetes! Usaremos minikube para crear un clúster local de un solo nodo.



Varias alternativas para ejecutar un clúster de Kubernetes local para el desarrollo utilizando Minikube existe. Aquí tienes algunos:

- tipo: ver <https://kind.sigs.k8s.io>
- Docker Desktop (con Kubernetes): consulte <https://docs.docker.com/escriptorio/características/kubernetes/>
- MicroK8s: consulte <https://microk8s.io>
- k0s: ver <https://k0sproject.io>
- k3s: <https://k3s.io>

Minikube se puede implementar en una máquina virtual, un contenedor o en hardware con diferentes controladores. Usaremos uno de los controladores preferidos, el controlador Docker, donde la instancia de minikube se ejecuta en un contenedor administrado por Docker Desktop en macOS y Windows con el Subsistema de Windows para Linux, v2 (WSL 2).



Para obtener información sobre los controladores disponibles en minikube, consulte <https://minikube.sigs.k8s.io/docs/drivers/>.

Docker y sus contenedores ya se están ejecutando en una instancia WSL 2 separada; consulte Sección Instalación de Docker Desktop para Windows en el Capítulo 22, Instrucciones de instalaciones para Microsoft Windows con WSL 2 y Ubuntu.

Una desventaja de ejecutar minikube como contenedor en Docker es que los puertos expuestos por minikube solo son accesibles en el host que ejecuta Docker. Para que los puertos estén disponibles para los clientes de Docker (por ejemplo, macOS o el servidor Linux que usaremos en WSL 2), podemos especificar las asignaciones de puertos al crear el clúster de minikube.

Antes de crear el clúster de Kubernetes, necesitamos aprender un poco sobre los perfiles de minikube, la herramienta CLI de Kubernetes conocida como kubectl y su uso de contextos.

Trabajar con perfiles de minikube

Para ejecutar varios clústeres de Kubernetes localmente, minikube viene con el concepto de perfiles. Por ejemplo, si desea trabajar con varias versiones de Kubernetes, puede crear varios clústeres de Kubernetes con minikube. A cada clúster se le asignará un perfil de minikube independiente.

La mayoría de los comandos de minikube aceptan el indicador `--profile` (o `-p` para abreviar), que permite especificar a qué clúster de Kubernetes se aplicará el comando. Si planea trabajar con un perfil específico durante un tiempo, existe una alternativa más práctica: especifique el perfil actual con el siguiente comando:

```
perfil de minikube mi-perfil
```

Este comando establecerá el perfil `my-profile` como el perfil actual.

Para obtener el perfil actual, ejecute el siguiente comando:

```
configuración de minikube obtener perfil
```

Si no se especifica ningún perfil, ya sea mediante el comando `minikube profile` o el modificador `--profile`, se utilizará un perfil predeterminado llamado `minikube`.

Puede encontrar información sobre los perfiles existentes con el comando de lista de perfiles de minikube .

Trabajar con la CLI de Kubernetes, kubectl

kubectl es la herramienta CLI de Kubernetes. Una vez configurado el clúster, esta suele ser la única herramienta necesaria para administrarlo.

Para gestionar los objetos de la API, como describimos anteriormente en este capítulo, el comando kubectl apply es el único que necesita conocer. Es un comando declarativo; es decir, como operador, solicitamos a Kubernetes que aplique la definición de objeto que le asignamos. Kubernetes es quien debe determinar qué debe hacerse realmente.



Otro ejemplo de comando declarativo que esperamos resulte familiar a muchos lectores de este libro es la sentencia SQL SELECT , que permite unir información de varias tablas de la base de datos. En la consulta SQL, solo declaramos el resultado esperado, y el optimizador de consultas de la base de datos es quien determina el orden de acceso a las tablas y los índices que se deben usar para recuperar los datos de la forma más eficiente.

En algunos casos, se prefieren las sentencias imperativas que indican explícitamente a Kubernetes qué hacer. Un ejemplo es el comando `kubectl delete` , donde indicamos explícitamente a Kubernetes que elimine algunos objetos de la API. La creación de un objeto de espacio de nombres también se puede realizar fácilmente con el comando `kubectl create namespace` explícito .

El uso repetido de las sentencias imperativas provocará un fallo; por ejemplo, eliminar el mismo objeto de API dos veces con `kubectl delete` o crear el mismo espacio de nombres dos veces con `kubectl create`. Un comando declarativo (es decir, usar `kubectl apply`) no fallará por uso repetido; simplemente indicará que no hay cambios y saldrá sin realizar ninguna acción.

Algunos comandos comúnmente utilizados para recuperar información sobre un clúster de Kubernetes son los siguientes: Sigue:

- `kubectl get` muestra información sobre el objeto API especificado
- `kubectl describe` proporciona más detalles sobre el objeto API especificado
- `kubectl logs` muestra la salida del registro de los contenedores

¡Veremos muchos ejemplos de estos y otros comandos de kubectl en este y los próximos capítulos!

Si tiene dudas sobre cómo utilizar la herramienta `kubectl`, consulte la ayuda de `kubectl` y `kubectl <comando> --help`. Los comandos siempre están disponibles y proporcionan información muy útil. Otro comando útil es `kubectl explain`, que permite mostrar los campos disponibles al declarar un objeto de Kubernetes. Por ejemplo, ejecute el siguiente comando si necesita consultar los campos disponibles para describir un contenedor en la plantilla de un objeto de implementación:

```
kubectl explain la.implementación.spec.template.spec.containers
```

Trabajar con contextos de kubectl

Para poder trabajar con más de un clúster de Kubernetes, ya sea usando `minikube` localmente o clústeres de Kubernetes configurados en servidores locales o en la nube, `kubectl` incorpora el concepto de contextos. Un contexto es una combinación de los siguientes elementos:

- Un clúster de Kubernetes
- Información de autenticación de un usuario
- Un espacio de nombres predeterminado

De forma predeterminada, los contextos se guardan en el archivo `~/.kube/config`, pero este archivo se puede modificar mediante la variable de entorno `KUBECONFIG`. En este libro, usaremos la ubicación predeterminada, por lo que desconfiguraremos `KUBECONFIG` utilizando el comando `KUBECONFIG unset`.

Cuando se crea un clúster de Kubernetes en `minikube`, se crea un contexto con el mismo nombre que el perfil de `minikube` y se establece como el contexto actual. Por lo tanto, los comandos `kubectl` que se emiten... Una vez creado el clúster en `minikube`, se enviará a ese clúster.

Para enumerar los contextos disponibles, ejecute el siguiente comando:

```
configuración de kubectl obtener contextos
```

A continuación se muestra un ejemplo de respuesta:

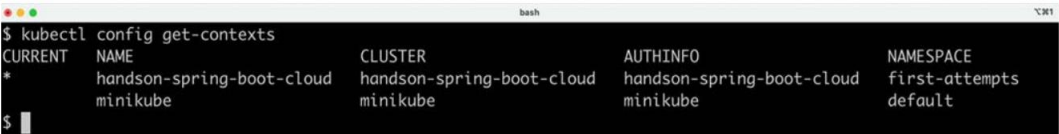


Figura 15.3: Lista de contextos de kubectl

El comodín, *, en la primera columna marca el contexto actual.



Solo verá el contexto `handson-spring-boot-cloud` en la respuesta anterior una vez que se haya creado el clúster, cuyo proceso describiremos en breve.

Si desea cambiar el contexto actual a otro contexto (es decir, trabajar con otro clúster de Kubernetes), ejecute el siguiente comando:

```
configuración de kubectl use-context mi-clúster
```

En este ejemplo, el contexto actual se cambiará a `my-cluster`.

Para actualizar un contexto (por ejemplo, cambiar el espacio de nombres predeterminado utilizado por `kubectl`), utilice el comando `kubectl config set-context`.

Por ejemplo, para cambiar el espacio de nombres predeterminado del contexto actual a `my-namespace`, utilice el siguiente comando:

```
configuración de kubectl establecer-contexto $(configuración de kubectl contexto-actual) --espacio de nombres  
mi espacio de nombres
```

En este comando, se utiliza `kubectl config current-context` para obtener el nombre del contexto actual.

Creación de un clúster de Kubernetes

Para crear un clúster de Kubernetes usando `minikube`, necesitamos ejecutar algunos comandos:

- Desactive la variable de entorno `KUBECONFIG` para garantizar que se cree el contexto `kubectl` en el archivo de configuración predeterminado, `~/.kube/config`.
- Crea el clúster usando el comando `minikube start`, donde también podemos especificar qué Versión de Kubernetes a utilizar y la cantidad de recursos de hardware que queremos asignar al cluster:

Para completar los ejemplos de los capítulos restantes de este libro, asigne 10 GB de memoria (es decir, 10 240 MB) al clúster. Los ejemplos también deberían funcionar si solo se asignan 6 GB (6144 MB) al clúster de `minikube`, aunque con mayor lentitud.

- Asigne la cantidad de núcleos de CPU y espacio en disco que considere adecuados; 4 núcleos de CPU y 30 GB de espacio en disco. En el siguiente ejemplo se utilizan GB de espacio en disco.
- Especifique la versión de Kubernetes que se utilizará. En este libro, usaremos la `v1.32.0`.
- Especificar que utilizaremos el controlador Docker como se describió anteriormente.

- Especifique las asignaciones de puertos requeridas. Los puertos 8080 y 8443 serán utilizados por el controlador Ingress. troller, y los puertos 30080 y 30443 serán utilizados por Servicios del tipo NodePort .



Consulte el Capítulo 16, Implementación de nuestros microservicios en Kubernetes, para obtener información sobre cómo el servidor Gateway implementa un servicio del tipo NodePort .

- Especifique el perfil de minikube que se usará para los próximos comandos de minikube . Usaremos handson-spring-boot-cloud como nombre del perfil.

Tras crear el clúster, usaremos el administrador de complementos de minikube para habilitar un controlador de Ingress y un servidor de métricas que viene incluido con minikube. El controlador de Ingress y el servidor de métricas se usarán en los siguientes capítulos.

Ejecute los siguientes comandos para crear el clúster de Kubernetes:

```
desarmar KUBECONFIG

inicio de minikube \
  --perfil=handson-spring-boot-cloud \
  --memoria=10240 \
  --cpus=4 \
  --tamaño-de-disco=30g \
  --kubernetes-version=v1.32.0 \
  --driver=docker \
  --puertos=8080:80 --puertos=8443:443 \
  --puertos=30080:30080 --puertos=30443:30443

Perfil de minikube handson-spring-boot-cloud

Los complementos de minikube habilitan el ingreso
Los complementos de minikube habilitan el servidor de métricas
```

Una vez completados los comandos anteriores, debería poder comunicarse con el clúster. Pruebe el comando `kubectl get nodes` . Debería responder con algo similar a lo siguiente:

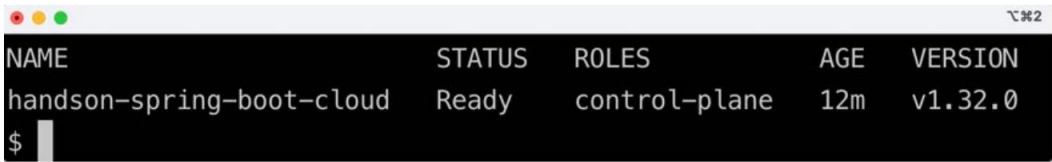


Figura 15.4: Lista de nodos en el clúster de Kubernetes

Una vez creado, el clúster se inicializará automáticamente en segundo plano, iniciando varios pods del sistema en los espacios de nombres kube-system e ingress-nginx . Podemos supervisar su progreso ejecutando el siguiente comando:

```
kubectl obtener pods --todos los espacios de nombres
```

Una vez completado el inicio, el comando anterior debe informar el estado de todos los Pods como En ejecución y el recuento de LISTO debe ser 1/1, lo que significa que un solo contenedor en cada Pod está en funcionamiento:

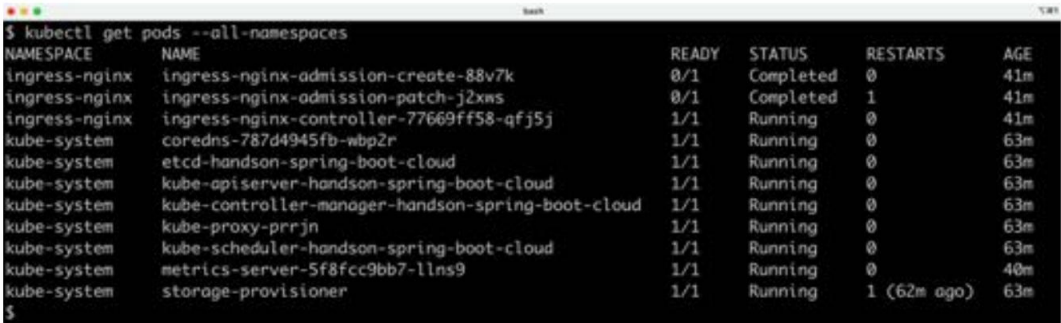


Figura 15.5: Lista de pods del sistema en ejecución

Tenga en cuenta que dos pods se reportan como Completados y no como En ejecución. Son pods creados por un trabajo. Objetos que se utilizan para ejecutar un contenedor un número fijo de veces, como un trabajo por lotes. Ejecute el comando `kubectl get jobs --namespace=ingress-nginx` para mostrar los dos objetos Job.

¡Ahora estamos listos para la acción!

Probando una implementación de muestra

Veamos cómo podemos hacer lo siguiente:

- Implementar un servidor web simple basado en NGINX en nuestro clúster de Kubernetes

- Aplicar algunos cambios a la implementación:
 - Cambie el estado actual eliminando el Pod y verifique que el ReplicaSet se haya creado.
come uno nuevo
 - Cambie el estado deseado escalando el servidor web a tres Pods y verifique que
ReplicaSet llena el vacío iniciando dos nuevos Pods
- Enrutar el tráfico externo al servidor web mediante un servicio con un puerto de nodo

Primero, cree un espacio de nombres, first-attempts, y actualice el contexto de kubectl para usar este espacio de nombres de manera predeterminada:

```
kubectl crear espacio de nombres primeros intentos kubectl
config establecer contexto $(kubectl config contexto-actual) --namespace=first-attempts
```

Ahora podemos crear una implementación de NGINX en el espacio de nombres usando el archivo kubernetes/first-tries/nginx-deployment.yaml . Este archivo tiene el siguiente aspecto:

```
apiVersion: apps/v1 tipo:
Implementación
metadatos:
  nombre: nginx-deploy
especulación:
  réplicas: 1
selector:
  etiquetas de coincidencia:
    aplicación: plantilla
nginx-app :
  metadatos:
    etiquetas:
      aplicación: nginx-app
  especulación:
    contenedores:
      - nombre: nginx-container
        imagen: nginx:latest
        puertos:
          - Puerto contenedor: 80
```

Expliquemos este código fuente con más detalle:

- Los atributos `kind` y `apiVersion` se utilizan para especificar que estamos declarando una implementación. objeto de ment.

La sección de metadatos se utiliza para describir el objeto de implementación. Por ejemplo, lo llamamos `nginx-deploy`.



Otros metadatos de uso común para un objeto de Kubernetes incluyen el nombre del espacio de nombres al que pertenece, las etiquetas y las anotaciones. Los veremos en este capítulo y en los siguientes.

A continuación viene una sección de especificaciones que define nuestro estado deseado para el objeto de implementación:

- réplicas: 1 especifica que queremos tener un Pod en funcionamiento.

La sección del selector especifica cómo la implementación encontrará los pods que administra. En este caso, buscará los pods que tengan la etiqueta de aplicación establecida en `nginx-app`.

- La sección de plantilla se utiliza para especificar cómo se crearán los Pods.
- La sección de metadatos especifica la etiqueta, `app: nginx-app`, que se utiliza para identificar los Pods, haciendo coincidir así el selector.
- La sección de especificaciones especifica detalles para la creación del contenedor único en el Pod, es decir, nombre, imagen y qué puertos utiliza.

Cree la implementación con los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 15
kubectl apply -f kubernetes/primeros-intentos/nginx-deployment.yaml
```

Veamos qué obtuvimos con el comando `kubectl get all` :



```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-deploy-79f9d4bcf5-vxmw7	1/1	Running	0	99s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx-deploy	1/1	1	1	99s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nginx-deploy-79f9d4bcf5	1	1	1	99s

```
$
```

Figura 15.6: Objetos de Kubernetes creados por la implementación de muestra

Como era de esperar, obtuvimos un objeto de Implementación, un Conjunto de Réplicas y un Pod. Tras un breve periodo, que depende principalmente del tiempo que tarde en descargarse la imagen de Docker de NGINX, el Pod estará en funcionamiento y se indicará como 1/1 en la columna LISTO , lo que significa que el estado deseado es igual al actual.

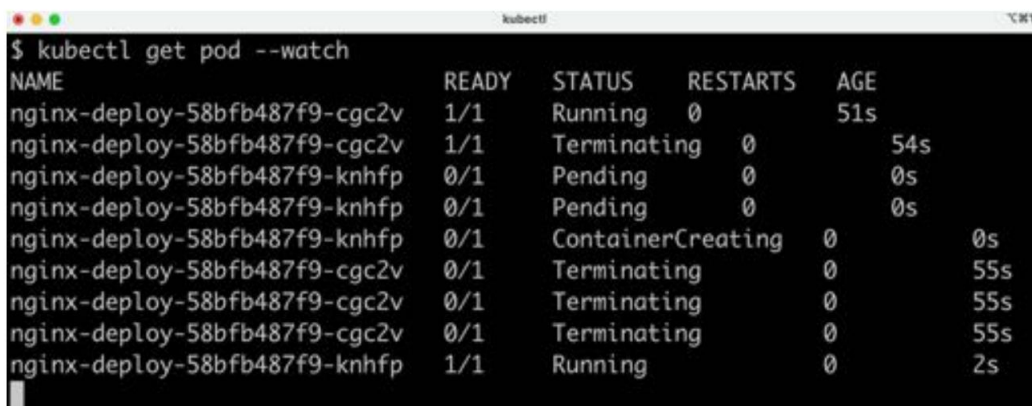
Ahora, cambiaremos el estado actual eliminando el Pod. Antes de eliminarlo, ejecute el comando `kubectl get pod --watch` en otra terminal. El uso de la opción `--watch` hace que el comando se bloquee, esperando los cambios de estado de los Pods en el espacio de nombres actual. Elimine el Pod con el siguiente comando:

```
kubectl eliminar pod --selector app=nginx-app
```

Dado que el Pod tiene un nombre aleatorio (nginx-deploy-79f9d4bcf5-vxmw7 en el ejemplo anterior), el Pod se selecciona según la etiqueta de la aplicación , que está configurada como nginx-app en el Pod.

Observe cómo `kubectl get pod --watch` informa cómo se termina el pod actual y, al mismo tiempo, se inicia uno nuevo. Es el ReplicaSet el que detecta la diferencia entre el estado deseado y el actual y, casi de inmediato, inicia un nuevo pod para compensar la desviación.

Los eventos reportados deberían verse como la siguiente captura de pantalla:



```
$ kubectl get pod --watch
NAME                                READY   STATUS             RESTARTS   AGE
nginx-deploy-58bfb487f9-cgc2v      1/1     Running            0          51s
nginx-deploy-58bfb487f9-cgc2v      1/1     Terminating      0          54s
nginx-deploy-58bfb487f9-knhfp      0/1     Pending            0          0s
nginx-deploy-58bfb487f9-knhfp      0/1     Pending            0          0s
nginx-deploy-58bfb487f9-knhfp      0/1     ContainerCreating  0          0s
nginx-deploy-58bfb487f9-cgc2v      0/1     Terminating      0          55s
nginx-deploy-58bfb487f9-cgc2v      0/1     Terminating      0          55s
nginx-deploy-58bfb487f9-cgc2v      0/1     Terminating      0          55s
nginx-deploy-58bfb487f9-knhfp      1/1     Running            0          2s
```

Figura 15.7: kubectl get pod --watch informa cambios en los Pods

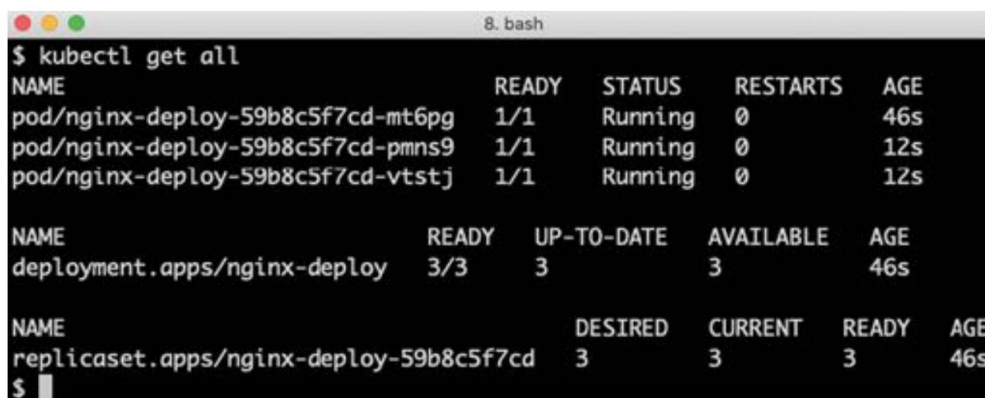
En la captura de pantalla, podemos ver que el Pod con un nombre que termina en cgc2v fue detenido por el comando delete y que ReplicaSet inmediatamente inició un nuevo Pod con un nombre que termina en knhfp.

Cambie el estado deseado estableciendo la cantidad de pods deseados en tres réplicas en kubernetes/first-attempts/nginx-deployment.yaml Archivo de implementación. Aplique el cambio en el estado deseado simplemente repitiendo el comando "kubectl apply" , como se mencionó anteriormente.

Nuevamente, tenga en cuenta que el comando kubectl get pod --watch informa que el ReplicaSet lanza nuevos Pods para obtener el estado actual equivalente al nuevo estado deseado, es decir, tres Pods.

Después de unos segundos, se informará que dos nuevos pods NGINX están en funcionamiento. Detenga el comando con Ctrl + C.

Ejecute el comando kubectl get all y espere una respuesta similar a la siguiente:



```
$ kubectl get all
NAME                                READY   STATUS             RESTARTS   AGE
pod/nginx-deploy-59b8c5f7cd-mt6pg   1/1     Running            0          46s
pod/nginx-deploy-59b8c5f7cd-pmns9   1/1     Running            0          12s
pod/nginx-deploy-59b8c5f7cd-vtstj   1/1     Running            0          12s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-deploy        3/3     3            3          46s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-deploy-59b8c5f7cd 3         3         3       46s
```

Figura 15.8: Nuevos pods iniciados por Kubernetes para alcanzar el estado deseado

Observe los tres pods y que el objeto de implementación indica 3/3. Esto se interpreta como tres pods listos y tres deseados, lo que significa que todos los pods deseados están listos para usarse.

Para habilitar la comunicación externa con los servidores web, cree un servicio utilizando kubernetes/

Archivo first-attempts/nginx-service.yaml . Su aspecto es el siguiente:

```

Versión de API: v1
tipo: Servicio
metadatos:
  nombre: servicio nginx
especificación:
  tipo: NodePort
  selector:
    aplicación: nginx-app
  puertos:
    - puerto de destino: 80
      puerto: 80
      puerto del nodo: 30080
```

Los atributos kind y apiVersion se utilizan para especificar que estamos declarando un objeto de Servicio .

La sección de metadatos se utiliza para describir el objeto de Servicio ; por ejemplo, para darle un nombre: servicio nginx.

A continuación viene una sección de especificaciones , que define el estado deseado del objeto de Servicio :

Con el campo de tipo , especificamos que queremos NodePort , es decir, un servicio accesible externamente en un puerto dedicado en cada nodo del clúster. Esto significa que una llamada externa puede acceder a los pods detrás de este servicio usando este puerto en cualquiera de los nodos del clúster, independientemente de en qué nodos se ejecuten los pods.

- El selector es utilizado por el Servicio para encontrar Pods disponibles, que, en nuestro caso, son Pods etiquetado con la aplicación: nginx-app.
- Finalmente, los puertos se declaran de la siguiente manera:
 - puerto: 80 especifica en qué puerto será accesible el Servicio, es decir, internamente en el cluster.
 - targetPort: 80 especifica el puerto en el Pod donde se reenviarán las solicitudes a.
 - nodePort: 30080 especifica el puerto al que se accederá externamente al servicio al usar cualquiera de los nodos del clúster. De forma predeterminada, un puerto de nodo debe estar entre 30000 y 32767.



Este rango de puertos se utiliza para minimizar el riesgo de colisión con otros puertos en uso. En un sistema de producción, se suele instalar un balanceador de carga delante del clúster de Kubernetes, lo que impide que los usuarios externos conozcan estos puertos y las direcciones IP de los nodos del clúster. Consulte el Capítulo 18, "Uso de una malla de servicios para mejorar la observabilidad y la gestión", específicamente la sección "Configuración del acceso a los servicios de Istio", para obtener más información sobre el uso de un servicio de Kubernetes con balanceo de carga.

Crea el servicio con el siguiente comando:

```
kubectl apply -f kubernetes/primeros-intentos/nginx-service.yaml
```

Para ver qué obtenemos, ejecute el comando `kubectl get svc`. Obtendrá una respuesta como la siguiente:



```
bash
$ kubectl get svc
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
nginx-service  NodePort    10.100.230.194 <none>        80:30080/TCP     30s
```

Figura 15.9: El servicio NodePort para nuestra implementación



`kubectl` admite nombres cortos para muchos objetos de la API como alternativa a su nombre completo. Por ejemplo, se usó "svc" en el comando anterior en lugar del nombre completo, "service". Ejecute el comando `kubectl api-resources` para ver todos los nombres cortos disponibles.

Para acceder al servidor web a través del puerto de nodo del Servicio, necesitamos conocer la dirección IP o el nombre de host del nodo individual de nuestro clúster. Al usar el controlador de Docker, el nombre de host es... formas localhost.

El puerto del nodo, 30080, se reenvía desde el motor de Docker mediante la opción `--ports` del comando `minikube start`. Consulte la sección anterior "Creación de un clúster de Kubernetes" para obtener más información. Esto significa que Se puede acceder al servicio en la dirección `localhost:30080`.



Los puertos abiertos en una instancia de WSL 2 son accesibles en Windows en localhost.

Con esta información, podemos dirigir un navegador web en macOS y Windows al servidor web implementado mediante la dirección `http://localhost:30080`. Se espera una respuesta como la siguiente:



Figura 15.10: Página web predeterminada de NGINX

¡Genial! Pero ¿qué pasa con la dirección IP y el puerto del clúster interno?

Una forma de verificar que el servidor web también sea accesible internamente en el clúster es iniciar un Pod pequeño que podemos usar para ejecutar curl desde dentro. El comando curl usará el interno

Dirección IP y puerto del clúster. No necesitamos usar la dirección IP interna; en su lugar, podemos usar un nombre DNS creado para el servicio en el servidor DNS interno. El nombre corto del DNS...

El nombre es el mismo que el nombre del servicio, es decir, `nginx-service`.



El nombre DNS completo de un servicio es `<service-name>.<namespace>.svc.cluster`.

Local. El nombre completo de este servicio es `nginx-service.first-attempts.svc.cluster.local`. Dado que ejecutaremos el siguiente comando en el mismo espacio de nombres, podemos usar el nombre corto.

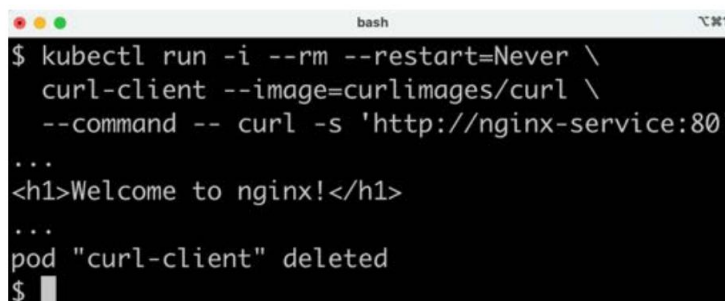
Ejecute el siguiente comando:

```
kubectll run -i --rm --restart=Nunca curl-client --image=curlimages/curl --comando -- curl -s 'http://nginx-service:80'
```

El comando parece un poco complejo, pero hará lo siguiente:

1. Cree un Pod con un contenedor pequeño basado en la imagen de Docker, `curlimages/curl`, que contiene el comando `curl`.
2. Ejecute el comando `curl -s 'http://nginx-service:80'` dentro del contenedor y redirija la salida a la terminal usando la opción `-i`.
3. Elimine el Pod usando la opción `--rm`.

Espera que la salida del comando anterior contenga la siguiente información (aquí solo mostramos partes de la respuesta):

A terminal window titled 'bash' with a dark background. The command entered is: `$ kubectl run -i --rm --restart=Never \ curl-client --image=curlimages/curl \ --command -- curl -s 'http://nginx-service:80'`. The output shows: `... <h1>Welcome to nginx!</h1> ... pod "curl-client" deleted`. The prompt `$` is visible at the bottom.

```
$ kubectl run -i --rm --restart=Never \
  curl-client --image=curlimages/curl \
  --command -- curl -s 'http://nginx-service:80'
...
<h1>Welcome to nginx!</h1>
...
pod "curl-client" deleted
$
```

Figura 15.11: Acceso a NGINX dentro del clúster de Kubernetes

¡Esto significa que el servidor web también es accesible internamente en el clúster!

Esto es básicamente todo lo que necesitamos saber para poder implementar nuestro panorama de sistemas.

Termine esto eliminando el espacio de nombres que contiene la implementación de nginx :

Primeros intentos de eliminar espacio de nombres en kubectl

Antes de finalizar este capítulo introductorio sobre Kubernetes, necesitamos aprender a administrar nuestro Clúster de Kubernetes.

Administrar un clúster local de Kubernetes

Un clúster de Kubernetes en ejecución consume muchos recursos, principalmente memoria. Por lo tanto, al terminar de trabajar con un clúster de Kubernetes en Minikube, debemos poder hibernarlo para liberarlo. Los recursos asignados. También necesitamos saber cómo reanudar el clúster cuando queramos.

Seguir trabajando con él. Eventualmente, también debemos poder eliminar permanentemente el clúster cuando ya no queramos conservarlo en el disco.

Minikube incluye un comando de parada que permite hibernar un clúster de Kubernetes. El comando de inicio que usamos para crear inicialmente el clúster de Kubernetes también permite reactivarlo desde su estado de hibernación. Para eliminar un clúster permanentemente, podemos usar el comando de eliminación. comando de minikube.

Hibernación y reanudación de un clúster de Kubernetes

Ejecute el siguiente comando para hibernar (es decir, detener) el clúster de Kubernetes:

```
parada de minikube
```

Ejecute el siguiente comando para reanudar (es decir, iniciar) el clúster de Kubernetes nuevamente:

```
inicio de minikube
```

Ejecutar comandos kubectl directamente después de reiniciar el clúster podría generar mensajes de error como este:

```
[E0428 09:44:16.333361 79175 memcache.go:106] No se pudo obtener la lista de recursos para metrics.k8s.io/v1beta1: el servidor actualmente no puede manejar la solicitud
```

Esto se debe a que el servidor de métricas tarda un poco en iniciarse; el mensaje de error desaparecerá después de un rato.



Al reanudar un clúster ya existente, el comando de inicio ignora los modificadores que se usaron al crear el clúster.

Tras reanudar el clúster de Kubernetes, el contexto de kubectl se actualizará para usar este clúster con el espacio de nombres utilizado actualmente como predeterminado. Si trabaja con otro espacio de nombres (por ejemplo, el espacio de nombres práctico que usaremos en el próximo capítulo, Capítulo 16, Implementación de nuestros microservicios en Kubernetes), puede actualizar el contexto de kubectl con el siguiente comando:

```
configuración de kubectl contexto-actual $(configuración de kubectl contexto-actual) --  
namespace=hands-on
```

Los comandos kubectl posteriores se aplicarán al espacio de nombres práctico cuando sea aplicable.



Minikube también incluye una alternativa más ligera y rápida a los comandos de parada e inicio : los comandos de pausa y reanudación . En este caso, los componentes del plano de control se pausan, no se detienen, lo que minimiza el consumo de CPU del clúster. Sin embargo, he observado problemas con estos comandos en los capítulos recientes, por lo que recomiendo usar los comandos de inicio y parada .

Terminar un clúster de Kubernetes

Si más adelante desea finalizar el clúster de Kubernetes, puede ejecutar el siguiente comando:

```
minikube eliminar --profile handson-spring-boot-cloud
```

Puedes ejecutar el comando de eliminación sin especificar el perfil, pero me parece más seguro especificarlo explícitamente. De lo contrario, podrías eliminar accidentalmente el clúster de Kubernetes equivocado.

Hemos aprendido a administrar un clúster de Kubernetes que se ejecuta en Minikube. Ahora sabemos cómo suspender y reanudar un clúster y, cuando ya no lo necesitamos, cómo eliminarlo definitivamente.

Resumen

En este capítulo, conocimos Kubernetes como un orquestador de contenedores.

Con Kubernetes, podemos gestionar un clúster de servidores como un gran servidor lógico que ejecuta nuestros contenedores. Declaramos un estado deseado para el clúster de Kubernetes, lo que garantiza que el estado real sea el mismo que el deseado en todo momento, siempre que haya suficientes recursos de hardware disponibles en el clúster.

El estado deseado se declara mediante la creación de recursos mediante el servidor de API de Kubernetes. El administrador de controladores de Kubernetes y sus controladores reaccionan a los diversos recursos creados por el servidor API y tomar medidas para garantizar que el estado actual coincida con el nuevo estado deseado. El

El programador asigna nodos a los contenedores recién creados, es decir, a los pods que contienen uno o más contenedores. En cada nodo, se ejecuta un agente (un kubelet) que garantiza que los pods programados en su nodo estén activos. El kube-proxy actúa como un proxy de red, lo que permite la abstracción del servicio al reenviar las solicitudes enviadas al servicio a los pods disponibles en el clúster.

Las solicitudes externas pueden ser manejadas por un balanceador de carga compatible con Kubernetes que puede proporcionar una dirección IP pública y/o un nombre DNS para el Servicio, un puerto de nodo que esté disponible en todos los nodos del clúster o a través de un recurso de Ingress dedicado.

También probamos Kubernetes creando un clúster local de un solo nodo con minikube. El clúster de minikube se ejecuta como un contenedor Docker mediante el controlador de Docker. Para que los puertos sean accesibles fuera del motor Docker, podemos usar la opción `--ports` en el comando de inicio de minikube. Con la herramienta CLI de Kubernetes, `kubectrl`, implementamos un servidor web simple basado en NGINX. Probamos las capacidades de resiliencia eliminando el servidor web y observamos que se recreaba automáticamente. Aprendimos a escalarlo manualmente solicitando que tres pods se ejecuten en el servidor web. Creamos un servicio con un puerto de nodo y verificamos que podíamos acceder a él tanto externamente como desde dentro del clúster.

Finalmente, aprendimos cómo administrar un clúster de Kubernetes que se ejecuta en minikube en términos de cómo hibernar, reanudar y finalizar el clúster.

Ya estamos listos para implementar el entorno de sistemas de los capítulos anteriores en Kubernetes. ¡Vaya al siguiente capítulo para descubrir cómo hacerlo!

Preguntas

1. ¿Qué sucede si ejecuta el mismo comando `kubectl create` dos veces?
2. ¿Qué sucede si ejecuta el mismo comando `kubectl apply` dos veces?
3. En términos de las preguntas 1 y 2, ¿por qué actúan de manera diferente la segunda vez que se ejecutan?
4. ¿Cuál es el propósito de un `ReplicaSet` y qué otro recurso crea un `ReplicaSet`?
5. ¿Cuál es el propósito de `etcd` en un clúster de Kubernetes?
6. ¿Cómo puede un contenedor averiguar la dirección IP de otro contenedor que se ejecuta en el mismo Pod?
7. ¿Qué sucede si crea dos implementaciones con el mismo nombre pero con nombres diferentes?
¿espacios?
8. ¿Qué configuración de dos Servicios con el mismo nombre puede hacer que fallen, incluso si
¿Se crean en dos espacios de nombres diferentes?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



16

Implementando nuestros microservicios en Kubernetes

En este capítulo, implementaremos los microservicios de este libro en Kubernetes. Para agrupar y configurar los microservicios para implementaciones en diferentes entornos de ejecución, se utilizará Helm, un gestor de paquetes para Kubernetes. Antes de ello, debemos repasar cómo se utiliza el descubrimiento de servicios. Dado que Kubernetes incluye compatibilidad integrada con el descubrimiento de servicios, parece innecesario implementar Netflix Eureka para este fin. Finalmente, probaremos algunas funciones de Spring Boot que facilitan el despliegue de microservicios en Kubernetes.

En este capítulo se tratarán los siguientes temas:

- Reemplazo de Netflix Eureka con objetos de servicio de Kubernetes y kube-proxy para el servicio descubrimiento
- Presentación de cómo se utilizará Kubernetes
- Utilizando el soporte de Spring Boot para un apagado elegante y sondas de actividad y preparación
- Uso de Helm para empaquetar, configurar e implementar los microservicios en diferentes entornos
- Verificar las implementaciones con el script de prueba, `test-em-all.bash`

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter16`.

Si desea ver los cambios aplicados al código fuente en este capítulo (es decir, ver el tiempo necesario para implementar los microservicios en Kubernetes), puede comparar este código fuente con el del Capítulo 15, Introducción a Kubernetes. Puede usar su herramienta de comparación preferida y comparar las dos carpetas: `$BOOK_HOME/Chapter15` y `$BOOK_HOME/Chapter16`.

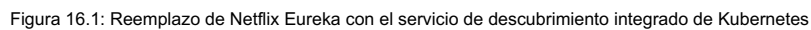
Reemplazo de Netflix Eureka con servicios de Kubernetes

Como se mostró en el capítulo anterior, Capítulo 15, Introducción a Kubernetes, Kubernetes incluye un servicio de descubrimiento integrado basado en objetos de servicio de Kubernetes y el componente de tiempo de ejecución kube-proxy . Esto evita la necesidad de implementar un servicio de descubrimiento independiente como Netflix Eureka, que utilizamos en los capítulos anteriores.

Una ventaja de usar el servicio de descubrimiento de Kubernetes es que no requiere una biblioteca cliente como Spring Cloud LoadBalancer, que hemos usado junto con Netflix Eureka. Esto hace que el servicio de descubrimiento de Kubernetes sea fácil de usar, independientemente del lenguaje o framework. El microservicio se basa en.

Una desventaja de usar el servicio de descubrimiento de Kubernetes es que solo funciona en un entorno Kubernetes . Sin embargo, dado que el servicio de descubrimiento se basa en kube-proxy, que acepta solicitudes al nombre DNS o la dirección IP de un objeto de servicio, debería ser bastante sencillo reemplazarlo con un servicio de descubrimiento similar, por ejemplo, uno que venga incluido con otro orquestador de contenedores.

Para resumir esto, eliminaremos el servidor de descubrimiento basado en Netflix Eureka de nuestro entorno de microservicios, como se ilustra en el siguiente diagrama:



- Netflix Eureka y la configuración específica de Spring Cloud LoadBalancer (cliente y servidor) se han eliminado del repositorio de configuración, config-repo.
- Se han eliminado las reglas de enrutamiento en el servicio de puerta de enlace al servidor Eureka.
archivo config-repo/gateway.yml .
- Se ha eliminado el proyecto del servidor Eureka, en la carpeta spring-cloud/eureka-server .
- El servidor Eureka se ha eliminado de los archivos y la configuración de Docker Compose.
gradle Archivo Gradle.
- Se ha eliminado la dependencia de spring-cloud-starter-netflix-eureka-client
en todos los archivos de compilación de clientes de Eureka, build.gradle.

- La configuración de la propiedad `eureka.client.enabled=false` se ha eliminado de todas las integraciones.

Pruebas de validación de antiguos clientes de Eureka.

El servicio de puerta de enlace ya no utiliza el enrutamiento basado en el balanceador de carga del lado del cliente en Spring Cloud LoadBalancer, utilizando el protocolo lb. Por ejemplo, `lb://product-composite`

El destino de enrutamiento ha sido reemplazado por `http://product-composite` en el archivo `config-repo/gateway.yml`.

- El puerto HTTP utilizado por los microservicios y el servidor de autorización se ha cambiado del puerto 8080 (9999 en el caso del servidor de autorización) al puerto HTTP predeterminado, 80. Esto se ha configurado en `config-repo` para cada servicio afectado, de la siguiente manera:

```
spring.config.activate.on-profile: docker
puerto del servidor: 80
```

Ninguna de las direcciones HTTP que utilizaremos se verá afectada por la sustitución de Netflix Eureka por los servicios de Kubernetes. Por ejemplo, las direcciones utilizadas por el servicio compuesto no se verán afectadas:

```
cadena final privada productServiceUrl = "http://producto";
cadena final privada recomendaciónServiceUrl = "http://recomendación";
cadena final privada reviewServiceUrl = "http://revisión";
```

Esto se debe a que cambiamos el puerto HTTP utilizado por los microservicios y el servidor de autorización al puerto HTTP predeterminado, 80, como se describió anteriormente.



El uso de Docker Compose todavía funciona, incluso aunque se haya eliminado Netflix Eureka.

La razón principal por la que esto funciona es que los nombres de los contenedores en los archivos de Docker Compose coinciden con los nombres de los servicios correspondientes utilizados en Kubernetes, lo que significa que los nombres DNS de los microservicios son los mismos en ambos entornos. Esto puede ser

Se utiliza para ejecutar pruebas funcionales de los microservicios sin implementarlos en Kubernetes; por ejemplo, se ejecuta `test-em-all.bash` junto con Docker Desktop, como en los capítulos anteriores. Sin embargo, al eliminar Netflix Eureka, ya no contamos con un servicio de descubrimiento al usar Docker simple y Docker Compose. Por lo tanto, el escalado de microservicios solo funcionará al implementarlos en Kubernetes.

En el Capítulo 17, Implementación de las características de Kubernetes para simplificar el entorno del sistema, en la sección "Verificación del funcionamiento de los microservicios sin Kubernetes", analizaremos la importancia de evitar que el código fuente de los microservicios dependa de la plataforma Kubernetes, evitando así la dependencia de un proveedor. También utilizaremos el script de prueba `test-em-all.bash`, junto con Docker Compose, para verificar que los microservicios no requieran Kubernetes desde una perspectiva funcional.

Ahora que nos hemos familiarizado con cómo se reemplazará Netflix Eureka con Kubernetes Services, presentemos los otros objetos de Kubernetes que usaremos.

Presentamos cómo se utilizará Kubernetes

Más adelante en el capítulo, veremos en detalle cómo se utilizan los distintos objetos de Kubernetes para implementar los microservicios y los administradores de recursos de los que dependen, como bases de datos y administradores de colas.

Antes de profundizar en todos los detalles, obtengamos una descripción general de los objetos de Kubernetes que se utilizarán:

Para cada microservicio, base de datos y gestor de colas que se implementará en Kubernetes, se crearán un objeto de implementación y un objeto de servicio. Para todos los componentes, excepto para el servidor perimetral llamado puerta de enlace, el objeto de servicio será del tipo ClusterIP . Para la puerta de enlace, el objeto de servicio será del tipo NodePort , que acepta HTTPS externo.

solicitudes en el puerto 30433.

- El servidor de configuración utilizará un ConfigMap, que contiene los archivos de configuración en config-repo.
- Para guardar las credenciales para el servidor de configuración y sus clientes, se crearán dos secretos: uno para el servidor de configuración y otro para sus clientes.

Ahora que hemos visto qué objetos de Kubernetes se crearán, conozcamos las características de Spring Boot que facilitan la implementación en Kubernetes.

Uso del soporte de Spring Boot para el apagado elegante y sondas de actividad y preparación

En Spring Boot v2.3, se agregaron un par de funciones útiles para respaldar las implementaciones en Kubernetes:

- Apagado ordenado: Siempre que sea necesario detener una instancia de microservicio (por ejemplo, en un escenario de actualización continua), existe el riesgo de que las solicitudes activas se vean afectadas al detenerse la instancia. Para minimizar este riesgo, Spring Boot ha añadido compatibilidad con el apagado ordenado. Al aplicar el apagado ordenado, un microservicio deja de aceptar nuevas solicitudes y espera un tiempo configurable para que se completen las solicitudes activas antes de apagar la aplicación. Las solicitudes que tardan más tiempo en completarse que el tiempo de espera de apagado se cancelarán. Estas solicitudes se considerarán casos excepcionales que un procedimiento de apagado no puede esperar antes de detener la aplicación.

El apagado elegante está habilitado de forma predeterminada y se ha configurado con un período de espera de 10 segundos para todos los microservicios agregando lo siguiente a la aplicación común.

Archivo yml en la carpeta config-repo :

```
spring.lifecycle.timeout-por-fase-de-apagado: 10 s
```

Para obtener más información, consulte <https://docs.spring.io/spring-boot/3.5/reference/web/apagado elegante.html>.

Sondas de actividad y preparación: Como se describe en el Capítulo 15, Introducción a Kubernetes, la correcta implementación de las sondas de actividad y preparación es esencial para que Kubernetes pueda gestionar nuestros pods. En resumen, una sonda de actividad indica a Kubernetes si un pod debe reemplazarse, y una sonda de preparación indica si su pod está listo para aceptar solicitudes. Para simplificar este trabajo, Spring Boot ha añadido soporte para implementar sondas de actividad y preparación. Las sondas se exponen en la URL `/actuator/health/Liveness` y `/actuator/health/readiness`, respectivamente. Pueden declararse por configuración o implementación en el código fuente si se requiere un mayor control en comparación con lo que proporciona la configuración. Al declarar las sondas por configuración, se puede declarar un grupo de estado para cada sonda, especificando los indicadores de estado existentes que debe incluir. Por ejemplo, una sonda de preparación debe informar "INACTIVO" si un microservicio no puede acceder a su base de datos MongoDB. En este caso, el grupo de estado de la sonda de preparación debe incluir el indicador de estado Mongo . Para consultar los indicadores de estado disponibles, consulte <https://docs.spring.io/spring-boot/3.5/reference/actuator/endpoints.html#actuator.endpoints.health>. indicadores de salud configurados automáticamente.

En este capítulo, declararemos las sondas utilizando la siguiente configuración en el archivo `com-mon application.yml` en la carpeta `config-repo` :

```
management.endpoint.health.probes.enabled: verdadero
administración.endpoint.health.group.readiness.include: estadodepreparación,
conejo, db, mongo
```

La primera línea de la configuración habilita las sondas de actividad y preparación. La segunda línea declara que las sondas de preparación incluirán indicadores de estado para las bases de datos RabbitMQ, MongoDB y SQL, si están disponibles. Para la sonda de actividad, no es necesario agregar indicadores de estado adicionales. Para el alcance de este capítulo, es suficiente que la sonda de actividad indique "UP" (activa) , siempre que la aplicación Spring Boot esté en funcionamiento.



Para obtener más información, consulte <https://docs.spring.io/spring-boot/3.5/referencia/actuador/endpoints.html#actuador.endpoints>. sondas kubernetes.

Probaremos estas funciones una vez que hayamos implementado nuestros microservicios en Kubernetes. Antes de hacerlo, necesitamos aprender sobre Helm y ver cómo nos ayuda a agrupar, configurar e implementar microservicios a Kubernetes.

Presentando Helm

Como se describió anteriormente, implementar un microservicio en Kubernetes requiere escribir archivos de manifiesto que declaren el estado deseado de un objeto de implementación y un objeto de servicio. Si también necesitamos agregar alguna configuración para los microservicios, se deben agregar manifiestos para ConfigMaps y Secrets. El enfoque de declarar un estado deseado y transferir la responsabilidad a Kubernetes para garantizar que el estado real sea siempre lo más cercano posible al estado deseado es muy útil.

Sin embargo, escribir y mantener estos archivos de manifiesto puede suponer una carga de trabajo considerable. Los archivos contendrán mucho código repetitivo, lo que significa manifiestos duplicados que tendrán el mismo aspecto para todos los microservicios. Además, resulta engorroso gestionar la configuración específica del entorno sin duplicar todo el conjunto de archivos de manifiesto, aunque solo sea necesario actualizar una fracción del contenido.

En el caso de unos pocos microservicios que solo se implementarán en unos pocos entornos, como un entorno de pruebas, control de calidad y producción, esto podría no ser un problema importante. Cuando el número de microservicios crece a decenas o incluso cientos y es necesario implementar diferentes grupos de microservicios en diferentes entornos de pruebas, control de calidad y producción, esto se convierte rápidamente en un problema de mantenimiento inmanejable.

Para solucionar estas deficiencias, utilizaremos Helm (<https://helm.sh>), Un gestor de paquetes de código abierto para Kubernetes. Helm incluye un lenguaje de plantillas que permite extraer la configuración específica de un microservicio o entorno a partir de definiciones genéricas de los distintos objetos de Kubernetes utilizados.



Para entornos de sistemas más pequeños con solo unos pocos objetos de implementación, pueden ser suficientes herramientas de creación de plantillas más sencillas. Por ejemplo, si ya está familiarizado con Ansible y sus plantillas Jinja2, se pueden usar en su lugar. Además, kubectl incluye compatibilidad con Kustomize, lo que ofrece una alternativa sin plantillas para personalizar. Archivos de manifiesto de Kubernetes.

En Helm, un paquete se conoce como diagrama. Un diagrama contiene plantillas, valores predeterminados para las plantillas y dependencias opcionales de las definiciones de otros diagramas. Cada componente que deba implementarse (es decir, los microservicios y los gestores de recursos de los que dependen, como bases de datos y gestores de colas) tendrá su propio diagrama que describe cómo implementarlo.

Para extraer definiciones repetitivas de los gráficos de los componentes, se utilizará un tipo especial de gráfico: el gráfico de biblioteca . Un gráfico de biblioteca no contiene definiciones implementables, sino solo plantillas que se espera que utilicen otros gráficos para los manifiestos de Kubernetes; en nuestro caso, para los objetos de implementación, servicio, mapa de configuración y secreto.

Finalmente, para describir cómo implementar todos los componentes en diferentes tipos de entornos (por ejemplo, para desarrollo y pruebas, o para staging y producción), se utilizará el concepto de diagramas principales y subdiagramas . Definiremos dos tipos de entornos: dev-env y prod- env. Cada entorno se implementará como un diagrama principal que depende de diferentes conjuntos de subdiagramas, por ejemplo, los diagramas de microservicios. Los diagramas de entorno también proporcionarán valores predeterminados específicos del entorno, como el número de pods solicitados, las versiones de imágenes de Docker, las credenciales, y las solicitudes y límites de recursos.

En resumen, tendremos un gráfico de biblioteca reutilizable, denominado común; un conjunto de gráficos específicos de microservicios y administradores de recursos, ubicados en la carpeta de componentes ; y dos gráficos principales específicos del entorno , ubicados en la carpeta de entornos . La estructura de archivos es la siguiente:

```
|-- común
| |-- Gráfico.yaml
| |-- plantillas
| |-- plantillas_org
| `-- valores.yaml
|-- componentes
| |-- servidor de autenticación
| |-- servidor de configuración
| |-- puerta de enlace
| |-- mongodb
| |-- mysql
| |-- producto
| |-- producto compuesto
| |-- conejomq
| |-- recomendación
| |-- reseña
| `-- servidor zipkin
`-- entornos
    |-- entorno de desarrollo
    `-- entorno de producción
```

Los archivos se pueden encontrar en la carpeta \$BOOK_HOME/Chapter16/kubernetes/helm .

Para compartir gráficos de Helm, se pueden publicar en un repositorio de gráficos de Helm. En este libro no publicaremos ningún gráfico, pero en el capítulo 17, «Implementación de funciones de Kubernetes para simplificar el entorno del sistema», instalaremos un componente llamado cert-manager usando un gráfico de Helm de un repositorio de gráficos.

Antes de aprender cómo se construyen los gráficos, aprendamos sobre los más utilizados. Comandos de Helm y cómo ejecutarlos.

Ejecución de comandos de Helm

Para que Helm haga algo por nosotros, usaremos su herramienta CLI, helm.

Algunos de los comandos de Helm más utilizados son los siguientes:

- crear: se utiliza para crear nuevos gráficos.
- Actualización de dependencias (dep up para abreviar): Resuelve dependencias en otros gráficos. Gráficos se colocan en la carpeta de gráficos y se actualiza el archivo Chart.lock .
- creación de dependencias: reconstruye las dependencias según el contenido del archivo Chart.lock .
- plantilla: representa los archivos de definiciones creados por las plantillas.
- install: Instala un gráfico. Este comando puede sobrescribir los valores proporcionados por un gráfico, ya sea usando el indicador --set para sobrescribir un solo valor o usando el indicador --values para proporcionar valores a su propio archivo YAML .
- install --dry-run: simula una implementación sin realizarla; es útil para verificar una implementación antes de ejecutarla.
- lista: enumera las instalaciones en el espacio de nombres actual.
- actualizar: actualiza una instalación existente.
- desinstalar: elimina una instalación.

Para obtener la documentación completa de los comandos que proporciona Helm, consulte <https://helm.sh/docs/helm/>.

Pongamos estos comandos de Helm en contexto y veamos de qué archivos consta un gráfico.

Mirando un gráfico de Helm

Una carta Helm tiene una estructura de archivos predefinida. Usaremos los siguientes archivos:

- Chart.yaml, que contiene información general sobre el gráfico y una lista de otros gráficos
Podría depender de.
- plantillas, que es una carpeta que contiene las plantillas que se utilizarán para implementar el cuadro.

- `values.yaml`, que contiene valores predeterminados para las variables utilizadas por las plantillas.
- `Chart.lock`, un archivo creado por Helm al resolver las dependencias descritas en el archivo `Chart.yaml`. Esta información describe con más detalle qué dependencias se utilizan. Helm lo utiliza para rastrear todo el árbol de dependencias, lo que permite recrearlo exactamente como se veía la última vez que el gráfico funcionó.
- `gráficos`, que es una carpeta que contendrá los gráficos de los que depende este gráfico después de Helm Ha resuelto las dependencias.
- `.helmignore`, que es un archivo de ignorados similar a `.gitignore`. Se puede usar para listar archivos que... Debe excluirse al construir el gráfico.

Ahora que entendemos la estructura dentro de un diagrama de Helm, aprendamos sobre una de las características principales:

Características de Helm: su mecanismo de plantilla y cómo pasarle valores.

Plantillas y valores de Helm

Las plantillas de Helm se utilizan para parametrizar los archivos de manifiesto de Kubernetes. Gracias a ellas, ya no es necesario mantener manifiestos de implementación extensos para cada microservicio. En su lugar, podemos definir una plantilla común que contiene marcadores de posición para indicar dónde se colocarán los valores específicos del microservicio en la plantilla cuando se renderice un manifiesto para un microservicio específico. Veamos un ejemplo, extraído de `kubernetes/helm/common/templates/_deployment.yaml`:

```
apiVersion: aplicaciones/v1
tipo: Implementación
metadatos:
  nombre: {{ include "common.fullname" . }}
especulación:
  réplicas: {{ .Values.replicaCount }}
  plantilla:
    especulación:
      contenedores:
        - nombre: {{ .Chart.Name }}
```

Es muy similar al manifiesto de implementación que vimos en el Capítulo 15, Introducción a Kubernetes, con la excepción del uso de las construcciones `{{ ... }}`, que se usan para insertar valores específicos del microservicio en la plantilla. La construcción `{{ include "common.fullname" . }}` se usa para invocar otras plantillas, como se explica brevemente. Las otras dos construcciones se usan para insertar valores mediante uno de los objetos integrados de Helm. Las partes más utilizadas de los objetos integrados son como sigue:

- **Valores:** se utiliza para hacer referencia a los valores en el archivo `values.yaml` del gráfico o a los valores suministrados cuando ejecutando un comando de Helm como `instalar`.
- **Versión:** Se utiliza para proporcionar metadatos sobre la versión actual que está instalada. Contiene
Contiene campos como los siguientes:
 - **Nombre:** El nombre del lanzamiento
 - **Espacio de nombres:** el nombre del espacio de nombres donde se realiza la instalación
 - **Servicio:** El nombre del servicio de instalación, que siempre devuelve Helm
- **Gráfico:** Se utiliza para acceder a la información del archivo `Chart.yaml`. Ejemplos de campos que pueden
Los siguientes son algunos ejemplos de metadatos útiles para una implementación:
 - **Nombre:** El nombre del gráfico
 - **Versión:** El número de versión del gráfico
- **Archivos:** Contienen funciones para acceder a archivos específicos de gráficos. En este capítulo, usaremos
las siguientes dos funciones en el objeto `Archivos` :
 - **Glob:** Devuelve los archivos de un gráfico según un patrón glob. Por ejemplo, el patrón `"config-repo/*"`
devolverá todos los archivos que se encuentren en la carpeta `config-repo`.
 - **AsConfig:** Devuelve el contenido de los archivos como un mapa YAML apropiado para declarar
valores en `ConfigMap`.
- **Capacidades:** Se puede usar para encontrar información sobre las capacidades del clúster de Kubernetes en el que
se realiza la instalación. Por ejemplo, una plantilla puede usar la información de este objeto para adoptar un
manifiesto basado en las versiones de API compatibles con el clúster de Kubernetes. No usaremos este objeto
en este capítulo, pero creo que nos conviene.
Tenga esto en cuenta para casos de uso más avanzados.

Para obtener más detalles sobre los objetos integrados, consulte https://helm.sh/docs/chart_template_guide/objetos_incorporados.

Todos los objetos son accesibles en un árbol donde, en la mayoría de los casos, el contexto raíz se puede acceder mediante el ámbito actual, representado por un punto, `"."`, también conocido como punto. En los ejemplos anteriores, podemos ver el uso del punto, por ejemplo, en `.Values.replicaCount` y `.Chart.Name`, donde se puede observar que los objetos integrados `Values` y `Chart` son accesibles directamente desde el ámbito actual. En la directiva de inclusión anterior, también podemos ver que el punto se usa como parámetro enviado a la plantilla denominada `common.fullname`, lo que significa que el árbol completo se envía a la plantilla. En lugar de enviar el árbol completo a una plantilla, se puede pasar un subárbol.

Al usar algunas funciones de Helm, el ámbito actual cambiará y ya no apuntará al contexto raíz . Por ejemplo, más adelante veremos la función de rango , que permite iterar sobre conjuntos de valores. Si necesitamos acceder al contexto raíz dentro del ámbito de una función de rango , podemos usar la variable predefinida \$.

Las plantillas de Helm también admiten la declaración de variables para hacer referencia a otros objetos, como en este ejemplo:

```
$nombre := .Nombre.de.lanzamiento
```

En este ejemplo, se ha declarado una variable, "nombre", que contiene el valor de la versión de Helm que se está procesando. Más adelante veremos cómo se usan las variables en construcciones más avanzadas.



Si reconoce el formato de las construcciones {{ ... }} al usar kubectl, tiene razón. En ambos casos, se basan en plantillas de Go. Para más información, consulte <https://golang.org/pkg/text/template/>.

Tras la introducción del mecanismo de plantillas, aprendamos cómo se construyen los tres tipos de gráficos. Comenzaremos con el gráfico más importante, el gráfico común , y luego explicaremos los gráficos de componentes y entornos .

El cuadro de la biblioteca común

Este gráfico contiene plantillas reutilizables, también conocidas como plantillas con nombre, para los cuatro tipos de manifiestos de Kubernetes que usaremos en este capítulo: Implementación, Servicio, ConfigMap y Secreto.

La estructura y el contenido del gráfico común se basan en la salida de una creación de Helm.

Comando. En concreto, se ha conservado el archivo de plantilla _helpers.tpl para reutilizar las mejores prácticas de nomenclatura.

Declara las siguientes plantillas que encapsulan las convenciones de nomenclatura:

- common.name: basado en el nombre del gráfico.
- common.fullname: Basado en una combinación del nombre del lanzamiento y el gráfico. En

En este libro, anularemos esta convención de nomenclatura y simplemente utilizaremos el nombre del gráfico.

- common.chart: basado en el nombre y la versión del gráfico.

Para obtener más detalles, consulte la implementación en el archivo _helpers.tpl .

Las plantillas con nombre, que solo serán utilizadas por otras plantillas y no se usarán para crear manifiestos, deben tener un nombre que comience con un guion bajo, "_". Esto se usa para evitar que Helm intente crear manifiestos usándolas únicamente.

Dado que las plantillas nombradas para los manifiestos de Kubernetes mencionados anteriormente contienen la parte principal de la lógica y, por lo tanto, la mayor parte de la complejidad en los diagramas de Helm, las revisaremos una por una.

La plantilla ConfigMap

Esta plantilla está diseñada para crear ConfigMaps a partir de los archivos de la carpeta `config-repo`. Cada ConfigMap contendrá todas las configuraciones no confidenciales requeridas por una implementación específica. El manifiesto de la implementación asignará el contenido del ConfigMap como un volumen en su plantilla de pod. Esto permitirá que los pods creados por la implementación accedan a la configuración como archivos en su sistema de archivos local. Consulte la sección "La plantilla de implementación" para obtener más información. El archivo `config-repo` La carpeta debe colocarse en los gráficos que utilizan el gráfico común.



En este capítulo, esta plantilla solo la usará el gráfico del servidor de configuración en la carpeta de componentes. En el siguiente capítulo, todos los demás microservicios también usarán esta plantilla para definir sus propios ConfigMaps, ya que se eliminará el servidor de configuración.

El archivo de plantilla se llama `_configmap_from_file.yaml` y tiene este aspecto:

```
{{- define "common.configmap_from_file" -}}
Versión de API: v1
tipo: ConfigMap
metadatos:
  nombre: {{ include "common.fullname" . }}
  etiquetas:
    app.kubernetes.io/nombre: {{ incluir "common.name" . }}
    helm.sh/chart: {{ incluir "common.chart" . }}
    app.kubernetes.io/administrado por: {{ .Release.Service }}
datos:
  {{ (.Files.Glob "config-repo/*").AsConfig | sangría 2 }}
{{- fin -}}
```

Una explicación de la plantilla es la siguiente:

La primera línea, `{{- define "common.configmap_from_file" -}}`, se utiliza para declarar el nombre de la plantilla reutilizable. El alcance de la plantilla termina con un `{{- end -}}`, la última línea de este ejemplo.

- Para establecer el nombre del ConfigMap, la plantilla `common.fullname` de `_helpers`.

Se utiliza el archivo `tpl`.

- A continuación, se definen una serie de etiquetas para facilitar la identificación del ConfigMap más adelante.

Nuevamente, se utilizan las plantillas del archivo `_helpers.tpl` para establecer el nombre y especificar el gráfico utilizado. Para indicar que este servicio se creó con Helm, se utiliza `app.kubernetes`.

La etiqueta `io/managed-by` se establece en el valor del campo `.Release.Service`. Según la descripción anterior del objeto `Release`, sabemos que siempre devuelve el valor de Helm.

- A continuación, se encuentra la parte principal del ConfigMap: su sección de datos. Para especificar la configuración en el ConfigMap, se utiliza la función `Glob` del objeto `Archivos` para obtener todos los archivos de la carpeta `config-repo`. A continuación, se aplica la función `AsConfig` al contenido de los archivos para crear un mapa YAML adecuado. El resultado se envía a la función de sangría, que garantiza una sangría correcta, en este caso, con dos caracteres.

Los guiones en `{{- y -}}` se utilizan para eliminar los espacios en blanco anteriores y finales que quedan después del procesamiento de la directiva dentro de las llaves.

Ejemplo de uso de la plantilla ConfigMap

En este capítulo, solo el servidor de configuración usará un ConfigMap. Consulte la sección "Gráficos de componentes" para obtener una descripción de cómo se usa esta plantilla.

Para ver el ConfigMap que Helm creará usando esta plantilla, ejecute el siguiente comando:

Manda:

```
cd $BOOK_HOME/Capítulo16/kubernetes/helm/componentes/config-server
Actualización de dependencia de timón.
plantilla de timón . -s plantillas/configmap_from_file.yaml
```

Espere un resultado del comando de plantilla helm como el siguiente:

```
---
# Fuente: config-server/templates/configmap_from_file.yaml
Versión de API: v1
tipo: ConfigMap
metadatos:
  nombre: servidor de configuración
  etiquetas:
    app.kubernetes.io/name: servidor de configuración
    helm.sh/chart: servidor de configuración-1.0.0
    app.kubernetes.io/administrado por: Helm
datos:
  aplicación.yaml: |-
```

```

aplicación:
  servidor de autenticación: localhost
...
auth-server.yml: |-
  puerto del servidor: 9999
...

```

El campo de datos contiene el contenido de todos los archivos en la carpeta config-repo .

Plantilla Secretos. Esta plantilla está

diseñada para crear secretos definidos por valores como las credenciales proporcionadas por los entornos de desarrollo y producción . Los secretos se asignarán como variables de entorno en los pods. Consulte la sección "Plantilla de implementación" para obtener más información. Dado que un entorno debe poder definir varios secretos, esta plantilla está diseñada para crear varios manifiestos de secretos mediante la función "range" de Helm. El archivo de plantilla se llama `_secrets.yaml` y tiene este aspecto:

```

{{- define "common.secrets" -}} {{- rango
$secretName, $secretMap := .Values.secrets }} apiVersion: v1

tipo: Secreto
metadatos:
  nombre: {{ $secretName }}
  etiquetas:
    app.kubernetes.io/nombre: {{ $secretName }} helm.sh/
    gráfico: {{ incluir "common.chart" $ }} app.kubernetes.io/
    administrado por: {{ $.Release.Service }}
tipo: Opaco
datos:
  {{- rango $clave, $valor := $mapasecreto }}
    {{ $key }}: {{ $val | b64enc }} {{- fin }}
...

{{- fin -}} {{- fin
-}}

```

Una explicación de la plantilla es la siguiente:

Tras la declaración de la plantilla en la línea 1, se utiliza la función `range` en la línea 2. Esta función asume que el campo `.Values.secrets` contiene un mapa de nombres de secretos y un mapa de sus pares clave/valor. La declaración del campo `Secrets` en uno de los archivos `values.yaml` del entorno se verá así:

```
misterios:
  un secreto:
    clave-1: valor-secreto-1
    clave-2: valor-secreto-2
  Otro secreto:
    clave-3: valor-secreto-3
```

Esta definición generará dos secretos, llamados "a-secret" y "other-secret". El rango La función asigna el nombre del secreto actual y su mapa a `$secretName` y `$secretMap` variables.

Dado que la función `range` modifica el alcance actual, ya no podemos usar la notación de punto para pasar el contexto raíz a la plantilla `common.chart`. En su lugar, se debe usar la variable `$`.

- En la sección de datos del manifiesto, se aplica una segunda función de rango para recorrer los pares clave-valor del secreto actual. Cada par clave-valor se asigna por rango. función a las variables `$key` y `$val`.
- Finalmente, los pares clave-valor del secreto se definen como una entrada de mapa en la sección de datos. El valor de la variable `$val` se canaliza a la función `b64enc` para codificarlo correctamente en Base64, según lo exige el manifiesto del secreto.

Los caracteres `---` se utilizan para separar los manifiestos secretos representados entre sí para que se procesen como documentos YAML separados.

Ejemplo de uso de la plantilla Secretos

Los secretos solo se definen mediante los gráficos de entorno `dev-env` y `prod-env`. Se utilizan para crear credenciales específicas del entorno. Consulte la sección "Gráficos de entorno" para obtener una descripción del uso de esta plantilla.

Para ver los secretos que Helm creará para `dev-env` usando esta plantilla, ejecute los siguientes comandos:

```
cd $BOOK_HOME/Capítulo16/kubernetes/helm para f en
componentes/*; hacer la actualización de la dependencia de Helm $f; hecho
actualización de dependencia de Helm entornos/dev-env plantilla de
Helm entornos/dev-env -s plantillas/secrets.yaml
```

Espere un resultado del comando de plantilla de timón como este:

```
---
# Fuente: dev-env/templates/secrets.yaml apiVersion: v1

tipo: Secreto
metadatos:
  nombre: config-client-credentials
  etiquetas:
    app.kubernetes.io/name: credenciales-del-cliente-de-configuración
    helm.sh/chart: dev-env-1.0.0
    app.kubernetes.io/administrado por: Helm
tipo: Opaco
datos:
  CONTRASEÑA DEL SERVIDOR DE CONFIGURACIÓN: ZGV2LXB3ZA==
  USUARIO DEL SERVIDOR DE CONFIGURACIÓN: ZGV2LXVzcg==
---

# Fuente: dev-env/templates/secrets.yaml
Versión de API: v1
tipo: Secreto
metadatos:
  nombre: config-server-secrets
  etiquetas:
    app.kubernetes.io/name: secretos del servidor de configuración
    helm.sh/chart: dev-env-1.0.0
    app.kubernetes.io/administrado por: Helm
tipo: Opaco
datos:
  CLAVE DE CIFRADO: bXktdmVyeS1zZW51cmUtZW5jcnlwdC1rZXk=
  NOMBRE DE USUARIO DE SPRING_SECURITY: ZGV2LXVzcg==
  CONTRASEÑA DE USUARIO DE SPRING_SECURITY: ZGV2LXB3ZA==
```

La plantilla de servicio

La plantilla de Servicio permite sobrescribir los valores predeterminados del gráfico común, con valores específicos para los gráficos que lo utilizan. El gráfico común, por ejemplo, proporcionará valores predeterminados para el tipo de Servicio y los puertos que este expondrá. Esto será útil para la mayoría de los microservicios, pero algunos necesitan poder sobrescribir estos valores predeterminados en su propio archivo `values.yaml`.

El archivo de plantilla se llama `_service.yaml` y comienza como las demás plantillas con nombre, con la declaración de su nombre, seguida de la implementación del mecanismo de anulación. Su aspecto es el siguiente:

```
{{- define "common.service" -}}
{{- $common := dict "Valores" .Valores.common -}}
{{- $noCommon := omit .Valores "comunes" -}}
{{- $overrides := dict "Valores" $noCommon -}}
{{- $noValues := omit . "Valores" -}}
{{- con fusión $noValues $overrides $common -}}
```

Esta construcción se puede explicar de la siguiente manera:

- Cuando un microservicio utiliza la plantilla `_service.yaml` para representar su manifiesto de servicio, los valores del archivo `values.yaml` del microservicio estarán disponibles en `.Values` objeto, y los valores del gráfico común estarán disponibles en el campo `.Values.common`.
- Por lo tanto, la variable `$common` hará referencia a un diccionario, creado por la función `dict`, con una clave, `Valores`, y su valor será el valor predeterminado del gráfico común. Estos valores se toman de la clave común del objeto `.Values`.
- La variable `$noCommon` contendrá todos los valores del microservicio excepto los valores bajo el clave común, especificada mediante la función `omit`.

La variable `$overrides` hará referencia a un diccionario, también con una clave, `Valores`, pero su valor será el de los valores del microservicio, excepto los valores comunes. Obtiene los valores de la variable `$noCommon` declarada en la línea anterior.

- La variable `$noValues` contendrá todos los demás objetos integrados, excepto el objeto `Values`.
- Ahora, aquí es donde ocurrirá la anulación; la función de combinación creará un diccionario basado en los diccionarios a los que hacen referencia las variables `$noValues`, `$overrides` y `$common`. En este caso, los valores encontrados en el diccionario `$overrides` tendrán prioridad sobre los valores del diccionario `$common`, anulando así sus valores.

- Finalmente, la función with cambiará el alcance del código de plantilla que sigue hasta alcanzar su definición `{{- end -}}`. Por lo tanto, el alcance actual, ".", se referirá ahora al diccionario fusionado.

Veamos un ejemplo para ver cómo funciona. El archivo `values.yaml` del gráfico común contiene la siguiente configuración predeterminada para el tipo de servicio y los puertos expuestos:

```
Servicio:
  tipo: ClusterIP
  puertos:
    - puerto: 80
      puerto de destino: http
      protocolo: TCP
      nombre: http
```

Esta configuración renderizará los objetos de servicio de tipo ClusterIP. Estos objetos expondrán el puerto 80 y reenviarán las solicitudes a los pods en su puerto, denominado http.

El servicio de puerta de enlace necesita exponer un NodePort y usar otras configuraciones de puerto. Para anular los valores predeterminados anteriores, declara lo siguiente en el archivo `values.yaml` de su gráfico :

```
servicio:
  tipo: NodePort
  puertos:
    - puerto: 443
      puerto de destino: 8443
      puerto del nodo: 30443
```

El archivo `values.yaml` de la puerta de enlace se puede encontrar en la carpeta `$BOOK_HOME/Chapter16/kubernetes/helm/components/gateway/values.yaml`.

El resto del archivo de plantilla de servicio se ve así:

```
Versión de API: v1
tipo: Servicio
metadatos:
  nombre: {{ include "common.fullname" . }}
etiquetas:
  app.kubernetes.io/nombre: {{ incluir "common.name". }} helm.sh/gráfico: {{ incluir
    "common.chart". }}
```

```

    app.kubernetes.io/administrado por: {{ .Release.Service }}

    especulación:
      tipo: {{ .Values.service.type }}
      puertos:
        {{ toYaml .Values.service.ports | sangría 4 }}
      selector:
        app.kubernetes.io/nombre: {{ incluir "common.name" . }}
    {{- fin -}}
    {{- fin -}}

```

Una explicación de la plantilla es la siguiente:

- Los campos de metadatos para el nombre y las etiquetas se definen de la misma manera que ya se vio para las plantillas anteriores.

- El tipo de servicio se establece mediante el campo `.Values.service.type`.

Los puertos expuestos por el Servicio se especifican mediante el campo `.Values.service.ports`. La función integrada `toYaml` se utiliza para formatear su valor como `yaml`, y el resultado se canaliza a la función `indent`, lo que garantiza una sangría correcta, en este caso, de 4 caracteres.

- Finalmente, se define el selector de pods. Se basa en la etiqueta `app.kubernetes.io/name` y se le asigna un nombre mediante la plantilla `common.name`.

Ejemplo de uso de la plantilla de Servicio

Cada componente utiliza la plantilla de servicio para crear su manifiesto de servicio. Como se describió anteriormente, los microservicios principales reutilizan la configuración del archivo `values.yaml` del gráfico común, mientras que los demás componentes sobrescriben estos valores en su propio archivo `values.yaml`.

Para ver el manifiesto de servicio generado para un componente principal del microservicio del producto, ejecute los siguientes comandos:

```

cd $BOOK_HOME/Capítulo16/kubernetes/helm
componentes/producto de actualización de dependencia de Helm
componentes de plantilla de timón/producto -s plantillas/servicio.yaml

```

Espere un resultado del comando de plantilla de timón como este:

```

# Fuente: producto/plantillas/servicio.yaml
Versión de API: v1
tipo: Servicio
metadatos:
  nombre: producto

```

```
etiquetas:
  app.kubernetes.io/name: producto helm.sh/chart:
  producto-1.0.0 app.kubernetes.io/managed-
  by: Helm
```

```
especificación: tipo: ClusterIP
```

```
puertos:
```

```
- nombre: http
  puerto: 80
  protocolo: TCP puerto
  de destino: http
```

```
selector:
```

```
  app.kubernetes.io/name: producto
```

Para ver el manifiesto de servicio generado para un componente que anula las configuraciones en el gráfico común para el componente de puerta de enlace , ejecute los siguientes comandos:

```
cd $BOOK_HOME/Capítulo 16/kubernetes/helm helm
dependencia actualización componentes/puerta de enlace
componentes de plantilla de timón/puerta de enlace -s plantillas/servicio.yaml
```

Espere un resultado del comando de plantilla de timón como este:

```
---
# Fuente: gateway/templates/service.yaml apiVersion: v1
```

```
tipo: Servicio
```

```
metadatos:
```

```
  nombre: puerta de enlace
```

```
  etiquetas:
```

```
    app.kubernetes.io/name: puerta de enlace helm.sh/
    chart: puerta de enlace-1.0.0 app.kubernetes.io/
    administrado por: Helm
```

```
  especificación:
```

```
    tipo: NodePort puertos:
```

```
      - nodePort: 30443
        puerto: 443
        puerto de destino: 8443
```

```
    selector:
```

```
      app.kubernetes.io/name: puerta de enlace
```

La plantilla de implementación. Finalmente, analicemos la plantilla para renderizar manifiestos de implementación. Esta es la plantilla más compleja, ya que debe gestionar muchas partes del manifiesto de implementación como opcionales. Cada componente utilizará distintas partes de un manifiesto de implementación. El archivo común chart values.yaml contiene valores predeterminados para estas configuraciones, aplicables a la mayoría de los componentes, lo que minimiza la necesidad de sobrescribirlas en el archivo chart values.yaml de cada componente. Las siguientes partes del manifiesto de implementación son opcionales para los componentes:

- Argumentos dados al contenedor cuando se inicia
- Variables de entorno
- Variables de entorno de Secrets
- La sonda de vitalidad
- La sonda de preparación
- Un ConfigMap y un volumen correspondiente

El archivo de plantilla se llama _deployment.yaml y sus primeras líneas se ven muy similares a la plantilla de Servicio, utilizando el mismo tipo de mecanismo de anulación:

```
{{- define "common.deployment" -}} {{-
$common := dict "Values" .Values.common -}} {{- $noCommon :=
omit .Values "common" -}} {{- $overrides := dict "Values"
$noCommon -}} {{- $noValues := omit . "Values" -}} {{- con
fusión $noValues $overrides $common -}}

apiVersion: apps/v1 tipo:
Implementación
metadatos:
  nombre: {{ include "common.fullname" . }}
  etiquetas:
    app.kubernetes.io/name: {{ include "common.name" . }} helm.sh/chart:
    {{ include "common.chart" . }} app.kubernetes.io/managed-by:
    {{ .Release.Service }}
```

Para obtener una explicación de esta parte de la plantilla, consulte la descripción anterior de la plantilla de Servicio.

Cuando se trata de la parte de especificaciones del manifiesto, comienza con lo siguiente:

```

especificación:
  réplicas: {{ .Values.replicaCount }}
  selector:
    etiquetas de coincidencia:
      app.kubernetes.io/nombre: {{ incluir "common.name" . }}
  plantilla:
    metadatos:
      etiquetas:
        app.kubernetes.io/nombre: {{ incluir "common.name" . }}
    especificación:
      contenedores:
        - nombre: {{ .Chart.Name }}
          imagen: "{{ .Values.image.repository }}/{{ .Values.image.name
        }}:{{ .Values.image.etiqueta }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}

```

Aquí podemos ver cómo se definen las partes principales de la especificación: el número de réplicas solicitadas, el selector para los pods y la plantilla utilizada para crear nuevos pods. La plantilla define etiquetas que coinciden con el selector y el nombre, la imagen de Docker y la política de extracción de imagen (imagePullPolicy) que se usará al iniciar un... recipiente.

A continuación vienen las distintas partes opcionales del manifiesto, como se describió anteriormente:

```

argumentos:
  {{- aYaml . {{- | sangría 12 }}
  fin }}
  {{- si .Values.env }}
    entorno:
      {{- rango $key, $val := .Values.env }} - nombre:
        {{ $key }}
        valor: {{ $val }} {{- fin }}

    {{- fin }} {{-
    si .Values.envFromSecretRefs }}
    envDe:
      {{- rango .Values.envFromSecretRefs }}

```

```

- referencia secreta:
    nombre: {{ . }}
{{- fin }} {{-
fin }}
{{- si .Values.livenessProbe_enabled }}
Sonda de vitalidad:
{{ toYaml .Values.livenessProbe | gui3n 12 }}
{{- fin }}
{{- si .Values.readinessProbe_enabled }}
Sonda de preparaci3n:
{{ toYaml .Values.readinessProbe | sangr3a 12 }}
{{- fin }}

```

Para las variables de entorno y los secretos asignados a variables de entorno, la funci3n de rango se utiliza de la misma manera que la plantilla de secretos . Las variables de entorno se pueden especificar a nivel de componente o de entorno, seg3n su caso de uso. Los secretos siempre se especifican mediante un gr3fico de entorno. Consulte las siguientes secciones sobre el componente. y gr3ficos ambientales.

El manifiesto concluye con la declaraci3n de los puertos que expone el contenedor, las solicitudes y l3mites de recursos y, finalmente, la declaraci3n opcional de un ConfigMap y un volumen correspondiente para mapear los archivos en el ConfigMap:

```

puertos:
{{ toYaml .Values.ports | sangr3a 12 }}
recursos:
{{ toYaml .Values.resources | sangr3a 12 }}
{{- si .Values.configmap.enabled }}
montajes de volumen:
- nombre: {{ include "common.fullname" . }} Ruta de
montaje: {{ .Values.configmap.volumeMounts.mountPath }}
vol3menes:
- nombre: {{ include "common.fullname" . }}
configMap:
nombre: {{ include "common.fullname" . }}
{{- fin }} {{- fin
-}}
{{- fin -}}

```

Desde el archivo `values.yaml` del gráfico común , podemos encontrar algunos valores predeterminados de interés, por ejemplo, cómo se definen los valores predeterminados para las sondas de actividad y preparación:

```
livenessProbe_enabled: falso
Sonda de vitalidad:
  Obtener http:
    esquema: HTTP
    ruta: /actuador/salud/vida
    puerto: 80
  initialDelaySeconds: 10
  periodoSegundos: 10
  tiempo de esperaSegundos: 2
  Umbral de fallo: 20
  Umbral de éxito: 1

readinessProbe_enabled: falso
Sonda de preparación:
  Obtener http:
    esquema: HTTP
    ruta: /actuador/salud/preparación
    puerto: 80
  initialDelaySeconds: 10
  periodoSegundos: 10
  tiempo de esperaSegundos: 2
  Umbral de fallo: 3
  Umbral de éxito: 1
```

De estas declaraciones podemos ver lo siguiente:

- Las sondas están deshabilitadas de forma predeterminada ya que no todas las implementaciones las utilizan.
- Las sondas se basan en solicitudes HTTP GET enviadas a los puntos finales expuestos por Spring Boot, como se describe en el apartado anterior `Uso del soporte de Spring Boot para el apagado elegante y las sondas para Sección de vitalidad y preparación` .
- Mientras el punto final responda con un código de respuesta 2xx o 3xx , la sonda se considera... para tener éxito.
- Las sondas se pueden configurar utilizando los siguientes parámetros:

- `initialDelaySeconds` especifica cuánto tiempo espera Kubernetes para sondear un contenedor

después de que se pone en marcha.

- `periodSeconds` especifica el tiempo entre solicitudes de sondeo enviadas por Kubernetes.
- `timeoutSeconds` especifica cuánto tiempo espera Kubernetes una respuesta antes de tratarla.
La sonda ha fallado.
- `FailureThreshold` especifica cuántos intentos fallidos realiza Kubernetes antes de abandonar. En el caso de una prueba de actividad, esto significa reiniciar el pod. En el caso de una prueba de preparación, significa que Kubernetes no enviará más solicitudes al contenedor hasta que las pruebas de preparación vuelvan a tener éxito.

El umbral de éxito especifica el número de intentos exitosos necesarios para que una sonda se considere exitosa nuevamente tras un fallo. Esto solo aplica a las sondas de preparación, ya que debe establecerse en 1 si se especifica para las sondas de actividad.



Encontrar configuraciones óptimas para las sondas puede ser un desafío; es decir, encontrar un equilibrio adecuado entre obtener una reacción rápida de Kubernetes cuando cambia la disponibilidad de un pod y no sobrecargar los pods con solicitudes de sondas.

En concreto, configurar una sonda de actividad con valores demasiado bajos puede provocar que Kubernetes reinicie pods que no necesitan reiniciarse; solo necesitan más tiempo para iniciarse. Iniciar un gran número de pods a la vez, lo que también resulta en tiempos de inicio demasiado largos, puede provocar muchos reinicios innecesarios.

Configurar valores de configuración demasiado altos en las sondas (excepto el valor `"successThreshold"`) hace que Kubernetes reaccione con mayor lentitud, lo cual puede resultar molesto en un entorno de desarrollo. Los valores adecuados también dependen del hardware disponible, lo que afecta los tiempos de inicio de los pods. Para el propósito de este libro, el valor `"failurethreshold"` de las sondas de actividad se establece en 20 para evitar reinicios innecesarios en equipos con recursos de hardware limitados.

Ejemplo de uso de la plantilla de implementación

Cada componente utiliza la plantilla de implementación para crear su manifiesto de implementación. Los microservicios principales reutilizan la mayor parte de la configuración del archivo `values.yaml` del gráfico común, lo que minimiza la necesidad de configuración específica del componente, mientras que los demás componentes sobrescriben la mayor parte de estos valores en su propio archivo `values.yaml`.

Para ver el manifiesto de implementación generado para un componente principal, ejecute los siguientes comandos para el microservicio del producto :

```
cd $BOOK_HOME/Capítulo16/kubernetes/helm
componentes/producto de actualización de dependencia de Helm
componentes de plantilla de helm/producto -s plantillas/implementación.yaml
```

Para ver el manifiesto de implementación generado para un componente que anula las configuraciones en el gráfico común, ejecute los siguientes comandos para el componente MongoDB:

```
cd $BOOK_HOME/Capítulo16/kubernetes/helm
actualización de dependencia de Helm componentes/mongodb
plantilla de Helm componentes/mongodb -s plantillas/deployment.yaml
```

Espere un resultado del comando de plantilla de timón como este:

```
---

# Fuente: mongodb/templates/deployment.yaml
apiVersion: apps/v1 tipo:
Implementación
metadatos:
  nombre: mongodb
  etiquetas:
    app.kubernetes.io/nombre: mongodb
    helm.sh/chart: mongodb-1.0.0 app.kubernetes.io/
    administrado por: Helm
  especulación:
    réplicas: 1
    selector:
      etiquetas de coincidencia:
        app.kubernetes.io/name: plantilla mongodb:

  metadatos:
    etiquetas:
      app.kubernetes.io/nombre: mongodb
    especulación:
      contenedores:
        - nombre: mongodb
          Imagen: "registry.hub.docker.com/library/mongo:8.0.5" Política de extracción de
          imagen: IfNotPresent
          puertos:
            - Puerto del contenedor: 27017
          recursos:
            límites:
              memoria: 400Mi
```

Con esto concluye el recorrido por las plantillas reutilizables con nombre en el gráfico común. Los archivos se encuentran en la carpeta `$BOOK_HOME/Chapter16/kubernetes/helm/common`.

A continuación, veamos cómo se definen los gráficos específicos de cada componente.

Los gráficos de componentes

Los gráficos de los microservicios y los administradores de recursos se almacenan en la carpeta de componentes y todos comparten la misma estructura de archivos:

- `Chart.yaml` expresa una dependencia de la biblioteca común `chart`.

La carpeta de plantillas contiene dos plantillas: `ployment.yaml` y `service.yaml`. Ambas aplican la plantilla con nombre correspondiente del gráfico común. Por ejemplo, la plantilla `Service.yaml` tiene este aspecto:

```
{{- plantilla "common.service" . -}}
```

- El archivo `values.yaml` contiene la configuración específica del microservicio. Por ejemplo, los valores del archivo para el gráfico del servidor de autenticación se ven así:

```
nombre completoAnulación: auth-server

imagen:
  nombre: auth-server

entorno:
  PERFILES DE PRIMAVERA ACTIVOS: "docker"

livenessProbe_enabled: verdadero

readinessProbe_enabled: verdadero
```

El servidor de autenticación solo necesita declarar su nombre, imagen de Docker, perfil de Spring y que desea utilizar la configuración predeterminada de las sondas de actividad y preparación.

El servidor de configuración se diferencia de los demás gráficos en que utiliza un `ConfigMap` para almacenar el repositorio de configuración, que contiene los archivos de configuración de todos los demás microservicios. En su carpeta de plantillas, define una plantilla para un `ConfigMap` basada en la plantilla especificada en el gráfico común para `ConfigMaps` que ya hemos visto:

```
{{- plantilla "common.configmap_from_file" . -}}
```

La plantilla espera encontrar los archivos de propiedades en la carpeta de gráficos, config-repo. Para evitar la duplicación de config -repo desde \$BOOK_HOME/Capítulo16/config-repo, se ha creado un enlace simbólico con el siguiente comando :

```
cd $BOOK_HOME/Capítulo16/kubernetes/helm/componentes/config-server
ln -s ../../../../config-repo config-repo
```



Dado que Git conserva los enlaces suaves, no es necesario volver a crear el enlace suave: el clon de Git ¡Command lo hace por ti!

Como ya se mencionó en el recorrido del diagrama común, el servicio de enlace se diferencia de los otros microservicios porque necesita exponer un servicio del tipo NodePort .

Además de los gráficos para los microservicios, la carpeta de componentes también contiene gráficos para las bases de datos, el agente de mensajes y el servidor Zipkin que utilizamos. Su estructura es la misma que la de los microservicios. Dado que las plantillas comunes se diseñaron para optimizar los gráficos para los microservicios, los demás gráficos deben sobrescribir más valores predeterminados en los archivos values.yaml que los de los microservicios. Para más detalles, consulte los archivos values.yaml en las siguientes carpetas: mongodb, mysql, rabbitmq y zipkin-server.

Los gráficos ambientales

Finalmente, los diagramas dev-env y prod-env de la carpeta de entornos integran todo para completar los paquetes de instalación de un entorno típico de desarrollo/pruebas o de ensayo/producción . Su archivo Charts.yaml contiene dependencias tanto del diagrama común como de los diagramas de la carpeta de componentes , y la carpeta de plantillas contiene una plantilla secrets.yaml para crear credenciales específicas del entorno como Secretos. Se basa en la plantilla con nombre para Secretos de

El gráfico común se ve así:

```
{{- plantilla "common.secrets" . -}}
```

Al observar el archivo values.yaml del gráfico dev-env , podemos encontrar los siguientes valores secretos definidos para el secreto config-server-secrets :

misterios:

secretos del servidor de configuración:

ENCRYPT_KEY: mi clave de cifrado muy segura

NOMBRE DE USUARIO DE SPRING_SECURITY: dev-usr

CONTRASEÑA DE USUARIO DE SPRING_SECURITY: dev-pwd

Esto dará como resultado que el secreto config-server-secrets contenga tres valores secretos, todos Base64-encodificado. Su manifiesto se verá así:

```

Versión de API: v1
tipo: Secreto
metadatos:
  nombre: config-server-secrets
  etiquetas:
    ...
tipo: Opaco
datos:
  CLAVE DE CIFRADO: bXktdmVyeS1zZW51cmUtZW5jcnlwdC1rZXk=
  NOMBRE DE USUARIO DE SPRING_SECURITY: ZGV2LXVzeg==
  CONTRASEÑA DE USUARIO DE SPRING_SECURITY: ZGV2LXB3ZA==

```



Tenga en cuenta que este archivo values.yaml contiene información confidencial, por ejemplo, la clave de cifrado utilizada por el servidor de configuración y la contraseña de acceso. Este archivo debe almacenarse de forma segura. Si no es adecuado almacenarlo de forma segura, una alternativa es eliminar la información confidencial y proporcionarla cuando se ejecuta el comando helm install .

Para utilizar el secreto en el manifiesto de implementación para el servidor de configuración, se define lo siguiente en el archivo values.yaml del gráfico dev-env :

```

servidor de configuración:
  envFromSecretRefs:
    - secretos del servidor de configuración

```

La plantilla de implementación descrita anteriormente utilizará esto para agregar el secreto como variables de entorno en el manifiesto de implementación para el servidor de configuración.

El gráfico del entorno de producción sobrescribe más valores que el gráfico del entorno de desarrollo . Por ejemplo, values.yaml

El archivo del diagrama prod-env especifica que se debe usar un perfil Spring adicional, prod, y qué versión usar para las imágenes de Docker. Esto se ve así para el microservicio de producto :

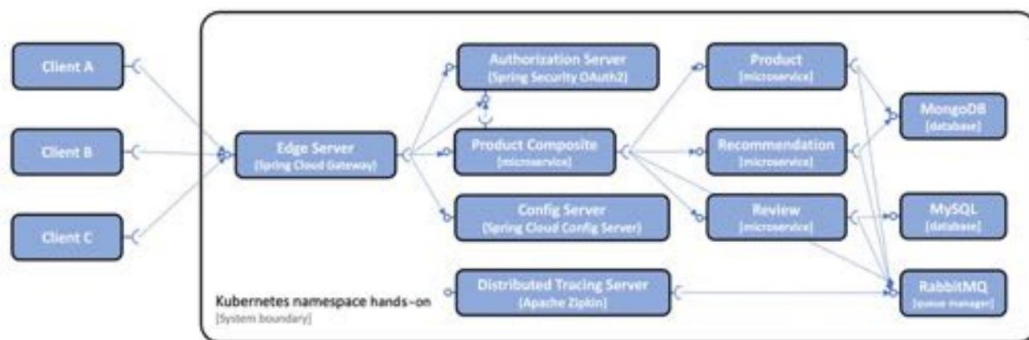
```

producto:
  imagen:
    etiqueta: v1
  entorno:
    PERFILES DE PRIMAVERA_ACTIVOS: "docker,prod"

```

Implementación en Kubernetes para desarrollo y pruebas

Para ejecutar pruebas funcionales, implementaremos los microservicios junto con los administradores de recursos que requieren en el mismo espacio de nombres, lo que llamaremos "hands-on". Esto facilita la configuración de un entorno de pruebas y su eliminación una vez finalizado. Simplemente podemos eliminar el espacio de nombres para eliminar todos los recursos utilizados por el entorno de pruebas. Este escenario de implementación se ilustra en el siguiente diagrama:



Antes de poder implementar el panorama del sistema, necesitamos construir nuestras imágenes de Docker y resolver las dependencias para nuestros gráficos de Helm.

Creación de imágenes de Docker

Normalmente, debemos enviar imágenes a un registro de Docker y configurar Kubernetes para que las extraiga del registro. En nuestro caso, con un clúster local de un solo nodo, podemos simplificar este proceso apuntando nuestro cliente Docker al motor de Docker en minikube y ejecutando el comando `docker compose build`. Esto hará que las imágenes de Docker estén disponibles inmediatamente en Kubernetes. Para el desarrollo, usaremos la versión más reciente de la imagen de Docker para los microservicios.

Puede crear imágenes de Docker desde la fuente de la siguiente manera:

```
cd $BOOK_HOME/Capítulo16 ./
gradlew compilación
eval $(minikube docker-env) docker
compose compilación
```

El comando eval \$(minikube docker-env) indica al cliente Docker local que se comunice con el motor Docker en minikube.

El archivo docker-compose.yml se ha actualizado para especificar un nombre para las imágenes de Docker que crea.

Por ejemplo, para el producto Servicio, tenemos lo siguiente:

```
producto:
  construir: microservicios/producto-servicio imagen:
    práctica/producto-servicio
```



latest es la etiqueta predeterminada para un nombre de imagen de Docker, por lo que no se especifica.

Con las imágenes de Docker creadas, es momento de crear los gráficos de Helm.

Resolución de dependencias del gráfico de Helm

Primero, actualizamos las dependencias en la carpeta de componentes :

```
para f en kubernetes/helm/components/*; hacer helm dep up $f; hecho
```

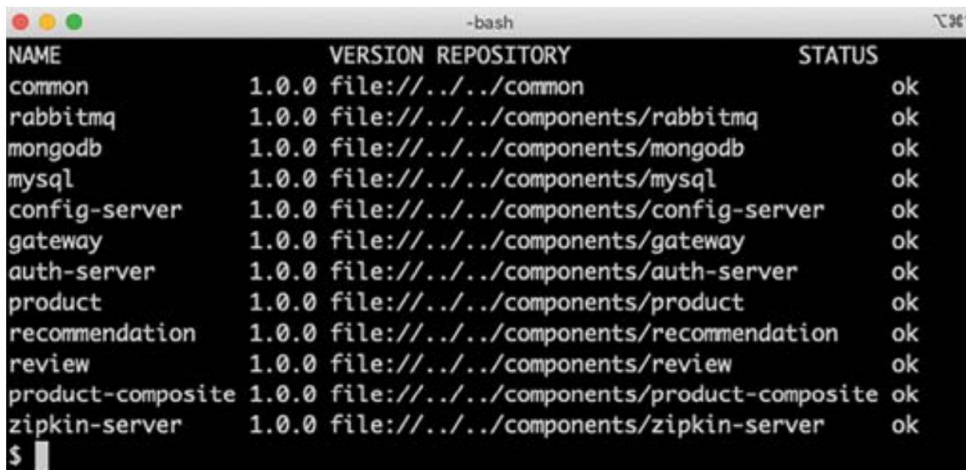
A continuación, actualizamos las dependencias en la carpeta de entornos :

```
para f en kubernetes/helm/environments/*; hacer helm dep up $f; hecho
```

Por último, verificamos que las dependencias para la carpeta dev-env se vean bien:

```
helm dep ls kubernetes/helm/environments/dev-env/
```

Espere que el comando responda con lo siguiente:



A terminal window titled '-bash' showing the output of a Helm dependency resolution command. The output is a table with four columns: NAME, VERSION, REPOSITORY, and STATUS. The table lists various components and their resolved paths, all with a status of 'ok'.

NAME	VERSION	REPOSITORY	STATUS
common	1.0.0	file:///../../common	ok
rabbitmq	1.0.0	file:///../../components/rabbitmq	ok
mongodb	1.0.0	file:///../../components/mongodb	ok
mysql	1.0.0	file:///../../components/mysql	ok
config-server	1.0.0	file:///../../components/config-server	ok
gateway	1.0.0	file:///../../components/gateway	ok
auth-server	1.0.0	file:///../../components/auth-server	ok
product	1.0.0	file:///../../components/product	ok
recommendation	1.0.0	file:///../../components/recommendation	ok
review	1.0.0	file:///../../components/review	ok
product-composite	1.0.0	file:///../../components/product-composite	ok
zipkin-server	1.0.0	file:///../../components/zipkin-server	ok

Figura 16.3: Dependencias del gráfico de Helm resueltas

¡Con ambas imágenes de Docker creadas y las dependencias de Helm resueltas, podemos comenzar a implementar en Ku-bernetes!

Implementación en Kubernetes

Implementar en Kubernetes implica crear o actualizar objetos de Kubernetes. Usaremos Helm para realizar la implementación, siguiendo estos pasos:

1. Para evitar un proceso de implementación lento debido a que Kubernetes descarga imágenes de Docker (lo que potencialmente causa que las sondas de actividad que describimos anteriormente reinicien nuestros Pods), ejecute los siguientes comandos docker pull para descargar las imágenes con anticipación:

```
evaluar $(minikube docker-env)
Docker pull mysql:9.2.0
Docker pull mongo:8.0.5
Administración de RabbitMQ:4.0.7
docker pull openzipkin/zipkin:3.5.0
```

2. Antes de usar los gráficos de Helm, renderice las plantillas utilizando el comando de plantilla de Helm

Para ver cómo se verán los manifiestos:

```
plantilla de timón kubernetes/helm/environments/dev-env
```

Tenga en cuenta que no se realizó ninguna interacción con el clúster de Kubernetes, por lo que la información del clúster será falsificada y no se ejecutarán pruebas para verificar si el clúster aceptará el manifiesto representado.

3. Para verificar también que el clúster de Kubernetes realmente aceptará el manifiesto renderizado, se puede realizar una ejecución en seco de la instalación pasando `--dry-run` al comando `helm install`. Al pasar el indicador `--debug`, también se mostrarán los valores proporcionados por el usuario y calculados que Helm usará al renderizar los manifiestos. Ejecute el siguiente comando para realizar una prueba:

```
helm install --dry-run --debug entorno de desarrollo práctico \
kubernetes/helm/entornos/dev-env
```

4. Iniciar la implementación del panorama completo del sistema, incluida la creación del espacio de nombres práctico, ejecute el siguiente comando:

```
instalación de helm en un entorno de desarrollo práctico
kubernetes/helm/entornos/dev-env \
-n práctico \
--crear-espacio de nombres
```



Tenga en cuenta que aquí es donde entra en juego la maquinaria Helm. Utilizará los gráficos que revisamos en la sección anterior Introducción a Helm para representar y aplicar los manifiestos de Kubernetes, lo que da como resultado la creación de los objetos de Kubernetes necesarios para la implementación.

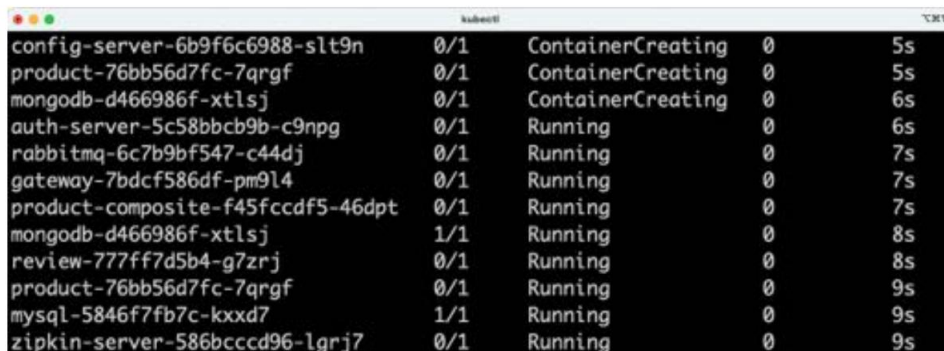
5. Establezca el espacio de nombres recién creado como el espacio de nombres predeterminado para kubectl:

```
configuración de kubectl contexto-actual $(configuración de kubectl contexto-actual) --
namespace=hands-on
```

6. Para ver los Pods iniciándose, ejecute el siguiente comando:

```
kubectl obtener pods --watch
```

Este comando informará continuamente cuando se estén ejecutando nuevos Pods y, si algo falla, informará el estado, como Error y CrashLoopBackOff. Después de un tiempo, es posible que vea errores en los Pods de puerta de enlace, compuesto de producto y servidor Zipkin. Esto se debe a que todos dependen de recursos externos que deben ser accesibles durante el inicio. De lo contrario, se bloquearán. El servicio de puerta de enlace y compuesto de producto depende del servidor de autenticación, y el servidor Zipkin depende del acceso a RabbitMQ. Normalmente, se inician más rápido que los recursos de los que dependen, lo que provoca esta situación. Sin embargo, Kubernetes detectará los Pods bloqueados y los reiniciará. Una vez que los recursos estén en funcionamiento, todos los Pods se iniciarán y se informará que están listos, mostrando 1/1 en la columna LISTO. Un ejemplo de salida del comando se ve así:



Pod Name	Namespace	Phase	Age
config-server-6b9f6c6988-slt9n	0/1	ContainerCreating	0 5s
product-76bb56d7fc-7qrgf	0/1	ContainerCreating	0 5s
mongodb-d466986f-xtlsj	0/1	ContainerCreating	0 6s
auth-server-5c58bbcb9b-c9npg	0/1	Running	0 6s
rabbitmq-6c7b9bf547-c44dj	0/1	Running	0 7s
gateway-7bdcf586df-pm9l4	0/1	Running	0 7s
product-composite-f45fccdf5-46dpt	0/1	Running	0 7s
mongodb-d466986f-xtlsj	1/1	Running	0 8s
review-777ff7d5b4-g7zrj	0/1	Running	0 8s
product-76bb56d7fc-7qrgf	0/1	Running	0 9s
mysql-5846f7fb7c-kxxd7	1/1	Running	0 9s
zipkin-server-586bcccd96-lgrj7	0/1	Running	0 9s

Figura 16.4: Pods reiniciados hasta que las dependencias externas estén listas

Después de ver una salida como la anterior, interrumpa el comando con Ctrl + C.

7. Espere a que todos los pods en el espacio de nombres estén listos con el siguiente comando:

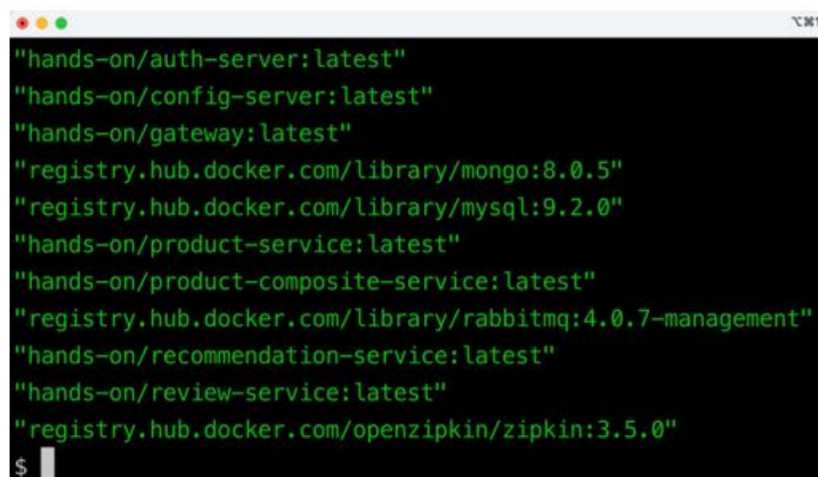
```
kubectl wait --timeout=600s --for=condición=listo pod --all
```

Se espera que el comando responda con 11 líneas de registro como pod/... condición cumplida, donde los tres puntos (...) se reemplazan con el nombre del Pod real que se informa que está listo.

8. Para ver las imágenes de Docker que se utilizan, ejecute el siguiente comando:

```
kubectl obtener pods -o json | jq .items[].spec.containers[].imagen
```

La respuesta debería verse así:



```
"hands-on/auth-server:latest"
"hands-on/config-server:latest"
"hands-on/gateway:latest"
"registry.hub.docker.com/library/mongo:8.0.5"
"registry.hub.docker.com/library/mysql:9.2.0"
"hands-on/product-service:latest"
"hands-on/product-composite-service:latest"
"registry.hub.docker.com/library/rabbitmq:4.0.7-management"
"hands-on/recommendation-service:latest"
"hands-on/review-service:latest"
"registry.hub.docker.com/openzipkin/zipkin:3.5.0"
```

Figura 16.5: Imágenes de Docker utilizadas en un entorno de prueba

Tenga en cuenta que las imágenes de Docker tienen la etiqueta de versión establecida en la última versión para los microservicios.

¡Ya estamos listos para probar nuestra implementación! Sin embargo, antes de hacerlo, debemos realizar los cambios necesarios en el script de prueba para su uso con Kubernetes.

Cambios en el script de prueba para su uso con Kubernetes

Para verificar la implementación, como de costumbre, ejecutaremos el script de prueba `test-em-all.bash`. Para trabajar con Kubernetes, las pruebas de disyuntores se han modificado ligeramente. Estas pruebas llaman a los endpoints del actuador en el servicio del componente del producto para comprobar su estado y acceder a los eventos del disyuntor. Dado que este endpoint no está expuesto externamente, en los capítulos anteriores se utilizó el comando `docker compose exec` para ejecutar un comando `curl` dentro del componente del producto.

Servicio para realizar las pruebas.

A partir de este capítulo, el script de prueba puede usar el comando `docker compose exec` o el comando `kubectl` correspondiente, `kubectl exec`, dependiendo de si ejecutamos los microservicios usando Docker Compose o Kubernetes.

Para saber qué comando usar, se ha añadido un nuevo parámetro al script, `USE_K8S`. Su valor predeterminado es falso. Para más detalles, consulte la función `testCircuitBreaker()` en el script de prueba.

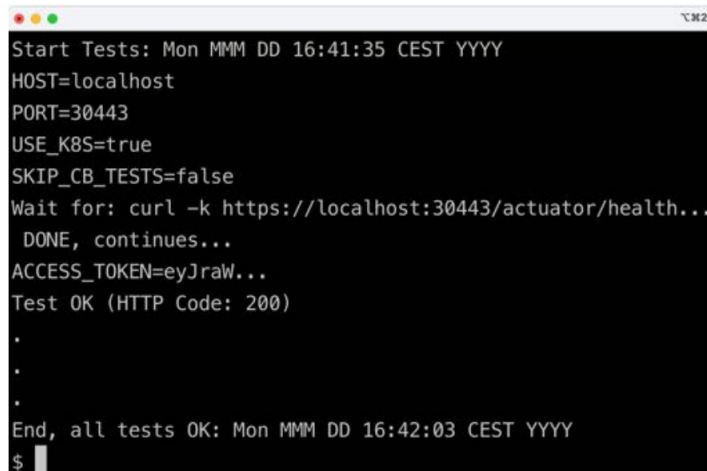
Prueba de la implementación

Al ejecutar el script de prueba, debemos asignarle la dirección del host que ejecuta Kubernetes (es decir, nuestra instancia de minikube) y el NodePort donde nuestro servicio de puerta de enlace escucha las solicitudes externas. Se puede acceder a la puerta de enlace mediante el puerto 30443. Como se mencionó en el Capítulo 15, dado que usamos el controlador Docker de minikube, el nombre de host siempre es `localhost`. Dado que el nombre de host es el mismo que al ejecutar pruebas con Docker Compose, no es necesario especificarlo; solo se debe especificar el puerto junto con el parámetro `USE_K8S`.

Inicie las pruebas con el siguiente comando:

```
PUERTO=30443 USE_K8S=verdadero ./test-em-all.bash
```

En la salida del script, podemos ver cómo se utiliza NodePort, pero aparte de eso, todo se ve igual que cuando usamos Docker Compose en los capítulos anteriores:



```
Start Tests: Mon MMM DD 16:41:35 CEST YYYY
HOST=localhost
PORT=30443
USE_K8S=true
SKIP_CB_TESTS=false
Wait for: curl -k https://localhost:30443/actuator/health...
DONE, continues...
ACCESS_TOKEN=eyJraW...
Test OK (HTTP Code: 200)
.
.
.
End, all tests OK: Mon MMM DD 16:42:03 CEST YYYY
$
```

Figura 16.6: Salida de las pruebas automatizadas del panorama del sistema

Con las validaciones del panorama del sistema realizadas, veamos cómo podemos probar las nuevas características de Spring Boot: apagado elegante y las pruebas de actividad y preparación.

Prueba de la compatibilidad de Spring Boot con el apagado elegante y sondas de actividad y preparación

En esta sección, probaremos las nuevas funciones de Spring Boot y veremos cómo interactúan con otros componentes en Kubernetes.

Comencemos probando la compatibilidad de Spring Boot con el apagado ordenado, donde la aplicación, durante su fase de apagado, esperará un tiempo configurable a que se completen las solicitudes activas. Recuerde que no se permiten nuevas solicitudes durante la fase de apagado.

Para probar el mecanismo de apagado ordenado, ejecutaremos un cliente que envía solicitudes continuamente al servicio compuesto. Primero, lo usaremos para enviar solicitudes que tardan 5 segundos, un tiempo menor que el tiempo de espera de apagado. El tiempo de espera está configurado en 10 segundos. A continuación, lo usaremos para enviar solicitudes que tardan más (15 segundos) para ver cómo se gestionan.

Como cliente de prueba, utilizaremos Siege, una herramienta de prueba de carga basada en línea de comandos.

Para poder ejecutar solicitudes de prueba que tardan tanto en completarse, debemos deshabilitar temporalmente el tiempo de espera en el servicio compuesto del producto. De lo contrario, se activará su disyuntor y nos impedirá ejecutar las solicitudes largas. Para poder deshabilitar Resilience4j, se ha añadido una clase de configuración, ResilienceConfig, al servicio compuesto. Si `app.resilience.enabled` Si la propiedad se establece en falso, se deshabilitan todas las funciones de Resilience4j. Para deshabilitar Resilience4j en el servicio compuesto, siga estos pasos:

1. Agregue lo siguiente en la sección compuesta del producto en el archivo de valores para dev-env entorno kubernetes/helm/environments/dev-env/values.yaml:

```
entorno:

  APP_RESILIENCE_ENABLED: "falso"
```

Después del cambio, el archivo de configuración debería verse así:

```
producto-compuesto:
  entorno:
    APP_RESILIENCE_ENABLED: "falso"
  envFromSecretRefs:
    - credenciales del cliente de configuración
```



Mientras esta configuración esté activa, el disyuntor realizará pruebas en modo prueba-em-all. bash ya no funcionará porque asume un tiempo de espera de 2 segundos.

2. Actualice la instalación de Helm con el comando de actualización de Helm , usando el indicador --wait para garantizar que la actualización se complete cuando finalice el comando:

```
Actualización de Helm, práctica de desarrollo y entorno -n práctica
kubernetes/helm/environments/dev-env --wait
```

Ahora podemos ejecutar las pruebas, procediendo con los siguientes pasos para probar con solicitudes que sean más cortas que el período de espera de apagado:

1. Obtenga un token de acceso:

```
TOKEN DE ACCESO=$(curl -d tipo_concesión=credenciales_del_cliente \
  -ks https://escritor:escritor-secreto@localhost:30443/oauth2/token \
  -d scope="producto:leer producto:escribir" \
  | jq .access_token -r)
```

Asegúrese de tener un token de acceso ejecutando el comando `echo $ACCESS_TOKEN` . Si está vacío, debe consultar el comando `curl` anterior y los registros de la puerta de enlace. y el servidor de autenticación.

2. Realice una solicitud de prueba y solicite un retraso de 5 segundos utilizando el parámetro de consulta de retraso :

```
tiempo curl -kH "Autorización: Portador $ACCESS_TOKEN" \
https://localhost:30443/producto-compuesto/1?delay=5
```

Si obtiene una respuesta normal y el comando de tiempo informa un tiempo de respuesta de 5 segundos, ¡los cambios de configuración del tiempo de espera aumentado funcionaron!

3. Use Siege para iniciar solicitudes que tardan 5 segundos en completarse, con 5 usuarios simultáneos enviando solicitudes con un retraso aleatorio de entre 0 y 2 segundos para distribuir ligeramente las solicitudes:

```
asedio -c5 -d2 -v -H "Autorización: Portador $ACCESS_TOKEN" \
https://localhost:30443/producto-compuesto/1?delay=5
```

Espere un resultado de la herramienta para cada solicitud completada como este:

```
HTTP/1.1 200 5,04 segundos: 771 bytes ==> GET /product-
composite/1?delay=5
```

4. Observe la salida del registro del Servicio del producto en una ventana de Terminal separada con el siguiente comando:

```
registros de kubectl -f --tail=0 -l app.kubernetes.io/nombre=producto
```

5. Ahora solicitaremos a Kubernetes que reinicie la implementación del producto . El reinicio iniciará primero un nuevo pod antes de que se apague el anterior, lo que significa que ninguna de las solicitudes enviadas por Siege se verá afectada. Son especialmente interesantes las pocas solicitudes que procesa el pod anterior cuando comienza a apagarse. Si el apagado correcto funciona correctamente, ninguna de las solicitudes activas debería fallar. Para reiniciar, ejecute el siguiente comando en una ventana aparte:

```
kubectl rollout restart desplegar/producto
```

6. Asegúrese de que solo se informen solicitudes exitosas en el resultado de la prueba de carga. herramienta, asedio, reportando 200 (OK).

- En el registro del pod de producto detenido, debería ver que se permitió que todas las solicitudes finalizaran correctamente antes de que se detuviera la aplicación. Al final del registro, se espera un resultado similar al siguiente:



```

YYYY-MM-DDT14:12:30.877Z INFO 1 --- [ionShutdownHook]
[product,,] o.s.b.w.embedded.netty.GracefulShutdown :
Commencing graceful shutdown. Waiting for active requests to complete
.
.
.
YYYY-MM-DDT14:12:35.245Z INFO 1 --- [netty-shutdown]
[product,,] o.s.b.w.embedded.netty.GracefulShutdown :
Graceful shutdown complete

```

Figura 16.7: Apagado elegante donde se permite que se completen todas las solicitudes

En concreto, observe el tiempo transcurrido entre los dos mensajes de registro (4 segundos, en este caso), lo que indica que el procedimiento de apagado realmente esperó a que se completara la última solicitud.

Ahora, ejecutemos la segunda prueba, con solicitudes que tardan más tiempo en completarse que el período de espera de apagado:

- Reinicie Siege, solicitando tiempos de respuesta más largos, superando el límite de espera de 10 segundos. Inicie 15 usuarios simultáneos, solicitando un tiempo de respuesta de 15 segundos y un retraso aleatorio entre solicitudes de 0 a 15 segundos. Detenga Siege con Ctrl + C y ejecute el siguiente comando:

```

asedio -c15 -d15 -v -H "Autorización: Portador $ACCESS_TOKEN" \
https://localhost:30443/producto-compuesto/1?delay=15

```

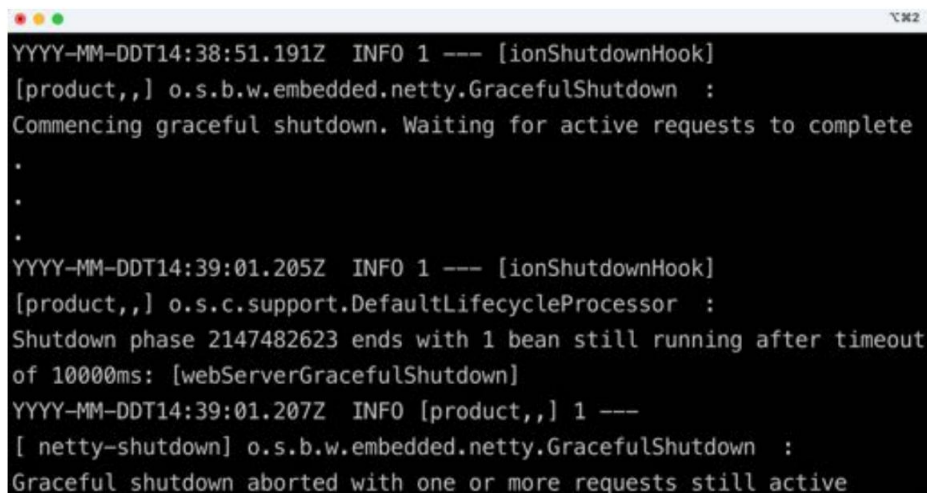
- Observe la salida del registro del pod del producto con el siguiente comando:

```
registros de kubectl -f --tail=0 -l app.kubernetes.io/nombre=producto
```

- Reinicie la implementación del producto :

```
kubectl rollout restart desplegar/producto
```

- Siga la salida del registro del pod del producto. Una vez apagado, debería poder ver que no todas las solicitudes pudieron finalizar correctamente antes de que se detuviera la aplicación. Al final, se espera una salida similar a la siguiente:



```

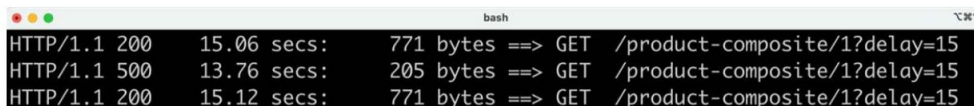
YYYY-MM-DDT14:38:51.191Z INFO 1 --- [ionShutdownHook]
[product,,] o.s.b.w.embedded.netty.GracefulShutdown :
Commencing graceful shutdown. Waiting for active requests to complete
.
.
.
YYYY-MM-DDT14:39:01.205Z INFO 1 --- [ionShutdownHook]
[product,,] o.s.c.support.DefaultLifecycleProcessor :
Shutdown phase 2147482623 ends with 1 bean still running after timeout
of 10000ms: [webServerGracefulShutdown]
YYYY-MM-DDT14:39:01.207Z INFO [product,,] 1 ---
[ netty-shutdown] o.s.b.w.embedded.netty.GracefulShutdown :
Graceful shutdown aborted with one or more requests still active

```

Figura 16.8: Apagado elegante donde se cancelan algunas solicitudes de larga duración

El mensaje de registro Apagado elegante abortado con una o más solicitudes aún activas indica que al menos una solicitud no pudo completarse antes de que se detuviera la aplicación.

5. En la salida de la herramienta de prueba de carga, Siege, ahora deberían aparecer una o varias solicitudes fallidas que informan 500 (Error interno del servidor) como se muestra a continuación:



```

HTTP/1.1 200 15.06 secs: 771 bytes ==> GET /product-composite/1?delay=15
HTTP/1.1 500 13.76 secs: 205 bytes ==> GET /product-composite/1?delay=15
HTTP/1.1 200 15.12 secs: 771 bytes ==> GET /product-composite/1?delay=15

```

Figura 16.9: Las solicitudes de larga duración fallan durante el apagado

Esto demuestra cómo se desarrolla el procedimiento de apagado después del tiempo de espera configurado y que las solicitudes de ejecución prolongada restantes se cancelan, como se esperaba.

Esto completa las pruebas del mecanismo de apagado elegante de Spring Boot, que claramente es útil para evitar que las solicitudes de clientes normales se vean afectadas por los Pods que se detienen, por ejemplo, como resultado de una reducción de escala o una actualización continua que se está realizando.

Limpieza después de las pruebas:

1. Detenga la herramienta de prueba de carga Siege con Ctrl + C.
2. Revierte la última versión de Helm para eliminar el tiempo de espera aumentado:

```
reversión de helm hands-on-dev-env -n hands-on --wait
```



El comando helm rollback también es útil para revertir una actualización fallida.

- Además, elimine la configuración de tiempo de espera aumentado en `kubernetes/helm/environments/dev-Archivo env/values.yaml` .
- Ejecute `test-em-all.bash` para verificar que la configuración se haya revertido:

```
PUERTO=30443 USE_K8S=verdadero ./test-em-all.bash
```

Finalmente, veamos qué información reportan las sondas de actividad y preparación de Spring Boot. Usaremos el servicio del producto , pero también pueden probar las sondas para otros servicios:

- Ejecute el siguiente comando para obtener la salida de la sonda de actividad del servicio del producto :

```
kubect! exec -it implementar/producto -- \
  wget -qO - localhost/actuador/salud/actividad | jq .
```

- Espere que responda con lo siguiente:

```
bash  ~%1
{
  "status": "UP"
}
$
```

Figura 16.10: Respuesta de una sonda de actividad

- Ejecute el siguiente comando para obtener la salida de la sonda de preparación del servicio del producto :

```
kubect! exec -it implementar/producto -- \
  wget -qO - localhost/actuador/salud/preparación | jq .
```

Esperemos que su respuesta sea un poco más extensa:

A terminal window with a dark background and light-colored text. It displays a JSON object representing the response of a probe. The object has a 'status' field set to 'UP', a 'components' field which is an object containing 'mongo' and 'rabbit' sub-objects, and a 'readinessState' field set to 'UP'. The 'mongo' sub-object has its own 'status' ('UP') and 'details' (including 'maxWireVersion': 25). The 'rabbit' sub-object has its 'status' ('UP') and 'details' (including 'version': '4.0.7').

```
{
  "status": "UP",
  "components": {
    "mongo": {
      "status": "UP",
      "details": {
        "maxWireVersion": 25
      }
    },
    "rabbit": {
      "status": "UP",
      "details": {
        "version": "4.0.7"
      }
    }
  },
  "readinessState": {
    "status": "UP"
  }
}
```

Figura 16.11: Respuesta de una sonda de preparación

A partir del resultado anterior, podemos confirmar que la disponibilidad del producto ahora depende de su acceso tanto a MongoDB como a RabbitMQ. Esto es previsible, ya que configuramos el grupo de estado de disponibilidad para incluir indicadores de estado para las bases de datos RabbitMQ, MongoDB y SQL, si están disponibles. Consulte la sección "Uso de la compatibilidad de Spring Boot con el apagado ordenado y las sondas de actividad y disponibilidad" para un resumen, si es necesario.

Antes de continuar, limpiemos lo que tenemos instalado en el entorno de desarrollo. Podemos hacerlo simplemente eliminando el espacio de nombres. Al eliminarlo, se eliminarán recursivamente los recursos que contiene, incluida la información sobre la instalación de Helm.

Elimina el espacio de nombres con el siguiente comando:

```
Eliminación práctica de espacios de nombres en kubectl
```



Si solo desea desinstalar lo que instaló el comando `helm install`, puede ejecutar el comando `helm uninstall hands-on-dev-env`.

Una vez eliminado el entorno de desarrollo, podemos avanzar y configurar un entorno orientado a la prueba y la producción.

Implementación en Kubernetes para pruebas y producción

En esta sección, implementaremos los microservicios en un entorno de prueba y producción. El entorno de prueba se utiliza para realizar el control de calidad (QA) y las pruebas de aceptación del usuario (UAT) como último paso antes de lanzar una nueva versión a producción. Para verificar que una nueva versión cumple no solo con los requisitos funcionales, sino también con los no funcionales (por ejemplo, en términos de rendimiento, robustez, escalabilidad y resiliencia), el entorno de prueba se configura para que sea lo más similar posible al entorno de producción.

Al implementar en un entorno de prueba o producción, se requieren una serie de cambios en comparación con la implementación para desarrollo o pruebas:

- Los administradores de recursos deben ejecutarse fuera del clúster de Kubernetes: Es técnicamente viable ejecutar bases de datos y gestores de colas para producción en Kubernetes como contenedores con estado, mediante StatefulSets y PersistentVolumes. Al momento de escribir este libro, lo desaconsejo, principalmente porque la compatibilidad con contenedores con estado es relativamente nueva y aún no está probada en Kubernetes. En su lugar, recomiendo usar los servicios de base de datos y gestores de colas existentes localmente o como servicios administrados en la nube, dejando que Kubernetes haga lo que mejor sabe hacer: ejecutar contenedores sin estado. En el marco de este libro, para simular un entorno de producción, ejecutaremos MySQL, MongoDB y RabbitMQ como contenedores Docker sin estado fuera de Kubernetes, utilizando los archivos de Docker Compose ya existentes.
- **Bloqueo:** Por razones de seguridad, es necesario restringir aspectos como los endpoints de los actuadores y los niveles de registro en un entorno de producción. Los endpoints expuestos externamente también deben revisarse desde una perspectiva de seguridad. Por ejemplo, el acceso al servidor de configuración probablemente debería bloquearse en un entorno de producción, pero lo mantendremos expuesto en este libro para mayor comodidad. Es necesario especificar las etiquetas de imagen de Docker para poder rastrear qué versiones de los microservicios se han implementado.
 - **Ampliar los recursos disponibles:** Para cumplir con los requisitos de alta disponibilidad y mayor carga, necesitamos ejecutar al menos dos pods por implementación. También podríamos necesitar aumentar la cantidad de memoria y CPU permitidas por pod. Para evitar la falta de memoria en la instancia de minikube, mantendremos un pod por implementación, pero aumentaremos la memoria máxima permitida en el entorno de producción.

- Configurar un clúster de Kubernetes listo para producción: Esto queda fuera del alcance de este libro, pero, si es posible, recomiendo usar uno de los servicios de Kubernetes administrados que ofrecen los principales proveedores de nube. Para el alcance de este libro, implementaremos en nuestra instancia local de Minikube.



Esta no pretende ser una lista exhaustiva de cosas que deben tenerse en cuenta al configurar un entorno de producción, pero es un buen comienzo.

Nuestro entorno de producción simulado se verá así:

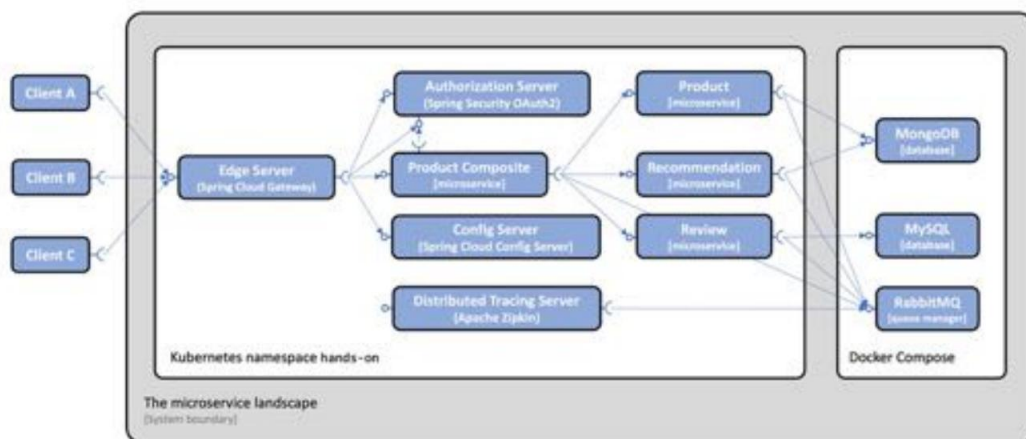


Figura 16.12: Administradores de recursos implementados fuera de Kubernetes

Cambios en el código fuente

Se han aplicado los siguientes cambios al código fuente para prepararlo para la implementación en un entorno que se utiliza para ensayo y producción:

- Se ha agregado un perfil Spring llamado prod a los archivos de configuración en el repositorio de configuración. repositorio de configuración:

```
spring.config.activate.on-profile: prod
```

- En los perfiles de producción se ha agregado lo siguiente:
 - URL a los administradores de recursos que se ejecutan como contenedores Docker simples:

```
spring.rabbitmq.host: 172.17.0.1
spring.data.mongodb.host: 172.17.0.1
spring.datasource.url: jdbc:mysql://172.17.0.1:3306/review-db
```



Usamos la dirección IP 172.17.0.1 para acceder al motor Docker en la instancia de minikube. Esta es la dirección IP predeterminada para el motor Docker al crearlo con minikube, al menos para minikube hasta la versión 1.18.

Se está trabajando para establecer un nombre DNS estándar que los contenedores puedan usar si necesitan acceder al host Docker en el que se ejecutan, pero al momento de escribir este artículo, este esfuerzo de trabajo no se ha completado.

- Los niveles de registro se han establecido en una advertencia o superior, es decir, error o fatal. Aquí está Un ejemplo:

```
logging.level.root: ADVERTENCIA
```

- Los únicos puntos finales del actuador que se exponen a través de HTTP son los puntos finales de información y estado que utilizan las sondas de actividad y preparación en Kubernetes, así como el punto final circuitbreakerevents que utiliza el script de prueba, test-em-all.bash:

```
administración.puntos finales.web.exposición.incluye:
salud, información, eventos de disyuntores
```



En un entorno de producción real, también deberíamos haber cambiado la configuración de imagePullPolicy: Never a IfNotPresent para descargar imágenes de Docker desde un registro de Docker. Sin embargo, dado que implementaremos la configuración de producción en la instancia de minikube, donde compilamos y etiquetamos manualmente las imágenes de Docker, no actualizaremos esta configuración.

Implementación en Kubernetes

Para simular el uso de administradores de recursos de producción, MySQL, MongoDB y RabbitMQ se ejecutarán fuera de Kubernetes mediante Docker Compose. Los iniciaremos como hicimos en los capítulos anteriores:

```
evaluar $(minikube docker-env)
docker composer -d mongodb mysql rabbitmq
```

También necesitamos etiquetar las imágenes Docker existentes con v1, usando los siguientes comandos:

```
etiqueta docker hands-on/auth-server hands-on/auth-server:v1
etiqueta docker hands-on/config-server hands-on/config-server:v1
etiqueta docker práctica/puerta de enlace práctica/puerta de enlace:v1
etiqueta docker práctica/servicio-compuesto-de-producto práctica/servicio-compuesto-de-producto:v1

etiqueta docker práctica/servicio de producto práctica/servicio de producto:v1
etiqueta docker servicio práctico/de recomendación servicio práctico/de recomendación:v1

etiqueta docker servicio práctico/de revisión servicio práctico/de revisión:v1
```

Desde aquí, los comandos son muy similares a cómo los implementamos en el entorno de desarrollo:

1. Implementar usando Helm:

```
Instalación de Helm en un entorno de producción práctico \ kubernetes/helm/environments/prod-env \
-n práctico --crear-espacio-de-nombres
```

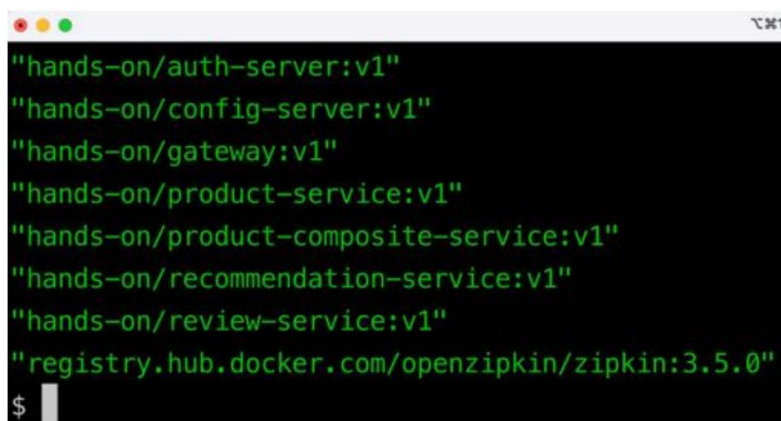
2. Espere a que las implementaciones estén en funcionamiento:

```
kubectl wait --timeout=600s --for=condición=listo pod --all
```

3. Para ver las imágenes de Docker que se utilizan actualmente en el entorno de producción, ejecute el comando siguiente comando:

```
kubectl obtener pods -o json | jq .items[].spec.containers[].imagen
```

La respuesta debería verse así:

A terminal window with a dark background and green text. It lists several Docker images: "hands-on/auth-server:v1", "hands-on/config-server:v1", "hands-on/gateway:v1", "hands-on/product-service:v1", "hands-on/product-composite-service:v1", "hands-on/recommendation-service:v1", "hands-on/review-service:v1", and "registry.hub.docker.com/openzipkin/zipkin:3.5.0". A prompt character "\$" is visible at the bottom left.

```
"hands-on/auth-server:v1"
"hands-on/config-server:v1"
"hands-on/gateway:v1"
"hands-on/product-service:v1"
"hands-on/product-composite-service:v1"
"hands-on/recommendation-service:v1"
"hands-on/review-service:v1"
"registry.hub.docker.com/openzipkin/zipkin:3.5.0"
$
```

Figura 16.13: Imágenes de Docker utilizadas en un entorno de producción

¡Tenga en cuenta la versión v1 de las imágenes de Docker!

Tenga en cuenta también que los Pods del administrador de recursos para MySQL, MongoDB y RabbitMQ desaparecieron; estos se pueden encontrar con el comando `docker compose ps`.

4. Ejecute el script de prueba, `test-em-all.bash`, para verificar el entorno de producción simulado:

```
CONFIG_SERVER_USR=usr_prod \
CONFIG_SERVER_PWD=contraseña de producción
PUERTO=30443 USE_K8S=verdadero ./test-em-all.bash
```



Espere el mismo tipo de resultado que obtuvimos cuando se ejecutó el script de prueba en el entorno de desarrollo.

Eso completa las pruebas; limpiemos para que el entorno de Kubernetes esté listo para el próximo capítulo.

Limpiando

Para eliminar los recursos que utilizamos, ejecute los siguientes comandos:

1. Eliminar el espacio de nombres:

```
Eliminación práctica de espacios de nombres en kubectl
```

2. Cierre los administradores de recursos que se ejecutan fuera de Kubernetes:

```
evaluar $(minikube docker-env)
Docker Compose Down
evaluar $(minikube docker-env -u)
```



El comando eval \$(minikube docker-env -u) indica al cliente Docker local que se comunique con el motor Docker local y que ya no se comunique con el motor Docker en minikube.

Como ya se describió anteriormente en este capítulo, el comando `kubectl delete namespace` eliminará recursivamente todos los recursos de Kubernetes existentes en el espacio de nombres, y el comando `docker compose down` detendrá MySQL, MongoDB y RabbitMQ. Con el entorno de producción eliminado, hemos llegado al final de este capítulo.

Resumen

En este capítulo, aprendimos cómo implementar los microservicios de este libro en Kubernetes usando Helm. Hemos visto cómo usar Helm para crear plantillas reutilizables, minimizando el código repetitivo necesario para crear los manifiestos de Kubernetes. Las plantillas reutilizables se almacenan en un gráfico común, mientras que los gráficos específicos de cada microservicio proporcionan valores específicos para cada uno. En el nivel superior, tenemos gráficos principales que describen cómo se debe implementar un entorno de desarrollo/prueba y de fase/producción mediante los gráficos de microservicios, junto con gráficos para administradores de recursos como bases de datos y gestores de colas.

También hemos visto cómo podemos beneficiarnos del uso de las funciones de Spring Boot para facilitar las implementaciones en Kubernetes. La compatibilidad de Spring Boot con el apagado ordenado permite que las solicitudes activas se completen antes de que se detenga un microservicio basado en Spring Boot, como durante una actualización continua. La compatibilidad con sondas de actividad y preparación facilita la declaración de sondas que conocen la disponibilidad de recursos externos de los que depende un microservicio específico.

Finalmente, para poder implementar nuestros microservicios en Kubernetes, tuvimos que reemplazar Netflix Eureka con el servicio de descubrimiento integrado en Kubernetes. El cambio del servicio de descubrimiento se realizó sin modificar el código fuente de Java; solo tuvimos que aplicar cambios a las dependencias de compilación y a parte de la configuración.

En el siguiente capítulo, veremos cómo podemos seguir utilizando Kubernetes para reducir la cantidad de servicios de soporte que necesitamos implementar. Continúe leyendo para ver cómo podemos eliminar la necesidad del servidor de configuración y cómo podemos reemplazar nuestro servidor perimetral con un controlador de ingreso de Kubernetes.

Preguntas

1. ¿Por qué eliminamos el servidor Eureka del panorama de microservicios al implementar?
¿Está en Kubernetes?
2. ¿Con qué reemplazamos el servidor Eureka y cómo quedó el código fuente del micro-
¿Servicios afectados por este cambio?
3. ¿Cuál es el propósito de las sondas de actividad y preparación?
4. ¿Cómo es útil el mecanismo de Spring Boot para el apagado elegante?
5. ¿Cuál es el propósito de las siguientes directivas de plantilla de Helm?

```
{{- $common := dict "Valores" .Valores.common -}}
{{- $noCommon := omitir .Valores "comunes" -}} {{- $overrides :=
dictar "Valores" $noCommon -}}
{{- $noValues := omitir . "Valores" -}} {{- con la fusión
$noValues $anula $common -}}
```

6. ¿Por qué fallaría la siguiente plantilla Helm?

```
{{- define "common.secrets" -}} {{- rango
$secretName, $secretMap := .Values.secrets }} apiVersion: v1

tipo: Secreto
metadatos:
  nombre: {{ $secretName }}
  etiquetas:
    app.kubernetes.io/nombre: {{ $secretName }}
tipo: Opaco
datos:
{{- rango $clave, $val := $secretMap }} {{ $clave }}: {{ $val |
b64enc }}
{{- fin }} {{- fin -}}

{{- fin -}}
```

7. ¿Por qué los siguientes manifiestos no funcionarían juntos?

```

Versión de API: v1
tipo: Servicio
metadatos:
  nombre: revisión
  etiquetas:
    app.kubernetes.io/name: revisión
especulación:
  tipo: ClusterIP puertos:
    - nombre: http
      puerto: 80
      protocolo: TCP
      puerto de destino: http
  selector:
    app.kubernetes.io/pod-name: revisión
---
apiVersion: apps/v1 tipo:
Implementación
metadatos:
  nombre: revisión
  etiquetas:
    app.kubernetes.io/name: revisión
especulación:
  réplicas: 1
  selector:
    etiquetas de coincidencia:
      app.kubernetes.io/name: plantilla de revisión :
    metadatos:
      etiquetas:
        app.kubernetes.io/name: revisión
    especulación:
      contenedores:
        - nombre: revisión
          Imagen: "Servicio de revisión práctica: más reciente"
          puertos:

```

- Puerto contenedor: 80

nombre: puerto http

protocolo: TCP

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



17

Implementación de Kubernetes Características para simplificar el sistema Paisaje

El panorama actual de microservicios contiene varios servicios de soporte que implementan patrones de diseño importantes requeridos en un panorama de microservicios a gran escala, por ejemplo, un servidor de borde, un servidor de configuración, un servidor de autorización y un servicio para rastreo distribuido. Para un resumen, consulte el Capítulo 1, Introducción a los microservicios. En el capítulo anterior, reemplazamos la implementación del patrón de diseño para el descubrimiento de servicios, basado en Netflix Eureka, con el servicio de descubrimiento integrado en Kubernetes. En este capítulo, simplificaremos aún más el panorama de microservicios al reducir la cantidad de servicios de soporte que se requieren para implementar. En su lugar, los patrones de diseño correspondientes serán manejados por capacidades integradas en Kubernetes. El Spring Cloud Config Server será reemplazado por Kubernetes ConfigMaps y Secrets. Spring Cloud Gateway será reemplazado por un objeto Kubernetes Ingress, que puede actuar como un servidor de borde de la misma manera que Spring Cloud Gateway.

En el Capítulo 11, "Seguridad del acceso a las API", presentamos el uso de certificados para proteger la API externa. Los certificados se aprovisionaban manualmente, lo cual era lento y propenso a errores, especialmente al recordar rotarlos antes de que caduquen. En este capítulo, aprenderemos sobre cert-manager y cómo se puede usar para automatizar el proceso de creación, aprovisionamiento y rotación de certificados.

Cuando se utilizan cada vez más funciones en una plataforma, como Kubernetes, es importante garantizar que el código fuente de los microservicios no dependa de la plataforma.

Para garantizar que aún podamos usar los microservicios sin implementarlos en Kubernetes, concluiremos el capítulo implementando el panorama de microservicios usando Docker Compose y ejecutando el script de prueba `test-em-all.bash` para verificar que los microservicios aún funcionan desde una perspectiva funcional sin usar Kubernetes.

En este capítulo se tratarán los siguientes temas:

- Reemplazo del servidor de configuración de Spring Cloud con ConfigMaps y secretos de Kubernetes
- Reemplazo de Spring Cloud Gateway con un objeto Ingress de Kubernetes
- Uso de cert-manager para aprovisionar certificados automáticamente
- Implementación y prueba del panorama de microservicios en Kubernetes
- Implementar y probar el entorno de microservicios utilizando Docker Compose para garantizar que El código fuente de los microservicios no está bloqueado en Kubernetes.

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte lo siguiente:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter17`.

Si desea ver los cambios aplicados al código fuente en este capítulo (es decir, ver qué se necesitó para reemplazar Spring Cloud Config Server y Spring Cloud Gateway con las funciones correspondientes en Kubernetes, y usar cert-manager para aprovisionar certificados), puede compararlo con el código fuente del Capítulo 16, "Implementación de nuestros microservicios en Kubernetes". Puede usar su herramienta de comparación preferida y comparar las dos carpetas: `$BOOK_HOME/Chapter16` y `$BOOK_HOME/Chapter17`.

Reemplazo del servidor de configuración de Spring Cloud

Como vimos en el capítulo anterior, ConfigMaps y Secrets pueden usarse para almacenar información de configuración de nuestros microservicios. Spring Cloud Config Server añade características como mantener todas las configuraciones en un solo lugar, control de versiones opcional mediante Git y la capacidad de cifrar información confidencial en el disco. Sin embargo, también consume una cantidad considerable de memoria (como cualquier aplicación basada en Java y Spring) y añade una sobrecarga significativa durante el arranque.

Por ejemplo, al ejecutar pruebas de integración automatizadas, como el script de prueba que usamos en este libro, `test-em-all.bash`, todos los microservicios se inician simultáneamente, incluido el servidor de configuración. Dado que los demás microservicios deben obtener su configuración del servidor de configuración, deben esperar a que este esté activo para poder iniciarse. Esto genera un retraso significativo al ejecutar las pruebas de integración. Si, en cambio, usamos ConfigMaps y Secrets de Kubernetes, se elimina este retraso, acelerando la ejecución de las pruebas de integración automatizadas. Por lo tanto, tiene sentido usar Spring Cloud Config Server cuando la plataforma subyacente no ofrece una capacidad similar, pero al implementar en Kubernetes, es mejor usar ConfigMaps y Secrets.

Usar ConfigMaps y Secrets de Kubernetes en lugar de Spring Cloud Config Server acelerará el inicio del entorno de microservicios y reducirá el consumo de memoria. Además, simplificará el entorno de microservicios al eliminar un servicio de soporte: el servidor de configuración. Al realizar el reemplazo, es importante hacerlo de forma que el código fuente de los microservicios no se vea afectado, evitando así una dependencia innecesaria de Kubernetes.

Este cambio se ilustra en el siguiente diagrama:

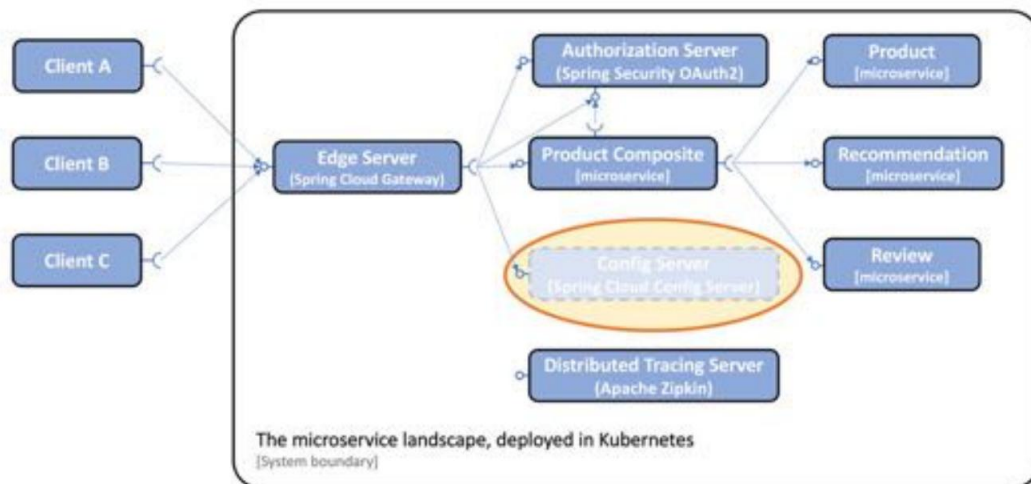


Figura 17.1: Reemplazo del servidor Spring Cloud Config con ConfigMaps integrados de Kubernetes y secretos

Veamos qué se requiere para reemplazar Spring Cloud Config Server con Kubernetes ConfigMaps ¡Y secretos!



Tenga en cuenta especialmente que solo cambiamos la configuración; ¡no se requieren cambios en el código fuente de Java!

Cambios necesarios para reemplazar Spring Cloud Config Servidor

Se han aplicado los siguientes cambios en la configuración del código fuente para reemplazar el

Servidor de configuración de Spring Cloud con mapas de configuración y secretos de Kubernetes:

1. Hemos eliminado el proyecto `spring-cloud/config-server` y también hemos eliminado el proyecto del archivo de compilación `settings.gradle`.
2. Hemos eliminado el gráfico de Helm para el servidor de configuración.
3. Hemos eliminado las pruebas específicas del servidor de configuración del script de prueba `test-em-all.bash`.
4. Hemos eliminado la siguiente configuración de todos los microservicios:
 - La dependencia `spring-cloud-starter-config` en los archivos de compilación `build.gradle`
 - Los archivos `application.yml` en las carpetas `src/main/resource` en cada proyecto, que se utilizaron para conectarse al servidor de configuración
 - La configuración de la propiedad `spring.cloud.config.enabled=false` en las pruebas de integración, ya que ya no es necesario
5. Los cambios en los archivos de configuración en la carpeta `config-repo` incluyen lo siguiente:

Hemos eliminado propiedades con información confidencial, como las credenciales de MongoDB, MySQL, RabbitMQ y la contraseña del certificado TLS utilizado por el servidor perimetral. Se utilizarán los secretos de Kubernetes para gestionar la información confidencial.

 - Se ha eliminado la ruta a la API del servidor de configuración en la configuración del servidor de borde.
6. Los cambios en los gráficos de Helm de los microservicios en `kubernetes/helm/components` incluyen la siguiente:

Se ha añadido una carpeta "config-repo" a cada gráfico. Se han creado enlaces simbólicos en la carpeta "config-repo" del gráfico de Helm para los archivos de configuración necesarios desde la carpeta "config-repo" común. Para cada microservicio, se ha creado un enlace simbólico al archivo de configuración común, `application.yaml`, y al archivo de configuración específico del microservicio.



Para obtener un resumen sobre cómo se crean los enlaces blandos, consulte la sección Gráficos de componentes en el Capítulo 16, Implementación de nuestros microservicios en Kubernetes.

- El archivo `values.yaml` se ha actualizado con lo siguiente:

Una variable de entorno para una propiedad de Spring que indica qué archivos de configuración usar. Por ejemplo, la propiedad se ve así para el microservicio de producto :

```
UBICACIÓN_DE_CONFIGURACIÓN_DE_SPRING: archivo:/config-repo/application.
yml,archivo:/config-repo/producto.yml
```

- Un `ConfigMap` que los microservicios usarán para encontrar sus archivos de configuración. Este `ConfigMap` estará disponible dentro del contenedor, en la ruta `/config-repo` . La declaración se ve así:

```
mapa de configuración:
  habilitado: verdadero
montajes de volumen:
  ruta de montaje: /config-repo
```

- Para crear el `ConfigMap`, se ha agregado una plantilla que se basa en la plantilla denominada `common.configmap_from_file`, del gráfico común .

7. Los cambios en los gráficos de Helm de los entornos en `kubernetes/helm/environments` incluyen la siguiente:

- Hemos eliminado la dependencia del gráfico del servidor de configuración.
- Se ha actualizado el archivo `values.yaml` .
- Los secretos para el servidor de configuración y sus clientes se han reemplazado por secretos para los administradores de recursos, MongoDB, MySQL y RabbitMQ, y sus clientes, como en este ejemplo:

```
credenciales-zipkin-de-rabbitmq:
  USUARIO_CONEJO: usuario-conejo-dev
  CONTRASEÑA DE CONEJO: conejo-pwd-dev

credenciales de mongodb:
```

BASE DE DATOS DE AUTENTICACIÓN DE SPRING_DATA_MONGODB: administrador

SPRING_DATA_MONGODB_USERNAME: mongodb-user-dev

CONTRASEÑA DE MONGODB DE DATOS DE SPRING: mongodb-pwd-dev



Como se recuerda en el capítulo anterior, este archivo values.yaml contiene información confidencial, como las contraseñas del ejemplo anterior. Por lo tanto, debe almacenarse de forma segura. Si no es adecuado almacenarlo de forma segura, una alternativa es eliminar la información confidencial y proporcionarla al ejecutar el comando helm install .

- A cada componente se le asignan los secretos que requiere.



Resumen del capítulo anterior: los Secretos se mapearán en cada Pod como variables de entorno.

Por ejemplo, el servicio del producto necesita acceso tanto a MongoDB como a RabbitMQ y, por lo tanto, se le asignan los siguientes dos secretos:

```
producto:
  envFromSecretRefs:
    - credenciales de rabbitmq
    - credenciales de mongodb
```

La mayoría de los cambios en los archivos values.yaml de los gráficos de Helm se incluyen en los manifiestos de Kubernetes para los objetos de implementación . Por ejemplo, el objeto de implementación del microservicio de producto tendrá el siguiente aspecto:

```
apiVersion: aplicaciones/v1
tipo: Implementación
metadatos:
  nombre: producto
especificación:
  plantilla:
    especificación:
      contenedores:
        - nombre: producto
```

```

entorno:

- nombre: SPRING_CONFIG_LOCATION valor:
  archivo:/config-repo/application.yml,archivo:/config-
    repositorio/producto.yml

- nombre: SPRING_PROFILES_ACTIVE valor:
  docker

envDe:

- referencia secreta:
  nombre: credenciales-de-rabbitmq

- referencia secreta:
  nombre: credenciales-mongodb

montajes de volumen:

- nombre: producto
  mountPath: /config-repo

volúmenes:

- nombre: producto
  configMap:
    nombre: producto

```

 Consejo rápido: Mejora tu experiencia de programación con las funciones AI Code Explainer y Quick Copy . Abre este libro en el Packt Reader de última generación. Haz clic en el botón Copiar.

(1) para copiar rápidamente el código en su entorno de codificación, o haga clic en el botón Explicar

(2) para que el asistente de IA le explique un bloque de código.



 El lector Packt de próxima generación se incluye de forma gratuita con la compra de este libro.

Escanee el código QR O vaya a packtpub.com/unlock, Luego, usa la barra de búsqueda para encontrar este libro por nombre. Revisa la edición que se muestra para asegurarte de encontrar la correcta.



Tenga en cuenta que las partes del manifiesto que no se han visto afectadas por los cambios anteriores se han omitido para mejorar la legibilidad.

Si desea renderizar usted mismo el manifiesto de Kubernetes de un componente, puede hacerlo aplicando el comando de plantilla de Helm en el gráfico correspondiente. También debe agregar al comando de plantilla los valores del archivo values.yaml del entorno aplicables al componente . Tomemos como ejemplo el servicio del producto . El archivo values.yaml del gráfico dev-env contiene las siguientes configuraciones aplicables al servicio del producto :

```
producto:
  envFromSecretRefs:
    - credenciales de rabbitmq
    - credenciales de mongodb
```

Para agregar estas configuraciones al comando de plantilla , podemos usar el indicador --set .



También hay un indicador --values que se puede usar para agregar un archivo values.yaml cuando se ejecuta el comando.

Al agregar valores de un gráfico de entorno, debemos recordar que son gráficos primarios de los gráficos de componentes. Esto significa que el nombre del gráfico de componentes debe eliminarse de la configuración al aplicarlo directamente. En este caso, debemos agregar los siguientes valores al comando de plantilla para representar correctamente el gráfico de productos :

```
envFromSecretRefs:
  - credenciales de rabbitmq
  - credenciales de mongodb
```

Las matrices YAML (como la anterior) se pueden definir con el indicador --set , enumerando los elementos entre llaves, por ejemplo, "{a,b,c}" . El gráfico de productos se puede representar con el siguiente comando:

```
plantilla de timón kubernetes/helm/components/product \
--establecer envFromSecretRefs= \
  "{credenciales-rabbitmq, credenciales-mongodb}"
```

El resultado contendrá manifiestos para el microservicio del producto , un ConfigMap, un Servicio y, finalmente, un objeto de Implementación .

Esto es lo que se requiere para reemplazar el servidor de configuración con Kubernetes ConfigMaps y

Secretos. En la siguiente sección, aprenderemos cómo reemplazar Spring Cloud Gateway con un objeto Ingress de Kubernetes.

Reemplazo de Spring Cloud Gateway

En esta sección, simplificaremos aún más el panorama de microservicios al reemplazar Spring Cloud Gateway con el objeto Ingress integrado en Kubernetes, lo que reducirá la cantidad de servicios de soporte que deben implementarse.

Como se presentó en el Capítulo 15, Introducción a Kubernetes, un objeto Ingress puede usarse en Kubernetes para actuar como servidor perimetral, al igual que Spring Cloud Gateway. Spring Cloud Gateway ofrece una funcionalidad de enrutamiento más completa que un objeto Ingress. Sin embargo, Ingress forma parte de la plataforma Kubernetes, por lo que no requiere implementaciones adicionales y también puede ampliarse mediante cert-manager para aprovisionar certificados automáticamente, como veremos más adelante en este capítulo.

También hemos utilizado Spring Cloud Gateway para proteger nuestros microservicios de solicitudes no autenticadas, requiriendo un token de acceso OAuth 2.0/OIDC válido de un servidor de autorización OAuth o un proveedor OIDC de confianza. Consulte el Capítulo 11, "Seguridad del acceso a las API", si necesita un resumen. Generalmente, los objetos de Ingress de Kubernetes no son compatibles con esto. Sin embargo, implementaciones específicas del controlador de Ingress podrían serlo.

Finalmente, la comprobación de estado compuesta que agregamos a la puerta de enlace en el Capítulo 10, Uso de Spring Cloud Gateway para ocultar microservicios detrás de un servidor perimetral, se puede reemplazar por las sondas de preparación y actividad de Kubernetes definidas en el manifiesto de implementación de cada microservicio.

Por lo tanto, al igual que con Spring Cloud Config Server, conviene usar Spring Cloud Gateway cuando la plataforma subyacente no ofrece una capacidad similar. Al implementar en Kubernetes, es recomendable usar objetos Ingress.

En este capítulo, delegaremos la responsabilidad de validar que la solicitud contenga un token de acceso válido al microservicio compuesto por producto . Esto se realiza mediante el reenvío del encabezado HTTP que contiene el token de acceso en las solicitudes al microservicio compuesto por producto , y este realizará la validación de los tokens de acceso OAuth como en los capítulos anteriores.

El próximo capítulo presentará el concepto de una malla de servicios, donde veremos una implementación alternativa de un Ingress que admite totalmente la validación de tokens de acceso OAuth codificados con JWT.



En la sección Verificar que los microservicios funcionan sin Kubernetes , seguiremos usando Spring Cloud Gateway junto con Docker Compose, por lo que no eliminaremos el proyecto.

El siguiente diagrama muestra que Spring Cloud Gateway se elimina del panorama de microservicios cuando se implementa en Kubernetes:

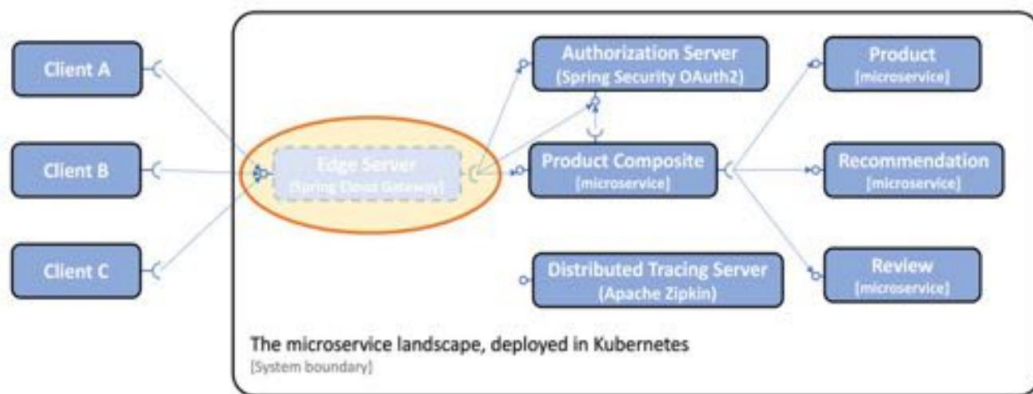


Figura 17.2: Reemplazo de Spring Cloud Gateway con el controlador Ingress integrado de Kubernetes

¡Veamos qué se requiere para reemplazar Spring Cloud Gateway con un objeto Ingress de Kubernetes!



Tenga en cuenta especialmente que solo cambiamos la configuración; es decir, ¡no se requieren cambios en el código fuente de Java!

Cambios necesarios para reemplazar Spring Cloud Gateway

Se han aplicado los siguientes cambios a la configuración del código fuente para reemplazar el

Spring Cloud Gateway con un objeto Ingress de Kubernetes:

1. Hemos eliminado el gráfico de Helm para Spring Cloud Gateway.
2. Hemos agregado una plantilla con nombre para los manifiestos de Ingress y algunos valores predeterminados para el Ingreso en el gráfico común .

La plantilla nombrada, `kubernetes/helm/common/templates/_ingress.yaml`, comienza con una declaración que reconocemos del capítulo anterior:

```

{{- define "common.ingress" -}}
{{- $common := dict "Valores" .Valores.common -}}
{{- $noCommon := omit .Valores "comunes" -}}
{{- $overrides := dict "Valores" $noCommon -}}
{{- $noValues := omit . "Valores" -}}
{{- con fusión $noValues $overrides $common -}}
Versión de API: networking.k8s.io/v1
tipo: Ingress
metadatos:
  nombre: {{ include "common.fullname" . }}
  etiquetas:
    app.kubernetes.io/nombre: {{ incluir "common.name" . }}
    helm.sh/chart: {{ incluir "common.chart" . }}
    app.kubernetes.io/administrado por: {{ .Release.Service }}
{{- con .Values.ingress.annotations }}
  anotaciones:
{{ toYaml . }}
fin }}

```

El nombre de la plantilla es `common.ingress`, y `apiVersion` y `kind` se configuran como `networking.k8s.io/v1` e `Ingress` para identificarla como un manifiesto de Ingress. La mayor parte del resto de la plantilla anterior es idéntica a la observada en otros manifiestos donde se requieren parámetros de sobrescritura, como la plantilla de Implementación o Servicio.

La única parte nueva es que la plantilla permite agregar anotaciones, si es necesario, utilizando el campo `ingress.annotations` en el archivo `values.yaml`.

El resto de la plantilla de Ingress contiene la parte principal del manifiesto y la parte de especificaciones. se ve así:

```

especificación:
  tls:
    - anfitriones:
        - {{ .Values.ingress.host | cita }}
      nombreSecreto: {{ .Values.ingress.tls.nombreSecreto }}
  normas:
    - host: {{ .Values.ingress.host | cita }}

```

```

    http:
      caminos:
        {{- rango .Valores.entrada.rutas }}
        - ruta: {{ .path }}
          pathType: Prefijo
          backend:
            servicio:
              nombre: {{ .service }}
              puerto:
                nombre: http
            {{- fin }}
        {{- fin }}
    {{- fin -}}

```

Primero viene una sección `tls` donde el manifiesto declara que Ingress solo acepta tráfico HTTPS y que el nombre de host aceptado se especificará con la clave `ingress.host` en los archivos `values.yaml`. El certificado utilizado para atender las solicitudes HTTPS se almacenará en un secreto cuyo nombre se especifica en los archivos `values.yaml` mediante la clave `ingress.tls.secretName`.

A continuación, se presentan las reglas de enrutamiento declaradas en la sección `"rules"`. Primero, se incluye el nombre de host utilizado para el enrutamiento. Este será el mismo que el de la sección `"tls"` anterior. A continuación, se presenta una lista de rutas. Estas se completarán mediante la sección `"ingress.paths"` del archivo `values.yaml`.

Cada entrada contiene una ruta y el nombre del servicio al que se dirigirán las solicitudes a esa ruta. Se espera que cada servicio tenga el nombre de su puerto configurado como `http`.

El archivo `values.yaml` del gráfico común proporciona los siguientes valores predeterminados para Ingress manifiesto:

```

ingreso:
  anotaciones:
    cert-manager.io/issuer: autofirmado
  tls:
    secretName: certificado-tls

```

Primero, se declara una anotación, `cert-manager.io/issuer`, para el objeto Ingress, que indica que `cert-manager` debe gestionar el certificado requerido para este objeto Ingress mediante un emisor autofirmado. Puede encontrar más información sobre esto en la sección `"Automatización del aprovisionamiento de certificados"`. A continuación, se encuentra el secreto que contendrá el certificado, dado el valor predeterminado. nombre `tls-certificate`.

3. Hemos añadido plantillas y configuraciones adicionales a los gráficos de entorno, dev-env y prod-env, para los manifiestos de Ingress. Las plantillas se llaman ingress.yml y se basan en la plantilla con nombre del gráfico común descrito anteriormente:

```
{{- plantilla "common.ingress" . -}}
```

4. Los valores restantes necesarios para generar un manifiesto de Ingress (un nombre de host y las rutas reales utilizadas para el enrutamiento) se especifican en los archivos values.yaml de cada diagrama de entorno . La declaración se ve así:

```
ingreso:
  anfitrión: minikube.me
  caminos:
    - ruta: /oauth2
      servicio: servidor de autenticación
    - ruta: /login
      servicio: servidor de autenticación
    - ruta: /error
      servicio: servidor de autenticación
    - ruta: /producto-compuesto
      servicio: producto-compuesto
    - ruta: /actuador/salud
      servicio: producto-compuesto
    - ruta: /openapi
      servicio: producto-compuesto
```

Desde la configuración, podemos ver que utilizaremos el nombre de host minikube.me y que tres rutas están definidas para auth-server, mientras que el resto de las rutas declaradas se enrutarán al servicio compuesto por producto .



Registraremos el nombre de host minikube.me en el archivo local /etc/hosts más adelante, en la sección Pruebas con Kubernetes ConfigMaps, Secrets, Ingress y cert-manager .

Los cambios anteriores generarán un manifiesto de Ingress mediante Helm. Dado que la plantilla de Ingress solo se utiliza en los gráficos de entornos, necesitamos generar uno de ellos para ver el manifiesto de Ingress.

Ejecute el siguiente comando para representar manifiestos utilizando el gráfico dev-env :

```
plantilla de timón kubernetes/helm/environments/dev-env
```

Busca kind: Ingress en la salida y encontrarás el manifiesto de Ingress. Su aspecto es el siguiente:

```
apiVersion: networking.k8s.io/v1 tipo: Ingress

metadatos:
  nombre: NOMBRE-DE-VERSIÓN-dev-env
  etiquetas:
    app.kubernetes.io/nombre: dev-env
    helm.sh/chart: dev-env-1.0.0
    app.kubernetes.io/administrado por: Helm
  anotaciones:
    cert-manager.io/issuer: autofirmado

especificación:
  tls:
    - anfitriones:
        - "minikube.me"
      secretName: certificado-tls
  normas:
    - anfitrión: "minikube.me"
      http:
        rutas:
          - ruta: /oauth2
            pathType: Prefijo
            backend:
              servicio:
                nombre: auth-server
              puerto:
                nombre: http
          - ruta: /producto-compuesto
            pathType: Prefijo
            backend:
              servicio:
                nombre: producto compuesto
              puerto:
                nombre: http
```

```
- ruta: /actuador/salud
  pathType: Prefijo
  backend:
    servicio:
      nombre: producto compuesto
      puerto:
        nombre: http
```

Tenga en cuenta que se han eliminado algunas de las reglas de enrutamiento para mejorar la legibilidad.

La última pieza que falta es cómo se crea el secreto que contiene el certificado; veámoslo.

próximo.

Automatizar el aprovisionamiento de certificados

La herramienta cert-manager (<https://cert-manager.io/docs/>) Es un controlador de gestión de certificados para Kubernetes. Facilita la creación, el aprovisionamiento y la rotación automatizados de certificados.

Admite varias fuentes para los certificados, como las siguientes:

- Compatible con RFC8555 (<https://tools.ietf.org/html/rfc8555>) Servidor ACME como como Let's Encrypt (<https://letsencrypt.org>)
- Motor de secretos PKI de HashiCorp Vault (<https://www.vaultproject.io/docs/secrets/pki>)
- Certificados autofirmados, emitidos por el propio administrador de certificados

Para obtener una lista completa de los emisores disponibles, consulte <https://cert-manager.io/docs/configuration/>.

Dado que los certificados autofirmados no requieren comunicación con recursos externos, son una buena opción para su uso durante el desarrollo. Los utilizaremos en el marco de este libro.



El uso de cert-manager en producción generalmente requiere el uso de un emisor, como Let's Encrypt, que puede emitir certificados para las API externas en las que confiarán los clientes de API (por ejemplo, navegadores web y sistemas externos).

Tras instalar cert-manager en un clúster de Kubernetes, se debe registrar al menos un emisor. Un emisor puede ser local en un espacio de nombres o accesible en todo el clúster. En la práctica, utilizaremos un emisor local registrado en el espacio de nombres existente .

Será responsabilidad de los gráficos de entorno, dev-env y prod-env, registrar un emisor adecuado. Ambos entornos utilizarán el emisor autofirmado. Una plantilla con nombre, `_issuer.yaml`, tiene Se ha añadido al gráfico común . Se ve así:

```
{{- define "common.issuer" -}}
Versión de API: cert-manager.io/v1
tipo: Emisor
metadatos:
  nombre: autofirmado
especificación:
  autofirmado: {}
{{- fin -}}
```

Los campos `apiVersion` y `kind` especifican que se trata de un emisor definido por `cert-manager`. Su nombre es `selfsigned` . En la sección anterior " Cambios necesarios para reemplazar Spring Cloud Gateway" , vimos cómo se usó este nombre para anotar el manifiesto de Ingress:

```
ingreso:
  anotaciones:
    cert-manager.io/issuer: autofirmado
  tls:
    secretName: certificado-tls
```

Esto es todo lo que se necesita para que `cert-manager` se active y proporcione un certificado para el objeto Ingress. `cert-manager` escucha el registro de los objetos Ingress que están anotados con `cert-manager.io/issuer` y comienza a emitir certificados utilizando el emisor referenciado en el valor de la anotación, en este caso, `autofirmado`.

La herramienta `cert-manager` usará el emisor para crear un certificado y lo almacenará en un secreto cuyo nombre corresponde al objeto Ingress. En nuestro caso, el nombre es `tls-certificate`. También se creará un objeto Certificado con el mismo nombre, que contiene información administrativa, como cuándo debe `cert-manager` renovar el certificado.

Dado que la plantilla con nombre, `common.issuer`, no admite ninguna configuración, para aplicarla en los gráficos de desarrollo y producción basta con agregar una plantilla a cada gráfico que la utilice. La plantilla se llama `issuer.yaml` y tiene este aspecto:

```
{{- plantilla "common.issuer" . -}}
```

Con esto, tenemos todo lo necesario para reemplazar Spring Cloud Config Server y Gateway con componentes nativos de Kubernetes y `cert-manager`. ¡Vamos a implementar y ejecutar algunas pruebas!

Pruebas con Kubernetes ConfigMaps, Secrets, Ingress y administrador de certificados

Con los cambios descritos, estamos listos para probar el entorno del sistema con Spring Cloud Config Server y Spring Cloud Gateway, reemplazados por Kubernetes ConfigMaps, Secrets, un objeto Ingress y cert-manager. Al igual que antes, al usar Spring Cloud Gateway como servidor perimetral, la API externa estará protegida por HTTPS. Con esta implementación, será el controlador Ingress el que utilice el certificado proporcionado por cert-manager para proteger la API externa con HTTPS. Esto se ilustra en el siguiente diagrama:

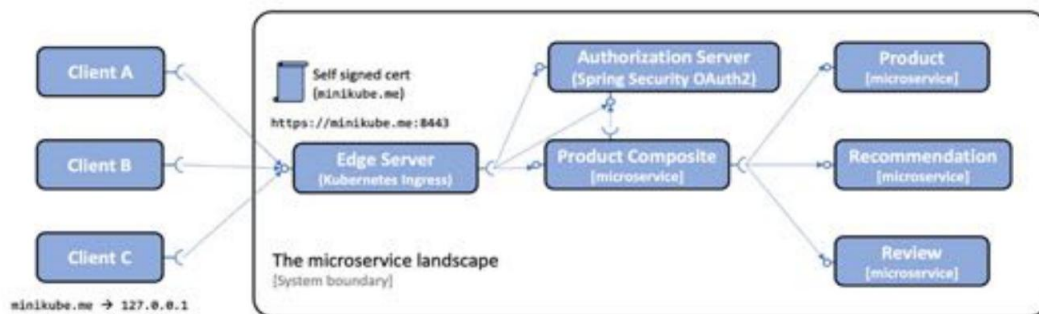


Figura 17.3: Protección del acceso externo mediante HTTPS

El controlador de Ingress se expone en el puerto HTTPS predeterminado, 443, en la instancia de minikube. En el host donde ejecutamos la instancia de minikube como contenedor Docker, nos comunicamos con ella a través de localhost. Al crear la instancia de minikube, se configuró el redireccionamiento de puertos del puerto 8443 en localhost al puerto 443 en la instancia de minikube. El controlador de Ingress se instaló al ejecutar el comando "minikube addons enable ingress".



Para obtener un resumen de la configuración de la instancia de minikube, consulte Creación de un clúster de Kubernetes sección en el Capítulo 15, Introducción a Kubernetes.

Una pregunta interesante es: ¿cómo puede el controlador de Ingress usar el puerto 443 en la instancia de minikube? Hemos visto el uso de servicios del tipo NodePort que pueden asignar un puerto a partir del 30000, así que ¿cómo puede el controlador de Ingress usar el puerto estándar para HTTPS, el 443?

El controlador de Ingress consta de un objeto de implementación , ingress-nginx-controller, en el espacio de nombres ingress-nginx . La respuesta a la pregunta es que el objeto de implementación configura su pod usando hostPort para asignar el puerto 443 del host de Kubernetes (es decir, la instancia de minikube) al puerto 443 del contenedor que se ejecuta en el pod. Las partes centrales de la definición de la implementación

El objeto se parece al siguiente:

```
apiVersion: aplicaciones/v1
tipo: Implementación
metadatos:
  nombre: controlador-nginx-de-ingreso
especulación:
  plantilla:
    especulación:
      contenedores:
        imagen: registry.k8s.io/nginx-ingress/controller:v1.5.1
        puertos:
          - Puerto del contenedor: 443
            puerto del host: 443
```



Esta configuración funciona para un clúster de Kubernetes de un solo nodo, utilizado para desarrollo y pruebas. En un clúster de Kubernetes de varios nodos, se utilizan balanceadores de carga externos para exponer un controlador de Ingress y lograr alta disponibilidad y escalabilidad.

El objeto de implementación usa el mismo tipo de comandos que en el Capítulo 16, " Implementación de nuestros microservicios en Kubernetes"; consulte la sección "Implementación en Kubernetes para desarrollo y pruebas" . En esta sección, también instalaremos cert-manager y agregaremos una entrada al archivo /etc/hosts para... nombre de host minikube.me.

Ejecute los siguientes pasos para implementar el panorama del sistema y verificar que funcione como se espera:

1. Instale cert-manager en su espacio de nombres y espere a que se complete la implementación. Antes de instalar cert-manager, debemos agregar su repositorio Helm. Ejecute los siguientes comandos:

```
Repositorio de Helm: agregar Jetstack https://charts.jetstack.io
```

```
actualización del repositorio de Helm
```

```
helm instalar cert-manager jetstack/cert-manager \
```

```
--crear-espacio-de-nombres \
```

```
--namespace administrador de certificados \
```

```
--versión v1.11.0 \
--establecer installCRDs=verdadero \
--esperar
```



La herramienta cert-manager también incluye un conjunto de definiciones de recursos personalizados (CRD) de Kubernetes, como el objeto emisor que se presentó anteriormente. Las CRD se utilizan en Kubernetes para ampliar su API, es decir, para añadir nuevos objetos a ella. El indicador `--set installCRDs=true` del comando anterior garantiza que estas definiciones de objeto se instalen al instalar

administrador de certificados.

Verifique que haya tres Pods listos en el espacio de nombres cert-manager con el siguiente comando:

Mand:

```
kubectl obtener pods --namespace administrador de certificados
```

Espere una respuesta como esta:

```
bash
NAME                                READY   STATUS    RESTARTS   AGE
cert-manager-64f9f45d6f-2dml         1/1     Running   0           10h
cert-manager-cainjector-56bbdd5c47-krzpw 1/1     Running   0           10h
cert-manager-webhook-d4f4545d7-zrhvf    1/1     Running   0           10h
$
```

Figura 17.4: Pods en el espacio de nombres cert-manager

2. Asigne minikube.me a la dirección IP que podemos usar para llegar a la instancia de minikube agregando

Una línea al archivo `/etc/hosts` . Ejecute el siguiente comando:

```
sudo bash -c "echo 127.0.0.1 minikube.me | tee -a /etc/hosts"
```



Tenga en cuenta que el comando `sudo` probablemente le solicitará su contraseña.

Verifique el resultado con el comando `cat /etc/hosts` . Se espera una línea que contenga `127.0.0.1 minikube.me`.

Si su archivo `/etc/hosts` contiene varias líneas para minikube.me (por ejemplo, de intentos anteriores), deberá eliminar las antiguas manualmente.

3. Puedes crear imágenes de Docker a partir del código fuente de la siguiente manera:

```
cd $BOOK_HOME/Capítulo 17
evaluar $(minikube docker-env -u)
./gradlew compilación
evaluar $(minikube docker-env)
compilación de docker compose
```



El comando eval \$(minikube docker-env -u) se utiliza para garantizar que el comando de compilación ./gradlew utilice el motor Docker del host y no el de la instancia de minikube. El comando de compilación utiliza Docker para ejecutar los contenedores de prueba.

4. Resuelva las dependencias del gráfico de Helm:

- a. Primero, actualizamos las dependencias en la carpeta de componentes :

```
para f en kubernetes/helm/components/*; hacer helm dep up $f; hecho
```

- b. A continuación, actualizamos las dependencias en la carpeta de entornos :

```
para f en kubernetes/helm/environments/*; hacer helm dep up $f; hecho
```

5. Establezca el espacio de nombres práctico como el espacio de nombres predeterminado para kubectl:

```
configuración de kubectl contexto-actual $(configuración de kubectl contexto-actual) --
namespace=hands-on
```

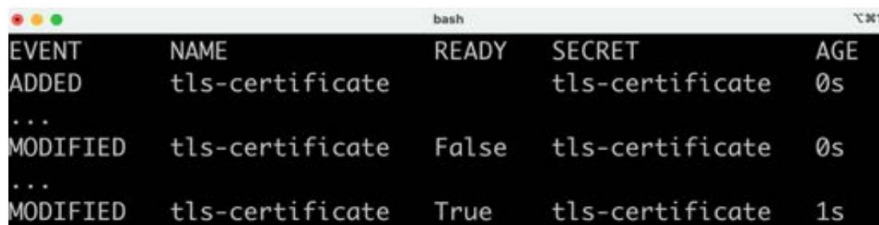
6. En una ventana de terminal independiente, ejecute el siguiente comando para supervisar cómo cert-manager crea los objetos de certificado:

```
kubectl obtener certificados -w --output-watch-events
```

7. Implemente el entorno del sistema usando Helm y espere a que se completen todas las implementaciones:

```
instalación de helm en un entorno de desarrollo práctico
kubernetes/helm/entornos/dev-env \
-n práctico \
--crear-espacio-de-nombres \
--esperar
```

8. Observe cómo cert-manager crea el certificado durante la implementación. Espere siguiente salida del comando `kubectl get certificados` :



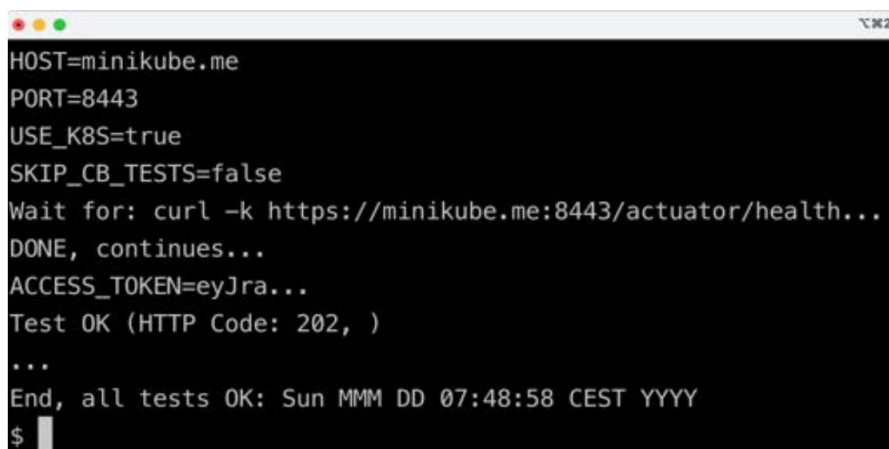
EVENT	NAME	READY	SECRET	AGE
ADDED	tls-certificate		tls-certificate	0s
...				
MODIFIED	tls-certificate	False	tls-certificate	0s
...				
MODIFIED	tls-certificate	True	tls-certificate	1s

Figura 17.5: Eventos del administrador de certificados que aprovisiona un certificado

9. Detenga el comando `kubectl get certificados` con `Ctrl + C`.
10. Ejecute la prueba para verificar que el entorno del sistema funciona como se espera:

```
HOST=minikube.me PUERTO=8443 USE_K8S=verdadero ./test-em-all.bash
```

Espere resultados de las pruebas similares a los que obtuvimos en el capítulo anterior (en formato condensado):



```
HOST=minikube.me
PORT=8443
USE_K8S=true
SKIP_CB_TESTS=false
Wait for: curl -k https://minikube.me:8443/actuator/health...
DONE, continues...
ACCESS_TOKEN=eyJra...
Test OK (HTTP Code: 202, )
...
End, all tests OK: Sun MMM DD 07:48:58 CEST YYYY
$
```

Figura 17.6: Verificación del panorama del sistema creado por el diagrama de Helm dev-env

Antes de finalizar dev-env, probemos el objeto de certificado que creó cert-manager y veamos cómo se puede utilizar para afectar el tiempo de retención del certificado.

Certificados rotativos

Comencemos a familiarizarnos con el objeto de certificado emitiendo el siguiente comando:

```
kubectl describe cert tls-certificate
```

Al final de la salida del comando, encontraremos la siguiente información respecto a el tiempo durante el cual será válido el certificado:

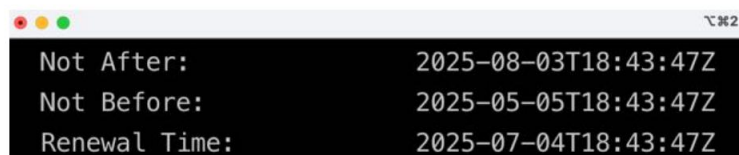


Figura 17.7: Periodo de validación del certificado y tiempo de renovación

Podemos ver que el certificado tiene una validez de 90 días (No después - No antes) y que cert-manager intentará renovarlo después de 60 días (Tiempo de renovación - No antes). Dado que el emisor autofirmado que utilizamos no permite ninguna configuración, estos son los valores predeterminados que cert-manager utiliza: 90 días de vigencia y un proceso de renovación que se inicia después de 2/3 de la vigencia.

Pero no queremos esperar 60 días para observar la renovación del certificado. Si estudiamos la especificación de la API para el objeto de certificado en <https://cert-manager.io/docs/reference/api-docs/#cert-manager.io/v1.Certificate>, Encontraremos un campo interesante en la sección de especificaciones. Se llama "renewBefore" y permite especificar con qué anticipación cert-manager debe iniciar el proceso de renovación. Si queremos que el certificado se renueve cada minuto, podemos especificar que "renewBefore" sea 90 días – 1 minuto = 90 * 24 horas – 1 minuto = 2160 horas – 1 minuto = 2159 horas y 59 minutos.

Inicie el comando `kubectl get events -w` en una ventana de terminal separada y ejecute el siguiente comando de parche para agregar el campo `renewBefore` al certificado:

```

Certificado de parche de kubectl tls-certificate --type=json \
-p='[{"op": "add", "path": "/spec/renewBefore", "value": "2159h59m"}]'
  
```

En un minuto, el comando "get events" debería empezar a informar sobre las renovaciones de certificados. Para cada renovación, el comando "get events" debería imprimir lo siguiente :

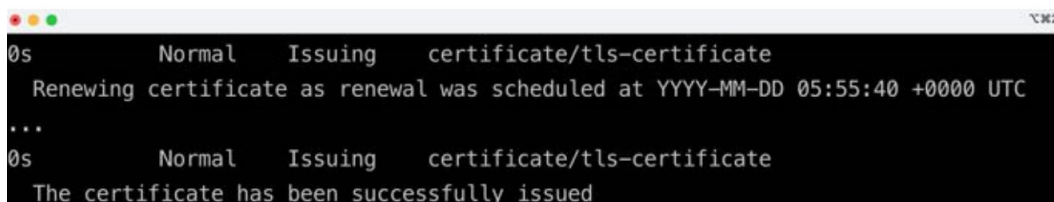


Figura 17.8: Eventos de cert-manager que rotan un certificado

Espere un par de minutos para verificar que el certificado se renueva cada minuto. Si tiene curiosidad por saber cuándo se realizará la próxima renovación, puede ejecutar el siguiente comando:

```
kubectl obtener certificado tls-certificate -o json | jq .status.renewalTime
```

Debería responder con una fecha como 2025-05-07T05:48:20Z.

Si ya no desea tener un tiempo de retención personalizado, puede eliminar el campo renovarAntes con el siguiente comando:

```
Certificado de parche de kubectl tls-certificate --type=json \
-p='[{"op": "eliminar", "ruta": "/spec/renewBefore"}]'
```

Con esto concluyen las pruebas que realizaremos en el panorama del sistema implementado utilizando el gráfico dev-env . Podemos eliminar el panorama del sistema con el siguiente comando:

```
Eliminación práctica de espacios de nombres en kubectl
```

¡Recapitulemos también cómo implementar el panorama del sistema usando el gráfico prod-env !

Implementación en Kubernetes para pruebas y producción

La implementación en un entorno de prueba y producción mediante el diagrama prod-env sigue los mismos pasos que en la sección "Implementación en Kubernetes para pruebas y producción" del capítulo 16, "Implementación de nuestros microservicios en Kubernetes". Los pasos se resumen aquí de forma compacta:

1. Inicie MySQL, MongoDB y RabbitMQ fuera de Kubernetes:

```
evaluar $(minikube docker-env)
docker componer -d mongodb mysql rabbitmq
```

2. Etiqueta las imágenes de Docker con las versiones v1 :

```
etiqueta docker hands-on/auth-server hands-on/auth-server:v1
etiqueta docker práctica/servicio-compuesto-de-producto práctica/servicio-
compuesto-de-producto:v1
etiqueta docker práctica/servicio de producto práctica/servicio de producto:v1
etiqueta docker servicio práctico/de recomendación servicio práctico/de recomendación:v1

etiqueta docker servicio práctico/de revisión servicio práctico/de revisión:v1
```

3. Implemente los microservicios utilizando el diagrama de Helm prod-env :

```
instalación de helm en un entorno de producción práctico
kubernetes/helm/entornos/prod-env \
-n práctica --crear-espacio-de-nombres \
--esperar
```

4. Ejecute la prueba para verificar que el entorno del sistema funciona como se espera:

```
HOST=minikube.me PUERTO=8443 USE_K8S=verdadero ./test-em-all.bash
```

Cuando haya terminado, limpie los recursos creados tanto en Kubernetes como en Docker usando los siguientes comandos:

1. Detenga los comandos `kubect! get cert -w` y `kubect! get events -w` si aún se están ejecutando usando `Ctrl + C`.
2. Elimine el espacio de nombres en Kubernetes con el siguiente comando:

```
Eliminación práctica de espacios de nombres en kubect!
```

3. Detenga MySQL, MongoDB y RabbitMQ con el siguiente comando:

```
evaluar $(minikube docker-env)
Docker Compose Down
```

Con esto, terminamos con todas las pruebas ejecutándose en Kubernetes. Veamos cómo verificar que... Los croservicios todavía funcionan sin Kubernetes.

Verificar que los microservicios funcionen sin Kubernetes

En este capítulo y en el anterior, vimos cómo las características de la plataforma Kubernetes, como ConfigMaps, Secrets, Services y objetos Ingress, pueden simplificar el desarrollo de un entorno de microservicios cooperativos. Sin embargo, es importante garantizar que el código fuente de los microservicios no dependa funcionalmente de la plataforma.

Evitar este bloqueo permite cambiar a otra plataforma en el futuro, si es necesario, con un mínimo esfuerzo. Cambiar de plataforma no debería requerir cambios en el código fuente, solo en la configuración de los microservicios.

Probar los microservicios con Docker Compose y ejecutar el script de verificación `test-em-all.bash` garantizará su funcionamiento sin Kubernetes. Dado que Kubernetes es una herramienta mucho más avanzada y compleja que Docker Compose, ejecutar pruebas de integración con algunos microservicios cooperativos mediante minikube lleva más tiempo que con Docker Compo

Esto hace que Docker Compose sea una herramienta para desarrolladores más conveniente. Sin embargo, minikube es una muy buena herramienta para verificar que los microservicios funcionen como se espera en un entorno local de Kubernetes antes de implementarlos en entornos de prueba, control de calidad y producción.

Al ejecutar microservicios sin Kubernetes, careceremos de las funciones no funcionales que Kubernetes nos proporciona, como la monitorización, el escalado y el reinicio de contenedores. Al usar Docker Compose, reemplazaremos las siguientes funciones de Kubernetes:

- En lugar de ConfigMaps, utilizaremos volúmenes que asignan los archivos de configuración directamente desde el sistema de archivos del host
- En lugar de utilizar secretos, mantendremos información confidencial, como credenciales, en el archivo .env de Docker Compose
- En lugar de un Ingress, utilizaremos Spring Cloud Gateway
- En lugar de Servicios, asignaremos los nombres de host utilizados por los clientes directamente a los nombres de host de los contenedores, lo que significa que no tendremos ningún descubrimiento de servicio implementado y No poder escalar contenedores

Usar Docker Compose de esta manera resultará en desventajas significativas desde una perspectiva no funcional en comparación con Kubernetes. Sin embargo, esto es aceptable, dado que Docker Compose solo se usará para ejecutar pruebas funcionales.

Repasemos los cambios en los archivos `docker-compose*.yaml` antes de ejecutar las pruebas con Docker Compose.

Cambios en los archivos de Docker Compose

Para ejecutar microservicios fuera de Kubernetes, utilizando Docker Compose, se han aplicado los siguientes cambios a los archivos `docker-compose*.yaml`:

- Hemos eliminado la definición del servidor de configuración.
- Hemos eliminado el uso de las siguientes variables de entorno del servidor de configuración:
`CONFIG_SERVER_USR` y `CONFIG_SERVER_PWD`
- Hemos mapeado la carpeta `config-repo` como un volumen en cada contenedor que necesita leer archivos de configuración del repositorio de configuración
- Hemos definido la variable de entorno `SPRING_CONFIG_LOCATION` para apuntar al con-
archivos de configuración en el repositorio de configuración
- Hemos almacenado información confidencial, como credenciales y contraseñas, en certificados TLS en el archivo .env de Docker Compose

- Hemos definido variables de entorno con credenciales para el acceso a los administradores de recursos utilizando las variables definidas en el archivo .env

Por ejemplo, la configuración del microservicio del producto se ve así en Docker:

componer.yml:

```
producto:
  construir: microservicios/producto-servicio imagen: práctica/
  producto-servicio
  ambiente:
    - SPRING_PROFILES_ACTIVE=docker -
    SPRING_CONFIG_LOCATION=archivo:/config-repo/application.yml,
                                archivo:/config-repo/producto.yml
    - SPRING_RABBITMQ_NOMBRE_USUARIO=${RABBITMQ_USR}
    - CONTRASEÑA_DE_CONEJO_SPRING_MQ=${CONEJO_DE_CONEJO}
    - BASE DE DATOS DE AUTENTICACIÓN DE SPRING_DATA_MONGODB=admin
    - SPRING_DATA_MONGODB_NOMBRE_USUARIO=${MONGODB_USR}
    - CONTRASEÑA DE MONGODB DE DATOS DE SPRING=${MONGODB_PWD}
  volúmenes:
    - $PWD/config-repo:/config-repo
```

Aquí hay una explicación del código fuente:

- La carpeta config-repo se asigna como un volumen en el contenedor en /config-repo • La variable de entorno SPRING_CONFIG_LOCATION le dice a Spring dónde encontrar los archivos de propiedades, en este caso, los archivos /config-repo/application.yml y /config-repo/product.yml

- Las credenciales para acceder a RabbitMQ y MongoDB se configuran como variables de entorno basado en el contenido del archivo `.env`

Las credenciales a las que se hace referencia en el código fuente anterior se definen en el archivo `.env` de la siguiente manera:

```
RABBITMQ_USR=conejo-usuario-prod
RABBITMQ_PWD=conejo-pwd-prod
MONGODB_USR=mongodb-usuario-prod
MONGODB_PWD=mongodb-pwd-prod
```

Pruebas con Docker Compose. Para realizar pruebas

con Docker Compose, usaremos Docker Desktop en lugar de minikube. Siga estos pasos:

1. Para indicarle al cliente Docker que use Docker Desktop en lugar de minikube, ejecute lo siguiente dominio:

```
evaluar $(minikube docker-env --unset)
```

2. Para evitar conflictos de puertos en el puerto 8443, debe detener la instancia de minikube:

```
parada de minikube
```

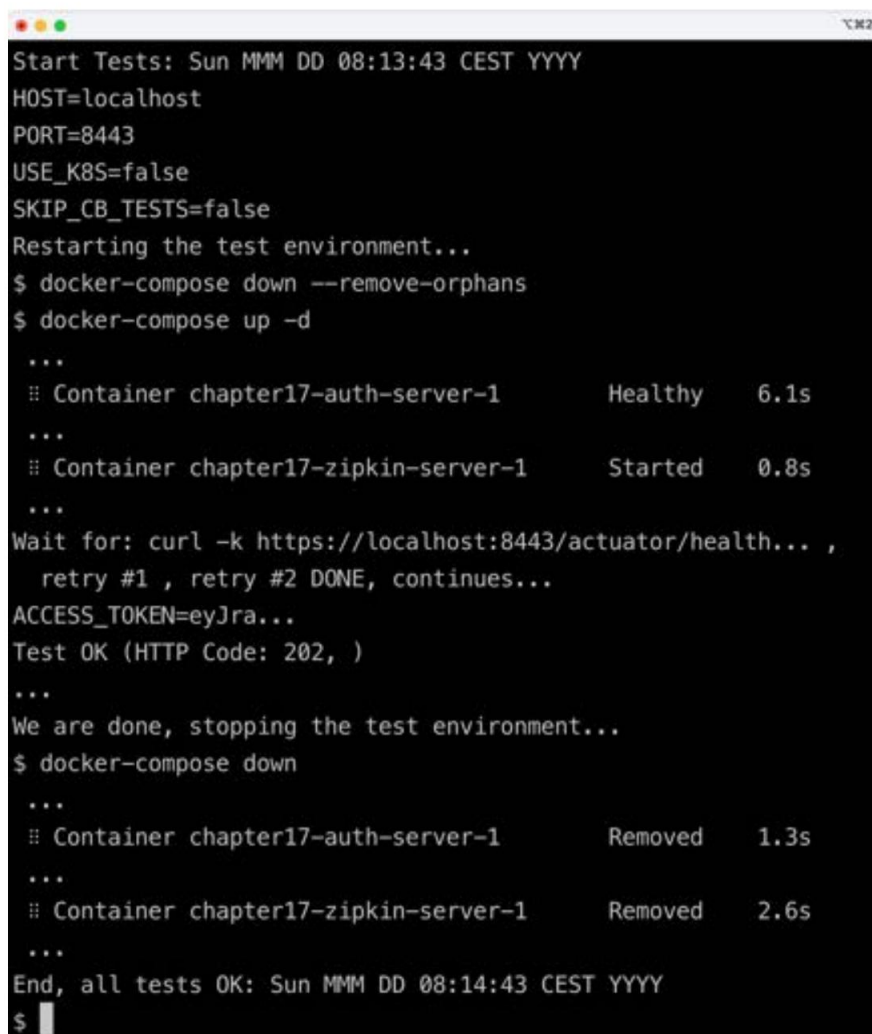
3. Cree las imágenes de Docker en Docker Desktop con el siguiente comando:

```
compilación de docker compose
```

4. Ejecute las pruebas usando RabbitMQ (con una partición por tema):

```
COMPOSE_FILE=docker-compose.yml ./test-em-all.bash inicio detención
```

5. Las pruebas deben comenzar iniciando todos los contenedores, ejecutándolos y, finalmente, deteniéndolos .
Se espera un resultado similar al de los capítulos anteriores (resultado reducido para facilitar la lectura):

A terminal window with a dark background and light text. It shows the execution of a test script. The script starts by displaying the current date and time, then sets environment variables like HOST, PORT, USE_K8S, and SKIP_CB_TESTS. It then restarts the test environment by running 'docker-compose down --remove-orphans' and 'docker-compose up -d'. This is followed by a series of status checks for containers 'chapter17-auth-server-1' and 'chapter17-zipkin-server-1'. A health check is performed using 'curl' on the 'actuator/health' endpoint. After a successful test (HTTP 202), the script stops the environment with 'docker-compose down' and shows the removal of the containers. The terminal ends with the message 'End, all tests OK: Sun MMM DD 08:14:43 CEST YYYY' and a prompt '\$'.

```
Start Tests: Sun MMM DD 08:13:43 CEST YYYY
HOST=localhost
PORT=8443
USE_K8S=false
SKIP_CB_TESTS=false
Restarting the test environment...
$ docker-compose down --remove-orphans
$ docker-compose up -d
...
:: Container chapter17-auth-server-1      Healthy   6.1s
...
:: Container chapter17-zipkin-server-1    Started   0.8s
...
Wait for: curl -k https://localhost:8443/actuator/health... ,
  retry #1 , retry #2 DONE, continues...
ACCESS_TOKEN=eyJra...
Test OK (HTTP Code: 202, )
...
We are done, stopping the test environment...
$ docker-compose down
...
:: Container chapter17-auth-server-1      Removed   1.3s
...
:: Container chapter17-zipkin-server-1    Removed   2.6s
...
End, all tests OK: Sun MMM DD 08:14:43 CEST YYYY
$
```

Figura 17.9: Verificación de la funcionalidad del panorama del sistema sin utilizar Kubernetes

6. Opcionalmente, ejecute las pruebas utilizando RabbitMQ con múltiples particiones por tema:

```
ARCHIVO_DE_COMPOSICIÓN=docker-compose-partitions.yml ./test-em-all.bash start
detener
```

Espere un resultado similar a la prueba anterior.

7. Como alternativa, ejecute la prueba utilizando Kafka con múltiples particiones por tema:

```
ARCHIVO_DE_COMPOSICIÓN=docker-compose-kafka.yml ./test-em-all.bash inicio detención
```

Espere un resultado similar a la prueba anterior.



Dado que el agente de Kafka puede tardar unos segundos en decidir qué partición asignar a las instancias de los grupos de consumidores, las pruebas pueden fallar, ya que la operación de reequilibrio aún está en curso al iniciarse. Si la prueba falla, vuelva a ejecutar el comando, pero sin el indicador de inicio :

```
ARCHIVO_DE_COMPOSICIÓN=docker-compose-kafka.yml ./test-em-all.  
parada de golpe
```

8. Inicie la instancia de minikube y configure el espacio de nombres predeterminado como práctico:

```
inicio de minikube  
  
configuración de kubectrl contexto-actual $(configuración de kubectrl contexto-actual) --  
namespace=hands-on
```

Con la ejecución exitosa de estas pruebas, hemos verificado que los microservicios también funcionan sin Kubernetes.

Resumen

En este capítulo, hemos visto cómo las capacidades de Kubernetes pueden utilizarse para simplificar un entorno de microservicios, lo que significa que reducimos la cantidad de servicios de soporte que deben desarrollarse e implementarse junto con los microservicios. Hemos visto cómo los ConfigMaps y Secrets de Kubernetes pueden utilizarse para reemplazar Spring Cloud Config Server y cómo un objeto Ingress de Kubernetes puede reemplazar Spring Cloud Gateway.

El uso de cert-manager nos permitió aprovisionar automáticamente certificados para puntos finales HTTPS expuestos por el controlador de Ingress, eliminando la necesidad de trabajo manual y engorroso.

Para verificar que el código fuente de los microservicios pueda ejecutarse en otras plataformas (es decir, que no esté bloqueado en Kubernetes), implementamos los microservicios usando Docker Compose y ejecutamos el script de prueba test-em-all.bash .

En el próximo capítulo, conoceremos el concepto de malla de servicios y aprenderemos cómo se puede utilizar un producto de malla de servicios, Istio, para mejorar la observabilidad, la seguridad, la resiliencia y el enrutamiento en un panorama de microservicios cooperativos que se implementan en Kubernetes.

¡Pasa al siguiente capítulo!

Preguntas

1. ¿Cómo se reemplazó el servidor Spring Cloud Config por recursos de Kubernetes?
2. ¿Cómo se reemplazó Spring Cloud Gateway por recursos de Kubernetes?
3. ¿Qué se necesita para que cert-manager aprovisiona automáticamente certificados para un Ingress?
¿objeto?
4. ¿Cómo se puede comprobar y actualizar el tiempo de retención de un certificado?
5. ¿Dónde se almacena el certificado real?
6. ¿Por qué ejecutamos las pruebas utilizando Docker Compose?

Desbloquea los beneficios exclusivos de este libro ahora

Escanee este código QR o vaya a packtpub.com/unlock, Luego busque este libro por nombre.

Nota: Tenga a mano la factura de compra antes de comenzar.



18

Usando una malla de servicios para Mejorar la observabilidad y Gestión

En este capítulo, se presentará el concepto de malla de servicios y se verá cómo sus capacidades pueden utilizarse para afrontar los desafíos de un entorno de sistemas de microservicios en áreas como la seguridad, la aplicación de políticas, la resiliencia y la gestión del tráfico. Una malla de servicios también puede utilizarse para proporcionar observabilidad, es decir, la capacidad de visualizar el flujo de tráfico entre microservicios.

Una malla de servicios se superpone parcialmente con las capacidades de Spring Cloud y Kubernetes que vimos anteriormente en este libro. Sin embargo, la mayor parte de la funcionalidad de una malla de servicios complementa a Spring Cloud y Kubernetes, como veremos en este capítulo.

En este capítulo se tratarán los siguientes temas:

- Una introducción al concepto de malla de servicios e Istio, una popular implementación de código abierto.
tación
- Implementación de Istio en Kubernetes
- Creación, observación y protección de una malla de servicios
- Garantizar que una malla de servicios sea resiliente
- Realizar actualizaciones sin tiempo de inactividad
- Probar el panorama de microservicios usando Docker Compose para garantizar que el código fuente en los microservicios no está bloqueado ni en Kubernetes ni en Istio

Requisitos técnicos

Para obtener instrucciones sobre cómo instalar las herramientas utilizadas en este libro y cómo acceder al código fuente

Para este libro, consulte los siguientes capítulos:

- Capítulo 21, Instrucciones de instalación para macOS
- Capítulo 22, Instrucciones de instalación para Microsoft Windows con WSL 2 y Ubuntu

Todos los ejemplos de código de este capítulo provienen del código fuente en `$BOOK_HOME/Chapter18`.

Si desea ver los cambios aplicados al código fuente en este capítulo, es decir, ver lo que se necesitó para crear una malla de servicios con Istio, puede compararlo con el código fuente del Capítulo 17, "Implementación de funciones de Kubernetes para simplificar el entorno del sistema". Puede usar su diff favorito.

herramienta y compare las dos carpetas, `$BOOK_HOME/Chapter17` y `$BOOK_HOME/Chapter18`.

Introducción a las mallas de servicios mediante Istio

Una malla de servicios es una capa de infraestructura que controla y observa la comunicación entre servicios, por ejemplo, microservicios. Las funcionalidades de una malla de servicios, como la observabilidad, la seguridad, la aplicación de políticas, la resiliencia y la gestión del tráfico, se implementan controlando y supervisando toda la comunicación interna dentro de la malla de servicios, es decir, entre...

Los microservicios en la malla de servicios.

Uno de los componentes principales de una malla de servicios es un componente proxy ligero, que se inyecta en cada microservicio que formará parte de ella. Todo el tráfico entrante y saliente de un microservicio se configura para pasar por su componente proxy. Los componentes proxy se configuran en tiempo de ejecución.

Mediante un plano de control en la malla de servicios, se utilizan las API expuestas por el proxy. El plano de control también recopila datos de telemetría de los proxies a través de estas API para visualizar el flujo de tráfico en la malla de servicios.

Una malla de servicios también contiene un plano de datos, compuesto por los componentes proxy junto con componentes independientes para gestionar el tráfico externo hacia y desde la malla de servicios, conocidos como puerta de enlace de entrada y puerta de enlace de salida, respectivamente. Los componentes de la puerta de enlace también se comunican con el plano de control mediante un componente proxy. Esto se ilustra en el siguiente diagrama:

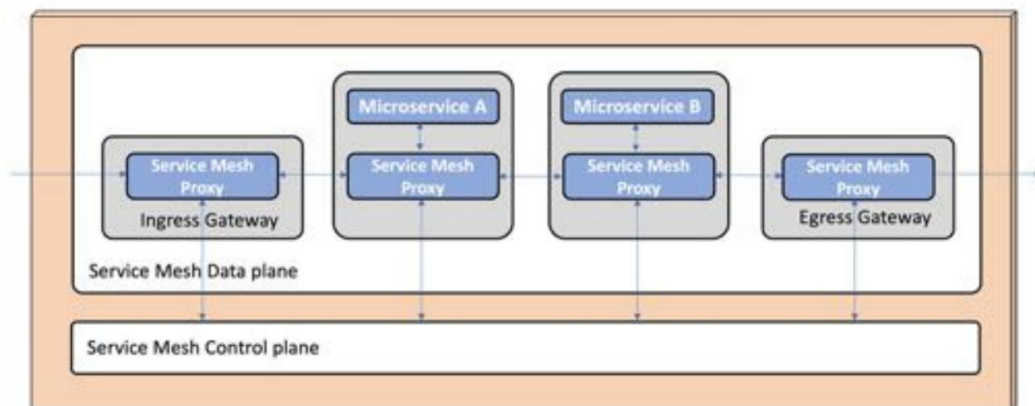


Figura 18.1: Malla de servicio con un plano de control y un plano de datos

La primera implementación disponible públicamente de una malla de servicios fue el proyecto de código abierto Linkerd, administrado por Buoyant (<https://linkerd.io>), que se originó en el proyecto Finagle de Twitter (<http://twitter.github.io/finagle>). Se lanzó en 2016 y, un año después, en 2017, IBM, Google y Lyft lanzaron el proyecto de código abierto Istio (<https://istio.io>). Desde entonces, se han lanzado varios proyectos de malla de servicios. Para obtener una descripción general de las implementaciones disponibles, consulte la categoría de malla de servicios en el mapa del panorama nativo de la nube de CNCF: <https://landscape.cncf.io/card-mode?category=service-mesh&grouping=category>. En este libro utilizaremos Istio.

Presentando Istio

Istio se puede implementar en varias distribuciones y plataformas de Kubernetes utilizando varias herramientas de instalación, como se describe en <https://istio.io/docs/setup>. Utilizaremos la herramienta CLI de Istio, `istioctl`, para instalar Istio en nuestro clúster Kubernetes de un solo nodo basado en Minikube.

Istio, como se explicó anteriormente, se divide en un plano de control y un plano de datos. Como operador, definiremos el estado deseado mediante la creación de objetos de Istio en el servidor de API de Kubernetes, por ejemplo, declarando reglas de enrutamiento. El plano de control leerá estos objetos y enviará comandos a los proxies en el plano de datos para que actúen según el estado deseado, por ejemplo, configurando reglas de enrutamiento. Los proxies gestionan la comunicación entre los microservicios y reportan datos de telemetría al plano de control. Estos datos se utilizan en el plano de control para visualizar lo que sucede en la malla de servicios.

Al implementar Istio en Kubernetes, la mayoría de sus componentes de ejecución se implementan en un espacio de nombres de Kubernetes independiente, `istio-system`. Para la configuración que utilizaremos en este libro, encontraremos las siguientes implementaciones en este espacio de nombres:

- `istiod`, el demonio de Istio que ejecuta todo el plano de control.



Dato curioso: Hasta la versión 1.4 de Istio, el plano de control se dividía en un conjunto de microservicios cooperativos. A partir de la versión 1.5, se consolidaron en un único binario ejecutado por `istiod`, lo que simplificó la instalación y configuración del plano de control en tiempo de ejecución. Además, se mejoraron características en tiempo de ejecución como el tiempo de arranque, el uso de recursos y la capacidad de respuesta. Esta evolución del plano de control de Istio es, en mi opinión, una lección interesante sobre el uso de microservicios de granularidad fina.

- `istio-ingressgateway` e `istio-egressgateway`, la puerta de enlace de entrada y salida de Istio
- Los componentes son parte del plano de datos.

Istio admite diversas integraciones con otros proyectos populares de código abierto para añadir funcionalidad adicional al plano de control. En este libro, integraremos los siguientes componentes:

- **Kiali**: proporciona observabilidad a la malla de servicio, visualizando lo que está sucediendo en la malla. Para obtener más información, consulte <https://www.kiali.io>.
- Rastreo** : Gestiona y visualiza información de rastreo distribuida basada en Jaeger o Zipkin. Usaremos Jaeger. Para más información, consulte <https://www.jaegertracing.io>.
- **Prometheus**: Ingiera y almacena datos basados en series temporales, por ejemplo, métricas de rendimiento. Para más información, consulte <https://prometheus.io>.
- **Grafana**: visualiza métricas de rendimiento y otros datos relacionados con series de tiempo recopilados por Prometheus. Para más información, consulte <https://grafana.com>.



En el Capítulo 20, Monitoreo de microservicios, exploraremos las capacidades de monitoreo del rendimiento usando Prometheus y Grafana.

Para obtener más información sobre la integración disponible en Istio, consulte <https://istio.io/latest/docs/operaciones/integraciones/>.

Los únicos componentes de Istio que se implementan fuera del espacio de nombres istio-system son los componentes proxy, que se inyectan en los microservicios que forman parte de la malla de servicios.

El componente proxy se basa en el proxy Envoy de Lyft (<https://www.envoyproxy.io>).

Los componentes de tiempo de ejecución en el plano de control y el plano de datos de Istio se resumen en el siguiente diagrama:

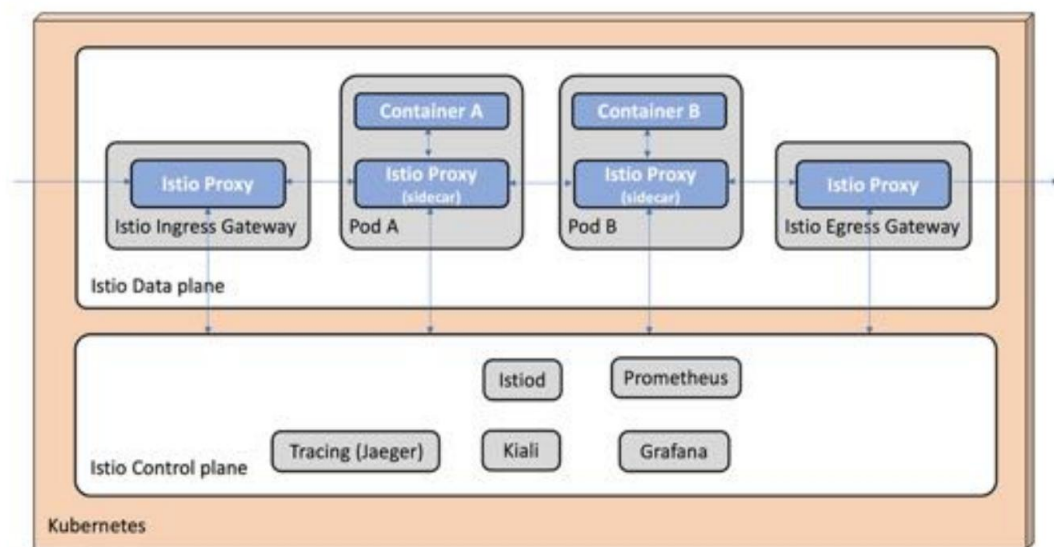


Figura 18.2: Componentes de tiempo de ejecución de Istio

Ahora que hemos tenido una introducción, veremos cómo se pueden inyectar estos objetos proxy en los microservicios.

Inyección de proxies de Istio en microservicios

Los microservicios que implementamos en Kubernetes en los capítulos anteriores se ejecutan como un único contenedor en un pod de Kubernetes (consulte la sección "Introducción a los objetos de la API de Kubernetes" del capítulo 15, "Introducción a Kubernetes") para obtener un resumen. Para que un microservicio se una a una malla de servicios basada en Istio, se inyecta un proxy de Istio en cada uno. Esto se logra añadiendo un contenedor adicional al pod que ejecuta el proxy de Istio.



Un contenedor agregado a un Pod con el objetivo de soportar el contenedor principal, como un proxy Istio, se denomina sidecar.

El siguiente diagrama muestra cómo se ha inyectado un proxy de Istio en un Pod de muestra, Pod A, como un sidecar:

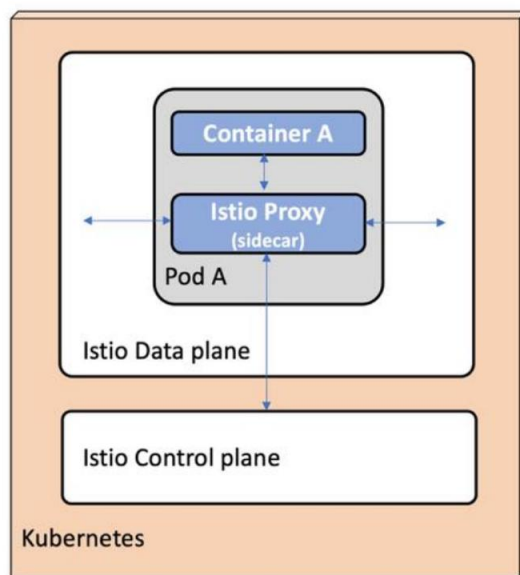


Figura 18.3: Proxy de Istio inyectado en el Pod A

El contenedor principal del Pod, el Contenedor A, está configurado para enrutar todo su tráfico a través de Istio apoderado.

Los proxies de Istio se pueden inyectar automáticamente al crear un objeto Pod o manualmente con la herramienta `istioctl`. Para que Istio inyecte automáticamente un proxy de Istio en los nuevos Pods de un espacio de nombres, este se puede etiquetar con `istio-injection: enabled`. Si se van a excluir algunos Pods del espacio de nombres de la inyección automática, se pueden anotar con `istio.io/inject: "false"`.

Para inyectar un proxy Istio manualmente en los pods de un objeto de implementación existente, haga lo siguiente. El comando se puede utilizar:

```
kubectl obtener implementación ejemplo-implementación -o yaml | istioctl kube-inject -f - | kubectl
aplicar -f -
```

Este comando puede parecer algo intimidante a primera vista, pero en realidad son solo tres comandos independientes. El comando anterior envía su salida al siguiente comando mediante barras verticales, es decir, el carácter `|`. Analicemos cada comando:

- El comando `kubectl getdeployment` obtiene la definición actual de una implementación denominada `sample-deployment` del servidor de API de Kubernetes y devuelve su definición en formato el formato YAML.

El comando `istioctl kube-inject` lee la definición del comando `kubectl getdeployment` y añade un contenedor adicional para un proxy de Istio en los pods gestionados por la implementación. La configuración del contenedor existente en el objeto de implementación se actualiza para que el tráfico entrante y saliente pase por el proxy de Istio.

- El comando `istioctl` devuelve la nueva definición del objeto de implementación, incluido un contenedor para el proxy Istio.

El comando `kubectl apply` lee la configuración actualizada del comando `istioctl kube-inject` y la aplica. Se iniciará una actualización de los pods pertenecientes a la implementación de la misma manera que vimos anteriormente (consulte la sección "Probar una implementación de ejemplo" del Capítulo 15, Introducción a Kubernetes).

En este libro, inyectaremos los proxies de Istio automáticamente aplicando la siguiente definición del espacio de nombres práctico:

```
Versión de API: v1
tipo: Espacio de nombres
metadatos:
  nombre: práctico
  etiquetas:
    inyección de istio: habilitada
```

De la definición anterior, podemos ver cómo al espacio de nombres se le asigna la etiqueta `istio-injection` con el valor `habilitado`.

Al momento de escribir este artículo, Istio no es totalmente capaz de actuar como proxy para MySQL, MongoDB y RabbitMQ, por lo que se excluirán de la malla de servicio agregando la siguiente anotación al archivo `values.yaml` de sus gráficos Helm :

```
anotaciones:
  sidecar.istio.io/inject: "falso"
```

Después de esta introducción sobre cómo se pueden inyectar los proxies de Istio en los pods, ahora podemos aprender sobre los objetos de la API de Istio utilizados en este libro.

Presentación de los objetos de la API de Istio

Istio también incluye un conjunto de Definiciones de Recursos Personalizados (CRD) de Kubernetes. Las CRD se utilizan en Kubernetes para ampliar su API, es decir, para añadir nuevos objetos. Consulte la sección "Introducción a los objetos de la API de Kubernetes" del Capítulo 15, "Introducción a Kubernetes", para obtener un resumen de la API de Kubernetes. En este libro, utilizaremos los siguientes objetos de Istio:

- La puerta de enlace se utiliza para configurar cómo gestionar el tráfico entrante y saliente de la malla de servicios. Una puerta de enlace depende de un servicio virtual que enruta el tráfico entrante a los servicios de Kubernetes. Utilizaremos un objeto de puerta de enlace para aceptar el tráfico entrante a nombres DNS que terminan en minikube.me mediante HTTPS. Los objetos de puerta de enlace de Istio reemplazarán a los objetos de Ingress utilizados en el capítulo anterior. Consulte la sección "Sustitución del controlador de Ingress de Kubernetes por una puerta de enlace de Istio" para obtener más información.
- **VirtualService** se utiliza para definir reglas de enrutamiento en la malla de servicios. Utilizaremos servicios virtuales para describir cómo enrutar el tráfico entrante desde una puerta de enlace de Istio a los servicios de Kubernetes y entre servicios. También utilizaremos servicios virtuales para inyectar fallos y retrasos con el fin de probar la fiabilidad y la resiliencia de la malla de servicios.
- **DestinationRule** se utiliza para definir políticas y reglas para el tráfico que se enruta (mediante un servicio virtual) a un servicio específico (es decir, un destino). Utilizaremos reglas de destino para configurar políticas de cifrado que encripten el tráfico HTTP interno y definir subconjuntos de servicios que describan las versiones disponibles de los servicios. Utilizaremos subconjuntos de servicios al realizar implementaciones sin tiempo de inactividad (azul-verde) desde una versión existente de un microservicio a... una nueva versión.



La división de responsabilidades entre **VirtualService** y **DestinationRule** puede parecer confusa al principio. Un objeto **VirtualService** se utiliza para configurar el enrutamiento a un servicio y **DestinationRule** para configurar cómo gestionar el tráfico de un servicio seleccionado. Por lo tanto, primero están los objetos **VirtualService**, que se utilizan para determinar a dónde enviar una solicitud. Una vez decidido esto, se aplica la **DestinationRule** del servicio receptor.

- **PeerAuthentication** se utiliza para controlar la autenticación de servicio a servicio dentro del servicio.
- Malla. Istio puede proteger la comunicación entre servicios en una malla de servicios mediante el aprovisionamiento automático de TLS mutuo (mTLS) para la autenticación de transporte. Los servicios cliente se autentican mediante un certificado de cliente proporcionado por Istio. Para permitir que Kubernetes invoque sondas de actividad y preparación mediante HTTP simple, configuraremos Istio para permitir una combinación de mTLS y HTTP simple, denominada modo PERMISIVO.

RequestAuthentication se utiliza para autenticar a los usuarios finales según las credenciales proporcionadas en una solicitud. Istio admite el uso de tokens web JSON (JWT) en general y, específicamente, cuando se utilizan según la especificación de OpenID Connect (OIDC) . Istio admite el uso del punto de acceso de descubrimiento estándar en OIDC para especificar dónde puede obtener el conjunto de claves públicas (JSON Web Key Set [JWKS]) para validar las firmas de los JWT. Configuraremos Istio para autenticar solicitudes externas mediante el servidor de autenticación, especificando su punto de acceso de descubrimiento JWKS. Para un resumen, consulte el capítulo 11, "Seguridad del acceso a las API".

- La Política de Autorización se utiliza para proporcionar control de acceso en Istio. No utilizaremos el acceso de Istio. En este libro, reutilizaremos el control de acceso existente implementado en el microservicio compuesto por producto . Por lo tanto, configuraremos una política de autorización. objeto que permite el acceso al microservicio compuesto por producto para cualquier usuario autenticado, es decir, para solicitudes que contengan un JWT válido en forma de token de acceso OIDC.

Ahora que hemos presentado los objetos API que usaremos, repasaremos los cambios aplicados al panorama de microservicios que surgen de la introducción de Istio.

Simplificando el panorama de microservicios

Como vimos en la sección anterior, Istio viene con componentes que se superponen con los componentes que se utilizan actualmente en el panorama de microservicios en términos de funcionalidad:

- La puerta de enlace de ingreso de Istio puede actuar como un servidor de borde, una alternativa a un servidor de ingreso de Kubernetes controlador
- El componente Jaeger que viene incluido con Istio se puede utilizar para el seguimiento distribuido en lugar del servidor Zipkin que implementamos junto con los microservicios.

En las siguientes dos subsecciones, obtendremos una descripción general de por qué y cómo se reemplaza el controlador de ingreso de Kubernetes con una puerta de enlace de ingreso de Istio, y por qué y cómo se reemplaza nuestro servidor Zipkin con el componente Jaeger que viene integrado con Istio.

Reemplazo del controlador de ingreso de Kubernetes con una puerta de enlace de ingreso de Istio

En el capítulo anterior, presentamos el controlador de entrada de Kubernetes como servidor perimetral (consulte la sección " Sustitución de Spring Cloud Gateway" del capítulo 17, " Implementación de funciones de Kubernetes para simplificar el entorno del sistema"). Una puerta de enlace de entrada de Istio ofrece varias ventajas sobre un controlador de entrada de Kubernetes:

- Puede informar datos de telemetría al plano de control para el tráfico que fluye a través de él.